From Calculus to the Money Number

The astrophysics, cosmology and mathematics behind this repository

Agentic Lensing — USF Computer Science

Contents

1. What this repository does, and what one number costs

This repository does two jobs that share almost nothing except the word "lens." The first job is a search: comb through tens of millions of galaxy images for the rare few whose light has been bent by something massive sitting in front of them. The second job is a fit: once a system is

confirmed as a genuine lens, extract a small number of physical parameters — a mass, a shape, a slope — from the pixels of its image. This guide takes you from calculus you already own to the point where you can derive the mathematics behind both jobs yourself, and then check it against numbers this repository has actually published. It has one destination: a single number, produced by the repository's most ambitious modeling campaign to date,

$$\gamma = 1.103 \pm 0.008$$

the density slope of one galaxy's mass profile . γ here is the exponent in $\rho \sim r^{-\gamma}$: how fast mass density falls off with radius. It is not the shear — this repository always spells shear γ_{ext}, or its components γ_1, γ_2, never a bare γ. That distinction gets restated at every chapter that could possibly confuse them, starting now, because the two most common ways to misread this repository's mathematics are conflating that γ with shear, and conflating source-plane position β with an inverse temperature (also usually called β everywhere outside lensing — this repository calls its tempering parameter λ instead, precisely to avoid that collision). By Chapter 25 you will know exactly what pipeline produced 1.103 ± 0.008, what the repository's own report claims about how surprising that number is, and where that claim's arithmetic actually goes when you do it yourself, one division at a time.

The two halves

Read `reproductions/REPRODUCTIONS.md` and the two halves fall out immediately: one set of lineages *finds* candidate lenses, another *models* the ones that survive. They are different disciplines wearing the same word.

Discovery starts from an imaging survey — the DESI Legacy Surveys' *grz* imaging, tens of millions of galaxy cutouts — and asks a yes/no question of each one: is this a lens? The tools are a convolutional classifier (`huang-2020`, `huang-2021`, `inchausti-2025`), a difference-imaging pipeline that catches a lensed transient by what changes between exposures (`sheu-2023`, `sheu-2024a`), and a friends-of-friends spectroscopic search that flags any sightline carrying two incompatible redshifts (`hsu-2025`, `dawes-2022`). The output of discovery is not a lens; it is a *candidate*, carrying a grade — a confidence label attached by a human, or by a machine standing in for one. Chapter 27 derives why this problem gets structurally harder as the survey's resolution runs out, and Chapter 28 (`reproductions/lensjudge/`) asks the question this guide considers its most consequential for a machine-learning audience: how good is that grade, actually? On the 130 systems (`reproductions/lensjudge/parity/FINDINGS.md:39`) where independent follow-up later confirmed or refuted the candidate, the human ordinal grade predicts that later truth at AUC 0.577, indistinguishable from a coin flip . Purity is flat across the grade itself — 95%, 92%, 91% for grades A, B, C — and two trained graders agree with each other at a quadratic-weighted kappa of only 0.42 . Whatever you train against a label this noisy inherits its ceiling. That is Chapter 28's whole argument, and it is why the outline puts it second, immediately after this one.

Modeling starts one step later: a lens is confirmed, its redshifts are measured, and the question is no longer "is it a lens" but "what, precisely, is doing the lensing." The lineage runs `gu-2022` (the method) through `cikota-2023` and `sheu-2024b` (single systems), `foundry-i` (the first genuine HMC posterior on a real HST system), to `claude-giga-lens` (`reproductions/REPRODUCTIONS.md:27`), the campaign that produces this guide's destination number. Where discovery is millions of low-dimensional decisions against a label nobody fully trusts, modeling is one system at a time against a forward model you can write down exactly —

a few dozen parameters, a render, a per-pixel comparison — but whose *posterior geometry* turns out to be vicious: nearly flat in some directions, curved by fourteen orders of magnitude in others (a condition number of 10^{14}, `reproductions/claude-giga-lens/papers/main.tex:352`), and in one documented case, a maximum-a-posteriori point that is not a maximum at all. Discovery stress-tests the label. Modeling stress-tests the likelihood and the sampler. Keep that distinction in view: a huge fraction of this guide (Parts I, IV, and V) exists to make modeling's failures derivable, and Chapter 28 exists to make discovery's failure *undeniable* — and, notably, dependent on no physics or calculus at all.

The Rosetta stone

Every idea in the modeling half has a name you already carry from machine learning. This repository's mathematics is not a new subject: it is a small, specific set of linear-algebra and probability operations, run on sky pixels instead of tensors. The table below is the guide's map from what you own to what this repository calls it; each row gets derived properly, once, at the chapter cited.

What you already know	What this repository calls it	Derived at
The change-of-variables factor $\lvert\det J\rvert$ in a normalizing flow	Magnification, $\mu = 1/\det A$	Ch. 4, Ch. 18
Cross-entropy loss, measured in nats	The Bayesian evidence $\log Z$, always quoted in nats	Ch. 8
Weight decay, an L_2 penalty on a linear layer	A Gaussian prior on the linear source amplitudes	Ch. 8, Ch. 22
A BIC-style complexity penalty	The $-\frac{1}{2}\log\det A$ Occam term that `gigalens` omits from its linear solve	Ch. 22, Ch. 23
The condition number of a loss Hessian	A parameter degeneracy: a flat direction the data cannot see	Ch. 5, Ch. 21
A saddle point a naive optimizer mistakes for a minimum	A real system's MAP fit, confirmed to be a saddle, not a mode	Ch. 5, Ch. 26
Whitening features before trusting a diagonal covariance	Whitening drizzled pixel noise before trusting a per-pixel Gaussian likelihood	Ch. 7, Ch. 24
A temperature schedule in simulated annealing	Tempered sequential Monte Carlo, $\lambda : 0 \to 1$	Ch. 23
A classifier's chosen operating point (TPR at fixed FPR)	The lens-finder's decision threshold, and the resolution wall behind it	Ch. 27
A label so noisy it caps a classifier's achievable AUC	A human "grade" that predicts truth barely above chance	Ch. 28

> **You already know this**
>
> The evidence $\log Z$ this repository tallies is in nats — the same unit a cross-entropy loss is reported in, because it is the same integral: a log-probability, summed in the natural base. A swing of 191 nats in $\log Z$ is a likelihood ratio of e^{191}. Jeffreys' scale calls anything past 5 nats

> of $\log Z$ "decisive" — this is 38 times that bar, before you have touched a single equation of gravitational lensing.

That box previews the entire shape of the guide's central device, the **Log-Det Ledger**: the quantity $\log|\det(\cdot)|$ of a matrix appears three times in this repository, in three costumes — magnification, a normalizing flow's change of variables, and a Bayesian Occam factor — and every chapter that meets one adds a row. Chapter 23 closes it, having shown all three are the same computation.

The money number

The report itself calls 1.103 ± 0.008 "the money number" (`reproductions/claude-giga-lens/papers/main.tex:7`) and the name is earned: it is the number the whole `claude-giga-lens` campaign was built to produce. Here is the shape of the argument, with no derivation yet — that is Chapters 20 through 25's job — only the destination.

A real HST system, `foundry-i`'s target, was drizzled (resampled) to three pixel scales for the same exposures. At the *binned* scale, a per-pixel diagonal noise model — the statistical core of every fast GPU lens code, including `gigalens` itself — produced a posterior for γ that split into two disconnected basins, with zero chains crossing between them. That diagonal likelihood was confident the "steep" basin was the right one: its own evidence favored steep over low by $+162.2$ nats . The campaign's contribution is a likelihood that models the drizzle noise as genuinely correlated between pixels rather than diagonal, built and validated in Chapter 24 on top of the marginalization machinery of Chapter 22. Applied to the same binned product, that correlated likelihood reverses the verdict outright: evidence now favors the low basin by -28.9 nats — a swing of 191.1 nats in total. The steep basin was a noise-covariance artifact of the diagonal likelihood, not a genuine second mode. That flip is the campaign's real-data verdict, and Chapter 25 calls it Hypothesis 1, confirmed.

But the correlated likelihood does not stop there — it also *moves* the recovered value of γ itself, and moves it further than the campaign expected. The restored low basin converges, under a 128-particle tempered SMC sampler (`reproductions/claude-giga-lens/papers/main.tex:761`), to $\gamma = 1.103 \pm 0.008$. The campaign's pre-registered anchor — a diagonal fit at the native, least-resampled pixel scale, the scale where a diagonal likelihood is closest to correct — sits at $\gamma = 1.433 \pm 0.034$. Those two numbers disagree by more than either of their quoted uncertainties can easily explain, and the report's own abstract puts a number on that disagreement: about 17σ (`reproductions/claude-giga-lens/papers/main.tex:98`). Hold that figure loosely. Chapter 25 has you compute the discrepancy yourself, from nothing but the four numbers already on this page, and it does not come out to 17 — under any of the three ways you might reasonably define "sigma" here. That gap between a headline claim and its own arithmetic is not a gotcha invented for this guide; the report's own footnote, `reproductions/claude-giga-lens/papers/main.tex:763`, records a second, smaller figure right next to the first.

Before you read any further into this guide: write down, in one sentence, your best guess right now at whether $\gamma = 1.103 \pm 0.008$ is a trustworthy measurement of this galaxy's density slope, and your one reason why. Do not look ahead. Chapter 25's closing section, "the verdict", is where you open it.

How to read this guide

The book has seven parts after this one, and they are not meant to be read in a straight line by default.

Part	Chapters	What it buys
I — The mathematical spine	2–8	Derivatives through probability, built from the calculus and linear algebra you already have
II — The physical universe	9–12	Arcseconds, galaxies, photons, redshifts: the vocabulary of an observation
III — Cosmology	13–15	Detachable — the lens-modeling likelihood in this repository uses no cosmology at all
IV — Gravitational lensing	16–21	The lens equation, magnification, the Einstein radius, mass profiles, degeneracies
V — This repository's science, decoded	22–26	The forward model, the samplers, the correlated-noise likelihood, and two dependency chains — Spine 1 (Ch. 25, the money number) and Spine 2 (Ch. 26, a saddle in the same campaign's optimization) — each ending in a verdict
VI — Discovery	27–28	The survey, the finders, the resolution wall, and the label that caps all of it
VII — Synthesis	29	Both ledgers close, a decoder from this repository's own reports to this guide, and where a CS professor has comparative advantage

One deliberate break from that order: **read Chapter 28 next.** It needs no physics and no calculus past what you already own — it is pure ML epistemics, a question about labels and AUC that you could ask of any classifier you have ever shipped — and it carries this repository's most striking result. Reading it second, before the calculus ramp of Part I, is the difference between doing homework and being on a case.

Two recurring devices carry the argument across chapters. A boxed **"You already know this"** tip, like the one above, marks a real bridge from ML to lensing at the exact point it becomes load-bearing — never decorative, never manufactured. And any chapter that constrains γ — starting

once the symbol has a precise meaning, in Chapter 20 — adds a row to a running γ **Ledger**, closed in Chapter 25, that tracks only one thing: what the evidence so far rules in or out about 1.103.

The guide's own credibility rests on one rule, and you can enforce it yourself: every computed number in this book is tagged with an HTML comment — `<!-- check: ch25.gamma_money = 1.103 ± 0.008 -->` is the one attached to this chapter's destination number — and every tag names a function in `site/guide_src/worked_examples.py` that computes it from scratch or pins it to a cited artifact. Run `~/.venvs/lensjudge/bin/python site/guide_src/worked_examples.py --check` yourself, right now if you like; it recomputes every pinned number in this guide and fails loudly if one has drifted. This repository's own final report carries a "$\sim 17\sigma$" claim that reconciles with none of the uncertainties it quotes elsewhere. This guide's answer to that is not a better adjective. It is a script you can run.

Connect to the repo

- `reproductions/REPRODUCTIONS.md:19`, `:29`, `:38` — the three lineage tables behind "the two halves": GIGA-Lens modeling, image-based finders, specialty discovery. Row `:27` is `claude-giga-lens` itself, the source of this chapter's destination number.
- `reproductions/claude-giga-lens/papers/main.tex:98` — the report's abstract, which states the whole campaign (and its $\sim 17\sigma$ claim) in one paragraph; `:756`–`:759` is the boxed money-number result itself; `:763` carries the footnote with the report's own arithmetic.
- `reproductions/claude-giga-lens/CAMPAIGN.md:133` — the dated gate record ("P1c MONEY NUMBER — 2026-07-14") every number in this chapter traces to; read the whole file once to see this repository's retraction culture in the raw, the register this guide tries to match.
- `reproductions/lensjudge/parity/FINDINGS.md:40` and `:108` — the grade-purity table and the headline AUC behind the discovery half's teaser in this chapter; Chapter 28 is where they get a full argument.
- `site/guide_src/worked_examples.py` — every tagged number in this guide, in one file, runnable; `site/guide_src/contract/outline.yml` is this guide's own chapter table, if you want to see the whole map at once instead of one part at a time.

Exercises

Exercise 1.1 — The sealed envelope

You already wrote this down once, above. If you skipped it, do it now: one sentence, your best guess at whether $\gamma = 1.103 \pm 0.008$ is a trustworthy measurement of the real slope of this galaxy's mass profile, and one reason why. Note the date. Do not revise it after reading further.

Solution

There is no answer to check here yet — only a receipt. Chapter 25 walks the same chain of evidence this chapter only named: the 191-nat basin flip, the cross-scale bracketing, and the sigma arithmetic the report's own abstract does not survive. When you reach "the verdict", reread your sentence before you read the chapter's own conclusion. The only way to fail this exercise is to have skipped writing the guess down before you knew where the argument was going.

Exercise 1.2 — Which half?

For each task below, say whether it belongs to discovery or modeling, and which Part of this guide you would need to have read to follow it in detail.

1. Deciding whether a 101×101-pixel DESI cutout contains a lens at all.
2. Recovering the axis ratio q and position angle ϕ of a confirmed lens's mass ellipse from its pixels.
3. Measuring the velocity dispersion of a candidate lens galaxy from its spectrum, to decide whether two objects on one sightline are really at different redshifts.
4. Deciding whether an evidence swing of 191 nats between two basins of a posterior is large enough to trust.

Solution

1. Discovery — Part VI (Ch. 27's finders and resolution wall).
2. Modeling — Part IV (Ch. 20's profiles and ellipticity) feeding Part V's inference machinery.
3. Discovery, though it leans on spectroscopy from Part II (Ch. 12) — this is exactly the friends-of-friends redshift-pair search `hsu-2025` and `dawes-2022` run, upstream of any imaging classifier.
4. Modeling — Part I's probability chapter (Ch. 8, evidence and nats) gives you the units; Part V (Ch. 23, Ch. 25) gives you the campaign's own answer.

Exercise 1.3 — Nats to a Bayes factor

The evidence swing between the two likelihoods on the binned product is 191.1 nats . A natural log and a base-10 log differ by a factor of $\ln 10 \approx 2.3026$. Convert 191.1 nats to a base-10 log-Bayes-factor by hand (divide by $\ln 10$), then say how many orders of magnitude past Jeffreys' "decisive" threshold of 5 nats (≈ 2.17 in $\log_{10}$) that leaves you.

Solution

$191.1/\ln 10 \approx 191.1/2.3026 \approx 83.0$. The correlated-likelihood basin flip is a Bayes factor of roughly 10^{83} — about 38 times past Jeffreys' decisive bar of $5/\ln 10 \approx 2.17$ in $\log_{10}$ units. Whatever else Chapter 25 finds wrong with this campaign's headline sigma claim, the basin flip itself is not a close call in any base you choose to log it in.

2. Derivatives, Taylor, and why linearization is the whole game

Everything hard in this book — the deflection field, the magnification, the Laplace approximation to a posterior, the mass matrix an HMC sampler needs to move efficiently — is the same two moves applied to a different function: (1) replace a nonlinear map by its best local *linear* model, and (2) when linear is not enough, add the best local *quadratic* correction. This chapter builds both moves in one dimension, where you can see every term, so that when they reappear in n dimensions (Ch. 4–5) as a Jacobian and a Hessian, the objects are already familiar and only the bookkeeping is new.

> **What you can skip**
>
> If you can state the derivative as a limit, prove the product and chain rules, and are comfortable with $O(h^2)$ meaning "shrinks like h^2 as $h \to 0$," skim Derivative as a linear map and slow down at Taylor to second order and Why second order — those two sections reframe familiar material as the object the rest of the book calls "the metric," "the Occam term," and "the saddle." This chapter does not redo limits or ϵ-δ arguments; you already own those.

Derivative as a linear map

The definition you know:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Rearrange it and you get the statement this book actually uses, over and over, under different names:

$$f(x+h) = f(x) + f'(x)\,h + o(h) \qquad \text{as } h \to 0,$$

where $o(h)$ means a remainder that shrinks *faster* than h itself — divide it by h and it still goes to zero. Read that equation as a claim about approximation, not about limits: near x, the graph of f is indistinguishable from the straight line through $(x, f(x))$ with slope $f'(x)$, up to an error that vanishes faster than the distance h you moved. The derivative is not a number that happens to fall out of a limit; it is *the slope of the unique line that makes this approximation work*. That line is the derivative's whole job description, and it is the only property of $f'(x)$ this book ever uses.

Two facts about that linear model matter more than the mechanics of computing it:

It composes. If $f(x+h) \approx f(x) + f'(x)h$ and $g(y+k) \approx g(y) + g'(y)k$, then composing the maps composes the linear models:

$$g(f(x+h)) \approx g(f(x)) + g'(f(x))\,f'(x)\,h,$$

which is the chain rule, and nothing more exotic. Every gradient your neural networks have ever used was produced by chaining exactly this identity, layer by layer, back through a computational graph.

It generalizes verbatim. For $f : \mathbb{R}^n \to \mathbb{R}^m$, the same statement holds with $f'(x)$ replaced by a matrix — the Jacobian — and h a vector:

$$f(x+h) = f(x) + Df(x)\,h + o(\|h\|).$$

Nothing about the *logic* changes; every entry of $Df(x)$ is still an ordinary 1-D derivative, one partial derivative at a time. Constructing that matrix, and reading geometric meaning (area, orientation) out of its determinant, is the business of Ch. 4. Here, file away only this: whatever a "Jacobian" turns out to be, it is asking exactly the question this section asks — what linear map best approximates this function near this point? — for a map with more than one input and output coordinate.

> **You already know this**
>
> A neural network's backward pass computes exactly this object, one layer at a time: the local linear map (the Jacobian) of that layer's function, chained through every layer by the identity above. `jax.grad` and `jax.hessian`, used throughout this repo's sampler code (`cgl/fitting.py:43`, `cgl/e1.py:797`, `cgl/e2.py:554`), compute nothing more exotic than the two objects this chapter builds by hand: the first-order and second-order local model of a function at a point.

A single worked derivative, done from the limit definition, sets up something you will need repeatedly: differentiating a power. For $f(x) = x^n$,

$$f'(x) = \lim_{h\to 0} \frac{(x+h)^n - x^n}{h}.$$

Expand $(x+h)^n$ by the binomial theorem: $(x+h)^n = x^n + nx^{n-1}h + \binom{n}{2}x^{n-2}h^2 + \cdots$. Subtract x^n, divide by h, and let $h \to 0$; every term past the first carries a positive power of h and vanishes, leaving

$$\frac{d}{dx}x^n = n\,x^{n-1}.$$

This holds for negative and non-integer n too (the proof needs a slightly different argument, but the result is the same), which is why it will apply directly to inverse-square-law and power-law profiles: the moment Ch. 3 introduces a density profile $\rho(r) \propto r^{-\gamma}$, you already have the derivative in hand.

Taylor to second order

A linear model is the *best possible* linear model, but "best possible" still leaves an error, and sometimes that error is the thing you care about. Taylor's theorem says how to shrink it further: add the next term.

$$f(x_0+h) = f(x_0) + f'(x_0)\,h + \frac{1}{2}f''(x_0)\,h^2 + O(h^3).$$

The pattern is: each successive term uses one more derivative and one higher power of h, and the *error* — the gap between f and the truncated polynomial — shrinks one power of h faster than the last term you kept. Keep only the first two terms and you recover last section's linear model,

with error $O(h^2)$. Keep three terms and the error drops to $O(h^3)$: a strictly better approximation, at the cost of one more derivative.

A concrete payoff: why the repo differentiates numerically by central differences. Ch. 4 will need the lens-equation Jacobian, and `lensing.py`'s implementation gets it not from an analytic formula but from finite differences:

```
bx_px, by_px = beta(x + h, y)
bx_mx, by_mx = beta(x - h, y)
...
a11 = (bx_px - bx_mx) / (2 * h)
```

(`lensing.py:101-123`, `lens_jacobian`). Why evaluate at *both* $x+h$ and $x-h$ and average, rather than the more obvious $(f(x+h)-f(x))/h$? Taylor's theorem answers it directly. Expand both points to third order:

$$f(x+h) = f(x) + f'(x)h + \tfrac{1}{2}f''(x)h^2 + \tfrac{1}{6}f'''(x)h^3 + O(h^4),$$

$$f(x-h) = f(x) - f'(x)h + \tfrac{1}{2}f''(x)h^2 - \tfrac{1}{6}f'''(x)h^3 + O(h^4).$$

Subtracting cancels every *even*-power term ($f(x)$, the h^2 term):

$$f(x+h) - f(x-h) = 2f'(x)h + \tfrac{1}{3}f'''(x)h^3 + O(h^5),$$

and dividing by $2h$ gives

$$\frac{f(x+h)-f(x-h)}{2h} = f'(x) + \frac{1}{6}f'''(x)\,h^2 + O(h^4).$$

The one-sided difference $(f(x+h)-f(x))/h$ keeps the h^2 term from the *first* expansion above and errs at $O(h)$. The centered difference cancels it by symmetry and errs at $O(h^2)$ — two more correct digits for the same number of function evaluations, for free, just by evaluating symmetrically around the point instead of stepping forward from it. That is the entire reason the repo's numerical Jacobian is centered and not forward: it is a direct, unglamorous consequence of Taylor's theorem, not a stylistic choice.

The same expansion generalizes to $\mathbb{R}^n \to \mathbb{R}$: gradient $\nabla f(x_0)$ replaces $f'(x_0)$, and the second-order term becomes a **quadratic form** built from the Hessian matrix H of second partial derivatives,

$$f(x_0+h) \approx f(x_0) + \nabla f(x_0)\cdot h + \frac{1}{2}h^{\mathsf{T}}Hh.$$

Building H and reading its eigenstructure is Ch. 5's job. What matters here is the *shape* of the answer: one vector (first-order information) and one symmetric matrix (second-order information) are all you ever get from a Taylor expansion, in any number of dimensions, and this book never needs anything past the quadratic term.

Why second order

Here is the payoff the whole chapter has been building toward, and it is worth stating baldly: **every hard object in this book is a first- or second-order local Taylor model of something**, and knowing that turns four topics that look unrelated — a bent light ray, a magnified image, a Bayesian evidence integral, and a Hamiltonian sampler's step size — into one idea applied four times.

- The **deflection field** is a nonlinear map from image position to source position, but at any single image location θ_0, an infinitesimal perturbation $d\theta$ produces $d\beta = A(\theta_0)\, d\theta$ — a strictly linear relationship, i.e. exactly this chapter's first-order model, with A the Jacobian (Ch. 17, Ch. 18).
- **Magnification** is a scalar built from that same Jacobian ($\mu = 1/\det A$) — a fact about a *derivative*, not a separate piece of physics (Ch. 18).
- The **Laplace approximation** to a Bayesian posterior is literally this chapter's second-order Taylor expansion, applied to $\log p(\theta)$ instead of a lensing map: expand around the MAP $\theta^\star$ (where $\nabla \log p = 0$, so the first-order term vanishes identically), and the Hessian of $\log p$ becomes the precision matrix of a Gaussian approximation to the whole posterior (Ch. 8).
- An **HMC sampler's mass matrix** is chosen to match the same Hessian, because a sampler that does not know the local quadratic shape of $\log p$ takes steps that are too small in narrow directions and too large in wide ones (Ch. 23).

That list is also a warning, and this repo lived it. At a **critical point** — anywhere the gradient is zero — the first-order term of a Taylor expansion vanishes *identically*. That is exactly what a gradient-based optimizer such as `jax.value_and_grad` (`cgl/fitting.py:43`, `cgl/e1.py:797,892`) is built to find: a point where it has nothing left to correct. But a vanishing gradient tells you a point is *flat* to first order; it says nothing about whether the function curves up, down, or up-in-some-directions-and-down-in-others there. That question belongs entirely to the second-order term — the Hessian — which is precisely why Ch. 5 exists.

The campaign's correlated-noise fit hit this directly. Its MAP-finding stage (`map_polish`) drove the gradient of the log-posterior to zero and stopped, reporting success. A second-order check — computing the Hessian of the log-posterior at that point with `jax.hessian` (`cgl/e2.py:554`) — showed the point was a **saddle, not a mode**: minimum eigenvalue -14.85, with five negative directions (main.tex, §6.3; CAMPAIGN.md, "P1c metric-fix attempts — 2026-07-10"). The sampler's step-size metric (the exact construction, and why "mass" and "covariance" are easy to swap by mistake, is Ch. 23's business) is built by reading a length scale off each Hessian eigenvalue λ_i — which only means what it is supposed to mean if $\lambda_i > 0$, i.e. if the log-posterior is bowl-shaped along that direction. Five of this Hessian's eigenvalues are negative: the log-posterior curves *away* from the stationary point along those directions, not toward it, so there is no local bowl for a length scale to describe. Two repairs were tried, and both were built specifically to *avoid* using $|\lambda_i|$ on an indefinite Hessian: one floored the raw diagonal $|H_{ii}|$ instead of the eigenvalues, one replaced the Hessian metric outright with an independently-fit SVI covariance. Both were positive-definite by construction, and both still left the sampler's $\widehat{R}$ diagnostic stuck at 21–22. The gradient alone could not have told you any of this; only the second-order term could, and only because someone thought to compute it. Ch. 26 tells this story in full, including how it was eventually resolved. File it away here as the concrete argument for why "why second order" is not academic throat-clearing: a real campaign, with real compute charged to it, spent time chasing a fix to a problem that the second-order term would have diagnosed immediately.

Connect to the repo

- `lensing.py:101-123` (`lens_jacobian`) — the numerical Jacobian is a **centered** finite difference, $(f(x+h)-f(x-h))/2h$, precisely because (as derived above) that cancellation of the even-order Taylor term buys an extra order of accuracy over a one-sided difference. (github)
- `cgl/fitting.py:43` and `cgl/e1.py:797,892` — `jax.value_and_grad`, the first-order local linear model, used to walk both the MAP optimizer and the SVI (variational) fit toward a stationary point. (github)
- `cgl/e2.py:554` — `jax.hessian(lp1)(z_map)`, the second-order local quadratic model of the log-posterior at the MAP; its eigenvalues (`w`, `n_neg` at `cgl/e2.py:557-558`) are the diagnostic that caught the saddle. (github)
- The saddle finding in full: main.tex §6.3, "Why sampling this posterior required tempered SMC".

Exercises

Exercise 2.1 — the power rule from the limit definition

Using the binomial theorem, show that $\frac{d}{dx}x^3 = 3x^2$ directly from the limit definition $f'(x) = \lim_{h\to 0}\left[(x+h)^3 - x^3\right]/h$. Which terms survive the limit, and why?

Solution

Expand $(x+h)^3 = x^3 + 3x^2h + 3xh^2 + h^3$. Subtract x^3 and divide by h:

$$\frac{(x+h)^3 - x^3}{h} = 3x^2 + 3xh + h^2.$$

Every surviving term after the first carries at least one positive power of h, so as $h \to 0$ they vanish, leaving $3x^2$. Nothing about this argument is special to the exponent 3 — the same cancellation, one binomial-theorem term at a time, produces nx^{n-1} for any positive integer n.

Exercise 2.2 — why the centered difference is one order better

A colleague suggests replacing `lensing.py`'s centered difference with the cheaper one-sided form $(f(x+h) - f(x))/h$, computing half as many evaluations. Using the Taylor expansions in Taylor to second order, state the leading-order error of each form and say by how many powers of h they differ.

Solution

From $f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + O(h^3)$, the one-sided difference is

$$\frac{f(x+h) - f(x)}{h} = f'(x) + \frac{1}{2}f''(x)\,h + O(h^2),$$

an error of order h (unless $f''(x_0) = 0$). The centered difference, derived in the text, has error $\frac{1}{6}f'''(x)h^2 + O(h^4)$ — order h^2. The centered form is one full power of h more accurate for the *same* step size, at the cost of one extra function evaluation (it needs $f(x-h)$ as well as $f(x+h)$). That is exactly the trade the repo makes in `lensing.py:115-122`.

Exercise 2.3 — a zero gradient is not a verdict

Consider $f(x, y) = x^2 - y^2$. Verify that $\nabla f = 0$ at the origin. Then evaluate f along the path $x = t, y = 0$ and along $x = 0, y = t$ for small $t \neq 0$. What do the two paths tell you that the gradient alone did not, and which repo finding does this mirror?

Solution

$\nabla f = (2x, -2y)$, which is exactly $(0, 0)$ at the origin — a bona fide critical point. Along $x = t, y = 0$: $f = t^2 > 0$, curving *upward*. Along $x = 0, y = t$: $f = -t^2 < 0$, curving *downward*. The same stationary point is a local max in one direction and a local min in another — the textbook definition of a saddle. The gradient could not distinguish this from a true minimum or maximum; only evaluating f off-origin (equivalently, the Hessian $\begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$, with one positive and one negative eigenvalue) reveals it. This is the toy version of exactly what `jax.hessian` caught in the campaign's correlated-noise MAP (min eigenvalue -14.85, five negative directions): a `map_polish` stage that stops once $\nabla \log p = 0$ has, by construction, no way to see this.

Exercise 2.4 — composing two linear models

The lens equation (Ch. 17) is a composition: image light depends on source position β, which depends on image position θ through $\beta(\theta) = \theta - \alpha(\theta)$. If a downstream likelihood is a function $L(\beta)$, write down the first-order (linear) model of $L(\beta(\theta))$ near a point θ_0, in terms of ∇L and the Jacobian of β. Do not evaluate anything — just apply the chain rule from Derivative as a linear map.

Solution

Let $A(\theta_0) = D\beta(\theta_0)$ be the lens Jacobian (Ch. 18). Composing the two linear models, exactly as in the chain-rule identity derived in the text,

$$L(\beta(\theta_0 + h)) \approx L(\beta(\theta_0)) + \nabla L(\beta(\theta_0))^{\mathsf{T}} A(\theta_0)\, h.$$

This is the same computation `jax.grad` performs automatically when it differentiates a likelihood through the forward model in Ch. 22 — one chain-rule multiplication, ∇L against the Jacobian of everything upstream of it, with no new mathematics past what this chapter derived.

3. Integrals, projection, and where the logarithm comes from

By the end of this chapter you can derive, from nothing but the definition of an integral, two facts that look like they come from nowhere in every lensing paper you will read: why a mass profile's density slope γ shifts by exactly one when you project it from three dimensions to two, and why the potential of a point mass sitting in a two-dimensional world is a logarithm rather than the familiar $1/r$. Both facts are downstream of doing an integral as an *accumulation* — stacking density along a sightline, summing flux around a loop — rather than treating "integral" as a symbol you look up a closed form for. By the last section you will have re-derived, from those two facts alone, why `lensing.py`'s singular-isothermal-sphere deflection has constant magnitude: a one-line code comment turns into a two-line proof.

> **What you can skip**
>
> You already know how to evaluate a definite integral, substitute $u = g(x)$, and check an improper integral for convergence with the p-test ($\int^{\infty} x^{-p}\,dx$ converges iff $p > 1$). None of that machinery is new here. What is new is what the integral *means* in this setting — a running total of mass, not an area under a curve — and two consequences that follow from computing it: an exponent that shifts by one, and a logarithm that appears in exactly one dimension count.

Integrals as accumulation

A Riemann sum partitions an interval, evaluates a function on each piece, and adds up the pieces times their width. Taking the partition to zero width gives the integral. Nothing about that procedure requires the function to be a curve on a page — it works exactly as well when the function is a mass density and the "area under the curve" is a mass.

The accumulation this guide returns to most often is a radial one: given a surface (2-D, sky-plane) mass density $\Sigma(R)$, the mass enclosed inside radius R is

$$M(<R) \;=\; \int_0^R \Sigma(R')\,(2\pi R')\,dR'.$$

The factor $2\pi R'\,dR'$ is itself a small accumulation: it is the area of a thin ring of radius R' and width dR', obtained by unrolling the ring into a rectangle of length (circumference) $2\pi R'$ and height dR' — the same "slice it thin, add up the slices" move as the outer integral, one level down. You will meet this exact integral again in Ch. 19, where dividing $M(<\theta)$ by the enclosed area turns it into the mean-convergence identity that defines the Einstein radius; the implementation, `mean_kappa_within` (`site/guide_src/lensing.py:179`), is this formula with Σ replaced by κ and normalized by the enclosed area.

That is accumulation along one direction (radius, in the sky plane). The next section needs accumulation along a second, orthogonal direction: straight through the galaxy, along the line of sight. That is where γ picks up its first concrete consequence.

The Abel projection

Take a spherically symmetric 3-D mass density that is a pure power law,

$$\rho(r) = A\, r^{-\gamma}, \qquad \gamma > 0,$$

with A collecting whatever normalization and scale radius the profile carries — $\gamma = 2$ is the isothermal sphere, the campaign's reference slope. A telescope cannot see $\rho(r)$; it sees the sky-plane surface density $\Sigma(R)$ obtained by adding up every bit of mass along the sightline at fixed projected radius R. Writing the sightline coordinate as z, so that the true 3-D radius is $r = \sqrt{R^2 + z^2}$, that accumulation is the integral

$$\Sigma(R) \;=\; \int_{-\infty}^{\infty} \rho\Big(\sqrt{R^2 + z^2}\Big)\, dz \;=\; 2A \int_0^{\infty} \left(R^2 + z^2\right)^{-\gamma/2} dz.$$

> **You already know this**
>
> Integrating a joint density over one of its coordinates to get the density of the rest is marginalization. Integrating $\rho(x, y, z)$ over z to get $\Sigma(x, y)$ is exactly that operation, with the line-of-sight coordinate playing the role of a nuisance variable. The campaign performs the identical move analytically on 28 shapelet source-light amplitudes (`reproductions/claude-giga-lens/cgl/likelihood.py:8`) — `reproductions/claude-giga-lens/cgl/marg` marginalizes them out of the posterior in closed form, via a Cholesky solve rather than a sightline integral, but it is the same integral in spirit: sum over what you cannot measure directly, keep what you can.

The integral over z still has R tangled up inside it, which hides the point. Substitute $z = Ru$ — a pure rescaling of the integration variable, nothing more:

$$dz = R\, du, \qquad R^2 + z^2 = R^2(1 + u^2),$$

$$\Sigma(R) = 2A \int_0^{\infty} \left(R^2(1 + u^2)\right)^{-\gamma/2} R\, du$$

which needs a moment of care with the algebra: $\left(R^2(1 + u^2)\right)^{-\gamma/2} = R^{-\gamma}(1 + u^2)^{-\gamma/2}$, and there is one more factor of R from dz, so

$$\Sigma(R) \;=\; 2A\, I(\gamma)\; R^{\,1-\gamma}, \qquad I(\gamma) \;=\; \int_0^{\infty} (1 + u^2)^{-\gamma/2}\, du. \tag{1}$$

Every trace of R inside the integral is gone — the substitution swept all of it out front as the single factor $R^{1-\gamma}$, and $I(\gamma)$ is just a number (a standard Beta-function integral once you evaluate it, though you do not need its value for anything that follows). (1) is the whole content of the Abel projection: **a 3-D power law of slope γ projects to a 2-D power law of slope $\gamma - 1$.** The exponent shifts by exactly one, every time, for any pure power-law ρ, no matter what A or the scale radius are.

The convergence of $I(\gamma)$ is not a technicality. For large u the integrand behaves like $u^{-\gamma}$, and by the p-test you already own, $\int^{\infty} u^{-\gamma}\, du$ converges only for $\gamma > 1$. Below that, the

sightline integral does not merely get large — it is infinite: an idealized, untruncated power-law halo with $\gamma \leq 1$ has infinite surface density at *every* radius. That is not a statement about real galaxies (which flatten and truncate at large r); it is a statement about this exact scale-free parameterization, which is precisely what the campaign's EPL mass profile is. The model's own prior on γ, `gamma=tfd.TruncatedNormal(2.0, 0.25, 1.0, 2.7)` (`reproductions/claude-giga-lens/cgl/e2.py:110`), happens to put its hard lower wall at exactly the boundary this section's math identifies as fatal. Nothing in the campaign's own notes says that convergence argument was the deliberate reason for choosing 1.0 rather than some other round number below the isothermal value — but the mathematics does not care why the wall was placed there: below $\gamma = 1$, the projection this section just derived stops being well-defined, not merely improbable, so a wall somewhere at or above 1.0 is not an arbitrary modeling choice, whether or not it was chosen for that reason.

That wall is worth a number. The campaign's headline result is $\gamma_{\text{binned}}(\text{corr}, \text{low}) = 1.103 \pm 0.008$ (`reproductions/claude-giga-lens/CAMPAIGN.md:134`) — a value this guide's later chapters spend real effort auditing (Ch. 20, Ch. 25). For now, treat it only as a number to plug into (1): it sits 12.9 of its own posterior widths above that hard wall — comfortably clear of it, but only (1) tells you *why* the wall is at 1.0 and not, say, 0.5 or 1.5.

Here is the same relationship, checked against real code rather than left as algebra. `epl_kappa` (`site/guide_src/lensing.py:81`) is the guide's implementation of the campaign's actual EPL convergence law — the same function Ch. 18 and Ch. 20 use:

```
kappa(R) = 0.5 * (3 - gamma) * (theta_E / R)**(gamma - 1)
```

The prefactor $(3-\gamma)/2$ and the scale θ_{E} are this formula's version of $2A\,I(\gamma)$ in (1) — where that prefactor comes from is Ch. 20's job, not this chapter's. What this chapter derived is the *exponent*, and a ratio at two radii cancels every multiplicative constant, prefactor included, and tests the exponent alone:

$$\frac{\kappa(R_2)}{\kappa(R_1)} = \left(\frac{R_1}{R_2}\right)^{\gamma-1}.$$

For the isothermal slope $\gamma = 2$, doubling the radius from $R_1 = 0.5''$ to $R_2 = 1.0''$ must exactly halve κ:

$$\kappa(1.0'')/\kappa(0.5'') = 0.5, \qquad \text{predicted } (0.5/1.0)^{2-1} = 0.5.$$

For the campaign's money number $\gamma = 1.103$, the same doubling barely moves κ at all:

$$\kappa(1.0'')/\kappa(0.5'') = 0.9311, \qquad \text{predicted } (0.5/1.0)^{1.103-1} = 0.9311.$$

An isothermal halo's surface density drops by half every time the radius doubles; a $\gamma = 1.103$ halo's surface density drops by seven percent. (1) says that difference is not a subtlety of the fit — it is the entire content of what γ means. Whether 1.103 is a trustworthy measurement of a real galaxy, or an artifact of a mis-specified likelihood, is exactly the question Ch. 25 spends its report card on; this chapter only establishes what the number would mean if you believed it.

Why a 2-D point mass has a logarithmic potential

Gauss's law is itself an accumulation statement — it says the net flux through a closed surface equals (a constant times) the mass enclosed. In 3-D, for a point mass and a spherical surface of radius r, spherical symmetry makes the flux integral trivial: the field $g(r)$ is the same everywhere on the sphere, so the flux is just $g(r)$ times the sphere's area,

$$g(r)\,(4\pi r^2) = -4\pi GM \quad \Longrightarrow \quad g(r) = -\frac{GM}{r^2},$$

the inverse-square law. Integrating $g = -d\Phi/dr$ gives the familiar Newtonian potential $\Phi(r) = -GM/r$.

Now run the identical argument with the enclosing surface confined to two dimensions — a circle instead of a sphere. This is not a toy: the lensing potential ψ genuinely lives in the 2-D sky plane, sourced by exactly the projected surface density $\Sigma(R)$ the previous section built, so a 2-D Gauss's law is the correct tool, not an analogy borrowed from one. A circle of radius R has circumference $2\pi R$ instead of a sphere's area $4\pi r^2$, so enclosing a mass M gives

$$g(R)\,(2\pi R) = -(\text{const})\,M \quad \Longrightarrow \quad g(R) \propto -\frac{M}{R},$$

inverse-linear, not inverse-square. Integrating $g = -d\Phi/dR$ against a $1/R$ field gives $\Phi(R) \propto \ln R$: a **logarithmic** potential — note the sign flip relative to 3-D: $\int R^{-2}\,dR = -R^{-1}$ carries a minus sign that $\int R^{-1}\,dR = \ln R$ does not, so the logarithm comes out positive even though g itself is negative (attractive). That is the entire reason a lens's potential shows up wearing a logarithm in every textbook derivation you will meet later — it is the fundamental (Green's) solution of the 2-D Laplacian. Ch. 6 derives it formally from the Laplacian directly, and checks numerically that it is harmonic everywhere except at the source, which is this section's result stated the other way around.

The two derivations in this chapter now click together. Take the isothermal case, $\gamma = 2$, where the previous section gave $\Sigma(R) \propto 1/R$. Feed that into the ring-accumulation integral from the first section:

$$M(<R) \;\propto\; \int_0^R \frac{1}{R'} \cdot 2\pi R'\,dR' \;=\; 2\pi \int_0^R dR' \;\propto\; R.$$

The enclosed mass of an isothermal profile grows *linearly* with radius — each new ring you add contributes the same amount of mass as the last one, because the ring's shrinking density ($1/R'$) exactly cancels its growing area (R'). Now put that into the 2-D field law just derived:

$$g(R) \;\propto\; -\frac{M(<R)}{R} \;\propto\; -\frac{R}{R} \;=\; \text{const}.$$

The radius cancels completely. An isothermal sphere's deflection has the same magnitude at every radius — the exact fact `sis_deflection` (`site/guide_src/lensing.py:54`) states as a one-line comment: "the flat rotation curve of a galaxy in one line: the deflection does not care how far out you are, only which way." You have now derived that sentence from two integrals, without reading

a single line of the code that implements it. (Two honesty notes for later chapters: this argument is Newtonian, and general relativity multiplies the whole thing by a factor of two that Ch. 16 derives; and turning g into an actual lens *equation* — a map from source position to image position — is Ch. 17's job.)

Connect to the repo

- `site/guide_src/lensing.py:81` — `epl_kappa`, the EPL convergence law whose exponent this chapter derived; `site/guide_src/lensing.py:54` — `sis_deflection`, the constant-magnitude deflection this chapter's two derivations combine to explain; `site/guide_src/lensing.py:179` — `mean_kappa_within`, the ring-accumulation integral of the first section, reused for the Einstein-radius identity in Ch. 19.
- `reproductions/claude-giga-lens/vendor/gigalens-sean/src/gigalens/jax/profiles/mass/epl.py:` — gigalens' own EPL code sets `t = gamma - 1`, the same exponent shift derived here, in the production profile this campaign actually fits.
- `reproductions/claude-giga-lens/cgl/e2.py:110` — the campaign's prior on γ, `TruncatedNormal(2.0, 0.25, 1.0, 2.7)`; its low wall at 1.0 is where the projection integral in (1) stops converging, not an arbitrary modeling choice.
- `reproductions/claude-giga-lens/CAMPAIGN.md:134` — the money number this chapter used as a worked example, $\gamma_{\text{binned}}(\text{corr}, \text{low}) = 1.103 \pm 0.008$.
- `reproductions/claude-giga-lens/cgl/marg.py:9` — the campaign's other integral-as-marginalization, done analytically over the 28 shapelet amplitudes counted at `reproductions/claude-giga-lens/cgl/likelihood.py:8`, rather than numerically over a sightline.

Exercises

Exercise 3.1 — the exponent, from scratch

Starting from $\Sigma(R) = 2A\int_0^\infty (R^2+z^2)^{-\gamma/2}\,dz$, carry out the substitution $z = Ru$ yourself and confirm (1). Then state, in one sentence, why the integral $I(\gamma)$ requires $\gamma > 1$ to exist at all.

Solution

$R^2 + z^2 = R^2(1+u^2)$ and $dz = R\,du$, so $(R^2+z^2)^{-\gamma/2}dz = R^{-\gamma}(1+u^2)^{-\gamma/2}\cdot R\,du = R^{1-\gamma}(1+u^2)^{-\gamma/2}\,du$. The factor $R^{1-\gamma}$ does not depend on u, so it comes straight out of the integral, leaving $\Sigma(R) = 2A\,R^{1-\gamma}\int_0^\infty (1+u^2)^{-\gamma/2}\,du$.

That is (1). For large u the integrand behaves like $u^{-\gamma}$; by the p-test, $\int^\infty u^{-\gamma}\,du$ converges only for $\gamma > 1$, so a pure power-law density with $\gamma \le 1$ has infinite surface density at every projected radius.

Exercise 3.2 — an intermediate slope

Without running any code, predict the ratio $\kappa(1.0'')/\kappa(0.5'')$ for $\gamma = 1.5$ — a slope roughly midway between the isothermal value and the money number. Then check it.

Solution

(1)'s exponent gives $(0.5/1.0)^{1.5-1} = (0.5)^{0.5} = 1/\sqrt{2} \approx 0.7071$. `epl_kappa` at $\gamma = 1.5$ gives the same ratio, 0.7071, to machine precision — the exponent match is 0 to 10^{-9}. A slope between 1.103 and 2 produces surface-density falloff between "barely drops" and "halves": the exponent interpolates exactly the way (1) says it must.

Exercise 3.3 — the prior wall, quantified

The campaign's γ prior is `TruncatedNormal(2.0, 0.25, 1.0, 2.7)` (`reproductions/claude-giga-lens/cgl` Using the money number $\gamma = 1.103 \pm 0.008$, how many of its *own* posterior widths separate it from the wall at 1.0? Is the wall clipping the posterior, or just sitting nearby?

Solution

$(1.103 - 1.0)/0.008 \approx 12.9$ posterior widths. A boundary that far from the bulk of the posterior mass is not clipping it — the posterior would look identical if the wall were moved further down. What the wall *does* constrain is the model itself: it is the smallest γ for which (1)'s projection integral converges, so no amount of data pressure could ever push a fit past it in a way that made mathematical sense.

Exercise 3.4 — deriving the flat rotation curve

Using only (a) $\Sigma(R) \propto 1/R$ for an isothermal profile and (b) the 2-D field law $g(R) \propto -M(<R)/R$, show that the isothermal deflection magnitude is independent of radius, and identify which of the two facts would have to change for that to fail.

Solution

$M(<R) = \int_0^R \Sigma(R')\, 2\pi R'\, dR' \propto \int_0^R dR' \propto R$ because the $1/R'$ density and the R' ring area cancel inside the integrand. Then $g(R) \propto -M(<R)/R \propto -R/R =$ constant. It fails for any $\gamma \neq 2$: the enclosed-mass integral would give $M(<R) \propto R^{2-\gamma}$ (from $\Sigma \propto R^{1-\gamma}$ and one more power of R from the ring area), and $g(R) \propto R^{2-\gamma}/R = R^{1-\gamma}$ would depend on radius for every slope except $\gamma = 2$. Isothermal is not "a" profile with a flat rotation curve — it is *the* profile whose projected exponent exactly cancels the ring-area exponent, and that cancellation is a single-value coincidence in γ.

4. Gradient, Jacobian, Hessian: det J is an area scaling

This chapter takes three objects you already compute — the gradient, the Jacobian, the Hessian — and asks what the determinant of a Jacobian actually *means*. The answer is not abstract linear algebra. $\det J$ is a literal area (or volume) ratio, and this repository computes that exact ratio, under three different names, in three different files: it is the lensing magnification $\mu = 1/|\det A|$ that Chapter 18 uses to explain why a background galaxy looks brighter than it should; it is the $\log|\det J_T|$ term every normalizing flow you have trained already adds to a log-density; and it is the $-\frac{1}{2}\log\det A$ Occam penalty buried in this campaign's marginal likelihood. By the end of this chapter you will be able to derive that reciprocal-determinant law from scratch, not just recognize it when you see it.

> **What you can skip**
>
> You already build and differentiate Jacobians for a living — every layer of a neural net is a Jacobian-vector or vector-Jacobian product, and you already invoke the change-of-variables theorem every time you compute $\log q(x) = \log p(u) - \log|\det J|$ under a bijector. Skip the review of *how* to compute a multivariable derivative if you want. What is not boilerplate here: the literal geometric reading of $\det J$ as an area ratio ("det J as an area scaling factor"), and the specific three places this repository's code computes it ("The Log-Det Ledger").

Gradient, Jacobian, Hessian

Chapter 2 defined the derivative of a scalar function of one variable as the best local *linear* approximation: $f(x+h) \approx f(x) + f'(x)\,h$. Nothing about that idea depends on there being one input or one output. For a vector-valued function of several variables, $F : \mathbb{R}^n \to \mathbb{R}^m$, the identical statement reads

$$F(\mathbf{x}+\mathbf{h}) \;\approx\; F(\mathbf{x}) + J_F(\mathbf{x})\,\mathbf{h}, \qquad (J_F)_{ij} = \frac{\partial F_i}{\partial x_j}.$$

J_F, the **Jacobian**, is an $m \times n$ matrix of partial derivatives, and () says exactly what Chapter 2 said: near $\mathbf{x}$, the nonlinear map F is well-approximated by the linear map $\mathbf{h} \mapsto J_F(\mathbf{x})\,\mathbf{h}$. When $m = 1$ (a scalar-valued function), J_F is a single row, and its transpose is the **gradient** ∇f you already know as "the direction of steepest ascent." The gradient is a special case of the Jacobian, not a different object.

Differentiate the gradient once more and you get the **Hessian**, the $n \times n$ matrix of second partials, $H_{ij} = \partial^2 f / \partial x_i\,\partial x_j$. It extends the Taylor expansion to second order exactly as Chapter 2 previewed:

$$f(\mathbf{x}+\mathbf{h}) \;\approx\; f(\mathbf{x}) + \nabla f(\mathbf{x}) \cdot \mathbf{h} + \tfrac{1}{2}\,\mathbf{h}^\top H(\mathbf{x})\,\mathbf{h}.$$

Two facts about H matter more than its definition. First, for any function whose second partials are continuous, mixed partials commute ($\partial^2 f/\partial x_i \partial x_j = \partial^2 f/\partial x_j \partial x_i$ — Schwarz's theorem), so **the Hessian is always symmetric**. Second, a zero gradient tells you that you are at a *stationary* point — it says nothing about whether that point is a minimum, a maximum, or a saddle. Only the Hessian's eigenvalues answer that (Chapter 2's own argument for why a second-order model is necessary; Chapter 5 makes it precise for exactly this repository's own saddle).

Here is why the Hessian is not a side character in this guide. The Jacobian of a *gradient field* is automatically a Hessian: if $F = \nabla\psi$ for some scalar potential ψ, then

$$(J_F)_{ij} = \frac{\partial F_i}{\partial x_j} = \frac{\partial}{\partial x_j}\left(\frac{\partial \psi}{\partial x_i}\right) = \frac{\partial^2 \psi}{\partial x_i\, \partial x_j} = H_{ij}(\psi).$$

Chapter 6 derives why the deflection field of a gravitational lens is exactly such a gradient, $\alpha = \nabla\psi$, for a scalar lensing potential ψ (Ch. 6). That single fact is why the Jacobian this repository differentiates at every image position — the matrix that Chapters 5, 18 and 20 spend three chapters decomposing — is guaranteed symmetric before you ever compute a single entry. A symmetric matrix has real eigenvalues and orthogonal eigenvectors; that is what makes the clean split into an isotropic part (convergence) and a shear part possible at all, and it is not a coincidence — it follows from () applied to a gradient field.

The change-of-variables theorem

Single-variable calculus has a substitution rule: $\int f(g(x))\, g'(x)\, dx = \int f(u)\, du$ with $u = g(x)$. The $g'(x)$ factor is a *local stretching factor* — it says how much a small interval dx is stretched or compressed when mapped through g. The multivariable generalization keeps that reading and replaces the scalar $g'(x)$ with the determinant of the Jacobian: for a smooth, invertible map $T : U \to V$,

$$\int_V g(\mathbf{y})\, d^n y \;=\; \int_U g(T(\mathbf{x}))\, \left|\det J_T(\mathbf{x})\right| d^n x.$$

() is the **change-of-variables theorem**. The next section derives *why* $|\det J_T|$ is the right local factor; for now, notice what () says about *densities*. If $\mathbf{x}$ has probability density p_U and $\mathbf{y} = T(\mathbf{x})$ has density p_X, conservation of probability mass on matching infinitesimal patches ($p_U(\mathbf{x})\, d^n x = p_X(\mathbf{y})\, d^n y$) combined with ()'s local scaling gives

$$p_U(\mathbf{x}) = p_X(T(\mathbf{x}))\, \left|\det J_T(\mathbf{x})\right|, \qquad \text{i.e.} \qquad \log p_U(\mathbf{x}) = \log p_X(T(\mathbf{x})) + \log\left|\det J_T(\mathbf{x})\right|.$$

> **You already know this**
>
> That last line is not an analogy — it is the identical arithmetic a normalizing flow performs to evaluate a pulled-back density. Compare it to the docstring of `reproductions/claude-giga-lens/cgl/flows.p` which writes the NeuTra pullback used by this campaign's samplers as `logp_u(u) = logp_target(T(u)) + log|det J_T(u)|`. You have been computing () every time you called `.log_prob()` on a bijected distribution; you were just calling the theorem by a different name.

det J as an area scaling factor

Where does the $|\det J|$ factor in () come from? Start with a *linear* map, $\mathbf{y} = A\mathbf{x}$, and ask what happens to the area of a unit square under it. Two edge vectors of the square, $(1, 0)$ and $(0, 1)$, map to the two columns of A. The area of the parallelogram spanned by two vectors (a_{11}, a_{21}) and (a_{12}, a_{22}) is $|a_{11}a_{22} - a_{12}a_{21}| = |\det A|$ — the shoelace formula applied to a parallelogram *is* the definition of a 2×2 determinant. So for a linear map, "area scales by $|\det A|$" is not a theorem to prove; it is what the determinant was built to compute.

A general smooth map T is not linear, but () says it is *locally* linear, with $J_T(\mathbf{x})$ as the local linear map. Shrink a patch of the domain down to an infinitesimal square at $\mathbf{x}$: its image is, to leading order, a parallelogram with area $|\det J_T(\mathbf{x})|$ times the original. Integrate that local scaling factor over every patch in U and you recover () exactly. The change-of-variables theorem is linearization (Ch. 2), applied pointwise, summed over a region.

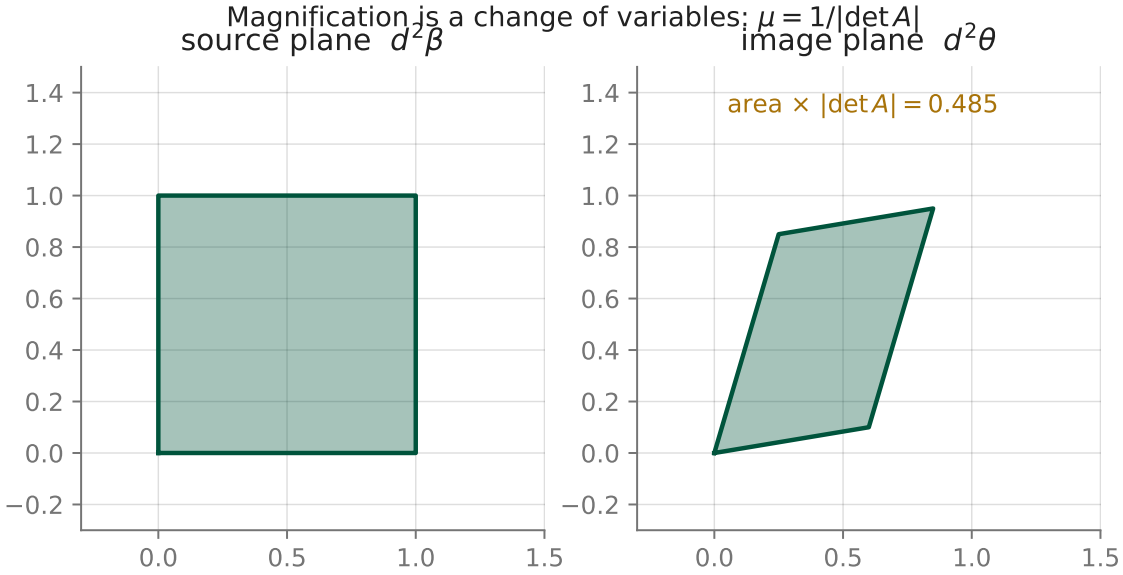

Figure 1: A unit square (left) pushed through the linear map $A = \begin{pmatrix} 0.6 & 0.25 \\ 0.1 & 0.85 \end{pmatrix}$ into the parallelogram on the right. Nothing is drawn freehand: the right-hand shape is literally A applied to the square's four corners. The annotated ratio is $\det A$, and it is the same number whether you compute it from the 2×2 determinant formula or from the shoelace-formula area of the polygon on the right.

Compute both routes yourself. The determinant of A above is $0.6 \times 0.85 - 0.25 \times 0.1 = 0.51 - 0.025 = 0.485$. Independently, push the square's four corners $(0,0), (1,0), (1,1), (0,1)$ through A and apply the shoelace formula to the resulting quadrilateral: the area comes out to 0.485 , matching the determinant to machine precision, because the shoelace formula and the 2×2 determinant are computing the same quantity two different ways. `worked_examples.py --show ch04` reproduces both numbers directly.

Two immediate uses of that reciprocal direction. **Magnification**: Chapter 18 defines the lensing Jacobian $A \equiv \partial\beta/\partial\theta$ — how a source-plane displacement β responds to an image-plane one θ — and shows that surface brightness is conserved along a light ray, so the flux ratio between an image and its true, unlensed source is exactly the *inverse* area ratio, $\mu \equiv 1/|\det A|$ (Ch. 18). A determinant smaller than one in the natural ($\theta \to \beta$) direction is a magnification bigger than one in the observed direction — the same $|\det J|$ arithmetic this section just verified, applied in reverse. **Normalizing flows**: a flow's *forward* Jacobian determinant is exactly the J_T of (), and its log-density correction is $+\log|\det J_T|$ on the base side, as derived above. Both are the identical operation; only the sign convention and which side of the map you call "base" differ.

The Log-Det Ledger

This repository computes $\log|\det(\cdot)|$ of a 2×2 (or larger) matrix in three unrelated-looking places. This chapter opens the ledger with the first two; Chapter 8 adds the third; Chapter 23 closes it by showing all three are, line for line, the same computation.

Log-Det Ledger — rows 1 and 2, opened here

#	Costume	Formula	Where
1	Lensing magnification	$\mu = 1/\lvert\det A\rvert$, $A = \partial\beta/\partial\theta$	`site/guide_src/lensing.py:139`
2	Normalizing-flow pullback density	$\log p_U(\mathbf{u}) = \log p_X(T(\mathbf{u})) + \log\lvert\det J_T(\mathbf{u})\rvert$	`reproductions/claude-giga-lens/c`
3	Gaussian-evidence Occam term (opens in Ch. 8)	$-\frac{1}{2}\log\det A$	`reproductions/claude-giga-lens/c`

Rows 1 and 2 are the *same* theorem — () — read in opposite directions: one turns an area ratio into a brightness ratio, the other turns it into a density-correction term. Row 3 is a different-looking object (it subtracts a log-determinant from a log-likelihood rather than adding one to a log-density) and Chapter 23 shows it collapses to the same arithmetic once you write the Gaussian evidence integral out explicitly. Ch. 23 closes the ledger.

`site/guide_src/lensing.py:139` is worth reading directly — its own docstring already states this chapter's thesis in one line: *"This is the SAME change-of-variables factor that a normalizing flow applies as $-\log|\det J|$ … Three log-dets, one idea."* This guide did not invent that framing; the repository's own numerics code did.

One notation hazard, flagged now because it recurs: `reproductions/claude-giga-lens/cgl/marg.py:48` also calls its own matrix A — but that A is $\tilde{X}^\top\tilde{X} + \Lambda$, the Gram matrix of a whitened design matrix plus a ridge term, and has nothing to do with the lensing Jacobian $A = \partial\beta/\partial\theta$ of row 1. The repository reuses the letter; this guide does not, past this sentence — from here on, A means the lensing Jacobian unless a chapter says otherwise.

Connect to the repo

Everything in this chapter is small enough to read directly.

- `site/guide_src/lensing.py:101-146` — `lens_jacobian` builds a 2×2 Jacobian numerically, by central differences, exactly as () defines it; `magnification` then takes $1/\det(\cdot)$ of it. Chapter 18 runs this on a real deflection field and checks the numerical result against a closed-form SIS magnification.
- `reproductions/claude-giga-lens/cgl/flows.py:1-21` — the module docstring is the pullback-density derivation of this chapter's "change-of-variables" section, in the form the campaign's NeuTra and glnt samplers actually run (Chapter 23).
- `reproductions/claude-giga-lens/cgl/marg.py:31-55` — the third ledger costume, `-1/2 logdetA`, computed via a Cholesky factor (`logdetA = 2 * sum(log(diag(chol)))`) rather

than a direct determinant, because a Cholesky log-determinant is numerically stable at the condition numbers this repository's real fits produce (Chapter 5 quantifies exactly how large those condition numbers get).

- `reproductions/claude-giga-lens/papers/main.tex:466-471` (`\label{eq:logl}`) — the typeset version of the Occam term, for when Chapter 8 sends you back to read it in context.
- `site/guide_src/figures.py:28-49` — Figure 4.1 above, generated (not drawn) by pushing a literal unit square through a literal NumPy matrix; change the matrix and the figure, the annotated determinant, and the caption's number all change together, because they are computed from the same four lines.

Exercises

Exercise 4.1 — det J, two ways

Using $A = \begin{pmatrix} 0.6 & 0.25 \\ 0.1 & 0.85 \end{pmatrix}$ from Figure 4.1: (a) compute $\det A$ directly from the 2×2 formula. (b) Push the unit square's four corners through A and compute the resulting parallelogram's area with the shoelace formula, area $= \frac{1}{2}\left|\sum_i (x_i y_{i+1} - x_{i+1} y_i)\right|$. Confirm the two answers agree, then check both against `worked_examples.py --show ch04`.

Solution

(a) $\det A = (0.6)(0.85) - (0.25)(0.1) = 0.51 - 0.025 = 0.485$.

(b) The corners map to $(0,0)$, $(0.6, 0.1)$, $(0.85, 0.95)$, $(0.25, 0.85)$ (each corner (x, y) maps to $(0.6x + 0.25y,\ 0.1x + 0.85y)$). The shoelace sum gives area $= 0.485$, matching (a) exactly — it must, since both computations are literally the same determinant, one written as a 2×2 formula and the other as a sum over edges. `worked_examples.py`'s `ch04_det_j_area` returns `det_A = 0.485` and `image_area_shoelace = 0.485` .

Exercise 4.2 — the reciprocal, by definition

Given $\det A = 0.485$ from Exercise 4.1, what is $\mu = 1/|\det A|$? If A stood for an actual lensing Jacobian $\partial\beta/\partial\theta$, would the corresponding image be brighter or dimmer than the source that produced it?

Solution

$\mu = 1/0.485 = 2.0619$. Since $\mu > 1$, the image would be *brighter*: a source-plane patch of fixed size corresponds to an image-plane patch about twice as large (Chapter 18's full argument is that surface brightness is conserved, so a bigger apparent patch at the same brightness-per-area means more total flux). A determinant less than one in the $\theta \to \beta$ direction always means magnification greater than one in the observed direction — the two are reciprocals by construction, not by coincidence.

Exercise 4.3 — a gradient's Jacobian is a Hessian, and it is symmetric

Let $\psi(x_1, x_2) = x_1^2 x_2 + x_2^3$. Compute $\nabla\psi$, then compute the Jacobian of $\nabla\psi$ (treating it as a map $\mathbb{R}^2 \to \mathbb{R}^2$) directly from its four partial derivatives. Confirm it equals the Hessian of ψ,

and confirm it is symmetric.

Solution

$\nabla\psi = (2x_1x_2,\ x_1^2 + 3x_2^2)$. Differentiating this vector field:

$$J_{\nabla\psi} = \begin{pmatrix} \partial(2x_1x_2)/\partial x_1 & \partial(2x_1x_2)/\partial x_2 \\ \partial(x_1^2+3x_2^2)/\partial x_1 & \partial(x_1^2+3x_2^2)/\partial x_2 \end{pmatrix} = \begin{pmatrix} 2x_2 & 2x_1 \\ 2x_1 & 6x_2 \end{pmatrix}.$$

This is exactly $H(\psi)$: $\partial^2\psi/\partial x_1^2 = 2x_2$, $\partial^2\psi/\partial x_2^2 = 6x_2$, and both mixed partials equal $2x_1$. The off-diagonal entries agree ($2x_1 = 2x_1$) because mixed partials commute for any polynomial (Schwarz's theorem) — the matrix is symmetric for every (x_1, x_2), not just by luck at one point. This is the general fact Chapter 6 invokes for the lensing potential: $\alpha = \nabla\psi$ makes the deflection field's own Jacobian automatically symmetric, before any lens-specific physics enters.

Exercise 4.4 — when does μ flip sign?

$\mu = 1/\det A$ is *signed*, not just an absolute ratio. Under what condition on A's entries does $\det A < 0$? Given that $A = (1-\kappa)I - \Gamma$ for lensing (Chapter 5 derives this decomposition), can you say anything yet about what a negative μ might mean physically? (You are not expected to fully answer this — it is Chapter 18's job. State what you can derive now, and what you cannot.)

Solution

For a 2×2 matrix, $\det A = a_{11}a_{22} - a_{12}a_{21} < 0$ exactly when the product of the diagonal entries is smaller than the product of the off-diagonal entries — geometrically, when the map reverses orientation (a right-handed basis maps to a left-handed one). What you *can* derive now: since $\mu = 1/\det A$ and $\det A$ can be negative, μ can be negative too, and nothing in this chapter's area-ratio argument breaks — $|\det A|$ is still the area-scaling factor, and the sign is separate information. What you cannot yet derive: *why* a sign flip corresponds to a genuinely different physical image (a parity-flipped image, one of the tell-tale signs that you have crossed a critical curve). That requires the actual lens equation and its multiple solutions, which is exactly what Chapter 17 sets up and Chapter 18 (parity) resolves.

5. Eigenvalues, saddles, and conditioning

Every lens Jacobian this book differentiates turns out to be a *symmetric* 2×2 matrix, and a symmetric 2×2 matrix is one of the few objects in mathematics whose eigenproblem you can solve completely, by hand, in four lines. This chapter does that once and spends it three ways: reading convergence and shear directly out of a Jacobian's eigenvalues (which sets up Ch. 18's critical curves), saying precisely what a *saddle* is and why the campaign's own MAP-finder walked into one without noticing (which sets up Ch. 26), and putting a number — not an adjective — on how badly conditioned this repo's hardest real posterior actually is. Ch. 2 already showed you the crime scene; this chapter builds the tool that reads it.

> **What you can skip**
>
> If you can already state and prove the spectral theorem for real symmetric matrices, classify a Hessian as positive/negative (semi)definite or indefinite from its eigenvalues, and define a matrix's condition number as a ratio of singular values, skim Symmetric 2×2s for the lensing-specific reading of κ and shear and read Definiteness and saddles and Conditioning mainly for the repo's own numbers. This chapter reproves the 2×2 spectral theorem from the characteristic polynomial rather than citing the general n-dimensional version you already own — it is four lines, and the specific form pays for itself immediately.

Symmetric 2×2 matrices eigendecompose in closed form

Ch. 4 built the lens Jacobian $A = \partial\beta/\partial\theta$ and read its determinant as a local area ratio. This chapter asks a sharper question: not just how much area A scales, but *in which directions*, and by how much each.

A is not an arbitrary 2×2 matrix. Because the deflection is a gradient, $\alpha = \nabla\psi$, the lens equation gives $A = I - \mathrm{Hess}(\psi)$, and mixed partial derivatives of a smooth scalar commute — $\partial^2\psi/\partial\theta_1\partial\theta_2 = \partial^2\psi/\partial\theta_2\partial\theta_1$ (Ch. 6 states this properly). So $a_{12} = a_{21}$ always: the lens Jacobian is *symmetric*, every time, for every mass profile. That single fact is why the rest of this section works.

Write a general symmetric 2×2 matrix as

$$A = \begin{pmatrix} a & b \\ b & d \end{pmatrix}.$$

Its eigenvalues solve $\det(A - \lambda I) = 0$, which expands to the characteristic polynomial

$$\lambda^2 - (a + d)\,\lambda + (ad - b^2) = 0.$$

The quadratic formula gives

$$\lambda_{1,2} = \frac{a + d}{2} \pm \sqrt{\left(\frac{a - d}{2}\right)^2 + b^2}.$$

Two things fall out of this for free, and both are the spectral theorem for 2×2 matrices, proved rather than quoted. First, the quantity under the square root is a sum of two squares, so it is never negative — λ_1 and λ_2 are always real. (In more than two dimensions the same statement is

true but needs more machinery than the quadratic formula; here you get it for nothing.) Second, if $b = 0$ the eigenvectors are the coordinate axes, and rotating a symmetric matrix's frame to make $b = 0$ is always possible — the eigenvectors of a symmetric matrix are always orthogonal. You will use both facts without comment for the rest of this book.

Now specialize. Write the trace of A as an isotropic part and the remainder as a traceless part:

$$A = (1-\kappa)\, I \;-\; \Gamma, \qquad \Gamma = \begin{pmatrix} \gamma_1 & \gamma_2 \\ \gamma_2 & -\gamma_1 \end{pmatrix}.$$

κ (the trace half) is the isotropic squeeze: it scales both directions equally. Γ (the traceless remainder) is the *shear*: it stretches one direction while compressing the perpendicular one. Trace zero means these two effects cancel *to first order* — a determinant's linear-order response to a perturbation is its trace, so a traceless perturbation costs no area to first order — but they do not cancel exactly at finite shear; Exercise 5.1 has you check this on the figure's own numbers. This split is exactly what `lensing.py` computes (`kappa_gamma_from_jacobian`, `lensing.py:126-136`): $\kappa = 1 - \frac{1}{2}(a_{11} + a_{22})$ from the trace, $\gamma_1 = -\frac{1}{2}(a_{11} - a_{22})$ and $\gamma_2 = -\frac{1}{2}(a_{12} + a_{21})$ from what is left over. Because Γ is already traceless, its own eigenvalues are $\pm\sqrt{\gamma_1^2 + \gamma_2^2}$ directly from the formula above (the trace term vanishes, and $ad - b^2$ becomes $-\gamma_1^2 - \gamma_2^2$). Subtracting Γ from $(1-\kappa)I$ therefore gives

$$\lambda_{\mathrm{t}} = (1-\kappa) - \sqrt{\gamma_1^2 + \gamma_2^2}, \qquad \lambda_{\mathrm{r}} = (1-\kappa) + \sqrt{\gamma_1^2 + \gamma_2^2}.$$

These are named for what they turn out to mean physically: near a roughly-circular lens, the smaller eigenvalue λ_{t} governs the **tangential** direction (perpendicular to the line to the lens centre) and the larger λ_{r} the **radial** direction. That naming is not yet justified by anything in this chapter — it becomes the content of Ch. 18, where $\lambda_{\mathrm{t}} = 0$ turns out to be exactly the condition that draws the Einstein ring.

Worked example. Take $\kappa = 0.3$, $\gamma_1 = 0.3$, $\gamma_2 = 0$ — a lens with both an isotropic squeeze and a shear stretch of equal size. The shear magnitude is $\sqrt{\gamma_1^2 + \gamma_2^2} = 0.3$, so

$$\lambda_{\mathrm{t}} = 0.7 - 0.3 = 0.4, \qquad \lambda_{\mathrm{r}} = 0.7 + 0.3 = 1.0.$$

A unit source circle, pushed backward through A^{-1} into the image plane, comes out stretched by $1/\lambda_{\mathrm{t}} = 2.5$ along one axis and by $1/\lambda_{\mathrm{r}} = 1.0$ (unchanged) along the other — an ellipse, not a circle, which is the entire mechanism by which a lens turns a round source into an arc.

Definiteness, and why a saddle is not a mode

A symmetric matrix A is **positive definite** if every eigenvalue is positive (equivalently $\mathbf{x}^{\mathsf{T}} A\, \mathbf{x} > 0$ for every $\mathbf{x} \neq 0$), **negative definite** if every eigenvalue is negative, and **indefinite** if it has eigenvalues of both signs. For a 2×2 symmetric matrix you do not need the eigenvalues to tell which case you are in: since $\lambda_1 \lambda_2 = \det A$ and $\lambda_1 + \lambda_2 = \operatorname{tr} A$,

$$\det A > 0,\ \operatorname{tr} A > 0 \;\Rightarrow\; \text{PD}, \qquad \det A > 0,\ \operatorname{tr} A < 0 \;\Rightarrow\; \text{ND}, \qquad \det A < 0 \;\Rightarrow\; \text{indefinite}.$$

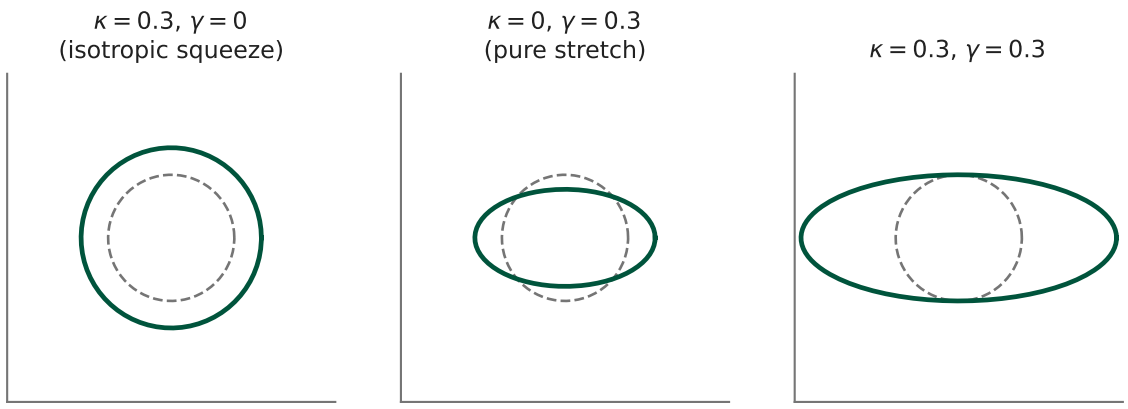

Figure 2: A unit source circle (dashed) pushed through A^{-1} for three cases: pure convergence ($\kappa = 0.3$, no shear — an isotropic circle, unchanged in shape), pure shear ($\kappa = 0$, $\gamma_1 = 0.3$, $\gamma_2 = 0$), and both together (the worked example above, right panel: $\lambda_t = 0.4$, $\lambda_r = 1.0$). The panel titles, generated by `figures.py`, write the shear magnitude as a bare "γ" for plot-space brevity — this guide reserves that symbol for the density slope, so read the figure's own label as $\sqrt{\gamma_1^2 + \gamma_2^2}$. It is a small, live example of exactly the overloading hazard this guide's notation contract exists to police.

An indefinite Hessian is what a **saddle** is, in coordinates: a point where the gradient is zero but the function curves up in some directions and down in others — Ch. 2, Exercise 2.3 works $f(x, y) = x^2 - y^2$ by hand and you should have that toy example in mind for what follows. The determinant test above is exactly why Ch. 4's determinant machinery was worth building before this chapter: you can diagnose a saddle from $\det A$ alone, with no eigendecomposition at all, the moment $\det A < 0$.

> **You already know this**
>
> "A vanishing gradient is not a verdict" is folklore in non-convex optimization for exactly this reason: in high dimensions, saddle points vastly outnumber true minima, and a first-order method can stall on one indefinitely. Dauphin et al. (2014), *"Identifying and attacking the saddle point problem in high-dimensional non-convex optimization,"* is the paper most ML researchers know for this. This repo's own `foundry-i` phase independently implemented that paper's fix — replace the Hessian H by its *absolute* curvature $|H| = V \operatorname{diag}(|\lambda_i|)\, V^\mathsf{T}$, so a Newton-style step *descends* along every eigendirection instead of climbing the negative-curvature ones — in `reproductions/foundry-i/32_saddlefree_newton.py`, on an earlier saddle in this same modelling pipeline. Hold onto that formula, $V \operatorname{diag}(1/|\lambda_i|)\, V^\mathsf{T}$: Ch. 26 shows it again, used for a different purpose, where it stops being correct.

The campaign hit the real thing directly. Its correlated-noise fit's MAP-finding stage drove $\nabla \log p$ to zero and reported success; a second-order check — `jax.hessian` of the log-posterior at that point (`reproductions/claude-giga-lens/cgl/e2.py:554`, eigendecomposed at `cgl/e2.py:557-558`) — showed the point was a **saddle**: minimum eigenvalue -14.85, five of forty-six eigenvalues negative (`reproductions/claude-giga-lens/CAMPAIGN.md`, "P1c metric-fix attempts — 2026-07-10"; main.tex, §6.3). A negative eigenvalue of H there means the log-posterior keeps *rising* as you move away from that point along that eigendirection — `map_polish` reported a local *trough* of the log-posterior along that one axis, not a peak, so nearby points along it score higher, not lower. The campaign's own ledger catches this concretely, on the same fit: the saddle-consistent point

$\gamma = 1.27$ scored $\log p = -4757$, while $\gamma = 1.10$ — a direction the saddle-seeded optimizer never explored — scored $\log p = -4683$, a full 74 nats higher. (Both γ values here are a mid-campaign snapshot, superseded by Ch. 25's converged answer; they are quoted only to make "a saddle is not the top" concrete on data instead of a toy.) A `map_polish` stage that stops the instant $\nabla \log p = 0$ has, by construction, no way to see any of this — exactly Ch. 2's point, now with the repo's own numbers attached. Ch. 26 tells the rest of the story: why this particular saddle resisted the standard fix, and how it was eventually sampled around rather than through.

Conditioning: cond $\sim 10^{14}$ is not an abstraction

For a symmetric matrix, define the **condition number** as the ratio of the largest to the smallest eigenvalue by absolute value, $\mathrm{cond}(A) = |\lambda_{\max}|/|\lambda_{\min}|$. (For a general, non-symmetric matrix the definition uses singular values instead; every matrix this book differentiates is symmetric, so the distinction never matters again.) It measures how much A can amplify a relative error: solving $A\mathbf{x} = \mathbf{b}$, a relative perturbation of size ε in $\mathbf{b}$ can produce a relative perturbation as large as $\mathrm{cond}(A) \cdot \varepsilon$ in $\mathbf{x}$ — and every number stored in float64 already carries a relative rounding error on the order of machine epsilon, $\varepsilon_{64} \approx 2.22 \times 10^{-16}$, whether you asked for it or not.

Float64 gives you about $-\log_{10} \varepsilon_{64} \approx 15.65$ significant decimal digits of headroom . Multiplying by a condition number of 10^{14} spends 14 of those digits purely amplifying input error, leaving

$$15.65 - \log_{10}(10^{14}) \approx 1.65$$

trustworthy decimal digits along the worst-conditioned direction — under two. That is the sense in which "cond $\sim 10^{14}$" is not a figure of speech: it is a direct statement that a naive Hessian inversion along that direction is barely more precise than a coin flip on the second digit.

This is not a hypothetical matrix. `foundry-i`'s real-data posterior — the same physical system this campaign's final report calls **T2**, the 46-dimensional marginalized real posterior of the actual DESI system DESI–165.4754–06.0423 — is, in its own words, "ultra-ill-conditioned (cond $\sim 10^{14}$): lens-light companion Sérsic centers at $H_{ii} \sim 10^{12}$, Sérsic indices n_{sersic} at $\sim 10^{-2}$" (`reproductions/foundry-i/README.md:114-117`). Take those two diagonal Hessian entries at face value and divide:

$$\frac{H_{ii}^{\max}}{H_{ii}^{\min}} = \frac{10^{12}}{10^{-2}} = 10^{14}.$$

(These are diagonal entries in the parameters' own coordinates, not literally the Hessian's eigenvalues — off-diagonal correlations mix parameters together in the true eigenbasis — but for a diagonally-dominant Hessian they set the scale, and here the naive ratio reproduces the quoted order of magnitude exactly.) The physical story behind the two numbers is simple: a Sérsic light centre is pinned by the data to a tiny fraction of a pixel, so the log-posterior curves *sharply* there; the Sérsic index is barely constrained at all, so it curves *almost not at all* — one parameter the data has opinions about, one it does not, in the same matrix. Marginalizing away the source's shapelet amplitudes does not fix this: the campaign's final report quotes the identical order of magnitude for the marginalized T2 target (§2.3, "The posterior zoo"). The ill-conditioning is not a bug a later processing step removed; it is a fact about which physical quantities a single pixel grid can and cannot pin down.

One more collision to flag by name, because this repo uses the letter twice more: the A above is the Laplace Hessian of the log-posterior — nothing to do with the lens Jacobian A from the first section, and a third, better-behaved A appears in Ch. 22: the ridge-regularized normal matrix `cgl/marg.py:48` builds while profiling out the source's linear shapelet amplitudes analytically,

$$A = X_{\mathrm{w}}^{\mathsf{T}} X_{\mathrm{w}} + \mathrm{diag}(\Lambda).$$

That A is positive definite *by construction*: adding $\mathrm{diag}(\Lambda)$ with every $\Lambda_i > 0$ floors every eigenvalue away from zero before anything can go wrong — the same move Ch. 8 will call a Gaussian prior and Ch. 22 will call ridge regression. At the campaign's own parity gate this A measures $\mathrm{cond}(A) = 1.37 \times 10^4$ (`reproductions/claude-giga-lens/CAMPAIGN.md`, P0 exit gate, 2026-07-06) — about 7.3×10^9 times friendlier than the raw Hessian above , purely because someone added a $+\Lambda$ before the trouble started. The same tension — a raw, ill-conditioned real-data Hessian versus a regularized, well-conditioned stand-in for it — is what Ch. 23 and Ch. 26 fight out for the sampler's own mass matrix.

Connect to the repo

- `site/guide_src/lensing.py:126-136` (`kappa_gamma_from_jacobian`) — the trace/traceless split this chapter derives, in six lines of numpy.
- `site/guide_src/figures.py:52-73` (`kappa_gamma_eigen`) — generates Figure 5.1 by constructing $A = (1-\kappa)I - \Gamma$ directly and mapping a circle through A^{-1}; nothing in it is hand-drawn.
- `reproductions/claude-giga-lens/cgl/e2.py:530-572` (`laplace_evidence`) — computes `H`, eigendecomposes it (`w`, `V`), and counts `n_neg`; this is the function that caught the saddle.
- `reproductions/foundry-i/32_saddlefree_newton.py` — an earlier saddle in the same pipeline, and the $V\,\mathrm{diag}(1/|\lambda_i|)V^{\mathsf{T}}$ formula Ch. 26 revisits.
- `reproductions/claude-giga-lens/cgl/marg.py:48` — the ridge-regularized normal matrix, `cond(A) = 1.37e4` at the parity gate.
- `reproductions/claude-giga-lens/CAMPAIGN.md`, "P1c metric-fix attempts — 2026-07-10" — the saddle diagnosis in the campaign's own words, retractions included.
- main.tex, §2.3, "The posterior zoo" and §6.3, "Why sampling this posterior required tempered SMC" — the published record of both numbers this chapter uses.

Exercises

> **Exercise 5.1 — the other two panels of Figure 5.1**
>
> Using $\lambda_{\mathrm{t}} = (1-\kappa) - \sqrt{\gamma_1^2 + \gamma_2^2}$ and $\lambda_{\mathrm{r}} = (1-\kappa) + \sqrt{\gamma_1^2 + \gamma_2^2}$, compute the eigenvalues for the left panel ($\kappa = 0.3$, $\gamma_1 = \gamma_2 = 0$) and the middle panel ($\kappa = 0$, $\gamma_1 = 0.3$, $\gamma_2 = 0$) of Figure 5.1. For each, say whether the image of a circle under A^{-1} is a circle or an ellipse, and why. Then check the text's claim that pure shear costs no area *to first order*: compute $\det A$ for the middle panel and compare it to 1.

Solution

Left panel: $\lambda_t = \lambda_r = 0.7$. Equal eigenvalues mean A^{-1} scales every direction by the same factor $1/0.7$ — the image is a *larger circle*, not an ellipse, because pure convergence has no preferred direction. Middle panel: $\lambda_t = 0.7$, $\lambda_r = 1.3$ — unequal, so the image is an ellipse, elongated along the λ_t eigendirection where A^{-1} stretches more. Only shear breaks the circle's symmetry; convergence alone never does, which is exactly why this chapter calls κ "isotropic" and Γ "traceless." For the area check: $\det A = 0.7 \times 1.3 = 0.91$, not 1 — at $\gamma_1 = 0.3$ the second-order term $-\gamma_1^2 = -0.09$ is not negligible. The claim was "no *linear-order* area cost," and $1 - \gamma_1^2$ has no linear term in γ_1 by construction; it is not a claim that finite shear leaves area untouched, and here it plainly does not ($\mu = 1/0.91 \approx 1.10$).

Exercise 5.2 — classify without eigendecomposing

A Hessian at a candidate fit point has $\operatorname{tr} H = -6$ and $\det H = 8$. Using only the trace/determinant test from Definiteness and saddles, classify it as PD, ND, or indefinite. Then find its two eigenvalues directly and confirm.

Solution

$\det H = 8 > 0$ and $\operatorname{tr} H = -6 < 0$, so the test gives **negative definite** — a genuine local maximum of whatever function H is the Hessian of, not a saddle. Solving $\lambda^2 - (-6)\lambda + 8 = 0$, i.e. $\lambda^2 + 6\lambda + 8 = 0$, factors as $(\lambda + 2)(\lambda + 4) = 0$, giving $\lambda = -2, -4$ — both negative, confirming ND without needing the factorization to know it in advance.

Exercise 5.3 — a local zero is not a global mean

$\lambda_t = 0$ says $\kappa + \sqrt{\gamma_1^2 + \gamma_2^2} = 1$ at a single image-plane point. Ch. 19 will show that $\bar{\kappa}(\theta_E) = 1$, the *mean* convergence inside the Einstein radius. Are these the same statement? If not, what is the precise relationship between the curve $\lambda_t = 0$ and the circle $\theta = \theta_E$ for a lens that is not perfectly circular?

Solution

No — one is local, one is global, and conflating them is a real trap. $\bar{\kappa}(\theta_E) = 1$ is an *average* over a disc of radius θ_E; it holds by definition for any profile, circular or not, and says nothing about any single point on the boundary. $\lambda_t = 0$ is a pointwise condition on the Jacobian and traces out the **critical curve** — for a circular lens the critical curve is a circle of radius exactly θ_E and the two statements coincide, but for an elliptical or shear-bearing lens the critical curve is a non-circular closed curve while θ_E (defined through the mean) still names a single radius, usually chosen as the effective radius of the same-area circle. Ch. 18 draws the two apart explicitly.

Exercise 5.4 — how many digits does a friendlier matrix buy you

Using $15.65 - \log_{10}(\operatorname{cond} A)$, compute the stable digit count for the ridge-regularized marg

normal matrix, $\mathrm{cond}(A) = 1.37 \times 10^4$, and compare it to the ~ 1.65 digits left by the raw Hessian's cond $\sim 10^{14}$.

Solution

$\log_{10}(1.37 \times 10^4) \approx 4.14$, so $15.65 - 4.14 \approx 11.5$ trustworthy digits — essentially all of float64's headroom, versus under two for the raw Hessian. The $+\Lambda$ in `cgl/marg.py:48` is not a minor numerical nicety; it is the difference between a matrix you can safely invert and one you cannot.

6. Divergence, Laplacian, Green's functions: the potential trio

This chapter derives one equation and one convolution, and everything from Ch. 16 onward cashes them in without re-deriving them. The equation is $\nabla^2\psi = 2\kappa$, alongside $\alpha = \nabla\psi$: the deflection field is the gradient of a scalar potential, and the convergence (the mass, in the units Ch. 15 built) is half that potential's Laplacian. The convolution is the reason every lensing potential you will meet — point mass, SIS, SIE, EPL — is built from a logarithm: the logarithm is the Green's function of the 2-D Laplacian, the same object that turns "some mass distribution" into "the potential it sources." Divergence and the Laplacian are the last two vector-calculus tools Part IV needs; this chapter builds them from the gradient and Jacobian you already have (Ch. 4) and hands you back the single identity that the rest of the book calls "the trio."

> **What you can skip**
>
> If you remember the divergence theorem (flux through a closed surface equals the integral of divergence over the enclosed volume) from an undergraduate course, skip straight to Green's function of the 2-D Laplacian — the general theory isn't re-derived here, only the one 2-D consequence this repo needs. If you don't remember it, or never proved it, the short argument in the next section is the whole proof you need for this book; it is not restated in later chapters.

Divergence and the Laplacian

For a vector field $\mathbf{F}(x, y) = (F_x, F_y)$, the **divergence** is the scalar

$$\nabla \cdot \mathbf{F} = \frac{\partial F_x}{\partial x} + \frac{\partial F_y}{\partial y}.$$

Read it as a flux density: $\nabla\cdot\mathbf{F}$ at a point measures how much $\mathbf{F}$ is "spreading out" of an infinitesimal neighborhood of that point, per unit area. The divergence theorem makes this precise:

$$\oint_{\partial\Omega} \mathbf{F} \cdot \hat{n}\, d\ell = \int_{\Omega} \nabla \cdot \mathbf{F}\, dA$$

for any region Ω — but the intuition ("divergence = local source density") is what you need going forward.

The **Laplacian** of a scalar field f is the divergence of its gradient:

$$\nabla^2 f \;=\; \nabla \cdot (\nabla f) \;=\; \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}.$$

You already built the machinery for this in Ch. 4: $\nabla^2 f$ is exactly the **trace** of the Hessian of f — the sum of its diagonal entries, and (per Ch. 5's eigendecomposition of a symmetric matrix) the sum of its eigenvalues too, since trace is basis-independent. That single fact — Laplacian = trace of Hessian — is the entire content of the derivation in Poisson's equation for lensing below.

A field with $\nabla^2 f = 0$ everywhere is called **harmonic**. Harmonic functions have no local sources or sinks: the value at any point equals the average of f over any small circle centered there. That mean-value property is why a harmonic potential can only vary because of what's happening *somewhere else* — never because of anything at the point itself.

One more identity, used silently every time this repo differentiates a deflection field instead of writing down a potential: for any smooth scalar f, $\nabla \times (\nabla f) \equiv 0$ — the curl of a gradient is identically zero, because mixed partial derivatives commute ($\partial_x \partial_y f = \partial_y \partial_x f$). A field that comes from a potential can never rotate. This is why gigalens' EPL profile is allowed to hand you $\alpha(\theta_1, \theta_2)$ as a closed-form deflection and never construct ψ at all (`gigalens/jax/profiles/mass/epl.py:19`, linked in Connect to the repo): as long as $\partial \alpha_{\theta_1} / \partial \theta_2 = \partial \alpha_{\theta_2} / \partial \theta_1$ holds identically in the closed form, a ψ is guaranteed to exist whose gradient it is, even though nobody ever writes it down.

> **You already know this**
>
> A continuous normalizing flow (a Neural ODE, $dz/dt = f(z,t)$) evolves its log-density as $d(\log p)/dt = -\nabla_z \cdot f(z,t)$ — the divergence of the flow field, which is also the trace of its Jacobian. That is the continuous-time cousin of the $-\log|\det J|$ a discrete flow layer pays per step (already on this guide's Log-Det Ledger, opened in Ch. 4). Divergence is "how much a vector field spreads a density out." In this chapter it spreads mass out of a region of sky; in a flow it spreads probability out of a region of latent space. Same operation, different density.

Green's function of the 2-D Laplacian

A **Green's function** is the impulse response of a linear operator: solve $L[G] = \delta$ (a unit point source), and the solution for *any* source s is the convolution $f = G * s$. For the 2-D Laplacian, the Green's function is a logarithm, and you can derive it with nothing but the divergence theorem.

Put a unit point source at the origin. By symmetry, the field $\mathbf{E} = \nabla G$ it sources must be radial: $\mathbf{E}(r) = E(r)\,\hat{r}$. Apply the divergence theorem to a disk of radius r centered on the source. The enclosed source is 1 regardless of r (it's a point, and any disk around it encloses all of it), so the flux out through the boundary circle must also be 1 for every $r > 0$:

$$E(r) \cdot 2\pi r = 1 \quad \Longrightarrow \quad E(r) = \frac{1}{2\pi r}.$$

Integrating $E(r) = dG/dr$ gives

$$G(r) = \frac{1}{2\pi} \ln r + \text{const}.$$

This is the differential-equation route to the same fact Ch. 3 derived by integrating Gauss's law directly over a 2-D mass distribution: a 2-D point source has a *logarithmic* potential, not the $1/r$ you'd expect from 3-D intuition. () is harmonic everywhere except at $r = 0$ — checkable numerically, since "harmonic away from the source" is the testable half of "all the source sits at one point."

Worked check. Take the lensing potential of a point mass, $\psi_{\text{pt}}(\theta_1, \theta_2) = \theta_{\text{E}}^2 \ln r$ with $\theta_{\text{E}} = 1''$ and $r = \sqrt{\theta_1^2 + \theta_2^2}$ — the same functional form as (), with the point mass folded into the prefactor. Evaluate its Laplacian by central finite differences at $(\theta_1, \theta_2) = (1.2'', 0.9'')$, i.e. $r_0 = 1.5''$. The five-point stencil returns -2.2×10^{-8} : zero to floating-point precision, off by roughly 10^{-8} purely from `h = 1e-4` truncation error — confirming that away from the point, this potential carries no convergence at all. (This point-mass potential is *not* one of this repo's mass profiles — its galaxies are extended isothermal-like systems, Ch. 20's SIS/SIE/EPL family — but it is the cleanest

illustration of the Green's function, because it isolates the singular source term from everything else.)

Poisson's equation for lensing

Newtonian gravity obeys $\nabla^2\Phi = 4\pi G\rho$ — the same divergence- theorem argument as the last section, sourced by mass instead of a unit point charge. Ch. 16 works out how projecting this along the line of sight and rescaling by the distance factors and $\Sigma_{\rm cr}$ (Ch. 15) collapses the 3-D equation to a 2-D one for the dimensionless lensing potential $\psi(\theta)$:

$$\nabla^2\psi = 2\kappa, \qquad \alpha = \nabla\psi.$$

This is a *different* factor of two from Ch. 16's GR-is-twice-Newton deflection — two unrelated 2's sitting in the same subject, by coincidence of convention, not physics. Keep them apart.

You can derive exactly where ()'s "2" comes from using only algebra you already have, with no new physics. Since $\alpha = \nabla\psi$, the Jacobian of the deflection field is the Hessian of ψ: $\partial\alpha/\partial\theta = H_\psi$. The lens Jacobian (Ch. 17, defined by $A = \partial\beta/\partial\theta$ with $\beta = \theta - \alpha$) is therefore $A = I - H_\psi$. The notation contract also fixes $A = (1-\kappa)I - \Gamma$, with Γ the *traceless* shear matrix. Equating the two expressions for A:

$$H_\psi = I - A = \kappa I + \Gamma.$$

Take the trace of both sides. $\mathrm{tr}(\Gamma) = 0$ by construction (that is what "traceless" means), and $\mathrm{tr}(H_\psi) = \nabla^2\psi$ by definition from the previous section. So

$$\nabla^2\psi = \mathrm{tr}(H_\psi) = 2\kappa + \mathrm{tr}(\Gamma) = 2\kappa.$$

The "2" is not a physical constant to look up — it is the trace of an identity matrix in 2-D, appearing because κ was *defined* as the isotropic (trace) part of H_ψ and Γ was defined to soak up everything traceless. Equivalently, since $\alpha = \nabla\psi$, you can skip the Hessian and go straight for divergence: $\nabla\cdot\alpha = \nabla\cdot(\nabla\psi) = \nabla^2\psi = 2\kappa$.

Worked check. `site/guide_src/lensing.py`'s `L.sis_deflection` is the SIS deflection $\alpha = \theta_{\rm E}\,\hat{\theta}$ (Ch. 20 works out why an isothermal profile gives a *constant-modulus* deflection). Its analytic convergence, $\kappa_{\rm SIS}(\theta) = \theta_{\rm E}/(2\theta)$, is the same form Ch. 19 uses as `lambda t: 0.5 * 1.0 / t`. At $\theta = 1.5''$ with $\theta_{\rm E} = 1''$:

$$\kappa_{\rm SIS}(1.5) = \frac{1}{2\times 1.5} = \frac{1}{3} \approx 0.3333.$$

So $2\kappa \approx 0.6667$. Now compute $\nabla\cdot\alpha$ with no shortcut: finite-difference `L.sis_deflection` itself at four points around $(1.2'', 0.9'')$ and sum $\partial\alpha_{\theta_1}/\partial\theta_1 + \partial\alpha_{\theta_2}/\partial\theta_2$. The result is 0.6667 , matching 2κ to a residual of about 2×10^{-9} — pure finite-difference truncation, nothing else. This is not a new computation dressed up: it is *exactly* what `L.kappa_gamma_from_jacobian` already does at `site/guide_src/lensing.py:133` (`kappa = 1.0 - 0.5 * (a11 + a22)`), which you met in Ch. 5 as an eigenvalue decomposition and in Ch. 18 as a magnification ingredient. That line

differentiates β, not α, and gets κ by $1 - \mathrm{tr}(A)/2$ rather than $\mathrm{tr}(H_\psi)/2$ — but $\mathrm{tr}(A) = 2 - \mathrm{tr}(H_\psi)$, so it is the identical arithmetic, one substitution away.

The synthesis of this chapter's two sections: κ is the *source*, ψ is what the Green's function convolution of that source produces,

$$\psi(\theta) = \frac{1}{\pi} \int \kappa(\theta') \, \ln|\theta - \theta'| \, d^2\theta' + (\text{linear terms}),$$

and $\alpha = \nabla\psi$ differentiates that convolution. gigalens never performs this integral — SIS, SIE and EPL all have closed-form deflections precisely because someone already did the convolution by hand — but the closed form is only trustworthy because () guarantees it corresponds to *some* κ in the first place.

Connect to the repo

- `site/guide_src/lensing.py:101` (`lens_jacobian`) and `site/guide_src/lensing.py:133` (`kappa_gamma_from_jacobian`) are this chapter's identity, discretized: a finite-difference Hessian of the deflection field, traced and halved.
- `reproductions/claude-giga-lens/vendor/gigalens-sean/src/gigalens/jax/profiles/mass/epl.py` is the production EPL deflection: a closed-form α that never materializes ψ, relying silently on curl-free-ness for its own consistency.
- The letter A is doing two unrelated jobs in this repo: the lens Jacobian here, and the Gaussian-normal matrix in `reproductions/claude-giga-lens/cgl/marg.py`'s Occam term (Ch. 22). Same symbol, different matrix — the notation contract flags this collision on purpose.

Exercises

Exercise 6.1 — Trace is basis-independent, so κ doesn't care which way you point your axes

Let R be a 2-D rotation matrix. Show that $\mathrm{tr}(R^\top H_\psi R) = \mathrm{tr}(H_\psi)$ for any matrix H_ψ. What does this tell you about computing κ in a rotated coordinate system — say, aligned with a galaxy's major axis instead of RA/Dec?

Solution

Trace is invariant under conjugation: $\mathrm{tr}(R^\top H_\psi R) = \mathrm{tr}(H_\psi R R^\top) = \mathrm{tr}(H_\psi I) = \mathrm{tr}(H_\psi)$, using the cyclic property of trace and $RR^\top = I$ for a rotation. Since $\nabla^2\psi = \mathrm{tr}(H_\psi)$, the Laplacian — and hence κ — is the same number no matter which orthonormal axes you differentiate in. This is why κ can be quoted as a single scalar per point without ever specifying an orientation, while γ_1, γ_2 (the *traceless* part) rotate into each other and must always be quoted with a position angle — exactly the q, ϕ machinery of Ch. 20.

Exercise 6.2 — The Green's function's flux doesn't care about the radius either

Using $E(r) = 1/(2\pi r)$ from (), show by direct computation that the flux $\oint \mathbf{E} \cdot \hat{n} \, d\ell$ through a circle of radius R is exactly 1 for *every* $R > 0$, not just as $R \to \infty$. Why does this have to be true for G to be a valid Green's function of a *point* source?

Solution

The boundary integral is $\oint E(R)\,d\ell = E(R)\cdot 2\pi R = \frac{1}{2\pi R}\cdot 2\pi R = 1$, independent of R. It has to be radius-independent because a point source has no size: any circle that encloses the point encloses *all* of it, so the enclosed "charge" (and hence the flux, by the divergence theorem) cannot depend on how big a circle you drew. This is the same radius-independence argument behind Ch. 19's claim that $\bar{\kappa}(\theta_{\rm E}) = 1$ is a *definition*, not a coincidence: it is a statement about enclosed mass, and enclosed mass inside a fixed boundary doesn't care about the profile's shape outside it.

Exercise 6.3 — Derive $\kappa_{\rm SIS}$ by hand, in polar form

The SIS potential is $\psi_{\rm SIS}(\theta) = \theta_{\rm E}\,\theta$ (radial, $\theta = |\theta|$). (a) Compute $\nabla\psi_{\rm SIS}$ and confirm it matches `L.sis_deflection`'s constant-modulus form. (b) The 2-D Laplacian of a purely radial function $f(\theta)$ is $f''(\theta) + f'(\theta)/\theta$. Use this to get $\nabla^2\psi_{\rm SIS}$, then $\kappa_{\rm SIS} = \frac{1}{2}\nabla^2\psi_{\rm SIS}$, and check it against the 0.3333 this chapter computed numerically at $\theta = 1.5''$.

Solution

(a) $\partial\psi/\partial\theta = \theta_{\rm E}$, and in 2-D $\nabla f(\theta) = f'(\theta)\,\hat{\theta}$, so $\nabla\psi_{\rm SIS} = \theta_{\rm E}\,\hat{\theta}$ — constant modulus $\theta_{\rm E}$, independent of θ, exactly `L.sis_deflection`'s "flat rotation curve" behavior. (b) $f(\theta) = \theta_{\rm E}\theta$ gives $f' = \theta_{\rm E}$, $f'' = 0$, so $\nabla^2\psi_{\rm SIS} = 0 + \theta_{\rm E}/\theta = \theta_{\rm E}/\theta$. Then $\kappa_{\rm SIS}(\theta) = \theta_{\rm E}/(2\theta)$. At $\theta = 1.5''$, $\theta_{\rm E} = 1''$: $\kappa = 1/3 \approx 0.3333$ — the same number the finite-difference check landed on, this time from four lines of calculus instead of four function evaluations.

Exercise 6.4 — Why a proposed deflection field can be *wrong* even if it looks reasonable

Suppose someone hands you a candidate deflection field $\alpha(\theta_1, \theta_2) = (\theta_2, -\theta_1)$ — smooth, well-defined everywhere, finite. Using the curl-free identity from Divergence and the Laplacian, determine whether any lensing potential ψ could produce it. What does this rule out about which vector fields are legitimate deflection fields at all?

Solution

Curl in 2-D is $\partial\alpha_2/\partial\theta_1 - \partial\alpha_1/\partial\theta_2$. Here that's $\partial(-\theta_1)/\partial\theta_1 - \partial\theta_2/\partial\theta_2 = -1 - 1 = -2 \neq 0$. Since $\nabla\times(\nabla\psi) \equiv 0$ for any smooth ψ, no potential can produce this field — it is a pure rotation (it sends every point to a 90-degree-rotated version of itself), and lensing deflections, being gradients of a scalar, can never rotate anything. This is a genuine, checkable constraint: any closed-form deflection a mass profile proposes (gigalens' EPL included) must satisfy $\partial\alpha_{\theta_1}/\partial\theta_2 = \partial\alpha_{\theta_2}/\partial\theta_1$ identically, or it does not correspond to any mass distribution at all.

7. Fourier, power spectra, and whitening

This chapter has no lensing in it. It is entirely about signals: what a convolution is, what its cross-correlation cousin is and why the difference bites, and what a power spectrum has to do with a covariance matrix. None of that appears in a standard general-relativity or lensing course, and none of it is needed until Part V — but without it, Methods I: the correlated-noise likelihood of the campaign's final report is unreadable notation: $C = D^{1/2} K D^{1/2}$, a "stationary correlation kernel," an "operator-norm whiteness gate $e_{\mathrm{op}} \leq 0.02$," a "positive-semidefinite-by-construction" kernel family. Every one of those phrases is Fourier analysis wearing an astronomy costume. By the end of this chapter you can read `reproductions/claude-giga-lens/cgl/whiten.py` start to finish, and Ch. 24 can spend its entire budget on the physics instead of re-teaching this.

> **What you can skip**
>
> The Fast Fourier Transform itself — the $O(N \log N)$ divide-and-conquer algorithm — is a standard algorithms-course result and is not re-derived here; every `np.fft.fft2` in this chapter (and in the repo) is used as a black box, the same way you already use it. If $\hat{x}[k] = \sum_n x[n] e^{-2\pi i k n/N}$ is already loaded, skim past its reintroduction below. What is *not* assumed: any prior exposure to power spectra, autocorrelation, or whitening as *statistical* objects rather than signal-processing ones, or the specific fact that the "convolution" every deep-learning framework applies is not the flip-and-slide convolution of a signal-processing textbook.

Convolution vs. cross-correlation

Ch. 6 handed you a convolution without dwelling on the word: the lensing potential is $\psi = 2(\kappa * G)$ — the factor of 2 is $\nabla^2 \psi = 2\kappa$'s, not a new one — the convergence smeared out by the Green's function of the Laplacian. For finite sequences f, g of length N (indices taken mod N — "circular," to keep the algebra finite; the repo zero-pads when it needs a genuinely *linear* convolution instead, and says so explicitly where it matters),

$$(f * g)[n] \;=\; \sum_{m=0}^{N-1} f[m]\, g[(n-m) \bmod N] \qquad \text{convolution.}$$

Cross-correlation looks almost identical, but does not flip the second sequence before sliding it:

$$(f \star g)[n] \;=\; \sum_{m=0}^{N-1} f[m]\, g[(m+n) \bmod N] \qquad \text{cross-correlation.}$$

Define the time-reversal $\tilde{g}[m] = g[(-m) \bmod N]$. Substituting shows $(f \star g)[n] = (f * \tilde{g})[n]$ exactly: correlation *is* convolution, against a flipped kernel. The two operations coincide, for every f, precisely when g is symmetric under negation ($g[-m] = g[m]$, an *even* kernel) — then $\tilde{g} = g$ and the flip changes nothing.

This is not a pedantic distinction. Every deep-learning framework's `Conv2d` is, by this definition, cross-correlation: no kernel is flipped before the sliding dot-product. This repo's own conv-whitening code hits the identical fact head-on and documents it in its own docstring (`reproductions/claude-giga-lens/cgl/whiten.py:25–29` (source)):

> Convolution semantics: `jax.lax.conv_general_dilated` with `'SAME'` padding is CROSS-CORRELATION (no kernel flip)… Symmetric whitening kernels make the distinction moot; asymmetric callers must pass h in correlation orientation.

`make_conv_whitener` (`reproductions/claude-giga-lens/cgl/whiten.py:39`) is therefore locked against exactly this hazard: `reproductions/claude-giga-lens/tests/test_marg.py:83–96` builds a *deliberately asymmetric* random 3×3 kernel and checks the JAX operator against `scipy.ndimage.correlate` (not `scipy.ndimage.convolve`) to machine precision. The whitening kernel that `build_whitener` actually constructs sidesteps the whole question by construction — it is forced symmetric, `h = 0.5 * (h + h[::-1, ::-1])` (`reproductions/claude-giga-lens/cgl/whiten.py:145`) — but the generic operator underneath it is correlation, not convolution, and the test exists because a future asymmetric kernel would silently get the wrong filter applied if that fact were ever forgotten.

> **You already know this**
>
> "Convolution" in a CNN is cross-correlation, full stop — you have almost certainly used this fact without naming it. What you may not have used before is the flip side: correlation is the tool for asking "how much does this pattern resemble itself, offset by Δ?" — which is a *statistical* question, not a filtering one. That statistical question is this chapter's real subject, and it is where the next section starts.

Power spectra and autocorrelation: the Wiener–Khinchin theorem

A sequence is **(weakly) stationary** if its statistics don't depend on *where* you look: the correlation between two positions depends only on their separation, never on their location. Write $R[\Delta] = \mathrm{Cov}(x[n], x[n+\Delta])$ — by stationarity this does not depend on n — and $\rho[\Delta] = R[\Delta]/R[0]$ for its normalized version ($\rho[0] = 1$). (This ρ is a *correlation*, not a density — Ch. 3's $\rho \sim r^{-\gamma}$ is a different, unrelated use of the same letter. Neither this guide nor the repo's own papers ever let the two contexts overlap.)

> **You already know this**
>
> Weight-sharing in a convolutional layer *is* an assumption of stationarity: the same kernel applies at every spatial location because the network is told, by construction, that image statistics don't privilege one pixel over another. This chapter's "stationary correlation kernel $\rho(\Delta)$" is the identical assumption, written down as a noise model instead of a weight tensor.

The **discrete Fourier transform** of a length-N real sequence is $\hat{x}[k] = \sum_{n=0}^{N-1} x[n]\, e^{-2\pi i k n/N}$. You don't need complex analysis for what follows — $e^{i\phi} = \cos\phi + i\sin\phi$ is bookkeeping, and its conjugate simply flips the sign of ϕ, $\overline{e^{i\phi}} = e^{-i\phi}$. Every quantity that survives to matter in this chapter (a power spectrum, a correlation, a variance) is real.

The convolution theorem, derived, not quoted: substitute $n' = n - m$ inside the double sum,

$$\widehat{f * g}[k] = \sum_n \sum_m f[m]\, g[(n-m)]\, e^{-2\pi i k n/N} = \Big(\sum_m f[m] e^{-2\pi i k m/N}\Big)\Big(\sum_{n'} g[n'] e^{-2\pi i k n'/N}\Big) = \hat{f}[k]\,\hat{g}[k].$$

Convolution in the index domain is pointwise multiplication in frequency. Exercise 7.2 asks you

to run the identical substitution on cross-correlation; here, take $g = f$ directly — the case that matters — and substitute $p = m + n$ into $R[n] = (x \star x)[n] = \sum_m x[m]\, x[m+n]$:

$$\hat{R}[k] = \sum_n \sum_m x[m]x[m+n]\, e^{-2\pi i k n/N} = \Big(\sum_m x[m]e^{2\pi i k m/N}\Big)\Big(\sum_p x[p]e^{-2\pi i k p/N}\Big) = \overline{\hat{x}[k]}\,\hat{x}[k] = |\hat{x}[k]|^2 =: S[k].$$

(The first factor picked up a + sign in the exponent from re-indexing; because x is real, that sum is exactly $\overline{\hat{x}[k]}$.) Equation () is the **Wiener–Khinchin theorem**: the power spectral density (PSD) is the Fourier transform of the autocorrelation, and it is *automatically non-negative*, because it is manifestly a squared magnitude. (The general, measure-theoretic statement of the same fact — a function is a valid autocorrelation if and only if its Fourier transform is everywhere non-negative — is Bochner's theorem; what you just derived is its four-line finite special case, and it is all this chapter needs.)

Computing $R[\Delta]$ at every lag directly costs $O(N^2)$ — every pair of positions, once. Equation () turns that into two $O(N \log N)$ FFTs and one $O(N)$ pointwise multiply — the same zero-padding trick you may know from FFT-based long-integer or polynomial multiplication, here keeping a *linear* autocorrelation from wrapping around a *circular* grid. This is not a textbook aside; it is the literal content of `reproductions/foundry-i/46_noise_audit.py:89`,

```
acf = np.real(np.fft.ifft2(np.abs(np.fft.fft2(pad_n)) ** 2))
```

read right to left: FFT the (zero-padded) residual image, take the squared magnitude — the PSD, by () — inverse-FFT it back, and every lag of the autocorrelation falls out at once. An all-pairs statistic, computed without ever forming a pair.

Worked example. `reproductions/claude-giga-lens/data/noise_kernel_report.json` stores exactly this calculation's output: the measured, model-subtracted, along-axis autocorrelation of the fine (`v3`, 0.04″) drizzle product.

lag Δ	0	1	2	3	4	5
$\rho(\Delta)$, fine product	1.000	0.815	0.547	0.357	0.259	0.229

and it keeps decaying only slowly out to lag 11, the report's cutoff. A crude estimator of the **integrated autocorrelation time**, $\tau_{\text{int}} = 1 + 2\sum_{k=1}^{K} \rho(k)$, truncated at the $K = 11$ lags actually measured, gives

$\tau_{\text{int}} \approx 7.45$. For a 1-D stationary sequence, $N_{\text{eff}}/N = 1/\tau_{\text{int}} \approx 0.134$: along one axis, only about 13% of pixels carry independent information. Pixel noise correlates along *both* image axes; if they were independent of each other, a 2-D pixel's independent fraction would be the square, ≈ 0.018 — within about 6% of the campaign's own headline figure, $N_{\text{eff}}/N \approx 0.017$ (`reproductions/claude-giga-lens/papers/main.tex:213`), from a fuller, radially-averaged 2-D audit this guide does not re-run. The gap is what truncating at 11 lags of an undecayed tail, plus approximate separability, should cost — four lines of arithmetic on a published table, landing within a few percent of a number from a completely different, more careful route.

τ_{int} has another name you will meet in Ch. 23: for a Markov chain instead of a pixel grid, the identical sum over the chain's own autocorrelation is the *integrated autocorrelation time*, and N/τ_{int}

is the *effective sample size* every $\hat{R}$/ESS diagnostic in this repo's sampler benchmark reports. Same formula, same reason it exists: correlated draws — in time, or in space — carry less information than their raw count suggests.

Whitening

A **whitening** transform $u = Wx$ makes $\mathrm{Cov}(u) = I$: every whitened coordinate unit-variance, zero cross-correlation with every other.

> **You already know this**
>
> ZCA/PCA whitening does this by eigendecomposing a covariance $\Sigma = V\Lambda V^\top$ and setting $W = V\Lambda^{-1/2}V^\top$ — divide by the square root of each eigenvalue, along its own eigenvector. What follows is the identical operation; the only thing that changes is which eigenbasis is in play.

That eigenbasis is not a coincidence you have to look up — it falls straight out of (). Let $e_k[n] = e^{2\pi ikn/N}$ be the k-th Fourier mode, and consider "convolve by a fixed kernel ρ" as a linear operator. Direct substitution shows

$$(\rho * e_k)[n] = \sum_m \rho[m]\, e_k[n-m] = e_k[n] \sum_m \rho[m]\, e^{-2\pi ikm/N} = \hat{\rho}[k]\, e_k[n].$$

Every Fourier mode is an eigenvector of "convolve by ρ," with eigenvalue $\hat{\rho}[k] = S[k]$ — the same power spectrum () built. A stationary covariance matrix $K_{ij} = \rho[i-j]$ (this is exactly what "stationary" means as a matrix statement) is therefore diagonalized by the Fourier modes, with eigenvalues $S[k]$: Ch. 5's eigendecomposition of a symmetric matrix, generalized from 2×2 to $N \times N$, with the eigenvectors handed to you for free instead of solved for numerically. So "divide by $\sqrt{\text{eigenvalue}}$ along each eigenvector" becomes: divide by $\sqrt{S[k]}$ at each frequency, then transform back —

$$\hat{h}[k] = \frac{1}{\sqrt{S[k]}}, \qquad h = \mathrm{IFFT}(\hat{h}).$$

which is `reproductions/claude-giga-lens/cgl/whiten.py:141` verbatim (`h_full = np.real(np.fft.ifft2(1 / np.sqrt(S)))`), after S is floored — the next section's subject.

An *exact* global divide-by-$\sqrt{S}$ is not what the production code applies, though: the likelihood needs whitening to be a local, differentiable, mask-aware JAX operation (`make_conv_whitener`, `reproductions/claude-giga-lens/cgl/whiten.py:39–72`), and a single monolithic FFT over a masked, irregular image is not that. `build_whitener` instead truncates h to a small $(2M+1)^2$ stencil — a short local filter, refined by Gauss–Newton and Lawson–IRLS rounds to push its worst-frequency error down — so it can no longer whiten *every* frequency exactly, and the construction is accepted only if its worst-case departure from perfect whitening clears a gate (writing the spectrum as $S(\omega)$, a function of continuous frequency $\omega = 2\pi k/N$, rather than $S[k]$'s bin index — what a filter's response actually has to cover):

$$e_{\mathrm{op}} \;=\; \max_{\omega} \big|\, S(\omega)\, |\hat{h}_M(\omega)|^2 - 1 \,\big| \;\le\; 0.02.$$

$\hat{h}_M^2 = 1/S$ exactly would give $S\,\hat{h}_M^2 \equiv 1$ at every frequency; e_{op} measures the worst single frequency's failure of that identity, and `reproductions/claude-giga-lens/cgl/whiten.py:147–149`'s `e_op_of` computes exactly ().

Worked example. For the binned product (`v3b`, 0.08″ — the "money" product the Ch. 25 chain samples), the accepted whitener uses a stencil half-width $M = 10$ and achieves $e_{\text{op}} = 0.0124$, comfortably inside the 0.02 gate — at the cost of eroding 9,273 of the product's pixels (any pixel whose $(2M+1)^2$ stencil would touch a mask or the border cannot be whitened cleanly and is dropped, not down-weighted), leaving 16,653 kept.

Positive semi-definiteness

A symmetric matrix K is **positive semi-definite** (PSD) if $x^\top K x \geq 0$ for every vector x, equivalently if every eigenvalue is ≥ 0. This is not an optional nicety for a covariance or correlation matrix — it is the entire content of what makes it *valid*: a negative eigenvalue would mean some linear combination of pixels has negative variance, which is not a statement about noise, it is nonsense. Combined with the last section's fact — a stationary K's eigenvalues *are* the samples $S[k]$ of its power spectrum — positive-semi-definiteness translates exactly into $S(\omega) \geq 0$ at every frequency: Ch. 5's linear algebra and this chapter's Fourier analysis stating the identical requirement in two languages.

() already showed that the *true* PSD of a genuine autocorrelation is automatically non-negative — a squared magnitude by construction. The practical difficulty is that the repo's $\rho(\Delta)$ isn't handed down as one; it is *fit* to noisy, model-subtracted pixel residuals by least squares, and a generic multi-parameter fit carries no such guarantee. The pre-registered plan tried the simplest such family first — a single Gaussian — and failed a more basic bar before PSD ever came up: it could not even match the measured correlation closely enough (worst residual 0.311 against a 0.05 gate, on the fine product; `reproductions/claude-giga-lens/papers/main.tex:383–384`). Its replacement — the kernel family at `reproductions/claude-giga-lens/papers/main.tex:389–394`, rendered in full in Methods I — is built the other way around: a weighted sum of a delta (flat spectrum, $S \equiv 1$, trivially non-negative), a drizzle-anchored kernel convolved with a bivariate Gaussian (a Gaussian's Fourier transform is itself a positive Gaussian, so this term's spectrum is a *product* of two non-negative spectra), and a second Gaussian pedestal — three pieces, each individually guaranteed PSD, combined with non-negative weights. The paper's own name for this is exact: "the minimal positive-semidefinite-by-construction extension."

Guaranteed-PSD-in-principle is not the end of the story, because the *fitted*, finite kernel's numerically estimated spectrum can still come arbitrarily close to zero at some frequency — a real near-degeneracy, not a fitting artifact: an oversampled grid genuinely carries almost no independent information at some spatial frequencies. () divides by $\sqrt{S}$, so a near-zero value there blows the whitening filter up. The fix is a floor:

$$S \;\leftarrow\; \max\!\left(S_{\text{raw}},\; s_{\text{floor}} \cdot \overline{S_{\text{raw}}}\right),$$

`reproductions/claude-giga-lens/cgl/whiten.py:137–138`, with a module default $s_{\text{floor}} = 0.05$ — clip any frequency's power to at least 5% of the spectrum's own mean.

Worked example. The fine product's raw spectral minimum is only 5.27% of its mean — barely inside that default floor. The binned product's is 2.41% — already *below* it. Clamping `v3b` to the fixed 0.05 floor anyway, rather than adapting it downward, is exactly the

mistake the campaign made and retracted: a flat floor biased the Monte-Carlo whitened variance to $\mathrm{Var}(u) = 0.981$ (`reproductions/claude-giga-lens/papers/main.tex:428`, `reproductions/claude-giga-lens/CAMPAIGN.md:622–623`) and failed the whiteness gate outright. Letting the floor *adapt* down to $s_{\text{floor}} = 0.02$ for this product is what the adopted whitener actually uses, landing at $\mathrm{Var}(u) \approx 1.00003$ — inside the gate. This is the same move Ch. 8 will name explicitly — ridge regression is a Gaussian prior is a refusal to trust a small estimated eigenvalue — applied here to a spectrum instead of a design matrix's Gram matrix, at the identical price: a small, deliberate bias, bought in exchange for not dividing by a number that measurement noise alone might have made zero.

One more quantity from `build_whitener` is worth banking now and cashing later: its `logdet_per_pix` output is $\overline{\log S}$, the frequency-average of the log-spectrum (-0.973 for `v3b`). By the Szegő limit theorem this average equals $\lim_{n\to\infty} \log\det(K)/n$ — so this one FFT-derived number *is*, per pixel, the log det that Ch. 22's Occam term (`reproductions/claude-giga-lens/cgl/marg.py:19`) needs for a covariance built this way. This guide has been running a ledger of $\log|\det(\cdot)|$ showing up in unrelated costumes since Ch. 4; here it shows up as the *average of a Fourier transform*.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/whiten.py` is this entire chapter, executable: `make_conv_whitener` (line 39) is the local, differentiable convolution operator; `build_whitener` (line 95) is the spectral construction (), the adaptive floor () (lines 136–138), and the e_{op} gate () (`e_op_of`, lines 147–149).
- `reproductions/claude-giga-lens/tests/test_marg.py:83` locks the convolution-vs-correlation semantics against `scipy.ndimage.correlate` with a deliberately asymmetric kernel.
- `reproductions/foundry-i/46_noise_audit.py:89` is the Wiener–Khinchin theorem, one line, computing an all-pairs autocorrelation without forming a pair.
- `reproductions/claude-giga-lens/data/noise_kernel_report.json` and `reproductions/claude-giga-` hold this chapter's worked numbers — the same files `site/guide_src/worked_examples.py`'s `ch07_fourier_whitening` reads.
- Methods I and its Convolutional whitening subsection are the report prose this chapter exists to make readable.
- Ch. 11 explains *where* the correlation this chapter treats as given data comes from (the drizzle kernel); Ch. 24 is where $C = D^{1/2} K D^{1/2}$ becomes a full pixel likelihood.

Exercises

> **Exercise 7.1 — Why the whitening kernel is forced symmetric**
>
> Using the time-reversal identity $(f \star g)[n] = (f * \tilde{g})[n]$ from this chapter, prove that convolution and cross-correlation by a kernel g agree for *every* input f if and only if $g[-m] = g[m]$ (an even kernel). Then explain, in one sentence, why `build_whitener` bothers to symmetrize its taps (`reproductions/claude-giga-lens/cgl/whiten.py:145`) even though the operator that ultimately applies them (`make_conv_whitener`) implements correlation, not convolution.

Solution

$(f \star g)[n] = (f * \tilde{g})[n]$ for every f iff $\tilde{g} = g$ as sequences (convolving against two different kernels agrees on every input iff the kernels are identical — take f to be a unit impulse to see this directly). $\tilde{g} = g$ means $g[(-m) \bmod N] = g[m]$ for all m: exactly the evenness condition. Symmetrizing the taps means the distinction this chapter opened with stops mattering for *this* kernel — correlating and convolving by it produce the same result — so the code no longer has to reason about which orientation it is implicitly using; the test at `reproductions/claude-giga-lens/tests/test_marg.py:83` still exists to catch anyone who ever calls `make_conv_whitener` with an asymmetric kernel and forgets that the underlying operator is correlation.

Exercise 7.2 — The general correlation theorem

Section Power spectra and autocorrelation derived $\widehat{x \star x}[k] = |\hat{x}[k]|^2$ by substituting $p = m + n$ into the autocorrelation's own definition. Repeat that substitution for two *different* sequences, $(f \star g)[n] = \sum_m f[m] g[m+n]$, and show that $\widehat{f \star g}[k] = \overline{\hat{f}[k]}\, \hat{g}[k]$.

Solution

$\widehat{f \star g}[k] = \sum_n \sum_m f[m] g[m+n]\, e^{-2\pi i k n/N}$. Substitute $p = m + n$, so $n = p - m$: $\sum_m \sum_p f[m] g[p]\, e^{-2\pi i k (p-m)/N} = \left(\sum_m f[m] e^{2\pi i k m/N}\right)\left(\sum_p g[p] e^{-2\pi i k p/N}\right) = \overline{\hat{f}[k]}\, \hat{g}[k]$, using that f is real so conjugating its transform flips the exponent's sign. Setting $g = f$ recovers () exactly, since $\overline{\hat{f}}\hat{f} = |\hat{f}|^2$.

Exercise 7.3 — A toy spectrum, and why the fine product needs such an aggressive floor

Model the fine product's along-axis correlation as a first-order AR process, $\rho(\Delta) = r^{|\Delta|}$, with $r = 0.8147$ the measured lag-1 value . Its power spectrum is the geometric-series closed form $S(\omega) = (1 - r^2)/(1 - 2r\cos\omega + r^2)$. Evaluate $S(0)$ (DC) and $S(\pi)$ (Nyquist), and compute their ratio.

Solution

At $\omega = 0$: $S(0) = (1 - r^2)/(1 - r)^2 = (1 + r)/(1 - r) \approx 9.79$. At $\omega = \pi$: $S(\pi) = (1 - r^2)/(1 + r)^2 = (1 - r)/(1 + r) \approx 0.102$. The ratio is ≈ 96: the DC power outweighs the Nyquist power by nearly two orders of magnitude. A flat 5%-of-mean floor sits far more aggressively relative to the *low* end of a spectrum this lopsided than it would for a milder correlation — which is exactly why the fine product (the most correlated of the three, Ch. 11) is also the one whose whitener needed the largest stencil, $M = 20$, to hold e_{op} under gate.

Exercise 7.4 — Redo the N_{eff} estimate for the binned product

`reproductions/claude-giga-lens/data/noise_kernel_report.json`'s binned (`v3b`) measured along-axis autocorrelation, out to its own 7-lag cutoff, is $\rho(\Delta) = 1.000, 0.615, 0.268, 0.193, 0.172, 0.143, 0.107$ for $\Delta = 0, \ldots, 7$. Compute τ_{int} and N_{eff}/N (1-D, along one axis) exactly as this chapter did for the fine product, and explain in one sentence why the binned number should come out so much less extreme than the fine one.

Solution

$\tau_{\text{int}} = 1 + 2(0.615 + 0.268 + 0.193 + 0.172 + 0.143 + 0.107 + 0.080) \approx 4.16$, so $N_{\text{eff}}/N \approx 0.241$ — nearly twice the fine product's 0.134. A 2×2 rebinning of the fine mosaic averages together exactly the pixels that were most correlated with each other in the first place, so the correlation length measured in *binned* pixels shrinks even as the correlation length measured in *sky* angle does not: binning does not create information, but it does stop double-counting the information the fine grid never had.

8. Likelihood, posterior, evidence, and the nat

This chapter turns "fit a Gaussian to pixel residuals" into a full Bayesian machine: a likelihood, a prior, and an evidence integral whose logarithm is measured in the same unit — the nat — as a cross-entropy loss. Two moves make that machine useful for a real lensing posterior. The first is mostly a name change: an ordinary least-squares fit is already a Gaussian log-likelihood, and its familiar diagnostic, chi-squared, follows from it in three lines. The second is a genuinely different move for a CS reader: Bayesian model comparison replaces "lower loss wins" with a full-volume Bayes factor, and this repo's own money number was decided by exactly one such comparison — a 191-nat swing that flipped a scientific verdict. Getting that comparison analytically, without brute-force integrating a 46-dimensional posterior, needs the Laplace approximation: the same second-order Taylor idea from Chapter 2, applied to a log-posterior instead of a loss surface. By the end of this chapter you will have derived that approximation, watched it become *exact* on a toy problem shaped exactly like this repo's own marginalization, and seen precisely how it breaks — not approximately, but formally undefined — at the kind of point this repo's real posterior turns out to have.

> **What you can skip**
>
> Bayes' theorem itself needs no derivation from you: $p(\theta \mid D) \propto p(D \mid \theta)\, p(\theta)$ is the one-line update rule behind every Bayesian classifier you have built, and the Bayes section states it and moves on. That a Gaussian log-likelihood reduces to a sum of squared, weighted residuals is also not new — it is the reason mean-squared error is the default regression loss. What *is* new here is the astronomy-specific packaging of that fact (chi-squared, reduced chi-squared, "goodness of fit"), the convention that model evidence is always quoted in nats, and the Laplace approximation as a literal 42-line function in this repo's code, not a textbook aside.

Bayes

Every posterior in this repository is built from one identity, which follows from nothing but the definition of a conditional probability, $p(A, B) = p(A \mid B)\, p(B) = p(B \mid A)\, p(A)$, rearranged:

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{p(D)}.$$

Nothing in Equation () is astronomy-specific. θ is whatever this repo happens to be fitting — for the money number it is the 46-dimensional parameter vector of an EPL mass profile, an external shear, four Sérsic lens-light components and a marginalized source, worked out in Chapter 22. D is the pixel data, one HST cutout's worth of numbers. $p(D \mid \theta)$ is the likelihood — how probable the data are, for a candidate θ, under the noise model of Chapter 11. $p(\theta)$ is the prior: whatever you believed about θ before this particular image, expressed as a distribution — this campaign's own EPL-slope prior is a `TruncatedNormal(2.0, 0.25, low=1.0, high=2.7)` (`reproductions/claude-giga-lens/cgl/e2.py:110`), and you will lean on that exact prior in Chapter 25. And $p(D)$, the evidence, is the number this chapter spends most of its remaining time on.

Notice what is *not* in Equation (): a loss function, a gradient step, an optimizer. It is a statement about probability distributions, not an algorithm. Turning it into one — MAP estimation, HMC, SMC — is the content of Chapters 22 through 26; this chapter builds the piece all of them share: what the right-hand side actually *is*, term by term.

Chi-squared

Assume the per-pixel noise is Gaussian and independent — the diagonal likelihood of Chapter 11, before this repo's central complication (correlated noise, Chapter 24) enters. Write the residual at pixel i as $r_i = (d_i - m_i(\theta))/\sigma_i$. Each $r_i \sim \mathcal{N}(0,1)$, and

$$\log p(D \mid \theta) = -\frac{1}{2}\sum_i r_i^2 - \sum_i \log(\sigma_i\sqrt{2\pi}) = -\frac{1}{2}\chi^2 + \text{const}, \qquad \chi^2 \equiv \sum_i r_i^2.$$

> **You already know this**
>
> χ^2 is twice a negative log-likelihood — the same object as a weighted mean-squared-error loss with per-example weights $1/\sigma_i^2$. Minimizing χ^2 over θ *is* maximum-likelihood fitting; it is the identical arithmetic to minimizing an MSE loss, wearing a name inherited from 1900s statistics instead of 2010s deep learning.

For n independent unit-variance residuals fit with k linear parameters, $\mathbb{E}[\chi^2] = n - k$ (Exercise 8.1 asks you to show this). The diagnostic astronomers actually report is the *reduced* chi-squared, $\chi^2_\nu \equiv \chi^2/(n-k)$, which should sit near 1 for a correctly specified Gaussian model. Too far above 1 means the model or the noise is wrong. Too far *below* 1 means — also that the model or the noise is wrong, just in the other direction, which is the easier of the two to forget.

This repo forgot it, twice, in opposite directions, and both incidents are load-bearing enough that the code now refuses to run without the fix. Foundry-I's earliest native-scale HST fits reported a floor of $\chi^2_\nu = 3.4$ — a badly fitting model, or so it looked. The real cause was a rendering convention bug: `lenstronomy`'s `subgrid_kernel` upsamples a PSF kernel internally by the supersampling factor, and the PSF handed to it had *already* been supersampled, so every native-scale fit was convolved with an effective PSF twice as broad as the true one. Fixing only that — same data, same noise model, same code otherwise — dropped χ^2_ν to 1.05 , a 3.2× improvement from correcting a convolution kernel's sampling convention, not the physics (`reproductions/foundry-i/README.md:19-36`; the guard that now refuses to let this recur is `reproductions/claude-giga-lens/cgl/guards.py:74-91`).

The opposite failure is subtler, because it looks like success. An early sky-noise calibration reported $\chi^2_\nu = 0.451$ — a suspiciously excellent fit, and "suspicious" turned out to be the correct reaction. The noise σ for that fit had been calibrated on *raw* image pixels, roughly 70% of which were diffuse lens-light flux, not sky background; an inflated σ shrinks every r_i, and therefore χ^2, exactly as it should when you divide by too large a number. Recalibrating σ on model-subtracted residuals — pixels with the lens light actually removed — gave the honest value, $\chi^2_\nu = 0.92$, a correction of 2.04× in the *other* direction (`reproductions/foundry-i/README.md:19-27`; guarded at `reproductions/claude-giga-lens/cgl/guards.py:129-142`), which is why the guard docstring still calls the retracted number "celebrated."

The lesson both incidents teach is the same one: χ^2_ν is a diagnostic *relative to the noise model you fed it*, not an absolute truth detector, and it can mislead in either direction.

Evidence and nats

Fitting one model to data answers "what parameters?" Comparing two models — or, as in this repo, two disconnected modes of the *same* posterior (a source can sit behind either a steep or a

shallow density slope with almost equally good pixel fits, Chapter 21) — needs a different question: which explanation, integrated over *all* of its parameters, accounts for more of the data's probability? That integral is the evidence,

$$Z(D) \equiv p(D) = \int p(D \mid \theta)\, p(\theta)\, d\theta,$$

and comparing two candidates H_1, H_2 with a Bayes factor $K = Z_1/Z_2$ is the Bayesian generalization of "lower loss wins." Because Z is a probability, this repo always reports $\log Z$, and always in **nats** — natural-log units, never $\log_{10}$ (dex) and never $\log_2$ (bits). That convention is not a stylistic accident: it is the same logarithm, and the same unit, as a cross-entropy loss.

> **You already know this**
>
> A cross-entropy loss summed over a dataset, $-\sum_i \log q(x_i)$, is already measured in nats whenever your framework's `log` is the natural log — PyTorch's and JAX's both are. $\log Z$ is the identical object applied to a whole model instead of one example, and the two are additive for the identical reason: $\log \prod_i p(x_i) = \sum_i \log p(x_i)$.

Because everything is additive in log-space, a *difference* in evidence is a sum of nats, and that sum is exactly this repo's headline result. Per-basin tempered-SMC evidence on the money-number posterior gives $\Delta \log Z = \log Z_{\text{steep}} - \log Z_{\text{low}} = +162.2$ nats under the diagonal likelihood (favouring the steep basin overwhelmingly), and -28.9 nats under the correlated likelihood of Chapter 24 (favouring the low basin instead) — a swing of 191.1 nats . Exponentiated, that swing is a probability-ratio factor of roughly 10^{83} : not a nudge, an inversion of which basin the data prefer. What "steep," "low," and "basin" mean, and why this flip is the campaign's central finding rather than a bug, is Chapter 25 in full. The only point here is what a few hundred nats of anything *mean* as a number — and that it is the same currency you already spend every time you report a training loss.

Laplace

When the evidence integral in Equation () cannot be done in closed form over every dimension — which is every real case in this repo except the linear amplitudes of the next section — one option is Monte Carlo (Chapter 23). A cheaper alternative, when it applies, borrows Chapter 2's Taylor expansion: fit the *shape* of the log-posterior near its peak, and integrate over that shape instead of the true one.

Let $\hat{\theta}$ be the mode (the MAP point), and expand $-\log p(\theta \mid D)$ to second order about it, exactly as in Chapter 2:

$$-\log p(\theta \mid D) \approx -\log p(\hat{\theta} \mid D) + \frac{1}{2}(\theta - \hat{\theta})^\top H (\theta - \hat{\theta}), \qquad H \equiv -\nabla\nabla \log p(\theta \mid D)\Big|_{\hat{\theta}}.$$

H is a Hessian — the Chapter 5 object again, and its eigendecomposition decides everything that follows. H is symmetric, so its eigenvalues are real, and if — and only if — every one is positive, H is positive-definite and $\hat{\theta}$ is a genuine local minimum of $-\log p$ (a mode, not a saddle). Approximating the un-normalized posterior near $\hat{\theta}$ as a multivariate Gaussian with covariance H^{-1}, and integrating that Gaussian exactly, gives the **Laplace approximation** to the evidence:

$$\log Z \approx \underbrace{\log p(D \mid \hat{\theta}) + \log p(\hat{\theta})}_{\text{best fit}} + \underbrace{\frac{d}{2}\log(2\pi) - \frac{1}{2}\log\det H}_{\text{Occam factor}}.$$

The first pair of terms is simply how good the best fit is. The second pair is new: it is a *volume*, not a fit quality — how much room the posterior has around $\hat{\theta}$, measured through a determinant, again. The Log-Det Ledger's row 1 was an area-scaling factor; row 2 was a normalizing flow's density correction; this is row 3, and Chapter 23 closes the ledger with all three side by side. A *larger* $\det H$ — sharper curvature, a model that must have its parameters tuned to a fine tolerance to fit at all — means a *smaller* Occam factor: a fussy model is penalized relative to a forgiving one that fits about as well without needing to be finely tuned. That is Occam's razor as arithmetic, not metaphor.

This repo runs Equation () verbatim. `reproductions/claude-giga-lens/cgl/e2.py:564`, inside `laplace_evidence()`, computes `log_ev = logp_map + 0.5 * ndim * np.log(2 * np.pi) - 0.5 * logdet_H` from the actual Hessian of the correlated log-posterior at its polished MAP. The same function's docstring states, in one sentence, exactly the failure mode Equation () predicts: `log_det H` needs every eigenvalue of H positive, and when H is "INDEFINITE… the log-evidence uses the floored-positive spectrum (a saddle evidence is only a rough basin-mass proxy; caveated)" (`reproductions/claude-giga-lens/cgl/e2.py:538-539`). A saddle has at least one *negative* eigenvalue — $\hat{\theta}$ is not a minimum of $-\log p$ in every direction, so the local Gaussian the whole derivation rests on does not exist there. The formula does not merely become inaccurate; it becomes undefined, since log of a non-positive number has no real value. The code's response is to floor every eigenvalue at a small positive constant before taking its log, trading a crash for a number honestly labeled a "rough proxy," not an evidence. Whether this repo's own money-number MAP actually *is* such a saddle — and what breaks, and what still works, when it is — is Chapter 26 in full.

You do not have to take the exactness of Equation () on faith. Whenever the log-posterior *is* exactly quadratic — a linear model with a Gaussian likelihood and a Gaussian prior, which is precisely what this repo's marginalized shapelet source amplitudes are (Chapter 22) — the Laplace approximation is not an approximation at all; it is the exact integral. Take the smallest possible instance of `reproductions/claude-giga-lens/cgl/marg.py`'s algebra: five points (x_i, y_i), one linear amplitude a with $y = ax$, ridge prior $a \sim \mathcal{N}(0, 1)$. `marg.py`'s formula gives — constants dropped, exactly as its own docstring says — a closed-form $\log L$ built only from $b = x^\top y$, $A = x^\top x + 1$, and $\log\det A = \log A = 4.025$. Restore the dropped normalization ($\frac{1}{2}\log 2\pi$ — the same additive constant the campaign's own report tracks explicitly as "+ const", `reproductions/claude-giga-lens/papers/main.tex:471`) and you get the full evidence, $\log Z = -3.183$. Now do it the hard way: integrate the un-normalized posterior over a *numerically*, on a fine grid, with no closed form assumed anywhere. The brute-force integral gives $\log Z = -3.183$, and the two agree not merely to three decimal places but to 6×10^{-15} — floating-point noise, nothing else. `marg.py` is not approximating an integral that would be too expensive to compute directly; for these particular (linear) parameters, it *is* the integral.

Ridge is a prior

Run the same Gaussian-times-Gaussian argument forward, instead of integrating it, and you get a familiar training-time identity. For a linear model $y = X\theta + \text{noise}$ (noise already whitened to unit variance) with an independent Gaussian prior $\theta \sim \mathcal{N}(0, \Lambda^{-1})$ per parameter, the log-posterior is

$$\log p(\theta \mid D) = -\frac{1}{2}\|y - X\theta\|^2 - \frac{1}{2}\theta^\top \Lambda \theta + \text{const},$$

and setting its gradient to zero gives

$$(X^\top X + \Lambda)\,\hat{\theta} = X^\top y.$$

> **You already know this**
>
> Equation () is ridge regression's normal equation, and Λ is the L2 penalty your optimizer calls `weight_decay`. A Gaussian prior on a parameter with precision $\lambda = 1/\sigma^2_{\text{prior}}$, and an L2 regularizer with coefficient λ, are not merely analogous — the MAP estimate under the first is the exact minimizer of the second.

This is the entire content of `reproductions/claude-giga-lens/cgl/marg.py`'s 55 lines (Chapter 22 runs it at full scale, marginalizing dozens of shapelet source amplitudes analytically instead of sampling them): $b = X_w^\top R_w$, $A = X_w^\top X_w + \Lambda$, $\hat{a} = A^{-1}b$ by a Cholesky solve — never a matrix inverse, recalling Chapter 5's conditioning warning: an ill-conditioned A does not forgive `np.linalg.inv`. And $\log\det A$ is the *same* Occam term as Equation ()'s $\log\det H$, because for these particular (linear) parameters the log-posterior genuinely is the exact quadratic Chapter 2 promised, and A *is* that quadratic's Hessian. This repo's own prior precision, `Lambda_ii = (i+1)/25` (`reproductions/claude-giga-lens/cgl/marg.py:38-39`), grows with shapelet order i — a smoothness prior, penalizing higher (wigglier, noisier) basis functions harder, the direct analogue of preferring a smaller network over a larger one at equal training loss.

The Occam term is audited, not merely derived. This repo's parity harness checks $-\frac{1}{2}\log\det A$ against `numpy.linalg.slogdet` at a stored validation point (`map_marg_pd.npz`) and finds agreement to 10^{-10}, with $\log\det A = 323.229$ and $\text{cond}(A) \approx 1.4\times10^4$ (`reproductions/claude-giga-lens/papers/mai` Table `tab:parity`, Gate E). `gigalens`'s own linear-amplitude solver omits this term entirely — it runs a plain least-squares solve and reports the best-fit likelihood with no Occam correction at all, which is the reason this campaign's evidence numbers (the previous section) exist as a contribution rather than a rerun.

> **Log-Det Ledger — row 3**
>
> A determinant appears a third time, in a third costume. Row 1 (Ch. 4) was the area-scaling factor of a change of variables; row 2 was a normalizing flow's $\log|\det J|$; row 3 is $-\frac{1}{2}\log\det A$, the Occam factor of a Gaussian evidence — the same object, up to a factor of $-\frac{1}{2}$, as the Laplace Hessian's $\log\det H$ above. Chapter 23 closes the ledger and puts all three side by side; Chapter 22 runs this exact term in production, on the real correlated-likelihood posterior.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/marg.py` (55 lines total) — the whole chapter's ridge/Occam algebra, compressed. Every line of `marg_loglik` (lines 31-55) is one term in Equations () and (); its module docstring is quoted almost verbatim above.
- `reproductions/claude-giga-lens/cgl/e2.py:531-572` — `laplace_evidence()`, the Laplace approximation as running code, floored-eigenvalue caveat included.

- `reproductions/claude-giga-lens/cgl/guards.py:74-142` — the two chi-squared retractions, encoded as literal, permanent guard functions: this repo's institutional memory refuses to let either bug recur silently.
- `reproductions/claude-giga-lens/papers/main.tex` — Table `tab:parity` (Gate E) carries the audited Occam-term number this chapter uses.
- `reproductions/foundry-i/README.md:19-42` — the PSF-convention fix and the sky-calibration retraction, in the campaign's own words.

Exercises

Exercise 8.1 — Why $\mathbb{E}[\chi^2] = n - k$

For n independent, unit-variance normal residuals fit by ordinary least squares against k linear parameters, show that $\mathbb{E}[\chi^2] = n - k$ rather than n.

Solution

Each individual $r_i \sim \mathcal{N}(0, 1)$ has $\mathbb{E}[r_i^2] = 1$, so a *raw* residual vector (no fitting) would give $\mathbb{E}[\chi^2] = n$. But least-squares fitting chooses the k parameters that make the residual vector orthogonal to the k-dimensional column space of the design matrix X — that orthogonality is the normal equation's geometric content. The residual is therefore confined to an $(n-k)$-dimensional subspace, and the sum of squares of a standard normal vector restricted to a $(n-k)$-dimensional subspace has expectation $n-k$, not n. Every fitted parameter "uses up" one degree of freedom's worth of expected χ^2.

Exercise 8.2 — Ridge from the posterior

Starting from the log-posterior $\log p(\theta \mid D) = -\frac{1}{2}\|y - X\theta\|^2 - \frac{1}{2}\theta^\top \Lambda \theta + \text{const}$, differentiate with respect to θ and set the gradient to zero to derive Equation ().

Solution

$\nabla_\theta[-\frac{1}{2}\|y - X\theta\|^2] = X^\top(y - X\theta)$, and $\nabla_\theta[-\frac{1}{2}\theta^\top\Lambda\theta] = -\Lambda\theta$ (using that Λ is symmetric — it is diagonal here). Summing and setting to zero: $X^\top y - X^\top X\theta - \Lambda\theta = 0$, i.e. $(X^\top X + \Lambda)\theta = X^\top y$, exactly Equation (). Set $\Lambda = 0$ and you recover the ordinary least-squares normal equation; a Gaussian prior is what turns $X^\top X$, which can be singular or ill-conditioned, into $X^\top X + \Lambda$, which — for any $\Lambda \succ 0$ — is provably invertible. Regularization as numerical hygiene and regularization as a Bayesian belief are the same Λ.

Exercise 8.3 — What omitting the Occam term costs you

Using this chapter's toy numbers (five points, one ridge-regularized amplitude a), compute what `marg.py`'s $\log L$ would have been *without* the Occam term — i.e. the plain best-fit likelihood $-\frac{1}{2}\|y - X\hat{a}\|^2$ that `gigalens`'s own least-squares path reports. How many nats does dropping the Occam term buy, and does that number depend on how tightly the data constrain a?

Solution

Dropping $-\frac{1}{2}\log\det A$ from `marg.py`'s formula gives $\log L_{\text{no Occam}} = -2.089$, compared with the correct $\log L_{\text{marg}} = -4.102$ — a gap of exactly $\frac{1}{2}\log\det A = 2.013$ nats , precisely half the Occam term this chapter has been tracking. Omitting the term always makes the reported likelihood *larger* (less negative), by an amount that grows with $\log\det A$ — which grows both with how many parameters are marginalized and with how tightly the data constrain them. A model with more free amplitudes, or amplitudes pinned down more tightly, collects a *bigger* uncorrected bonus for exactly the flexibility Occam's razor exists to penalize: the mechanism by which an evidence comparison without this term systematically favors more flexible models.

Exercise 8.4 — Why flooring an eigenvalue is not a fix

`cgl/e2.py`'s `laplace_evidence()` floors any Hessian eigenvalue below `eig_floor` before computing $\log\det H$. Explain in one sentence why this makes the function *not crash*, and in a second sentence why the number it then returns is not a real Bayesian evidence.

Solution

Flooring replaces every non-positive eigenvalue with a small positive constant, so every factor inside the log is positive and $\log\det H$ is a well-defined real number — the crash (a log of zero or a negative number) is avoided by construction. But the Laplace derivation assumed a genuine local Gaussian approximation around a *minimum* of $-\log p$; at a saddle, the true posterior surface curves *away* from $\widehat{\theta}$ in the negative-eigenvalue directions rather than toward it, so no Gaussian with a positive-definite covariance actually approximates the posterior there. The floored number is a well-defined arithmetic quantity, and a useful rough proxy for how much probability mass sits in that basin — but it is not the integral Equation () derived, because the assumption the derivation needed (an everywhere-positive-definite H) is false at the point being evaluated.

9. Arcseconds, magnitudes, and the units of the sky

Every number this guide computes from here on is quoted in one of four units: an angle in arcsec, a brightness in magnitudes, a surface brightness in magnitudes-per-square-arcsec (or counts per pixel), and a pixel scale in arcsec/pixel. None of the physics in Part IV needs anything harder than arithmetic once you have these pinned down — the hard part is that all four are unfamiliar conventions wearing familiar-looking symbols. This chapter buys you the ability to read a figure caption, a FITS header, or a line of this repo's code (`pixscale=0.262`, `m_z < 20`, `delta_pix=0.04`) and know exactly what physical statement it is making. Part III (cosmology) adds one more unit, the megaparsec, once distances stop being purely angular.

> **What you can skip**
>
> You already know what a logarithmic unit is — decibels, pH, the bits and nats you already use for cross-entropy. Skip any explanation of "why take a log of a wide-dynamic-range quantity." What is new here is not the concept but the specific, historically-frozen conventions: which base, which sign, which zero point, and why gravitational lensing cares about the difference between a flux and a surface brightness. If you are comfortable with the small-angle approximation from Ch. 2 (that $\sin\theta \approx \theta$ to first order), skip straight to the parsec derivation.

Angles on the sky

Nothing astronomical comes with a ruler. Every telescope measures a direction, not a distance, so the natural unit for anything on the sky is an angle: degrees, and for anything as small as a galaxy or a lensed arc, the much finer **arcsecond** ($1^\circ = 60' = 3600''$).

The reason arcseconds show up multiplied by 206265 everywhere is nothing more than the Taylor expansion you already derived in Ch. 2. One radian is, by definition, the angle whose arc length equals its radius, so converting radians to arcseconds is a matter of unit bookkeeping:

$$1 \text{ rad} = \frac{180}{\pi} \times 3600'' \approx 206264.8''.$$

This exact constant lives in the repo's own numerics as `ARCSEC_PER_RAD` — see `site/guide_src/lensing.py:25` (source) — because every deflection angle, Einstein radius, and image position in this repo's lens equation is carried in arcsec, never radians, and the conversion factor has to come from somewhere concrete.

The reason 206265 is worth deriving rather than memorizing: it is also a license to stop worrying about the small-angle approximation everywhere else in this guide. At $\theta = 1''$ (already large by lensing standards — most Einstein radii are 1–2″), the gap between $\sin\theta$ and θ itself is

$$\frac{|\sin\theta - \theta|}{\theta} \approx \frac{\theta^2}{6} \approx 3.9 \times 10^{-12},$$

twelve orders of magnitude smaller than the angle itself. This is why the lens equation in Ch. 17, $\beta = \theta - \alpha(\theta)$, is written as ordinary vector subtraction in a flat 2-D plane with no trigonometry in sight: at arcsecond scales, the curved sky and the flat tangent plane are the same surface to twelve decimal places.

The same approximation pins down the **parsec**, the distance unit every paper you will ever read (including this repo's own cosmology chapters, once you convert into physical Mpc) actually uses. A parsec is *defined* as the distance at which one astronomical unit — Earth's orbital radius, roughly 1.496×10^8 km — subtends an angle of exactly $1''$. Since $1''$ is minuscule, $\tan\theta \approx \theta$ applies immediately and the definition becomes pure division:

$$1\,\mathrm{pc} = \frac{1\,\mathrm{AU}}{\theta(1'')} = \frac{1.496 \times 10^8\,\mathrm{km}}{4.848 \times 10^{-6}} \approx 3.086 \times 10^{13}\,\mathrm{km} \approx 3.26\,\text{light-years}.$$

Deriving it this way — from one triangle and the same first-order Taylor term already in hand — reproduces astropy's own built-in parsec constant exactly, to the last bit of double-precision floating point.

Notice what this repo does *not* do with parsecs: the lens-modeling likelihood at the center of Parts IV–V never leaves arcsec (see `site/guide_src/cosmo.py:1–20` (source) — cosmology enters in exactly three places, and the fitting itself is "arcsec in, arcsec out"). Mpc reappears only once you convert an Einstein radius into a physical mass, which is Ch. 15's job. You need the parsec to read an abstract, not to run this repo's own fits.

Magnitudes

> **You already know this**
>
> A magnitude is a log-likelihood with the sign and the base changed. Sky surveys record fluxes spanning ten or more orders of magnitude in a single image — a nearby star and a faint background galaxy on the same CCD — exactly the dynamic-range problem that makes you compute a log-likelihood instead of a likelihood. Astronomy solved the same problem a century earlier, kept the sign backwards for historical reasons, and called the result a magnitude.

Flux is what a detector actually integrates: energy per unit time per unit collecting area. The **magnitude** system converts flux into a logarithmic, and famously *backwards*, scale: brighter objects get *smaller* (even negative) magnitude numbers. The definition, for two fluxes F_1, F_2:

$$m_1 - m_2 = -2.5 \log_{10}\left(\frac{F_1}{F_2}\right).$$

The -2.5 is Norman Pogson's 1856 fix to make the scale consistent with Hipparchus's ancient by-eye ranking of stars from 1 (brightest) to 6 (faintest): Pogson set five magnitudes to mean exactly a factor of 100 in flux, which is where $2.5 = 5/\log_{10}(100)$ comes from. The minus sign is the price of keeping "1st magnitude" attached to the brightest stars.

Modern digital surveys drop the "compare to a reference star" step and fix an absolute zero point instead. This repo's own imaging — the DESI Legacy Survey cutouts fetched throughout `reproductions/aion-1/` — uses the "nanomaggie" flux unit with zero point 22.5, so a single flux measurement converts to a magnitude with no reference object at all:

$$m = 22.5 - 2.5 \log_{10}(f_{\mathrm{nmgy}}).$$

This is exactly the formula at `reproductions/aion-1/03_fetch_provabgs.py:65–67` (source): `flux = 10.0 ** ((22.5 - mags) / 2.5)`. You can invert this yourself on a number this repo actually uses. The DR11-south discovery sweep's own faint-end quality cut is $m_z < 20$ in the z-band (`reproductions/dr11-campaign/papers/main.tex:94` (source)). Plugging $m = 20$ into the zero-point formula:

$$f = 10^{(22.5-20)/2.5} = 10^1 = 10 \text{ nanomaggies exactly.}$$

That the cut lands on a clean power of ten is not a coincidence you need to explain — it is Pogson's 2.5 doing exactly what it was built to do: two magnitudes of difference from the zero point is one clean order of magnitude in flux. (For the same reason, a source at 100 nanomaggies sits at $m = 22.5 - 2.5\log_{10}(100) = 17.5$.)

Surface brightness

Flux is a *total*: integrate a light distribution over the solid angle an object occupies and you get a flux. **Surface brightness** I is the integrand — flux per unit solid angle, e.g. magnitudes per square arcsec, or (as this repo's rendering code actually stores it) linear counts per pixel. The distinction matters here for one reason: gravitational lensing conserves surface brightness and only ever moves flux around by changing solid angle.

This is a restatement of Ch. 18's magnification in a new unit. Lensing bends light rays; it does not create or destroy photons, so the specific intensity carried along any single ray is unchanged: $I(\beta) = I(\theta)$, point for point, source to image. What magnification $\mu = 1/\det A$ changes is how much *solid angle* on the sky that unchanged intensity ends up covering — a lensed image looks brighter in total flux only because it is bigger, never because each patch of it got more photons per unit area than its unlensed source patch had. The repo's forward model renders every source (the Sersic profiles of Ch. 10, `site/guide_src/lensing.py:162–164` (source)) as exactly this kind of surface-brightness field, ray-traces it through the lens equation, and only afterward multiplies by pixel area to get the counts a detector would actually record.

That multiplication is worth naming explicitly, because it is another det-J moment for the Log-Det Ledger: converting a continuous surface brightness (flux per solid angle) into flux per discrete pixel is a change of variables, and its Jacobian determinant is the pixel's own area. At the campaign's finest pixel scale, `delta_pix` $= 0.04''$, that area is

$$\det(\texttt{delta\_pix} \cdot I) = (0.04'')^2 = 0.0016 \text{ arcsec}^2,$$

which is the literal line `conversion_factor = float(cfg["delta_pix"]) ** 2  # det(delta_pix*I)` at `reproductions/claude-giga-lens/cgl/e2.py:455` (source). The comment in the repo's own source names it a determinant because it is one — the same idea Ch. 4 introduces for magnification and Ch. 23 closes out for the flow and Occam log-dets.

The repo also tests the conservation directly rather than trusting it: the regression `test_mock_pipeline_matches_` in `reproductions/claude-giga-lens/tests/test_drizzle.py:198–201` (source) feeds the drizzle resampling operator a spatially uniform mock scene and asserts the output stays uniform at the same value — a resampling step that is not allowed to manufacture or destroy surface brightness, checked the way you would check that a change-of-variables in a normalizing flow leaves total probability mass at 1.

Pixel scale

Pixel scale is how many arcseconds one detector pixel spans — a fixed property of the telescope's optics, set once at design time. It is a different number from **seeing**, the width of the point-spread function (PSF) imposed by the atmosphere (or, for a space telescope, by diffraction), which varies exposure to exposure. Confusing the two is the single most common unit mistake in reading a lensing figure.

> **You already know this**
>
> The relationship between pixel scale and seeing is the Nyquist–Shannon sampling theorem, applied to a spatial signal instead of a time series. The PSF is the highest-frequency structure in the image; to resolve it without aliasing you need at least two samples (pixels) across its width. Fewer, and you are undersampled — real structure gets folded onto itself. Many more, and you are oversampled: extra pixels that do not carry extra independent information, only extra correlated noise. Ch. 11 turns that exact fact into the reason a naive diagonal likelihood is wrong on resampled data.

The DESI Legacy Survey — the discovery half of this repo's science — has a native pixel scale of $0.262''$/pixel (the `pixscale=0.262` default at `reproductions/aion-1/_ls_cutout.py:36` (source)). Its own g-band coadd PSF, measured directly rather than assumed, has FWHM $\approx 1.35''$ (`reproductions/cikota-2023/papers/main.tex:270` (source)). That is

$$\frac{1.35''}{0.262''/\mathrm{px}} \approx 5.15 \text{ pixels across the FWHM},$$

comfortably above the Nyquist floor of two: the survey is not undersampled by its own pixel grid. But 5.15 pixels of blur is not the same statement as "5.15 pixels of resolution" — the seeing disk itself, $1.35''$, is comparable to or larger than a typical Einstein radius. Sampling the blur finely does not un-blur it: `reproductions/cikota-2023`'s own PSF ablation shows that swapping in the source paper's sharper $0.6''$ PSF only closes about half the Einstein-radius discrepancy, because "the $1.35''$ Legacy seeing is baked into the pixels and cannot be deconvolved beyond what the survey resolved" (`reproductions/cikota-2023/README.md:41–43` (source)). Ch. 27 turns this exact seeing-vs-Einstein-radius comparison into the resolution wall that bounds automated lens discovery, and Ch. 28 shows it is the same wall that makes human labels noisy.

When this repo fits a lens model, it resamples the survey's own pixels onto finer synthetic grids so the model has room to render sub-pixel structure. The P1c money-number chain (Ch. 25) uses three such products, defined at `reproductions/claude-giga-lens/cgl/e2.py:55–59` (source):

product	pixel scale	grid	oversampling vs. the $0.262''$ survey pixel
fine	$0.04''$	260^2	$6.55\times$
binned	$0.08''$	130^2	$3.28\times$
native	$0.13''$	80^2	$2.02\times$

Finer pixels here are not finer *information* — they are the same photons, interpolated onto a denser grid, which is precisely why the fine product turns out to be the most internally correlated of the

three. That consequence, quantified, is Ch. 11's job; the point to take from this chapter is narrower: pixel scale is a unit conversion (arcsec per pixel), seeing is a physical measurement (arcsec FWHM), and no amount of resampling moves information between them.

Connect to the repo

- `site/guide_src/lensing.py:25` (source) — `ARCSEC_PER_RAD`, the constant this chapter derives from a Taylor expansion.
- `site/guide_src/cosmo.py:1–20` (source) — why this repo's lens-modeling likelihood never uses a parsec or an Mpc.
- `reproductions/aion-1/03_fetch_provabgs.py:65–67` (source) — the Pogson magnitude-to-flux conversion this repo actually runs.
- `reproductions/dr11-campaign/papers/main.tex:94` (source) — the $m_z < 20$ cut used to build the 53.8M-galaxy discovery parent sample.
- `reproductions/claude-giga-lens/tests/test_drizzle.py:198–201` (source) — the regression test that pins surface-brightness conservation under resampling.
- `reproductions/claude-giga-lens/cgl/e2.py:55–59` and `:455` (source) — the three drizzle pixel scales behind the money number, and the pixel-area Jacobian that converts surface brightness into counts.
- `reproductions/cikota-2023/papers/main.tex:270` and `README.md:38–43` (source) — the measured 1.35″ Legacy seeing and the PSF ablation showing it cannot be deconvolved away.
- Every number above reproduces with `~/.venvs/lensjudge/bin/python site/guide_src/worked_example --show ch09`.

Exercises

Exercise 9.1 — the parsec, from one triangle

Derive 1 pc in kilometers using only 1 AU $\approx 1.496 \times 10^8$ km and the small-angle approximation — no other astronomical constant allowed. Then explain in one sentence why the answer does not depend on which galaxy, star, or lens you are observing.

Solution

By definition, a parsec is the distance d at which 1 AU subtends 1″. Geometrically, $\tan(1'') = \mathrm{AU}/d$; because 1″ in radians is $\theta = 1/206264.8 \approx 4.848 \times 10^{-6}$, which is tiny, $\tan\theta \approx \theta$ to twelve decimal places (Ch. 2's Taylor expansion, quantified in this chapter). So

$$d = \frac{\mathrm{AU}}{\theta} = \frac{1.496 \times 10^8\,\mathrm{km}}{4.848 \times 10^{-6}} \approx 3.086 \times 10^{13}\,\mathrm{km},$$

which matches astropy's own `u.pc` exactly. It is purely a statement about the geometry of one right triangle (an AU and a 1″ angle) — nothing about the object being observed enters, which is exactly why it is usable as a universal distance unit rather than an object-specific one.

Exercise 9.2 — inverting Pogson's law

The DESI Legacy Survey's zero point is 22.5 (flux in nanomaggies). What magnitude corresponds to 100 nanomaggies? What flux corresponds to the DR11-south sweep's own faint cut, $m_z < 20$? Why does the second answer come out to a clean number without any special pleading?

Solution

Inverting $m = 22.5 - 2.5\log_{10} f$ for $f = 100$: $m = 22.5 - 2.5\log_{10}(100) = 22.5 - 5 = 17.5$. For $m = 20$: $f = 10^{(22.5-20)/2.5} = 10^1 = 10$ nanomaggies exactly. The cleanness is not a coincidence to explain away — Pogson fixed 2.5 so that every 2.5 magnitudes is exactly one power of ten in flux, by construction. Any cut that lands a whole multiple of 2.5 magnitudes from the zero point will always be a round number in flux.

Exercise 9.3 — is the survey oversampled or under-resolved?

DESI Legacy's native pixel scale is $0.262''$ and its measured g-band seeing is $1.35''$ FWHM. Compute pixels-per-FWHM and say whether the survey is Nyquist-sampled. Then explain why that answer alone does not tell you whether a $1.2''$-Einstein-radius lens will be *resolved*.

Solution

Pixels across the FWHM: $1.35/0.262 \approx 5.15$. Nyquist only requires ≥ 2 samples across the highest-frequency feature, so the survey's own pixel grid comfortably resolves its own PSF — it is not pixel-undersampled. But Nyquist sampling of the PSF is a statement about not losing information *the PSF already contains*; it says nothing about whether the PSF itself is narrow enough to separate a $1.2''$ ring from its $1.35''$-wide blur disk. Those are two different comparisons — pixel scale vs. PSF width, and PSF width vs. Einstein radius — and only the second one is the wall Ch. 27 derives.

Exercise 9.4 — the pixel-area Jacobian

The campaign's "fine" drizzle product has `delta_pix = 0.04`. Compute the pixel area in square arcsec, and explain in one sentence why `reproductions/claude-giga-lens/cgl/e2.py:455` labels this quantity a determinant rather than only an area.

Solution

Area $= (0.04'')^2 = 0.0016$ arcsec^2. Converting a surface brightness (flux per unit solid angle) into a per-pixel flux is a change of variables from continuous arcsec^2 to discrete pixels, and the scaling factor of any change of variables is the Jacobian determinant of the map between them — here delta_pix $\cdot\, I$ acting on a 2-D patch, whose determinant is delta_pix^2. It is the same object as the magnification $1/\det A$ in Ch. 18 and the flow log-det in Ch. 23 — an area-scaling factor, computed the same way every time.

10. Galaxies, Sersic profiles, and velocity dispersion

Every lens in this repository is a galaxy first and a set of equations second. Before the mass model, the sampler, and the evidence integral, there is a physical object: a massive elliptical galaxy, sitting close enough to the line of sight to a background source that its gravity bends the source's light into arcs and rings. This chapter is about that object — what kind of galaxy it is, how its light is described (the Sersic profile, and the ugly-looking constant b_n that makes it work), and how its stars' motion (the velocity dispersion σ_v) predicts the one number every later chapter cares about, the Einstein radius. It ends with a genuine physics puzzle: real massive ellipticals turn out to have a total mass profile close to *isothermal* ($\gamma = 2$), for reasons no one fully derives from first principles. That puzzle is the reason this repository's own prior on γ is centered exactly there — and the reason the money number, $\gamma = 1.103$, is worth being suspicious of.

> **What you can skip**
>
> You own standard deviation, exponentials, and simple ordinary differential equations already — skip past the notation, not the physics. If you already know what a de Vaucouleurs profile is and why b_n exists, skip to Velocity dispersion. If you already know that "isothermal" in "singular isothermal sphere" refers to a constant velocity dispersion, skip straight to The isothermal conspiracy.

Ellipticals

Galaxies split, broadly, into two dynamical families. Disk galaxies — spirals, most star-forming galaxies — are supported against their own gravity by *ordered rotation*: pick a star, and it is on a roughly circular orbit, moving in the same sense as its neighbors. Elliptical galaxies are supported by the opposite thing: *disordered* motion. Pick a star in an elliptical and it is on some eccentric, randomly oriented orbit; there is no coherent spin field, only a cloud of individual trajectories pointed every which way, the stellar equivalent of a hot gas. The light is correspondingly featureless — smooth, centrally concentrated, no spiral arms, no ongoing star formation, an old and red stellar population — because there is no organizing structure left to draw a pattern with.

Ellipticals are also, characteristically, the *most massive* galaxies in their environment: the deepest potential wells around. That is not a coincidence of which galaxies happen to appear in this repository's data — it is close to a selection effect. Bending light by a detectable amount over an arcsecond-scale region takes a deep, compact potential well, and ellipticals are the class of galaxy built to have one. When Cikota et al. describe the lens in the Einstein cross this repo reproduces, they call it plainly "a massive elliptical lens galaxy (L1)" (`reproductions/cikota-2023/papers/main.tex:69`) — and the same description fits essentially every lens galaxy this repository's campaigns touch.

To subtract that galaxy's own light from an image before you can see the arcs behind it (or to model it directly, as this repo's forward model does — Ch. 22), you need a compact functional form for how its surface brightness falls off with radius. Astronomers reach for one function almost universally: the Sersic profile.

The Sersic profile

Surface brightness $I(R)$ — light per unit solid angle at projected radius R (Ch. 9) — for a Sersic profile is

$$I(R) = I_e \exp\left\{-b_n\left[\left(\frac{R}{R_e}\right)^{1/n} - 1\right]\right\}.$$

Three quantities carry the physics. R_e, the *effective radius*, is defined so that exactly half the galaxy's total light comes from $R < R_e$ — it is a half-light radius by construction, not a fitted scale length. $I_e = I(R_e)$ is the surface brightness there. n, the *Sersic index*, is a shape parameter: $n = 1$ recovers the exponential profile of a disk galaxy; $n = 4$ recovers de Vaucouleurs's 1948 law for elliptical light, $I \propto \exp\{-b(R/R_e)^{1/4}\}$; other values of n interpolate and extrapolate between a shallow, extended profile and a sharply cuspy one.

b_n is the piece that looks like an arbitrary fitting constant and is not. Because R_e *must* enclose exactly half the light regardless of n, b_n is not a free knob — once you fix n, b_n is *determined* by that requirement. Write the total luminosity as an integral over the whole plane,

$$L = \int_0^\infty I(R)\, 2\pi R\, dR,$$

and substitute $x = b_n(R/R_e)^{1/n}$, so $R = R_e(x/b_n)^n$. The integral turns into a Gamma-function form, $L = 2\pi n R_e^2 I_e\, e^{b_n} b_n^{-2n}\, \Gamma(2n)$, and demanding that the light within R_e equal exactly half of L becomes a condition on the *regularized incomplete* Gamma function,

$$P(2n,\, b_n) = \frac{1}{2}.$$

This equation has no closed form in elementary functions — b_n is defined implicitly, one value per n, and has to be solved numerically. What everyone actually uses, including this repository's own gigalens vendor code (`reproductions/claude-giga-lens/vendor/gigalens-sean/src/gigalens/jax/profiles/` and this guide's own `site/guide_src/lensing.py:157`, is the Capaccioli/Ciotti–Bertin fitting formula, accurate to well under a percent for $1 \lesssim n \lesssim 10$ (it degrades fast below that — at $n = 0.5$ it is already off by nearly 3%, Exercise 10.1 quantifies the $n = 1$ end of that range):

$$b_n \approx 1.9992\, n - 0.3271.$$

At $n = 1$ (exponential disk) this gives $b_n = 1.6721$; at $n = 4$ (de Vaucouleurs) it gives $b_n = 7.6697$. The increase is a factor of about four and a half , which is the normalization constant's way of saying that a de Vaucouleurs profile is far more centrally cuspy — most of the light is crammed close to R_e from outside, with a long faint outer wing — than a disk's gentler exponential fall-off.

The campaign's own lens-light model does not commit to a single n up front. It fits *four* separate Sersic components for the lens galaxy's light (plus a Sersic-and-shapelet source), each drawing its Sersic index from a Uniform$(0.5, 8.0)$ prior (`reproductions/claude-giga-lens/cgl/e2.py:101`) — spanning everything from a disky $n = 0.5$ profile to a super-de-Vaucouleurs $n = 8$ cusp, because a real galaxy's light does not arrive pre-labeled with its own index. Ch. 22 covers the full four-component model this feeds into.

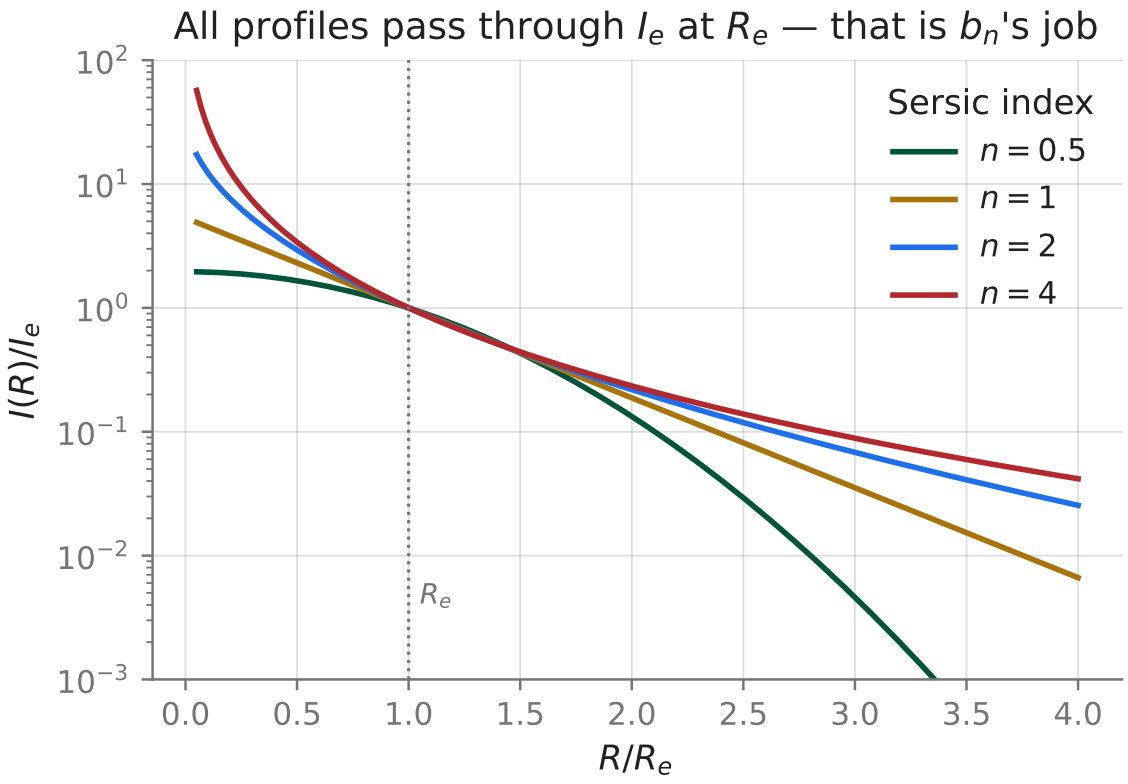

Figure 3: $I(R)/I_e$ for $n = 0.5, 1, 2, 4$, each rescaled by its own b_n. Every curve, regardless of n, passes through 1 at $R/R_e = 1$ — that is what solving $P(2n, b_n) = 1/2$ for b_n *buys you*: a single normalization convention under which R_e means the same thing (half the light) no matter how steep or shallow the profile is.

Velocity dispersion

> **You already know this**
>
> Velocity dispersion is exactly a standard deviation: the same statistical object as the σ in a Gaussian likelihood (Ch. 8) or the noise term in any regression loss. A galaxy with $\sigma_v = 300$ km/s has stars whose line-of-sight velocities are spread three times as wide as one with $\sigma_v = 100$ km/s. Nothing about the astrophysics changes what kind of object σ_v is.

Because an elliptical is not rotating in any organized way, you cannot read its mass off a rotation curve the way you can for a disk. What you *can* measure — from the width of an absorption line in its spectrum, the subject of Ch. 12 — is σ_v, the spread of its stars' line-of-sight velocities. The question this section answers is why that single number is enough to predict an Einstein radius.

Start from hydrostatic equilibrium for a self-gravitating, velocity-dispersion-supported sphere — astronomers call this the (isotropic) Jeans equation; it is the exact mechanical analogue of $dP/dr = -\rho\, GM(r)/r^2$ for a gas, with pressure P replaced by $\rho\sigma_v^2$:

$$\frac{d}{dr}\left(\rho(r)\,\sigma_v^2\right) = -\rho(r)\,\frac{GM(r)}{r^2}, \qquad M(r) = 4\pi\int_0^r \rho(r')\,r'^2\,dr'.$$

"Isothermal" means σ_v does not depend on r — literally the mechanical statement of a system held at constant temperature, since for an ideal gas $P = \rho\, k_B T/m$ and σ_v^2 plays the role $k_B T/m$ plays there. With σ_v pulled outside the derivative and no other length scale anywhere in the problem, dimensional analysis alone forces $\rho(r)$ to be a pure power law; try $\rho(r) = A/r^2$ and check that it is self-consistent. Then $M(r) = 4\pi A r$ (density $\times$ area integrates linearly in r), so

$$\text{LHS} = \sigma_v^2\frac{d}{dr}\left(\frac{A}{r^2}\right) = -\frac{2A\sigma_v^2}{r^3}, \qquad \text{RHS} = -\frac{A}{r^2}\cdot\frac{G\cdot 4\pi A r}{r^2} = -\frac{4\pi G A^2}{r^3}.$$

Both sides scale as r^{-3} — the ansatz was consistent — and equating the coefficients gives $A = \sigma_v^2/(2\pi G)$. So a constant-σ_v ("isothermal") sphere has density

$$\rho(r) = \frac{\sigma_v^2}{2\pi G\, r^2}.$$

That is exactly $\rho \sim r^{-\gamma}$ with $\gamma = 2$: the isothermal sphere *is* the $\gamma = 2$ profile, and this derivation is where the word "isothermal" earns its meaning rather than being a label attached to a formula. It also explains the other half of "singular isothermal sphere": the Abel projection of Ch. 3 turns $\rho \sim r^{-\gamma}$ into a surface density $\Sigma \sim R^{1-\gamma}$, so at $\gamma = 2$, $\Sigma(R) \propto 1/R$ — finite everywhere except a genuine divergence at $R = 0$. That divergence is the "singular."

The same profile also gives you a disk galaxy's flat rotation curve for free: the circular velocity satisfies $v_{\rm circ}^2(r) = GM(r)/r = 4\pi GA = 2\sigma_v^2$ — constant in r, independent of A's value. A flat rotation curve (rotation-supported systems) and a constant velocity dispersion (pressure-supported systems) are the *same* statement, $\rho \sim r^{-2}$, observed through two different kinematic windows.

Ch. 19 takes this density profile the rest of the way, combining it with the definition of $\theta_{\rm E}$ as the radius where the mean interior convergence equals exactly 1 to arrive at

$$\theta_{\mathrm{E}} = 4\pi \left(\frac{\sigma_v}{c}\right)^2 \frac{D_{\mathrm{ds}}}{D_{\mathrm{s}}}$$

(Hsu+2025 Eq. 1, implemented at `site/guide_src/cosmo.py:79` and `site/guide_src/lensing.py:170`; recall $D_{\mathrm{ds}} \neq D_{\mathrm{s}} - D_{\mathrm{d}}$, Ch. 15). You can already use the formula here, since its only missing ingredient beyond what you just derived is that one projection step. For a fiducial massive elliptical — $\sigma_v = 250$ km/s at $z_l = 0.5$, $z_s = 2.0$, the same system Ch. 19 uses — it gives $\theta_{\mathrm{E}} = 1.145''$. Because the formula is quadratic in σ_v, doubling the velocity dispersion — from 150 km/s to 300 km/s — exactly quadruples the Einstein radius, from $0.412''$ to $1.649''$, a ratio of 4.0 . That steep scaling is also why lensing is rare: only fairly massive ellipticals, with σ_v comfortably above ~ 150 km/s, produce an Einstein radius large enough to resolve against a survey's seeing at all — the same resolution constraint Ch. 27 derives from the pixel scale directly.

The isothermal conspiracy

Here is the puzzle the derivation above sets up but does not resolve. A real massive elliptical's gravity is built from two physically unrelated ingredients. The *stars* follow something like the Sersic profile just derived — not a power law at all, an exponential of a fractional power, shallow near the center and falling off far faster than r^{-2} at large radius. The *dark matter* — inferred, historically, from exactly the flat rotation curves derived above staying flat well past the edge of a disk galaxy's visible light, which under Newtonian gravity requires unseen mass out there — is typically modeled with an NFW profile, whose logarithmic slope is not constant either: shallower than isothermal near the center, steeper than isothermal far outside.

Neither component, on its own, is isothermal. And yet decades of joint lensing-plus-dynamics measurements of real massive ellipticals find that the *total* mass profile — stars and dark matter summed, which is all strong lensing and stellar dynamics ever weigh — comes out close to isothermal ($\gamma \approx 2$) over the radii those measurements actually probe, roughly the effective radius R_e this chapter's Sersic profile defines. Two physically unrelated components, with unrelated profile shapes, adding up to something close to a single clean power law is not a prediction of any first-principles argument. It is an empirical regularity astronomers have been trying to explain by appeal to galaxy-formation physics — feedback, adiabatic contraction — for decades, with no fully settled account. Some of the literature calls it the bulge–halo conspiracy; this book, following the outline's own name for it, calls it the isothermal conspiracy.

It is not a throwaway fact for this repository. It is written directly into the campaign's own mass model: the EPL slope prior (`reproductions/claude-giga-lens/cgl/e2.py:110`) is $\gamma \sim \mathrm{TruncatedNormal}(2.0,\ 0.25,\ \mathrm{low} = 1.0,\ \mathrm{high} = 2.7)$ — centered exactly on the conspiracy, with a width reflecting the real galaxy-to-galaxy scatter astronomers measure around it, and hard walls at 1.0 and 2.7 because slopes that shallow or that steep essentially never occur in real massive ellipticals. That prior is not a mathematical convenience chosen for this campaign. It is this section, compiled.

Hold the number 2.0 in mind. This repository's own headline measurement for one galaxy, $\gamma_{\mathrm{binned}}(\mathrm{corr}, \mathrm{low}) = 1.103$, sits 3.588 prior-sigma below that center — closer to the prior's own hard floor at 1.0 than to its mean. For comparison, the value Ch. 25 treats as the *trustworthy* anchor, $\gamma = 1.433$, sits only 2.268 prior-sigma below 2.0 — still a real departure from isothermal, but a smaller one. Whether 1.103's larger departure reflects a genuinely unusual galaxy or a mismodeled noise covariance is exactly what Chapters 20 through 26 exist to adjudicate.

Ledger

What this chapter rules in or out about $\gamma = 1.103$: nothing, yet — this chapter fixes the *expectation*, not the *verdict*. Real massive ellipticals cluster close to $\gamma = 2$ for real, if incompletely understood, physical reasons; that is why this repository's own prior is centered there rather than at some arbitrary number. Against that expectation, $\gamma = 1.103$ is a large-looking departure — 3.588 prior-sigma — but a large prior-sigma is a statement about the *prior*, not about the *data*. Whether the data actually demand a galaxy this far from isothermal is Ch. 25's question, and whether that demand survives a saddle-point posterior is Ch. 26's.

Connect to the repo

- `site/guide_src/lensing.py:157` (`sersic_bn`) and `:162` (`sersic`) — the Capaccioli fit, matching the campaign's production code exactly.
- `reproductions/claude-giga-lens/vendor/gigalens-sean/src/gigalens/jax/profiles/light/sersic` — gigalens' own `bn = 1.9992 * n_sersic - 0.3271`.
- `reproductions/claude-giga-lens/cgl/e2.py:100`–`123` — the four lens-light Sersic priors and the source's; line `101` ($n \sim \text{Uniform}(0.5, 8.0)$) and line `110` ($\gamma \sim \text{TruncatedNormal}(2.0, 0.25, 1.0, 2.7)$), the isothermal conspiracy written as code.
- `site/guide_src/cosmo.py:79` (`theta_e_from_sigma_v`) and `site/guide_src/lensing.py:170` (`theta_e_sis`) — the SIS $\theta_{\rm E}$–σ_v relation this chapter derives the origin of and Ch. 19 applies.
- `reproductions/cikota-2023/papers/main.tex:69` — "a massive elliptical lens galaxy (L1)," the description that fits nearly every lens in this repository's campaigns.

Exercises

Exercise 10.1 — b_n exactly, versus the fit

b_n is defined by $P(2n, b_n) = 1/2$, where P is the regularized lower incomplete Gamma function. In Python, `scipy.special.gammaincinv(a, p)` solves exactly this: it returns the x such that $P(a, x) = p$. Compute the *exact* b_n for $n = 1$ and $n = 4$ via `gammaincinv(2*n, 0.5)`, compare to the Capaccioli fit's 1.6721 and 7.6697, and explain — in terms of what the fit was tuned for — why one comparison is far tighter than the other.

Solution

```python
from scipy.special import gammaincinv
gammaincinv(2*1, 0.5)   # 1.6783
gammaincinv(2*4, 0.5)   # 7.6692
```

At $n = 4$ the fit (7.6697) and the exact value (7.6692) agree to four significant figures — a relative error under 0.01%. At $n = 1$ the fit (1.6721) misses the exact value (1.6783) by about 0.4%, two orders of magnitude worse. The Capaccioli/Ciotti–Bertin formula was fit to reproduce de Vaucouleurs's $n = 4$ profile — the one astronomers actually cared about getting right for elliptical galaxies — and is a linear approximation to a genuinely nonlinear function everywhere else. It is *good enough* across the whole range this repo's own $n \sim \text{Uniform}(0.5, 8.0)$ prior spans, but "good enough" is not "exact," and $n = 1$ is where that shows up first.

Exercise 10.2 — the flat rotation curve, from the same profile

Using $\rho(r) = \sigma_v^2/(2\pi G r^2)$ and $M(r) = 4\pi \int_0^r \rho(r')r'^2\,dr'$, show that the circular velocity $v_{\rm circ}(r) = \sqrt{GM(r)/r}$ is independent of r, and find it in terms of σ_v.

Solution

$M(r) = 4\pi \int_0^r \frac{\sigma_v^2}{2\pi G r'^2}\, r'^2\,dr' = \frac{2\sigma_v^2}{G}\, r$ — the r'^2 from the volume element exactly cancels the r'^{-2} in ρ, so the integrand is constant and $M(r)$ is linear in r. Then

$$v_{\rm circ}^2(r) = \frac{GM(r)}{r} = \frac{G}{r}\cdot\frac{2\sigma_v^2}{G} r = 2\sigma_v^2,$$

with no r left anywhere — flat, at $v_{\rm circ} = \sqrt{2}\,\sigma_v$. This is the same cancellation, for the same reason, that made the density ansatz self-consistent in the main text: an r^{-2} density is precisely the one power law whose enclosed mass grows linearly, which is precisely what a flat rotation curve (or a constant velocity dispersion) requires.

Exercise 10.3 — doubling sigma_v

Confirm, using `cosmo.theta_e_from_sigma_v` (`site/guide_src/cosmo.py:79`), that doubling σ_v at fixed z_l, z_s exactly quadruples $\theta_{\rm E}$, and say in one sentence why the exponent is 2 and not, say, 1.

Solution

```
import cosmo
cosmo.theta_e_from_sigma_v(150.0, 0.5, 2.0)   # 0.412"
cosmo.theta_e_from_sigma_v(300.0, 0.5, 2.0)   # 1.649"
# ratio: 4.0
```

That ratio is exactly the 4.0 from the main text . The formula $\theta_{\rm E} = 4\pi(\sigma_v/c)^2 D_{\rm ds}/D_{\rm s}$ is quadratic in σ_v because $\theta_{\rm E}$ traces back to $\rho(r) \propto \sigma_v^2/r^2$ — the σ_v^2 sitting in the density's normalization propagates linearly through $M(r)$, through $\Sigma(R)$, and through the mean-convergence definition of $\theta_{\rm E}$, none of which introduces another power of σ_v or cancels the one that is there.

Exercise 10.4 — how surprised should the anchor make you?

The campaign's γ prior is centered at 2.0 with $\sigma = 0.25$ (`reproductions/claude-giga-lens/cgl/e2.py:110`). Ch. 25 treats $\gamma = 1.433$ as the *anchor* — the value it is willing to trust. How many prior-sigma is the anchor from isothermal, and how does that compare to the money number's own 3.588?

Solution

$$\frac{2.0 - 1.433}{0.25} = 2.268$$

The anchor is already a real departure from isothermal — 2.268 prior-sigma is not nothing — but it is noticeably smaller than the money number's 3.588. Both numbers describe *surprise relative to a prior expectation*, not evidence from the data; neither, by itself, tells you which measurement to believe. That is a different question, with a different kind of arithmetic, and it is exactly what Ch. 25 works through.

11. From photons to pixels: PSF, noise, and drizzle

Every number this book eventually fits — a slope, a shear, an Einstein radius — comes from an array of floating-point numbers that started as photons hitting silicon. This chapter is the chain between the two: how a CCD turns light into counts, why the resulting image is never quite what the sky actually looked like (the point-spread function), why each pixel's error bar has the specific algebraic form it does, and why *HST*'s habit of combining several dithered exposures onto one output grid — drizzling — quietly correlates the noise between neighbouring pixels. That last fact is not a footnote. It is the first link in what this book calls **Spine 1**: drizzle correlates noise, a diagonal (independent-pixel) likelihood assumes it does not, that mismatch biases a fitted slope, and undoing it is what produces the 191-nat evidence swing you will compute yourself in Ch. 25. Everything in this chapter is arithmetic you can check on a calculator, and every check matches a real number this campaign published or retracted.

> **What you can skip**
>
> You already know that a Poisson-distributed count has variance equal to its mean, and you already know what a convolution is (Ch. 7 built it from scratch either way). What is genuinely new here is astronomy-specific: what a point-spread function *is* physically, the specific two-term shape of a CCD's noise model, and the drizzle resampling scheme — none of which has a direct ML analogue, so this chapter does not assume you have seen them.

CCDs and the point-spread function

A CCD (or the HgCdTe array HST's infrared channel actually uses — the physics is the same for this chapter's purposes) is a grid of light-sensitive wells. Over an exposure of duration $t_{\rm exp}$, each well accumulates photo-electrons: a photon that reaches the detector frees an electron with some wavelength-dependent probability (the quantum efficiency), and at readout the well's accumulated charge is reported as a count. Two facts about this process matter for everything that follows. First, it is a **counting** process — the number of photo-electrons in a well over a fixed exposure is a Poisson random variable, and for a Poisson variable with mean c, the variance is also c. Second, nothing about this process cares what the true sky brightness distribution looked like at scales finer than the telescope can resolve — which is the point-spread function's whole story.

No real telescope images a point source as a point. Diffraction at a finite aperture, small imperfections in the optics, and (for ground-based instruments) the atmosphere all spread a true point of light into a smooth, extended blob before it reaches the detector — the **point-spread function** (PSF). The image that lands on the array is not the sky; it is the sky *convolved* with the PSF. This is the exact convolution of Ch. 7, not an analogy: the forward model this repo fits (Ch. 22) literally renders a noiseless model image on a fine grid and convolves it with an empirical PSF kernel before comparing it, pixel by pixel, to the data. Get the kernel wrong and every downstream number — every χ^2, every posterior — inherits the error.

> **You already know this**
>
> A point-spread function is a fixed, non-learned convolution kernel — exactly a single frozen conv layer with one channel. "The kernel must be sampled at the data's own pixel scale" is exactly the ML bug family of feeding a fixed-resolution operator an input at the wrong resolution: nothing raises an exception, the array shapes still broadcast, and the output is

> silently wrong by a scale factor.

That last sentence is not hypothetical; it is the single largest defect this campaign's predecessor reproduction actually shipped for a while. `gigalens`'s simulator hands the PSF kernel to `lenstronomy`'s `subgrid_kernel`, which **assumes the kernel it receives is already sampled at the data's own pixel scale**, `delta_pix`, and upsamples it internally by the rendering `supersample` factor. `foundry-i` instead fed it an empirical PSF built directly from stars at its own, finer native sampling (0.065″) while telling the simulator `delta_pix=0.13`, `supersample=2` — so the function upsampled a kernel that was already oversampled, double-applying the refinement and broadening the *effective* PSF actually used in every native-scale fit by roughly 2×.

The tell was not a crash; it was a χ^2_ν **floor**. No matter how much extra flexibility the model was given — additional Sérsic light components, more shapelet terms in the source — the reduced chi-squared refused to drop below about $\chi^2_\nu \approx 3.4$. That distinction matters: underfitting is a problem more model flexibility relieves; a floor that flexibility cannot touch is the signature of a *systematic* — something structurally wrong that no amount of extra freedom in the light or source model can absorb, because the mismatch is in the instrument model, not the sky model. Fixing *only* the PSF sampling convention — same noise model, same data, same everything else — dropped the floor to $\chi^2_\nu = 1.051$, and simultaneously resolved what had looked like a sampler pathology: the slope parameter's effective sample size, which had been stuck near 260 after thousands of draws under the broadened kernel, jumped past 5,700 once the PSF was corrected. The sampler had not been struggling; the model had been structurally wrong in a way that also happened to flatten one posterior direction. `cgl/guards.py:74` encodes this incident as a hard check, `assert_psf_sampling`, so it cannot recur silently:

```
if abs(psf_pixel_scale - delta_pix) > atol:
    raise GuardError(
        f"PSF kernel sampled at {psf_pixel_scale}\" but delta_pix={delta_pix}\". "
        "subgrid_kernel upsamples internally; passing a supersampled kernel "
        "double-applies the refinement..."
    )
```

The noise model: background plus Poisson

Every pixel's error bar in this repo's likelihood comes from two independent noise sources added in quadrature. The first is a roughly constant background term — sky brightness, detector read noise, dark current — with RMS $\sigma_{\rm bkg}$, approximately the same in every pixel of an exposure. The second is the source's own photon-counting (Poisson) noise, and it is where the CCD physics above becomes an equation.

Let f be the model's predicted flux *rate* in a pixel (counts per unit time). Over an exposure of length $t_{\rm exp}$, the expected photon count is $c = f\,t_{\rm exp}$, and because counting is Poisson, $\mathrm{Var}[\mathrm{count}] = c = f\,t_{\rm exp}$. Converting back to a flux-rate estimate $\hat{f} = \mathrm{count}/t_{\rm exp}$ rescales the variance by $1/t_{\rm exp}^2$:

$$\mathrm{Var}[\hat{f}] \;=\; \frac{\mathrm{Var}[\mathrm{count}]}{t_{\rm exp}^2} \;=\; \frac{f\,t_{\rm exp}}{t_{\rm exp}^2} \;=\; \frac{f}{t_{\rm exp}}.$$

Two independent noise sources add in variance, not in RMS, so the total per-pixel error is

$$\text{err} \;=\; \sqrt{\sigma_{\text{bkg}}^2 \;+\; \frac{f}{t_{\text{exp}}}}.$$

That is the exact line the repo runs — `foundry-i/_hmc_lib.py:135` writes `err_map = jnp.sqrt(self.background_rms ** 2 + im_sim / self.exp_time)`, and `cgl/mocks.py:227`, the mock generator this book's own examples trace back to, writes the identical formula with its own constants: `SIGMA_BKG = 0.2`, `EXP_TIME = 100.0` (`cgl/mocks.py:65-66`). At a pixel where the model predicts flux $f = 1.0$ in those same units,

$$\text{err} = \sqrt{0.2^2 + 1.0/100} = \sqrt{0.05} = 0.2236,$$

a number you can check on a calculator against the repo's own constants before you ever look at a data file.

Notice what this buys, and what it costs. Because err depends on the *model* flux f and not on the data, brighter model pixels automatically get a larger error bar and therefore less weight — the likelihood is a **weighted** least squares with weight $\sqrt{W} = 1/\text{err}$ (`cgl/likelihood.py:326`), and that weight is heteroscedastic by construction, not fit separately. The cost is subtler: the error map is a function of the *parameters being fit*, recomputed at every forward pass, not a fixed quantity decided once from the data alone.

> **You already know this**
>
> An error map that is itself a function of the model is exactly what an aleatoric-uncertainty network predicts alongside its mean output — the "noise" you divide by is recomputed every forward pass, not looked up. Down-weighting noisy pixels by their own predicted variance ($\sqrt{W} = 1/\text{err}$) is the identical move to uncertainty-weighted regression loss.

Every data product the repo loads carries the same five-object shape — image, error map, keep-mask, PSF, metadata (`cgl/paths.py:81`, `load_product`) — but the error map's *origin* differs by survey: for the *HST* products above it is rebuilt from σ_{bkg} and t_{exp}; for the Euclid Q1 targets of Ch. 22, the survey delivers a per-pixel RMS map directly (`VIS_RMS`), so `err_map` is used as-is and t_{exp} never enters the log-posterior at all (`cgl/euclid_io.py:34-39`) — same interface, different physical origin.

A calibration this formula depends on is where the campaign's second real incident lives. Early in the reproduction, the fine-scale product's MAP fit reported $\chi^2_\nu = 0.451$ — comfortably, almost suspiciously, under the campaign's own < 1.1 quality bar. A χ^2_ν well below 1 is itself a red flag, not a free win: with correctly calibrated Gaussian noise, a good fit clusters χ^2_ν *near* 1, not far below it, because a $\chi^2_\nu \ll 1$ means the error bars you divided by were too large — the noise model overstated the noise. An audit found exactly that: σ_{bkg} had been calibrated on raw-image fluctuations at large radius that were actually about 70% diffuse light from the lens galaxy's own outer wings — starlight the segmentation had missed, not noise at all — inflating every downstream error bar. Recalibrating the identical kernel on the **model-subtracted** residual (where the lens light is already removed) gave the honest $\chi^2_\nu = 0.92$: still comfortably under the bar, but no longer suspicious. `cgl/guards.py:129`, `assert_model_subtracted_sky`, now refuses any noise-calibration artifact that is not explicitly tagged `model_subtracted=True`. Put the two incidents

side by side: a χ^2 that refuses to *improve* and a χ^2 that looks *too good* are equally diagnostic, in opposite directions, of the same kind of mistake — a wrong noise model, not a wrong sky model.

Drizzle

HST rarely takes one exposure of a target; it takes several, each offset by a fraction of a pixel (a *dither*), and combines them onto a common output grid — often finer than any single native exposure — with an algorithm called **drizzle** (Fruchter & Hook 2002). Each input pixel's flux is distributed over the output-pixel footprint its "drop" overlaps, scaled by a parameter called `pixfrac`. This campaign's target system, Foundry I, exists in three such drizzle products at different output scales (main.tex, Data and Targets): a fine $0.04''$/px MAST HAP mosaic (`v3`), a 2×2-binned $0.08''$/px rebuild of it (`v3b`), and the pipeline's own native-scale product (`v2d`) at close to the true WFC3/IR detector pixel.

Think of an output pixel's noise value as a **weighted average of the native pixels whose drop footprint overlaps it**. When the output grid is finer than native, two *adjacent* output pixels typically share at least one of those native parents — part of their noise comes from literally the same random draw, so they are not independent. This is the same mechanism Ch. 7 covers in general: a moving-average filter turns white noise into a correlated process, with correlation extending over the filter's width. The extreme case makes the mechanism plain: at a *fixed* sub-pixel phase with `pixfrac=1`, each output pixel would simply *copy* the one native pixel it falls inside — any two sharing that native parent perfectly correlated, any two that do not exactly independent (native pixels are drawn iid). Real dithers do not sit at one fixed phase; successive exposures land the output grid at different sub-pixel offsets, and averaging the covariance and the variance *separately* over all such unknown registrations (`cgl/noise.py:166`, `drizzle_acf`) is what turns that simple same-phase argument into the campaign's closed-form anchor for the along-axis, nearest-neighbour correlation:

$$t(1) \;=\; \frac{r-1}{r-1/3}, \qquad r \;=\; \frac{\text{native detector pixel}}{\text{output pixel}}.$$

This is the one line that opens Spine 1. `site/guide_src/lensing.py:194` is the guide's own copy of it. The true WFC3/IR detector pixel is a header fact, $0.1283''$ (`02_fit_noise_kernels.py:79`); for the fine skycell's output scale of $0.04''$/px, that gives

$$r_{\text{fine}} = 0.1283/0.04 = 3.2075,$$

and the closed form evaluates to

$$t(1) = \frac{3.2075-1}{3.2075-1/3} = 0.76805\text{—}$$

matched to three decimal places — $0.76805 - 0.76799 = 6\times10^{-5}$ — by numerically enumerating the actual drop-overlap operator on a test patch (`cgl/noise.py:166`; the campaign's own report quotes 0.76799 for that enumeration, Methods I, covariance model and drizzle anchor). That agreement between a closed-form phase-average and a full numerical enumeration of the real operator is what makes the number trustworthy rather than merely plausible — you can reproduce both sides yourself with `worked_examples.py --show ch11`.

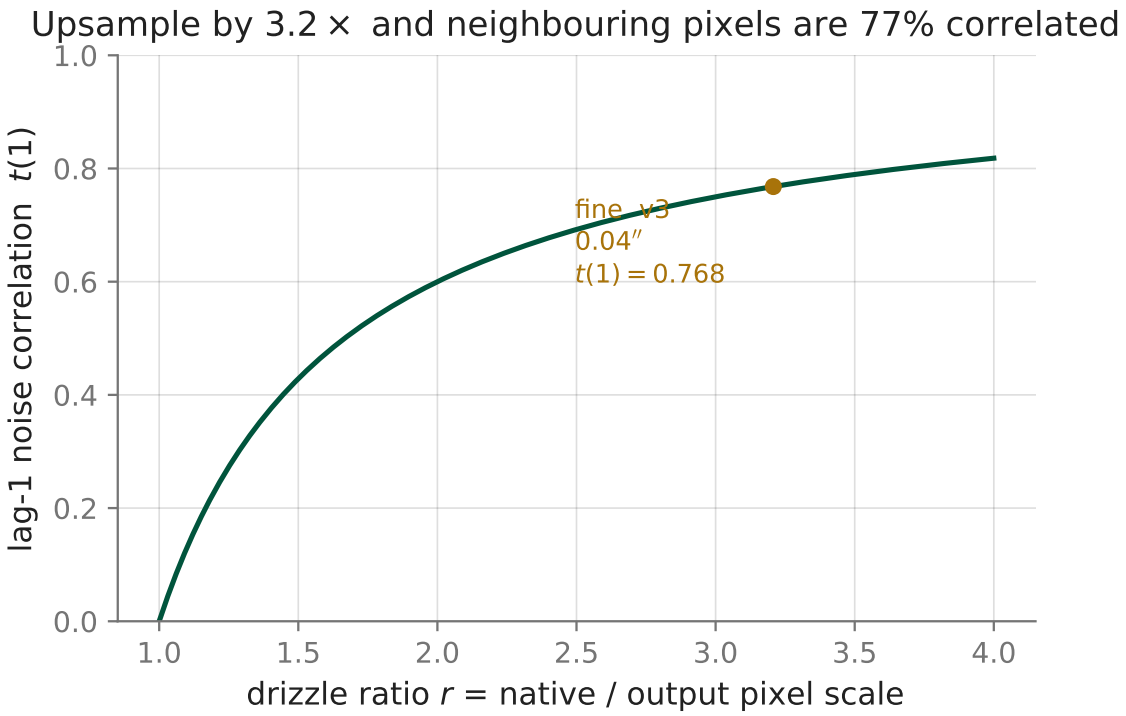

Figure 4: $t(1) = (r - 1)/(r - 1/3)$ against the pixel-scale ratio r, from just above 1 (almost no resampling) to 4 (aggressive oversampling). The campaign's fine skycell sits at $r = 3.2075$ — $0.04''$ output pixels built from a $0.1283''$ native detector pixel — where the curve gives $t(1) = 0.768$: over three-quarters of one output pixel's noise is shared with its neighbour. Nothing here is fit to real data; the curve is the closed form above, evaluated on a grid.

The closed form is not universal, and treating it as though it were is its own small lesson. It is derived for well-oversampled resampling and is only valid for $r \geq 2$ (`02_fit_noise_kernels.py:225-227`). The native product sits at $r = 0.1283/0.13 \approx 0.987$ — *below* that domain — and plugging it in anyway returns $t(1) \approx -0.02$: a negative "correlation" from a construction (a weighted average with non-negative weights) that can never produce one in the regime the formula was actually derived for. The formula does not raise an exception; it just stops meaning anything. That is exactly why $r \approx 1$ at native scale is not a defect to fix but the reason this product is called the campaign's *diagonal-trustworthy anchor*: almost no resampling happened there, so whatever residual correlation exists must be measured directly (the numerical enumeration), never read off this closed form.

Why drizzle correlates noise, and what it breaks

The closed form above is the pure drizzle-kernel mechanism; the *full* measured correlation on real data includes it plus other real structure. On the model-subtracted residual, the campaign measures an along-axis lag-1 correlation of $\rho(1) = 0.815$ on the fine product, 0.615 binned, and 0.305 native (main.tex, Table 1) — every one of them larger than the pure drizzle anchor at that scale, because real pixels also carry PSF and detector structure the idealized kernel does not model. The consequence is quantified as an **effective independent-pixel fraction**: on the fine product, $N_{\text{eff}}/N \approx 0.017$ — a 2×2 block of fine pixels is about 78% internally correlated, not four separate measurements.

This is the mismatch that makes a *diagonal* Gaussian likelihood mis-specified on drizzled data. A diagonal likelihood implicitly assumes it has N independent pixels; on the fine product it really has $N_{\text{eff}} \approx 0.017N$. Posterior widths shrink as the inverse square root of the effective sample size (the same central-limit scaling behind every standard error you have ever quoted), so believing N when the truth is N_{eff} makes the reported error bars too tight by a factor

$$\sqrt{\frac{N}{N_{\text{eff}}}} = \sqrt{\frac{1}{0.017}} \approx 7.67 :$$

almost eight times too confident, on this product alone. This is not a rounding correction; it is the wrong calibration entirely, and it is why a diagonal likelihood cannot simply be "corrected" with a fudge factor on its error bars — the correlation structure itself has to enter the likelihood. Ch. 24 builds exactly that: a correlated-noise likelihood $C = D^{1/2} K D^{1/2}$ with K a stationary kernel anchored on the closed form of this chapter, and convolutional whitening to make it tractable at these pixel counts. Once you correct this mismatch on the real system, the answer moves — by how much, and whether the correction is itself trustworthy, is the 191-nat evidence swing you derive in Ch. 25, The chain.

> **Ledger**
>
> This chapter never touches γ — nothing here constrains the EPL slope. What it establishes is the *reason a constraint on γ can be wrong at all*: the diagonal likelihood every fast lens-modeling code uses by default is misspecified on drizzled data by a measured factor of $\sim 7.7\times$ in its own error bars on the fine product. Every γ number that follows in this book inherits that fact until Ch. 24 fixes it.

Connect to the repo

- `cgl/guards.py:74-91` (`assert_psf_sampling`) and `cgl/guards.py:129-141` (`assert_model_subtracted_` — the two incidents this chapter teaches, hard-coded as guards so they cannot recur silently. (github)
- `cgl/mocks.py:65-66,225-227` (`add_noise_native`) — the noise model, err = $\sqrt{\sigma_{\rm bkg}^2 + f/t_{\rm exp}}$, instantiated with its own constants. (github)
- `foundry-i/_hmc_lib.py:135` — the identical formula in the earlier reproduction (`background_rms`, `exp_time`). (github)
- `cgl/paths.py:81-90` (`load_product`) and `cgl/likelihood.py:262-289,326` — the five-object product dict every driver script loads (`img`, `err_map`, `keep_mask`, `psf`, `meta`), and where $\sqrt{W} = 1/\text{err}$ becomes the likelihood's per-pixel weight. (github)
- `cgl/euclid_io.py:34-39` — the Euclid contrast: the survey delivers `err_map=VIS_RMS` directly, so `t_exp` never enters the log-posterior. (github)
- `site/guide_src/lensing.py:194-205` (`drizzle_lag1`) and `cgl/noise.py:117-213` (`drizzle_overlap_matrix_1d`, `drizzle_acf`) — the closed form and the numerical enumeration that validates it. (github)
- `02_fit_noise_kernels.py:79-83,202-227` — where $r_{\rm fine} = 3.2075$ is defined from a header fact, and the "closed form only valid for $r \geq 2$" note that flags the native product as out of domain. (github)
- main.tex: Data and Targets, the real system and its three drizzle products and Methods I, covariance model and drizzle anchor.
- The foundry-i final report: The sampling-convention rule and Why earlier attempts disagreed: the defect chain.

Exercises

Exercise 11.1 — the noise model, from counting statistics to a calculator

Starting from Var[count] = c for a Poisson-distributed photon count with mean $c = f\,t_{\rm exp}$, derive Var$[\hat{f}]$ for the flux-rate estimate $\hat{f} = \text{count}/t_{\rm exp}$, and show it reduces to $f/t_{\rm exp}$. Then, using the repo's own mock constants $\sigma_{\rm bkg} = 0.2$, $t_{\rm exp} = 100$ (`cgl/mocks.py:65-66`), compute err at a pixel where the model predicts $f = 1.0$.

Solution

Rescaling a random variable by a constant $1/t_{\rm exp}$ rescales its variance by the square of that constant:

$$\text{Var}[\hat{f}] = \text{Var}\left[\frac{\text{count}}{t_{\rm exp}}\right] = \frac{\text{Var}[\text{count}]}{t_{\rm exp}^2} = \frac{f\,t_{\rm exp}}{t_{\rm exp}^2} = \frac{f}{t_{\rm exp}}.$$

With $\sigma_{\rm bkg} = 0.2$, $t_{\rm exp} = 100$, $f = 1.0$:

$$\text{err} = \sqrt{0.2^2 + 1.0/100} = \sqrt{0.04 + 0.01} = \sqrt{0.05} \approx 0.2236$$

— matching `worked_examples.py --show ch11`'s `err_example` exactly, because it is the same formula run on the same two numbers.

Exercise 11.2 — a formula outside its own domain

The closed form $t(1) = (r-1)/(r-1/3)$ is only valid for $r \geq 2$ (`02_fit_noise_kernels.py:225-227`). Evaluate it at $r = 0.987$ (the native product) and interpret the sign. Given that a drizzled output pixel is a weighted average with non-negative weights, could a *true* correlation computed that way ever come out negative? What should you conclude when a formula that "should" only ever return values between 0 and 1 hands you a negative number without complaining?

Solution

$$t(1) = \frac{0.987 - 1}{0.987 - 1/3} = \frac{-0.013}{0.6537} \approx -0.020$$

A weighted average of non-negative weights cannot produce a negative correlation between two such averages that share any positively weighted term, so -0.020 cannot be a real correlation. The formula does not check its own domain — it is a closed form derived under an approximation ($r \geq 2$, well-oversampled resampling) that simply does not hold here, and it returns a syntactically valid, semantically meaningless number rather than an error. The lesson generalizes past drizzle: any closed form you did not derive yourself is only as trustworthy as your knowledge of the assumptions that produced it. At native scale the campaign does not extrapolate the closed form at all; it measures the correlation directly by enumerating the real drop-overlap operator (`cgl/noise.py:166`), which is non-negative by construction.

Exercise 11.3 — how overconfident is a diagonal likelihood, exactly?

The fine product's measured $N_{\text{eff}}/N \approx 0.017$. If a diagonal likelihood believes it has N independent pixels when only N_{eff} behave independently, and posterior widths shrink as the inverse square root of the effective sample size, by what factor does a diagonal fit on this product understate its own posterior width?

Solution

$$\frac{\sigma_{\text{diag, believed}}}{\sigma_{\text{true}}} = \sqrt{\frac{N_{\text{eff}}}{N}} \quad \Rightarrow \quad \text{understatement factor} = \sqrt{\frac{N}{N_{\text{eff}}}} = \sqrt{\frac{1}{0.017}} \approx 7.67$$

A diagonal fit on the fine product reports error bars roughly 7.7 times too tight — not a rounding issue but a fundamentally wrong calibration. This is precisely the miscalibration Ch. 24's correlated-noise likelihood exists to remove.

Exercise 11.4 — two chi-squared incidents, one lesson

Section The noise model described a χ^2_ν floor that extra model flexibility could not move, and a χ^2_ν that looked *too good* until the sky calibration was corrected. Both were eventually caught,

but neither would have been caught by simply "fitting harder." What single diagnostic habit would have caught both, and why does adding model flexibility fail to substitute for it?

Solution

Both incidents are visible in the *shape* of the residual, not its magnitude: a PSF-broadening defect leaves structured, PSF-scale residual power no source-model flexibility can absorb (the mismatch is in the instrument model, not the sky model), and a wing-contaminated sky calibration leaves a model-subtracted residual whose variance does not match the raw image's. Adding model flexibility only ever changes what is being fit; it cannot diagnose whether the *noise model being divided by* is correct, since a chi-squared statistic is a ratio of the two. The habit that catches both: before adding model complexity, inspect the model-subtracted residual itself — its autocorrelation, its variance by region — rather than only watching whether χ^2_ν moves. This is exactly what `assert_model_subtracted_sky` now enforces mechanically.

12. Redshifts and what a spectrum tells you

Every chapter so far has treated an image as the whole of the data: a grid of pixels, a point-spread function, a noise model. A spectrum measures the same object a different way — flux broken out by wavelength instead of by position — and it buys you two numbers no image can give you on its own: a redshift z, which tells you how far away something is, and a velocity dispersion σ_v, which tells you how much mass is moving around inside it. This chapter derives both from first principles and then reproduces them on a real DESI system. By the end you will have run, by hand, the same redshift-ratio test that DESI Strong Lens Foundry's discovery pipeline (Hsu et al. 2025) uses to flag lens candidates out of 28 million fiber spectra — before anyone looks at a picture.

> **What you can skip**
>
> You do not need atomic physics: take it as given that an electron dropping between two fixed energy levels emits or absorbs light at one specific, reproducible wavelength, and move on. You do not need spectrograph engineering (fibers, gratings, detectors) — for this chapter's purposes a spectrograph is a very high-resolution, wavelength-binned camera. You do already own the one piece of signal-processing math this chapter leans on: matching a template against data by sliding it and scoring the overlap is a cross-correlation, and you met it formally in Ch. 7.

What a spectrum is, and why it has lines

A galaxy's spectrum is a plot of flux against wavelength. Most of it is a smooth continuum — the light of a few hundred billion stellar photospheres, each close to a blackbody. Sitting on that continuum are sharp features at specific wavelengths: **emission lines**, where hot ionized gas (star-forming regions, an active nucleus) radiates at the exact energy of an electron transition, and **absorption lines**, where cooler gas or a stellar photosphere in front of the continuum removes light at that same energy. Because the transition energy is fixed by quantum mechanics and not by the local environment, every hydrogen atom in the universe emits Balmer-alpha (H , a name — not the deflection-angle α of later chapters) at the same rest-frame wavelength, every once-ionized oxygen ion produces the same [OII] doublet near 3727 Å, and every calcium ion in an old star's atmosphere absorbs at the same Ca II H & K pair near 3934/3969 Å. The rest wavelength is a physical constant of the transition — not a measurement of the object in front of you.

What *does* vary from object to object is where that fixed pattern shows up in the observed spectrum. If every line — from the reddest Balmer line to the bluest oxygen line — sits at the *same* multiplicative factor away from its rest wavelength, that object has a redshift. Measuring that one factor is the whole content of the next section.

A chapter-local notation note, made explicit because this book is careful about it elsewhere: this chapter writes wavelength as λ, the standard symbol in every spectroscopy text. That has nothing to do with the SMC tempering parameter of the same letter you will meet in Ch. 23 — the two never appear in the same equation, and λ means wavelength everywhere in this chapter and nowhere else in the book.

Measuring a redshift

Define the redshift operationally as

$$1 + z \equiv \frac{\lambda_{\text{obs}}}{\lambda_{\text{emit}}},$$

where λ_{emit} is a line's known rest-frame wavelength and λ_{obs} is where you find it in the measured spectrum. This is a definition, not yet a physical claim — whether the underlying cause is the expansion of space or a literal recession velocity is exactly the question Ch. 13 answers. For this chapter, treat z as a wavelength ratio you read off a spectrum, full stop.

In practice nobody eyeballs one line. A real pipeline (DESI's `redrock`, the tool behind every redshift in this section) carries a library of template spectra — quiescent galaxy, star-forming galaxy, quasar, star — and for each template scores, on a fine grid of trial redshifts, how well the redshifted template matches the observed spectrum. The redshift reported is the trial value that maximizes that score.

> **You already know this**
>
> Scoring a redshifted template against data and taking the arg-max over trial redshifts is the cross-correlation operation of Ch. 7, with the redshift playing the role of the lag. Matching the *whole* pattern of lines, not one feature, is what keeps the estimate honest — the spectroscopic analog of a matched filter rejecting a lone spurious peak — and it is why redshift catalogs carry a warning flag (`ZWARN`) for fits the pipeline itself does not trust.

A real pair, measured. Here is a real system recovered by exactly this kind of fit, matched to grade-A candidate DESI-004.5374+01.0382 in Hsu et al. (2025)'s Table 2. Two DESI fiber spectra, separated by under an arcsecond on the sky, carry measured redshifts

$$z_{\text{lens}} = 0.3266, \qquad z_{\text{src}} = 0.7497.$$

Feed the lens redshift into () with the Ca II K absorption line (rest wavelength 3933.66 Å — its depth traces old stars, so it shows up in a quiescent lens galaxy, not a star-forming background source):

$$\lambda_{\text{obs}} = 3933.66 \text{ Å} \times (1 + 0.3266) = 5218.47 \text{ Å}.$$

Do the same for the source with the [OII] doublet (rest wavelength 3727 Å, an emission feature of ionized gas — present in a star-forming galaxy, essentially absent in an old elliptical) at $z_{\text{src}} = 0.7497$:

$$\lambda_{\text{obs}} = 3727.42 \text{ Å} \times (1 + 0.7497) = 6521.90 \text{ Å}.$$

Both land inside an optical spectrograph's range with room to spare, which is why one exposure of one patch of sky yields both redshifts, from two entirely different atomic transitions. Try the line most people reach for first, Balmer-alpha (rest 6564.61 Å), on the *source* instead:

$$\lambda_{\text{obs}} = 6564.61 \text{ Å} \times 1.7497 = 11486.15 \text{ Å},$$

deep in the near-infrared, off the red end of a ground-based optical spectrograph entirely. That is precisely why a redshift pipeline searches a whole template library rather than a hard-coded line list: which lines are even observable depends on z itself.

The ratio that starts a discovery pipeline. Divide the two redshifts:

$$\frac{z_{\rm src}}{z_{\rm lens}} = \frac{0.7497}{0.3266} = 2.2954.$$

That ratio, and the threshold it clears, are not incidental to this example — they are the literal discovery criterion of Hsu et al. (2025)'s pairwise spectroscopic search (`reproductions/hsu-2025/05_run_full_fof.py:115`). Group every DESI fiber spectrum by sky position (a friends-of-friends match at 3), then keep any group whose maximum-to-minimum redshift ratio is at least 1.3. Two spectra that close together on the sky but that far apart in redshift are not one blended object — they are a foreground galaxy and a background galaxy along one line of sight, the geometric definition of a lens candidate. Run on the full DR1 catalog, the cut takes 28,425,963 raw fiber spectra down to 15,786,243 after basic quality cuts, and finally to 13,530 candidate groups (27,334 spectra) — a purely spectroscopic discovery channel that runs *before* any imaging classifier (Ch. 27) has seen a pixel.

Not every published "redshift" is this kind of measurement. A single-author paper on NISP spectroscopy of Euclid Q1 lenses (arXiv:2604.02726) was withdrawn at its sixth revision after it emerged that 385 of its 440 tabulated "deflector redshifts" — 87.5% of them — were not spectroscopic at all but *photometric* redshifts: estimates from broad-band colors, not from a resolved emission or absorption line. The paper's own validation against real (blind) spectroscopy recovered only about 35% of them correctly (`reproductions/lensjudge/parity/FINDINGS.md:91`). Calling a color fit a redshift measurement is exactly the substitution this section warns against: a photometric estimate is a guess dressed in the units of a measurement, and this one did not survive review.

σ_v from line width

A redshift measures where the *center* of a line sits: the bulk line-of-sight velocity of the whole galaxy. A velocity dispersion measures how *wide* the line is, and it comes from a different effect entirely: the stars inside the galaxy are not all moving at the galaxy's bulk velocity. Some orbit toward you, some away, with a spread of line-of-sight velocities σ_v around the mean. Each star's own light picks up its own small Doppler shift; sum the light of the whole population and a symmetric spread of shifts smears the line rather than displacing it.

For $v \ll c$ — every σ_v in this repo is a few hundred km/s, three orders of magnitude below c — the Doppler shift linearizes to

$$\frac{\Delta\lambda}{\lambda} = \frac{v}{c},$$

the same small-parameter linearization as Ch. 2's Taylor argument (Ch. 2), applied here to a relativistic formula instead of a lens equation. A line's intrinsic profile, convolved with a Gaussian kernel of standard deviation

$$\sigma_\lambda = \lambda_{\rm rest}\,\frac{\sigma_v}{c}$$

from (), and further convolved with the spectrograph's own instrumental resolution, is what a pipeline actually fits. Not read off a plot, for the same reason a redshift isn't: recovering a width against a comparable instrumental width needs a template and a model, not a ruler.

The scale of the effect, and why it needs a fit. Take the median velocity dispersion FastSpecFit recovers for the lens galaxies in Hsu et al. (2025)'s sample: $\sigma_v = 217.09$ km/s (the middle 68% of the measured population spans roughly 152.12–292.27 km/s). Relative to c,

$$\frac{\sigma_v}{c} = \frac{217.09}{299792.458} = 7.24 \times 10^{-4},$$

and on the same Ca II K line used above it broadens the line by only

$$\Delta\lambda = \lambda_{\rm rest}\,\frac{\sigma_v}{c} = 3933.66\ \text{Å} \times 7.24 \times 10^{-4} = 2.848\ \text{Å}.$$

Compare that to the *shift* computed in the last section for the same line: 5218.47−3933.66 = 1284.81 Å, about 450 times larger. That ratio is the whole reason a redshift is visible by eye on a plotted spectrum and a velocity dispersion is not: the shift moves a line across a large fraction of the visible band, while the width perturbs it by a few Ångstroms — comparable to the spectrograph's own resolution element. Recovering σ_v means fitting the line's shape against an instrumental-resolution template, which is exactly what a stellar-population code like FastSpecFit does.

A fitter's own trap. That fit does not always succeed, and how it fails is worth internalizing. Only 4,238 of the 13,530 candidate pairs — 31.3% — carry a σ_v that FastSpecFit itself trusts (`reproductions/hsu-2025/07_classify_einstein_dimple.py:145-153`): a positive inverse variance, `VDISP_IVAR > 0`. The remaining 9,292 either have no usable fit or return the fitter's own failure default of exactly 250.0 km/s with `VDISP_IVAR = 0` — a number that looks completely plausible for a massive galaxy and is, in fact, a placeholder the code emits when it gives up. There is no way to tell a real 250 km/s from a fake one except by checking the diagnostic that comes with it: exactly the discipline this guide's own numbers are held to, applied here to someone else's fitting code.

The payoff. Feed a trustworthy σ_v into the SIS Einstein-radius formula of Ch. 19 — the physical argument for *why* that formula works comes from galaxy dynamics, derived in Ch. 10 — and a spectrum alone gives you a mass estimate. For the 4,238 Hsu et al. pairs with a reliable σ_v, the median predicted Einstein radius is $\theta_{\rm E} = 0.6815''$, computed straight from catalog numbers, no image involved.

That is the payoff this chapter has been building toward. A spectroscopic redshift tells you two objects sit at different distances — not a blend, not a chance alignment, a genuine foreground-background pair. A spectroscopic velocity dispersion turns that pair into a mass estimate. An image without either number is a ring of light with an unknown cause: it could be a lens, a tidal tail, a face-on spiral, or two unrelated galaxies at the same redshift. That is the content of this chapter's destination sentence — a lens without redshifts is a picture, not a mass — and it is why an entire discovery channel (Hsu et al. 2025, and the DESI Foundry modeling papers downstream of it) exists purely to mine a spectroscopic catalog that DESI built for cosmology, not for lensing.

Connect to the repo

- `reproductions/hsu-2025/05_run_full_fof.py:40` sets `Z_RATIO_MIN = 1.3`; line `:115` computes `z_stats["z_ratio"] = z_stats["zmax"] / z_stats["zmin"]` and applies the cut. Run at full DR1 scale this is the 28.4M-fiber → 13,530-group funnel quoted above (GitHub).
- `reproductions/hsu-2025/07_classify_einstein_dimple.py:103` implements `theta_e_sis`, the same formula Ch. 19 derives; lines `:145-153` are the comment documenting FastSpecFit's `VDISP = 250.0` failure-mode cap, and the `VDISP_IVAR > 0` guard that catches it (GitHub).
- `reproductions/hsu-2025/data/classified_stats.json`, `dr1_stats.json`, and `xmatch_table2.json` carry every count and percentile quoted in this chapter's worked examples.
- `reproductions/lensjudge/tools/spectrum.py:32` (`sis_theta_e`) is an agentic grading tool computing this exact arithmetic live — a "does this pair make physical sense" check LensJudge runs during its own grading, not just a textbook formula (GitHub).
- `reproductions/lensjudge/parity/FINDINGS.md:91` documents the withdrawn NISP paper discussed above (GitHub).
- `reproductions/cikota-2023/README.md:118` is an honest converse example: that reproduction states plainly that it did **not** reproduce any spectroscopy, and uses the discovery paper's published redshifts only for a unit conversion, rather than quietly presenting them as its own measurement (GitHub).

Exercises

Exercise 12.1 — The ratio, by hand

Using $z_{\rm lens} = 0.3266$ and $z_{\rm src} = 0.7497$, compute $z_{\rm src}/z_{\rm lens}$ and state whether this DESI pair would survive Hsu et al. (2025)'s friends-of-friends discovery cut.

Solution

$0.7497/0.3266 = 2.2954$, comfortably above the threshold of 1.3. The pair survives — it is, in fact, one of the 13,530 groups the cut keeps at full DR1 scale.

Exercise 12.2 — Why Balmer-alpha doesn't work here

Compute where Balmer-alpha (rest wavelength 6564.61 Å) would land if you searched for it in the *source* spectrum at $z_{\rm src} = 0.7497$. Then explain why a redshift pipeline does not simply fail when one expected line falls outside the observable band.

Solution

$6564.61 \times 1.7497 = 11486.15$ Å , deep in the near-infrared — well outside where a ground-based optical spectrograph is sensitive, compared to the 5218–6522 Å range where the Ca II K and [OII] lines in this chapter's worked example actually landed. The pipeline does not fail, because it never depended on any one line: it cross-correlates a whole template (Ch. 7's operation) against the data, and at any given z some *other* line in the template — with a different rest wavelength — falls inside the band instead.

Exercise 12.3 — Shift vs. width, by the numbers

Using $\sigma_v = 217.09$ km/s on the Ca II K line, verify $\Delta\lambda_{\rm width} \approx 2.85$ Å. Compare it to the *shift* computed for the same line, $5218.47 - 3933.66$ Å. What is the ratio, and what does it tell you about why a redshift is visible by eye but a velocity dispersion is not?

Solution

$\Delta\lambda_{\rm width} = 3933.66 \times (217.09/299792.458) = 2.848$ Å . The shift is $5218.47 - 3933.66 = 1284.81$ Å. The ratio is about 450: the shift displaces a line across more than a thousand Ångstroms, while the width perturbs it by less than three — a scale comparable to the spectrograph's own resolution element. That is why z is legible on a plotted spectrum and σ_v requires fitting the line profile against an instrumental-resolution model.

Exercise 12.4 — The fitter's cap

9,292 of Hsu et al.'s 13,530 candidate pairs lack a trustworthy σ_v. What fraction of the *whole* sample is that? If an analysis used FastSpecFit's `VDISP` column without checking `VDISP_IVAR > 0`, why can't you simply look at the number 250.0 km/s and know it's a placeholder rather than a real measurement?

Solution

$9292/13530 = 0.687$, about 68.7% of the sample. You can't tell a real 250 km/s from the fitter's failure cap by inspecting the value alone — 250 km/s is a perfectly ordinary velocity dispersion for a massive elliptical (`reproductions/hsu-2025/07_classify_einstein_dimple.py:145-153`). The only way to distinguish them is the diagnostic that ships alongside the number, `VDISP_IVAR`: zero means the fit gave up and returned its template floor, not a measurement.

13. The expanding universe and redshift

This chapter answers a question every redshift number in this repository quietly assumes you can already read: what does z mean? Cikota's Einstein cross, the Carousel cluster's background sheet, Ch. 12's DESI fiber-pair search — every one of them reports z as if its meaning were obvious. By the end of this chapter you can say exactly what it asserts about the universe itself, derive Hubble's law from a single Taylor expansion — the same move Ch. 02 promised would recur through the whole book — and explain precisely why $H_0 = 70$ is not, on its own, an age or a speed limit.

> **What you can skip**
>
> Of the whole book, this chapter and the two after it (Ch. 14, Ch. 15) are the most detachable. The lens-modelling likelihood this repository actually optimises — the one Ch. 22 through Ch. 26 build up in full — is entirely angular: arcseconds in, arcseconds out, with no cosmological quantity anywhere in its gradient (`site/guide_src/cosmo.py:4`). Cosmology enters this repository's campaigns only where a physical, non-angular scale is required, and the money-number pipeline is not one of those places — see "Connect to the repo" below for the full, short list. If your only goal is $\gamma = 1.103$, skip straight to Ch. 16. Read this chapter instead before your first meeting with an astronomer: "the universe is expanding" is the one sentence every one of them assumes you already hold for the right reasons, and after this chapter, you will.

The scale factor

Cosmologists describe the universe's size with a single dimensionless function of time, the **scale factor** $a(t)$, normalized so that $a(t_0) \equiv 1$ **today**, at t_0. It carries no units and no meaning in isolation — only the *ratio* of a at two different times means anything, and that ratio is exactly what a measured redshift hands you, in the next section.

The picture underneath $a(t)$: lay a fixed grid over the universe, with galaxies (on average, ignoring their small individual peculiar motions) sitting at fixed grid coordinates for all time — their **comoving** positions, $\mathbf{x}_c$. Nothing on this grid moves. What changes is the ruler: the *physical* (proper) distance between two grid points at time t is

$$d_{\rm phys}(t) \;=\; a(t)\, d_c,$$

where d_c is the fixed comoving separation between them. At $t = t_0$, $a = 1$ and physical distance equals comoving distance — exactly why the normalization was chosen that way.

> **You already know this**
>
> A comoving coordinate is a fixed label — like a vertex stored in normalized device coordinates, unchanged by anything downstream. The scale factor $a(t)$ is the single global multiplier that turns those fixed labels into physical distances at time t, the same way a viewport transform turns NDC into pixels. The mesh — every galaxy's comoving position — never moves. Only the multiplier changes, uniformly, everywhere, at once.

$a(t)$'s actual functional form — whether it grows like $t^{2/3}$, accelerates, or does something else entirely — is set by how much matter and dark energy the universe holds, and deriving it is Ch.

14's job. This chapter needs only $a(t)$'s value today and its instantaneous rate of change right now — and that rate of change is exactly what "$H_0 = 70$" reports.

Hubble's law

Define the **Hubble parameter** as the log-derivative of the scale factor,

$$H(t) \;\equiv\; \frac{\dot{a}(t)}{a(t)} \;=\; \frac{d}{dt}\ln a(t).$$

This is exactly the quantity you would call a continuously-compounded growth rate in any other setting: the fractional change in size per unit time, evaluated instantaneously. $a(t)$ need not actually grow exponentially — $H(t)$ is a snapshot of its rate at t, not a claim about its long-run shape.

Taylor-expand $a(t)$ to first order around today, t_0, exactly as Ch. 02 does for every other hard object in this book:

$$a(t) \;\approx\; 1 \;+\; H_0\,(t - t_0), \qquad H_0 \equiv H(t_0),$$

using $a(t_0) = 1$. This is nothing but a derivative given a name: the slope of the line tangent to $a(t)$ at t_0.

Now put a comoving galaxy at fixed comoving distance d_c. Its proper distance at any nearby time is $d(t) = a(t)\,d_c$, so its recession velocity is

$$v(t) \;=\; \frac{d}{dt}\big[a(t)\,d_c\big] \;=\; \dot{a}(t)\,d_c \;=\; \frac{\dot{a}(t)}{a(t)}\,\big[a(t)\,d_c\big] \;=\; H(t)\,d(t).$$

Evaluated today, this is **Hubble's law**, $v = H_0\,d$. It is not an empirical fit stapled onto the expansion after the fact; it is ()'s linear term, restated as a velocity instead of a scale factor.

Everywhere this repository touches cosmology, it fixes $H_0 = 70$ km/s/Mpc (`site/guide_src/cosmo.py:29`). What else the model carries — Ω_m, dark energy — is Ch. 14's subject; only H_0 matters here. The units are the tell: velocity over distance is 1/time, a rate, not a speed and not a length. Two readings of that rate:

- **Hubble time**, $1/H_0 = 13.97$ Gyr — roughly the age of the universe, though not exactly; Ch. 14 derives why the two differ once matter and dark energy get a vote in $a(t)$'s history.
- **Hubble distance**, $c/H_0 = 4283$ Mpc — the characteristic length scale of the observable universe.

Equivalently, H_0 itself is 0.0716 per Gyr: at today's rate, every comoving distance grows about seven percent larger per billion years — the same $d\ln a/dt$ from above, in units a human timescale can hold.

Redshift as expansion

A photon is a wave, and as it crosses expanding space its wavelength is carried along by the same scale factor that carries a comoving distance:

$$\frac{\lambda_{\rm obs}}{\lambda_{\rm emit}} \;=\; \frac{a(t_{\rm obs})}{a(t_{\rm emit})} \;=\; \frac{1}{a(t_{\rm emit})},$$

using $a(t_{\rm obs}) = a(t_0) \equiv 1$, since "observed" means "now." Ch. 12 already showed you how z gets measured — as exactly this wavelength ratio, off a spectral line whose *rest* wavelength is known from the lab. Define it the same way here:

$$1 + z \;\equiv\; \frac{\lambda_{\rm obs}}{\lambda_{\rm emit}} \;=\; \frac{1}{a(t_{\rm emit})}, \qquad a(t_{\rm emit}) = \frac{1}{1+z}.$$

That is the whole content of "redshift is expansion": a measured z is a direct readout of how large the universe was when the light in front of you left its source, relative to how large it is now. Not a velocity through space — a size, then, compared against a size, now.

Four redshifts already sitting in this repository's worked examples translate directly:

system	z	$a(t_{\rm emit}) = 1/(1+z)$
Cikota+2023 Einstein-cross lens	0.271	0.787
Cikota+2023 Einstein-cross source	0.897	0.527
Carousel cluster lens (Ch. 15)	0.49	0.671
Carousel reference plane	1.432	0.411

The last row is the sharpest teaching example. It is tempting to read z as a velocity via $v \approx cz$ — and at low z that is a good approximation, since it is exactly ()'s linear term again, this time along the photon's path instead of a nearby galaxy's worldline. Push it past where the linear approximation holds and it breaks outright: read the Carousel's own $z_{\rm ref} = 1.432$ as "cz" and you get 429,303 km/s, i.e. $1.432\,c$ — faster than light. Nothing travels that fast, and nothing has to: no object is moving *through* space at all, space itself is stretching, and there is no speed limit on how fast a purely geometric distance can grow — the same way there is no upper bound on how fast a resized window can grow even though nothing drawn inside it is "moving." The correct statement was never a velocity. It is $a(t_{\rm emit}) = 0.411$: the universe was 41 percent its current size when that light left.

Connect to the repo

- `site/guide_src/cosmo.py:29` fixes the repo-wide choice, `COSMO = FlatLambdaCDM(H0=70, Om0=0.3)` — every number in this chapter that uses H_0 uses exactly this value.
- `site/guide_src/cosmo.py:4` states the audit this chapter's skip-box leans on: the lens-modelling likelihood is entirely angular, and cosmology enters campaigns in this repository only at a short, enumerated list of places:
  - `reproductions/hsu-2025/07_classify_einstein_dimple.py:50` — $\theta_{\rm E}$ from σ_v, the calculation Ch. 19 derives and Cikota's Einstein cross exercises.
  - `reproductions/sheu-2024b/04_setup_multiplane.py:35` — $\Sigma_{\rm cr}$ and $M(<\theta_{\rm E})$ for the Carousel cluster, worked in full in Ch. 15.

  - `reproductions/sheu-2023/05_lightcurve_salt3.py:57` — a lensed supernova's Hubble-residual distance modulus, the one place in this repository H_0 sets an actual physical distance rather than an angle.
- `reproductions/claude-giga-lens/cgl/euclid_io.py:54` — the money-number pipeline itself fixes $z_l = 0.5$, $z_s = 1.0$ as inert defaults, "only the angular Einstein radius is meaningful." Nothing in that pipeline's gradient depends on either number — this chapter's skip-box claim, confirmed from the campaign that matters most.

Exercises

Exercise 13.1 — Hubble's law, both directions

Starting from (), derive $v = H_0 d$ for a nearby comoving galaxy (two lines of algebra suffice). Then use it to predict the recession velocity of a galaxy at $d = 100$ Mpc, and invert $v \approx cz$ to get the redshift that recession velocity implies.

Solution

Differentiate $d(t) = a(t)\, d_c$: $\dot{d} = \dot{a}\, d_c = (\dot{a}/a)\,(a\, d_c) = H(t)\, d(t)$. At $t = t_0$, $H(t_0) = H_0$ and $d(t_0) = d$, so $v = H_0 d$.

At $d = 100$ Mpc: $v = 70 \times 100 = 7000$ km/s. Inverting $v \approx cz$: $z \approx v/c \approx 0.0233$. This is the same linear regime the naive-cz trap in "Redshift as expansion" breaks out of once z stops being small.

Exercise 13.2 — Two scale factors you will need again

Compute $a(t_{\text{emit}}) = 1/(1+z)$ for $z = 0.5$ and $z = 2.0$. Ch. 15 uses exactly this pair to show that angular-diameter distances do not subtract. State in one sentence what each scale factor means physically.

Solution

$a(0.5) = 1/1.5 = 0.667$. $a(2.0) = 1/3 = 0.333$. The $z = 0.5$ light left when the universe was two-thirds its current size; the $z = 2.0$ light left when it was one-third — a factor of two in redshift is not a factor of two in "how far back," because a and z are related by $1/(1+z)$, not linearly.

Exercise 13.3 — Why c/H_0 has units of length

H_0 is quoted in km/s/Mpc. Show, by unit algebra alone, that c/H_0 has units of length, then compute it and compare it to the Hubble distance quoted above.

Solution

$[c]/[H_0] = (\text{km/s})/(\text{km s}^{-1}\,\text{Mpc}^{-1}) = \text{Mpc}$ — the km/s cancels completely, leaving a pure length. Numerically, $c/H_0 = 4283$ Mpc, matching the Hubble distance quoted in "Hubble's law" above. Run the same cancellation on $1/H_0$ instead and what is left is a pure time — the Hubble time.

Exercise 13.4 — The naive-cz trap, generalised

The Carousel's reference redshift, read as cz, gave $1.432\,c$ — faster than light. At roughly what redshift does the naive Newtonian reading $v = cz$ first predict a superluminal recession, and why does crossing that threshold not violate special relativity?

Solution

$v = cz$ exceeds c as soon as $z > 1$ — no computation needed beyond noticing $cz > c \iff z > 1$. Both the Carousel's reference plane ($z_{\text{ref}} = 1.432$) and the naive-cz speed derived from it (429,303 km/s) sit past that threshold. Special relativity bounds the speed of anything moving *through* space; it says nothing about how fast a purely geometric distance between two points can grow while nothing moves through the space in between. $v = cz$ is only ever the low-z approximation to $a(t_{\text{emit}}) = 1/(1+z)$ — the actual, always-valid statement — so its failure at high z is the approximation's fault, not relativity's.

14. FRW and the Friedmann equations

Chapter 13 gave you the scale factor $a(t)$ and showed that redshift is a record of how much a has grown since a photon left its source. This chapter gives you the equation that decides how $a(t)$ actually evolves — the Friedmann equation — and the handful of numbers, Ω_m and Ω_Λ, that fix its trajectory. By the end you can write down `FlatLambdaCDM(H0=70, Om0=0.3)`, the one line this repository's cosmology begins and ends with, and say exactly what each of its two arguments asserts, including the third number it fixes that you never typed.

> **What you can skip**
>
> If you already have a GR or cosmology course behind you: skip the Newtonian derivation of the Friedmann equation below (you already have the real one from Einstein's field equations) and the review of $F = GMm/r^2$. Go straight to The density parameters, where the only repo-specific content lives — what `FlatLambdaCDM(70, 0.3)` fixes, and the fact that $\Omega_m + \Omega_\Lambda = 1$ is a *definition*, not a measurement (the measurement is that $\Omega_k = 0$). If you're willing to take the Friedmann equation on faith entirely, skip to Connect to the repo: none of this chapter touches γ, and that fact is worth knowing on its own.

The FRW metric

A metric is a distance formula: a rule for turning a small coordinate separation into a physical (proper) length. In flat 2-D Euclidean space you already know it as $ds^2 = dx^2 + dy^2$ — Pythagoras, applied to infinitesimally close points. General relativity's only real complication is that the coefficients in that formula are allowed to depend on where, and when, you are.

> **You already know this**
>
> A metric is a (possibly position- or time-dependent) quadratic form that turns coordinate differences into a distance. That is exactly the job a covariance-weighted norm does when you whiten a feature vector, and it is exactly the job the mass matrix does in Ch. 23's HMC sampler, which turns a gradient into a step in parameter space. The FRW metric below is the same kind of object, applied to spacetime.

The **cosmological principle** — on large scales the universe looks the same at every point (homogeneous) and the same in every direction from that point (isotropic) — is an assumption, well tested but unprovable in general. It is also an enormous constraint: it says the distance formula for the whole universe can depend on time through only *one* function and on space through only *one* constant. That function is $a(t)$. The constant is a curvature index k. The result is the **Friedmann–Robertson–Walker (FRW) metric**:

$$ds^2 = -c^2\,dt^2 + a(t)^2\left[\frac{dr^2}{1-kr^2} + r^2\,d\Omega^2\right]$$

Here t is cosmic time — the clock every observer at rest relative to the overall expansion agrees on — and r is a *comoving* radial coordinate: fixed for an object that moves only with the Hubble flow, exactly the coordinate whose separation $a(t)$ stretches into a physical one in Ch. 13. The term $d\Omega^2$ is the ordinary solid-angle piece from spherical coordinates — the same combination you'd get unrolling latitude and longitude on a globe. This guide does not write out its two angles

individually: those two symbols are reserved elsewhere for the lens-plane position (Ch. 17), and this repo's overloading habit is exactly what this notation is trying to avoid repeating.

k takes one of three values (after an overall rescaling of r): $k = -1$ (open, hyperbolic, infinite volume), $k = 0$ (flat, ordinary Euclidean geometry), or $k = +1$ (closed, a 3-sphere, finite volume). This repository never considers the first or the third: every cosmology object it builds is `FlatLambdaCDM`, which hard-codes $k = 0$ and does not expose a curvature argument at all. Comoving coordinates are the ones a galaxy catalog is built in; the *proper* separation between two comoving points at a fixed time is $a(t)$ times their (constant) comoving separation — the same idea Ch. 13 introduced for a single photon's wavelength, now embedded in an actual distance formula. Ch. 15 turns this into the angular-diameter distance the lensing code calls directly.

Deriving this metric from Einstein's field equations requires the tensor calculus this guide deliberately skips. You do not need that derivation to use the metric — you need it to know what makes $a(t)$ move, which is the subject of the next section.

The Friedmann equation, derived as energy conservation

You can get the right equation for $\dot{a}$ without any tensor calculus, using nothing but Newtonian energy conservation — a genuine, standard trick, not a hand-wave, though it comes with a caveat stated at the end of this section.

Pick any comoving observer as an origin and consider a sphere around them of fixed comoving radius r_c, small enough that space inside it looks uniform. Its physical radius is $R(t) = a(t)\, r_c$, and by homogeneity it encloses a uniform matter density $\rho(t)$, hence a mass $M(R) = \frac{4}{3}\pi R^3 \rho$. Put a test particle of mass m on its surface. By Newton's shell theorem the sphere pulls on that particle exactly as if all its mass sat at the center, so ordinary energy conservation gives

$$\frac{1}{2} m \dot{R}^2 - \frac{GM(R)m}{R} = E$$

with E constant — nothing is adding or removing energy from the particle. Substituting $M(R)$ and writing the (still-constant) E in the suggestive form $E \equiv -\frac{1}{2} m k c^2 r_c^2$:

$$\frac{1}{2}\dot{R}^2 - \frac{4}{3}\pi G \rho R^2 = -\frac{1}{2} k c^2 r_c^2$$

Divide through by $\frac{1}{2}R^2 = \frac{1}{2}a^2 r_c^2$. The comoving radius r_c — which was arbitrary, the radius of a sphere you drew, not a physical scale — cancels completely, because $\dot{R}/R = \dot{a}/a$ for *any* r_c. That cancellation is not a bookkeeping convenience; it is required, because the resulting law has to hold for every comoving observer, not just the one who happens to sit at the center of your particular sphere. What survives is the first **Friedmann equation**:

$$H^2 \equiv \left(\frac{\dot{a}}{a}\right)^2 = \frac{8\pi G}{3}\rho - \frac{kc^2}{a^2} \tag{2}$$

with the same curvature index k that appears in the metric. A denser universe (ρ larger) expands faster; positive curvature ($k = +1$) is a literal drag term that can eventually halt and reverse the expansion, exactly as positive total energy versus a negative one decides whether a thrown ball escapes to infinity or falls back.

The honest caveat. This derivation is Newtonian, and Newtonian gravity has no curvature and no dynamical role for pressure. That (2) comes out *exactly* right is a genuine feature of general relativity applied to a homogeneous, isotropic spacetime (a cousin of Birkhoff's theorem), not a coincidence you should distrust — but it is also not a proof, and it does not extend to the *second* Friedmann equation, the one governing $\ddot{a}$, which needs pressure to source gravity. Pressure gravitating at all has no Newtonian analogue; it is a genuinely relativistic effect, in the same family as the factor of two you'll meet in Ch. 16 when light bends twice as far as Newton predicts. This repo never needs the second equation: it treats dark energy as a fixed extra density, not as the solution of a dynamical equation, which is exactly the next paragraph.

Adding dark energy and matter dilution. Treat the cosmological constant Λ as an extra density component that never dilutes, $\rho_\Lambda \equiv \Lambda c^2/(8\pi G) = \text{const}$ — this capital Λ is unrelated to the lowercase tempering parameter λ you'll meet in Ch. 23; case is the only thing distinguishing them, so read carefully. Ordinary matter, by contrast, does dilute: mass conservation in a fixed comoving sphere, $M = \rho_m(t) \cdot \frac{4}{3}\pi R(t)^3 = \text{const}$ (nothing creates or destroys matter, only accumulation the way Ch. 3 already had you doing over a volume), together with $R \propto a$, forces $\rho_m(t) \propto a(t)^{-3}$. Put both pieces into (2) with $\rho = \rho_m + \rho_\Lambda$.

A reference density. At any moment, relabel the right-hand side's overall scale as a density: $\rho_{\text{crit}}(t) \equiv \dfrac{3H(t)^2}{8\pi G}$. This is not a special density in the physics — it is what ρ would have to equal for $k = 0$ to hold exactly at that instant, given the H that instant actually has. It is the natural yardstick for the next section, and it is already enough to compute two numbers with only H_0 in hand. This repository fixes $H_0 = 70$ km/s/Mpc in every one of the three places cosmology appears (the next section names them), giving

$$\rho_{\text{crit},0} = \frac{3H_0^2}{8\pi G} \approx 1.360 \times 10^{11}\ M_\odot/\text{Mpc}^3$$

— roughly one solar mass smeared over a cube 634 light-years on a side, which is the density the entire observable universe would need, on average, to be spatially flat. Ch. 13 already showed you the companion length and time scales that come from H_0 alone, the Hubble distance and Hubble time; $\rho_{\text{crit},0}$ is the third such scale, and it is the one that needs a density, not just a rate, to define.

Omega_m, Omega_Lambda, and what flat means

Normalize any density by ρ_{crit} and you get a dimensionless **density parameter**: $\Omega_m \equiv \rho_{m,0}/\rho_{\text{crit},0}$, $\Omega_\Lambda \equiv \rho_\Lambda/\rho_{\text{crit},0}$, and, folding the curvature term into the same units, $\Omega_k \equiv -kc^2/(H_0^2 a_0^2)$ (with the convention $a_0 \equiv a(t_0) = 1$ today). Evaluate (2) at $t = t_0$ and divide every term by H_0^2:

$$\Omega_m + \Omega_\Lambda + \Omega_k = 1$$

This identity costs you nothing: it is what dividing the Friedmann equation by H_0^2 *always* produces, for any universe, whatever its actual curvature. It is not evidence that the universe is flat. **Flatness is the separate, empirical claim that $\Omega_k = 0$** — that curvature's contribution happens to vanish. Once you assert that, the identity above collapses to $\Omega_m + \Omega_\Lambda = 1$, and specifying Ω_m alone fixes Ω_Λ for free.

What `FlatLambdaCDM(70, 0.3)` actually asserts. Three independent places in this repository build the identical cosmology object: `FlatLambdaCDM(H0=70, Om0=0.3)`, at

`reproductions/hsu-2025/07_classify_einstein_dimple.py:50` (source), `reproductions/sheu-2024b/04_se` (source), and `reproductions/sheu-2023/05_lightcurve_salt3.py:57` (source). That one line asserts three numbers, only one of which you typed:

$H_0 = 70$ and $\Omega_{m,0} = 0.3$ are the two you chose. `astropy`'s `FlatLambdaCDM` class does not even accept a curvature argument — it forces $\Omega_{k,0} = 0$ by construction — so $\Omega_{\Lambda,0}$ is not independently set, it is *computed*, as $1 - \Omega_{m,0} - \Omega_{k,0} = 0.7$. Check the sum yourself and you get exactly 1, as the identity above guarantees it must.

With Ω_m and Ω_Λ in hand, (2) becomes a concrete, checkable function of redshift. Using $1+z = 1/a$ (Ch. 13) and the matter-dilution result from the previous section,

$$H(z)^2 = H_0^2 \left[\Omega_m (1+z)^3 + \Omega_\Lambda\right]$$

Build the right-hand side yourself from nothing but $H_0 = 70$, $\Omega_m = 0.3$, $\Omega_\Lambda = 0.7$, at $z = 0.5$, and you should get

in km/s/Mpc. That is not a number this guide asserts and asks you to trust — it is `astropy`'s own `FlatLambdaCDM.H(0.5)`, computed independently from the metric machinery the library actually runs, and the two agree to machine precision:

The same H_0, Ω_m, Ω_Λ also fix the age of the universe — the time integral of $1/[(1+z)H(z)]$ back from today to $a = 0$ — at

billion years. None of the three numbers above, nor the age, nor $\rho_{\mathrm{crit},0}$, ever enters the lens-modelling likelihood this guide spends Part V decoding: `site/guide_src/lensing.py` runs entirely in arcsec, with no distances at all, and this chapter's whole apparatus enters this repository in exactly the three files cited above — Einstein radii from velocity dispersions, Σ_{cr} and enclosed mass, and one supernova's distance modulus. The money number $\gamma_{\mathrm{binned}}(\mathrm{corr,\ low}) = 1.103 \pm 0.008$ does not know that $H_0 = 70$; it would come out identical if this repository had asserted $H_0 = 67$ instead. That scoping is worth holding onto precisely because it tells you where a wrong cosmology *would* and would not bite.

Connect to the repo

`site/guide_src/cosmo.py:29` (source) is the one line this chapter's whole worked example runs against: `COSMO = FlatLambdaCDM(H0=70, Om0=0.3)`. The module's own docstring, starting at `site/guide_src/cosmo.py:5` (source), makes the same scoping argument as the paragraph above — cosmology enters this repository in exactly three places — and is the reason this guide's cosmology Part is three chapters, not ten: that proportion is itself a finding about where to spend your attention, not an omission.

This repository never builds its own Friedmann-equation solver. All three files construct `astropy.cosmology.FlatLambdaCDM`; two of them call `.angular_diameter_distance` directly — `reproductions/hsu-2025/07_classify_einstein_dimple.py:50` and `reproductions/sheu-2024b/04_setup_` cited above — and the third, `reproductions/sheu-2023/05_lightcurve_salt3.py:57`, hands the object straight to `sncosmo`, which draws the supernova's luminosity distance from it internally. What this chapter buys you is the ability to read `.H`, `.age`, `.critical_density0`, and `.angular_diameter_distance` — the four methods this chapter's own worked example (`ch14_frw_friedmann` in `site/guide_src/worked_examples.py`) exercises directly — as physics rather than as API surface: `.H(z)` is (2) evaluated at a redshift, `.age(0)` is that same equation

integrated back to $a = 0$, and `.critical_density0` is the reference density this chapter derived from H_0 alone. Ch. 15 is next: it turns the same `COSMO` object into the two distances, $D_{\mathrm{d}}, D_{\mathrm{s}}, D_{\mathrm{ds}}$, that this repository's lensing calculations actually consume, and into the non-additivity gotcha that bites everyone once.

Exercises

Exercise 14.1 — Dimensional analysis on the Friedmann equation

Confirm, from units alone, that $\frac{8\pi G}{3}\rho$ in (2) has the same units as H^2. You will need Newton's constant's units, $[G] = \mathrm{m}^3\,\mathrm{kg}^{-1}\,\mathrm{s}^{-2}$, and a mass density's units, $[\rho] = \mathrm{kg}\,\mathrm{m}^{-3}$.

Solution

$$[G][\rho] = \left(\mathrm{m}^3\,\mathrm{kg}^{-1}\,\mathrm{s}^{-2}\right)\left(\mathrm{kg}\,\mathrm{m}^{-3}\right) = \mathrm{s}^{-2}$$

which is exactly $[H]^2$, since $H = \dot{a}/a$ is a rate (one over a time). The $8\pi/3$ is dimensionless, so it does not affect the check. The kc^2/a^2 term must independently carry units of s^{-2} as well: $[c^2] = \mathrm{m}^2\mathrm{s}^{-2}$, so k itself must carry units of inverse length squared unless a is defined with units of length (some textbooks set $a_0 \neq 1$ precisely to make k dimensionless ± 1 or 0; this guide follows the convention $a_0 = 1$, in which case k silently absorbs the missing length dimension). Either convention gives the same physics; just be consistent about which one a given equation is using.

Exercise 14.2 — Flatness is a measurement, not an identity

Explain, in a sentence or two, why $\Omega_m + \Omega_\Lambda + \Omega_k = 1$ can be stated with total confidence before a single measurement is made, while $\Omega_k = 0$ cannot.

Solution

The sum identity falls straight out of *dividing the Friedmann equation by* H_0^2 — it is true by the definitions of Ω_m, Ω_Λ, and Ω_k as H_0^2-normalized pieces of an equation that already balances. It holds for any values those three symbols take, in any universe, flat or not. $\Omega_k = 0$ is a different kind of claim entirely: it says the *actual* curvature term is zero, which is a statement about which universe we are in, checkable only by measuring Ω_m, Ω_Λ (or $H(z)$ directly) independently and seeing whether they leave any room for a curvature term. `FlatLambdaCDM` doesn't measure this — it assumes it, by refusing to expose a curvature argument at all.

Exercise 14.3 — H(z) at a redshift this chapter didn't show you

Using only $H_0 = 70$, $\Omega_m = 0.3$, $\Omega_\Lambda = 0.7$, and $H(z)^2 = H_0^2\left[\Omega_m(1+z)^3 + \Omega_\Lambda\right]$, compute $H(z=1)$ by hand. Then check your answer against `astropy`'s own value.

Solution

$$H(1)^2 = 70^2\left[0.3 \cdot 2^3 + 0.7\right] = 4900 \times 3.1 = 15190$$

$$H(1) = \sqrt{15190} \approx 123.25 \text{ km/s/Mpc}$$

which matches `cosmo.COSMO.H(1.0)` to machine precision:

Notice H nearly *doubled* between $z = 0.5$ (91.6 km/s/Mpc, in the main text) and $z = 1$ (123.25 km/s/Mpc) — the matter term $\Omega_m(1+z)^3$ is starting to dominate over the constant Ω_Λ term as you look back in time, exactly as $\rho_m \propto a^{-3}$ says it must.

Exercise 14.4 — Where pressure would have to enter

This chapter's derivation of (2) never once used pressure, and it doesn't need to: the equation it produces is exactly right. Where, physically, would you expect a Newtonian argument like this one to start failing if you tried to extend it to the *second* Friedmann equation (the one for $\ddot{a}$)?

Solution

Newton's shell theorem and $F = GMm/r^2$ know about mass, full stop — pressure does not appear anywhere in Newtonian gravity as a source of attraction. In general relativity it does: the source of gravity is (loosely) $\rho + 3p/c^2$, not ρ alone, so a medium under positive pressure gravitates *more*, and a negative-pressure component (which is exactly what makes something behave like a cosmological constant) can drive $\ddot{a}$ positive — accelerated expansion — with no Newtonian counterpart at all. This is the same family of "GR adds a term Newton has no room for" as the factor of two in Ch. 16, where light bends twice as far as a naive Newtonian calculation predicts. Neither this chapter nor this repository ever needs the pressure-sourced equation explicitly — dark energy is supplied as a fixed density, not derived from an equation of state — but it is worth knowing where the Newtonian trick this section leaned on would have quietly stopped working.

15. Distances that do not add, and $\Sigma_{\rm cr}$

Chapter 14 gave you one object, `FlatLambdaCDM(H0=70, Om0=0.3)`, and the equation that makes it tick. This chapter turns that object into distances you can act on — and there turn out to be three of them, not one, because "how far away" depends on whether you mean light-travel geometry, angular size, or flux. You will derive all three from the same flat FRW metric Chapter 14 wrote down, meet the one operation on them that is silently illegal — subtracting two angular-diameter distances — and use the survivor, $\Sigma_{\rm cr}$, to reproduce this repository's own published numbers for a real cluster lens: its critical surface density and the mass enclosed inside its Einstein radius, both computed from nothing but two redshifts and five lines of code.

> **What you can skip**
>
> If comoving, angular-diameter, and luminosity distance are already second nature — including the fact that they differ by explicit factors of $(1+z)$, not by anything mysterious — skip **Three distances, one universe** and go straight to **D_ds != D_s - D_d**. That section is not a subtle GR effect; it is an algebra trap this repository's own code has to dodge every time it builds a lens model, and it is worth reading even if you could write down $D_{\rm A}(z)$ from memory. If you also already know $\Sigma_{\rm cr}$, skip straight to **The Carousel**, where the payoff is two real, checkable numbers.

Three distances, one universe

In a flat, static, Euclidean space "distance" needs no qualifier. In an expanding one it does, because the three operations you might use a distance for — timing a light ray, measuring an angular size, and measuring a flux — stop agreeing with each other once redshift is not tiny. This section derives the three you need, in order, each one built on the last.

Comoving distance. Ch. 14 wrote down the FRW metric; this repository only ever uses its flat ($k=0$) case, where the radial part simplifies to $ds^2 = -c^2dt^2 + a(t)^2\left[dr^2 + r^2\,d\Omega^2\right]$. A photon travels on a null geodesic, $ds^2=0$, and along a purely radial path ($d\Omega=0$) that forces $c\,dt = a(t)\,dr$. Define the **comoving distance** as the coordinate distance such a photon covers:

$$D_{\rm C}(z) \;\equiv\; \int_{t_e}^{t_0} \frac{c\,dt}{a(t)}$$

Change the integration variable from cosmic time to redshift using Ch. 13's $a(t_e) = 1/(1+z)$ and Ch. 14's $H \equiv \dot{a}/a$: differentiating $a=(1+z)^{-1}$ gives $da/dz = -(1+z)^{-2}$, and $dt = da/\dot{a} = da/(aH)$, so $dt = -dz/[(1+z)H(z)]$. Absorbing the sign by integrating from 0 to z instead of t_e to t_0:

$$D_{\rm C}(z) \;=\; c\int_0^{z} \frac{dz'}{H(z')}$$

That is Ch. 14's Friedmann equation, integrated along the line of sight. `astropy` performs () numerically — there is no closed form once both Ω_m and Ω_Λ are nonzero — and its `comoving_distance` method is exactly this integral. At $z=2.0$:

$$D_{\rm C}(2.0) \approx 5179.862 \text{ Mpc}$$

Angular-diameter distance. The metric's transverse piece, $a(t)^2 r^2\, d\Omega^2$, is a genuine 2-sphere of *physical* (proper) radius $a(t)\, r$ at fixed t — this is Ch. 13's "proper distance $= a(t)\times$ comoving distance" rule, now applied to the transverse direction instead of the radial one. An object of physical transverse size ℓ, sitting at comoving radius r at the moment t_e its light was emitted, therefore subtends $\delta\theta = \ell/[a(t_e)\, r]$ — plain Euclidean geometry on a sphere of that radius, nothing relativistic beyond the metric itself. Define $D_{\rm A} \equiv \ell/\delta\theta = a(t_e)\, r$. In a flat universe the comoving radial coordinate r *is* $D_{\rm C}(z)$ — no curvature-dependent sin or sinh correction, since this repository never leaves $k = 0$ — so, using $a(t_e) = 1/(1+z)$:

$$D_{\rm A}(z) \;=\; \frac{D_{\rm C}(z)}{1+z}$$

This is `astropy`'s `.angular_diameter_distance`, and the one function this repository actually calls: `cosmo.d_a` at `site/guide_src/cosmo.py:32` (source). At $z = 2.0$: $D_{\rm A}(2.0) = D_{\rm C}(2.0)/3 \approx 1726.621$ Mpc.

Luminosity distance. Put a source of luminosity L at comoving distance $D_{\rm C}(z)$, radiating isotropically. By the time its light is observed it has spread over a sphere of *physical* area $4\pi D_{\rm C}(z)^2$ (using $a_0 \equiv a(t_0) = 1$, today). Ordinary inverse-square dilution alone would give $F = L/[4\pi D_{\rm C}(z)^2]$, but two more factors of $(1+z)$ cut the observed flux further: each photon's energy is itself redshifted, $E_{\rm obs} = E_{\rm emit}/(1+z)$, and the arrival *rate* of photons is time-dilated by the same factor — photons emitted a proper time δt_e apart arrive spaced by $\delta t_e\,(1+z)$. Combining all three:

$$F = \frac{L}{4\pi D_{\rm C}(z)^2 (1+z)^2}$$

Flux is *defined* via $F \equiv L/(4\pi D_{\rm L}^2)$, so matching the two expressions gives $D_{\rm L}(z) = (1+z)\, D_{\rm C}(z) = (1+z)^2\, D_{\rm A}(z)$. At $z = 2.0$: $D_{\rm L}(2.0) = 9 \times 1726.621 \approx 15539.586$ Mpc

— matching `astropy`'s own `luminosity_distance` to better than a millimeter in 15.5 billion parsecs:

$D_{\rm L}$ is what a Type Ia supernova's distance modulus, $m - M = 5\log_{10}(D_{\rm L}/10\,{\rm pc})$, actually consumes — the third and final place cosmology enters this repository, `reproductions/sheu-2023/05_lightcurve_salt3.py` (source), a lensed-supernova campaign this guide does not otherwise follow. Everything else in this guide — every $\theta_{\rm E}$, every $\Sigma_{\rm cr}$ — uses only $D_{\rm A}$, under the repository's own subscripted names $D_{\rm d}$ (to the lens), $D_{\rm s}$ (to the source), and $D_{\rm ds}$ (between them) — never $D_{\rm s} - D_{\rm d}$, which is the next section's whole point.

D_ds != D_s - D_d

State it as plainly as the notation contract this guide follows demands: $D_{\rm ds}$ is **not** $D_{\rm s} - D_{\rm d}$. Angular-diameter distances do not add, and the reason is entirely algebraic, not a mystery of curved spacetime — you already have every piece needed to derive it.

The general angular-diameter distance between two redshifts, in a **flat** universe, is

$$D_{\rm A}(z_1, z_2) \;=\; \frac{D_{\rm C}(z_2) - D_{\rm C}(z_1)}{1+z_2}$$

— the formula `astropy`'s `angular_diameter_distance_z1z2` evaluates for `FlatLambdaCDM`, and exactly what `cosmo.d_ds` at `site/guide_src/cosmo.py:37` (source) calls. Two things about () are worth noticing before touching a number. First, it reduces correctly at $z_1 = 0$ (Exercise 15.2 makes you check this): $D_{\mathrm{A}}(0, z_2) = D_{\mathrm{C}}(z_2)/(1 + z_2) = D_{\mathrm{A}}(z_2)$, exactly (). Second — and this is the trap — D_{ds} divides by $(1 + z_s)$ *once*, applied to the *difference* of two comoving distances. It does not divide $D_{\mathrm{C}}(z_l)$ by $(1 + z_l)$ and $D_{\mathrm{C}}(z_s)$ by $(1 + z_s)$ separately and then subtract. But that second, wrong operation is exactly what $D_{\mathrm{s}} - D_{\mathrm{d}}$ is:

$$D_{\mathrm{s}} - D_{\mathrm{d}} \;=\; \frac{D_{\mathrm{C}}(z_s)}{1 + z_s} - \frac{D_{\mathrm{C}}(z_l)}{1 + z_l}$$

> **You already know this**
>
> Two quantities that were each independently rescaled by a *different* factor do not combine correctly under subtraction — this is the same reason you cannot take two activations that were each standardized by a different layer's own running mean and variance, subtract them, and expect the result to be meaningfully normalized. D_{d} and D_{s} are each already divided by their *own* $(1 + z)$; the fix is to undo that per-term rescaling — work in D_{C}, which really is additive — and apply a single, shared rescaling only at the end.

Subtracting () and the naive expression above, the $D_{\mathrm{C}}(z_s)$ terms cancel identically, leaving a closed form entirely in terms of the lens distance:

$$D_{\mathrm{ds}} - (D_{\mathrm{s}} - D_{\mathrm{d}}) \;=\; D_{\mathrm{C}}(z_l)\left[\frac{1}{1 + z_l} - \frac{1}{1 + z_s}\right] \;=\; D_{\mathrm{d}}\,\frac{z_s - z_l}{1 + z_s}$$

(using $D_{\mathrm{C}}(z_l) = D_{\mathrm{d}}(1 + z_l)$ in the last step). Now the numbers, for $z_l = 0.5$, $z_s = 2.0$ — the same pair Ch. 13's exercises already flagged as this chapter's test case, chosen specifically because $(1+z)$ is nowhere near 1 for either redshift:

$$D_{\mathrm{d}} \approx 1259.084 \text{ Mpc}, \qquad D_{\mathrm{s}} \approx 1726.621 \text{ Mpc}$$

The correct D_{ds}, from () or equivalently `cosmo.d_ds(0.5, 2.0)`:

$$D_{\mathrm{ds}} \approx 1097.079 \text{ Mpc}$$

Build it the long way, from the comoving distances directly — $D_{\mathrm{C}}(0.5) \approx 1888.625$ Mpc and $D_{\mathrm{C}}(2.0) \approx 5179.862$ Mpc, subtract, then divide once by $1 + z_s = 3$:

$$\frac{5179.862 - 1888.625}{3} \approx 1097.079 \text{ Mpc}$$

and it lands on the same number to machine precision:

The naive subtraction, by contrast:

$$D_{\mathrm{s}} - D_{\mathrm{d}} \approx 467.537 \text{ Mpc}$$

— less than half of $D_{\rm ds}$, a factor of

2.3465 off, silently. The closed-form gap derived above, $D_{\rm d}\,(z_s - z_l)/(1+z_s) = 1259.084 \times 1.5/3 \approx 629.542$ Mpc

matches the actual gap between the two numbers above exactly:

This is not a rounding error or a subtle relativistic correction to chase down — it is what happens whenever you subtract two numbers that were each divided by something different before you got them.

Sigma_crit: the natural unit of surface density

Convergence, $\kappa \equiv \Sigma/\Sigma_{\rm cr}$, is the surface mass density Σ of a lens measured in units of a reference density $\Sigma_{\rm cr}$ built entirely from c, G, and the three distances of the previous section:

$$\Sigma_{\rm cr} \;=\; \frac{c^2}{4\pi G}\,\frac{D_{\rm s}}{D_{\rm d} D_{\rm ds}}$$

Dimensional analysis alone forces this shape to be plausible before you know anything about *why* it is the right one: $[c^2/G] = (\mathrm{m^2\,s^{-2}})/(\mathrm{m^3\,kg^{-1}\,s^{-2}}) = \mathrm{kg/m}$, and the distance ratio $D_{\rm s}/(D_{\rm d}D_{\rm ds})$ carries units $\mathrm{m/m^2} = \mathrm{m^{-1}}$, so the product is $\mathrm{kg/m^2}$ — a surface density, exactly as the name promises. Dimensional analysis cannot tell you *why* this particular combination of distances is the physically correct one, only that whatever the correct formula turns out to be, it had better have this shape — the actual physics (the point-mass deflection law and the geometry of a lens, source, and observer in a line) is Ch. 16–Ch. 17's job, and the fact that $\kappa = 1$ marks the boundary between one image and several is Ch. 19's. What matters here is narrower: $\Sigma_{\rm cr}$ is a well-defined, computable number the moment you have $D_{\rm d}$, $D_{\rm s}$, and $D_{\rm ds}$ in hand, and — because $D_{\rm ds}$ appears in the denominator — it inherits the previous section's non-additivity trap directly: get $D_{\rm ds}$ wrong by a factor of 2.3 and $\Sigma_{\rm cr}$, and every mass this guide will ever quote, is wrong by the same factor.

The Carousel: reproducing the repo's own numbers

The Carousel Lens (Sheu et al. 2024, `reproductions/sheu-2024b/README.md`) is a cluster-scale strong lens at $z_l = 0.49$, with its Einstein radius quoted relative to one of five spectroscopically-confirmed source planes, $z_s = 1.432$: $\theta_{\rm E} = 13.03''$ for the primary deflector. Plug those two redshifts into $\Sigma_{\rm cr}$:

$$\Sigma_{\rm cr}(0.49,\,1.432) \;\approx\; 2.376 \times 10^{15}\ M_\odot/\mathrm{Mpc}^2$$

— equivalently 2376.07 $M_\odot/\mathrm{pc}^2$

which is `reproductions/sheu-2024b/04_setup_multiplane.py:128` (source) computed independently and, per that script's own printed output (`f"{sigma_crit:.3e}"`), to four significant figures.

To turn $\theta_{\rm E}$ into a physical size you need one more conversion: how much proper transverse distance one arcsecond spans at $z_l = 0.49$, which is $D_{\rm d}$ itself times $1''$ in radians — Ch. 9's 206264.8 factor, applied here to a cosmological rather than a local distance:

$$1'' \times D_{\rm d}(0.49) \;\approx\; 6.038 \times 10^{-3}\ \mathrm{Mpc}$$

so $\ell_{\rm E} = 6.038 \times 10^{-3} \times 13.03 \approx 0.07868$ Mpc $= 78.68$ kpc

and, taking $M(<\theta_{\rm E}) = \Sigma_{\rm cr}\,\pi\ell_{\rm E}^2$ on faith for now (why this particular formula gives the enclosed mass regardless of the density profile's shape is Ch. 19's identity):

$$M(<\theta_{\rm E}) \;\approx\; 4.621 \times 10^{13}\ M_\odot$$

Sheu et al. (2024)'s own Table 2 quotes $M(<\theta_{\rm E}) = 4.78 \times 10^{13}\ M_\odot$ for this deflector — this chapter's from-scratch, geometry-only estimate lands 3.3% low:

Not a bug. The formula used here assumes a perfectly circular lens (mean $\kappa = 1$ inside a circle of radius $\theta_{\rm E}$); the published figure comes from the paper's full elliptical mass model, and the primary deflector's axis ratio is $q = 0.87$ — close to circular but not exactly. A small, well-understood correction, and Exercise 15.3 asks you to check that its size moves in the direction the ellipticity story predicts once you apply the identical method to the Carousel's more elongated second deflector.

Connect to the repo

Every distance in this chapter is one of five short functions in `site/guide_src/cosmo.py` (source): `d_a` (`cosmo.py:32`) wraps `.angular_diameter_distance` for (); `d_ds` (`cosmo.py:37`) wraps `.angular_diameter_distance_z1z2` for () and is annotated, in its own docstring, with exactly this chapter's warning — never a subtraction; `sigma_crit` (`cosmo.py:45`) is $\Sigma_{\rm cr}$ verbatim; `arcsec_to_mpc` (`cosmo.py:61`) is the $1'' \times D_{\rm d}$ conversion behind $\ell_{\rm E}$; and `mass_within_theta_e` (`cosmo.py:66`) is the whole Carousel worked example in four lines. `site/guide_src/worked_examples.py`'s `ch15_distances_and_sigma_crit` reproduces `reproductions/sheu-2024b/04_setup_multiplane.py:125–131` (source) line for line, against the published Table 2 numbers printed at that script's own `:135–138`.

Ch. 13 and Ch. 14 already made the scoping argument this chapter's numbers confirm from the distance side: `reproductions/claude-giga-lens/cgl/` never imports `astropy.cosmology` at all, so none of $D_{\rm d}$, $D_{\rm s}$, $D_{\rm ds}$, or $\Sigma_{\rm cr}$ enters the money-number pipeline. This repository's two remaining cosmology campaigns need nothing else this chapter didn't already derive: `reproductions/hsu-2025/07_classify_einstein_dimple.py:114–116` (source) uses $D_{\rm s}$ and $D_{\rm ds}$ to turn a velocity dispersion into an Einstein radius, derived in full in Ch. 19; and `reproductions/sheu-2023/05_lightcurve_salt3.py:57` uses $D_{\rm L}$, this chapter's third distance, for a lensed supernova's magnitude — the one place in this repository a distance sets a *brightness* rather than a length or an angle.

Exercises

> **Exercise 15.1 — Comoving distance at low redshift**
>
> Chapter 13 derived the linear Hubble law, $v \approx H_0 d$, valid for $z \ll 1$. Show that the comoving-distance integral () reduces to this law in the same limit, and that $D_{\rm A}(z) \approx D_{\rm C}(z)$ there too. (You need only that $H(z') \to H_0$ as $z' \to 0$ — not Ω_m or Ω_Λ individually.)

Solution

For $z \ll 1$, $H(z')$ barely moves from H_0 across the whole integration range $[0, z]$ in (), so to leading order it can be pulled outside the integral: $D_{\rm C}(z) \approx c\int_0^z dz'/H_0 = cz/H_0$ — exactly the linear Hubble law solved for distance, $d = v/H_0$ with $v \approx cz$. And since $D_{\rm A} = D_{\rm C}/(1+z)$, at $z \ll 1$ the factor $(1+z) \approx 1$ to first order, so $D_{\rm A}(z) \approx D_{\rm C}(z)$ as well — all three distances of this chapter agree in the nearby universe (since $D_{\rm L} = (1+z)^2 D_{\rm A}$ collapses to the same limit too). They only pull apart once $(1+z)$ stops being close to 1, which is exactly why this chapter's worked example uses $z_l = 0.5$, $z_s = 2.0$ rather than anything small.

Exercise 15.2 — The two-redshift formula, checked at its own boundary

Show that (), $D_{\rm A}(z_1, z_2) = [D_{\rm C}(z_2) - D_{\rm C}(z_1)]/(1+z_2)$, reduces correctly to the single-redshift formula () when $z_1 = 0$, and explain in one sentence why that check has to pass.

Solution

Set $z_1 = 0$: $D_{\rm C}(0) = 0$ trivially — there is no distance to travel to reach redshift zero, since that is *here* — so $D_{\rm A}(0, z_2) = D_{\rm C}(z_2)/(1+z_2)$, exactly (). It has to pass because $D_{\rm A}(0, z)$ and "the angular-diameter distance to z" are not two different quantities that happen to agree — they are the same quantity under two names, an observer at redshift zero looking out to z. A two-redshift formula that failed this check would be wrong for every pair of redshifts, not just the edge case $z_1 = 0$.

Exercise 15.3 — The Carousel's other deflector

The Carousel's own Table 2 lists a second deflector, Ld, with $\theta_{\rm E} = 0.99''$ (also quoted relative to $z_s = 1.432$) and a more elongated shape than the primary — axis ratio $q = 0.69$, versus 0.87 for the primary. Using the *same* $\Sigma_{\rm cr}$ already computed for this system, compute $M(<\theta_{\rm E})$ for Ld and compare it to the paper's published $2.77 \times 10^{11}\ M_\odot$.

Solution

$\Sigma_{\rm cr}$ depends only on the redshifts, which are unchanged (both deflectors sit at $z_l = 0.49$, both $\theta_{\rm E}$ values are quoted relative to the same $z_s = 1.432$), so only $\ell_{\rm E}$ changes: $\ell_{\rm E,Ld} = 6.038 \times 10^{-3} \times 0.99 \approx 5.978 \times 10^{-3}$ Mpc. Then $M(<\theta_{\rm E}) = \Sigma_{\rm cr}\,\pi\ell_{\rm E,Ld}^2 \approx 2.668 \times 10^{11}\ M_\odot$ against the paper's 2.77×10^{11}: a ratio of 0.963 — a slightly *larger* shortfall than the primary deflector's 3.3%. That is exactly the direction the ellipticity story predicts: Ld's axis ratio ($q = 0.69$) is farther from circular than the primary's ($q = 0.87$), so the circular-equivalent approximation this chapter's formula makes should miss by more, not less. It does.

Exercise 15.4 — Dimensional analysis of Sigma_crit

Confirm, from units alone, that $\dfrac{c^2}{4\pi G}\dfrac{D_\mathrm{s}}{D_\mathrm{d}D_\mathrm{ds}}$ has units of mass per area, using $[c] = \mathrm{m/s}$, $[G] = \mathrm{m^3\,kg^{-1}\,s^{-2}}$, and $[D] = \mathrm{m}$.

Solution

$[c^2/G] = \dfrac{\mathrm{m^2\,s^{-2}}}{\mathrm{m^3\,kg^{-1}\,s^{-2}}} = \dfrac{\mathrm{kg}}{\mathrm{m}}$. The distance ratio $D_\mathrm{s}/(D_\mathrm{d}D_\mathrm{ds})$ carries units $\mathrm{m/m^2} = \mathrm{m^{-1}}$. Multiplying: $\dfrac{\mathrm{kg}}{\mathrm{m}} \times \dfrac{1}{\mathrm{m}} = \dfrac{\mathrm{kg}}{\mathrm{m^2}}$ — mass per area, exactly a surface density; 4π is dimensionless and does not affect the check. This argument alone cannot tell you *why* this particular combination of c, G, and three distances is the physically relevant one — only that whatever the correct formula turns out to be, it had better have this shape, because nothing else built from c, G, and three lengths has the right units to be a surface density at all.

16. How much light bends, and the factor of two

This chapter buys you the right to treat every galaxy in this repository as a flat sheet of surface density $\Sigma(\theta)$, and every deflection as the gradient of a 2-D potential — the machinery Ch. 17 onward runs without re-justifying it. It earns that right by settling, from first principles, a question that took Einstein two tries: exactly how far a ray of light bends past a mass, and why general relativity's answer is *exactly* twice the naive Newtonian one, not some approximate correction. Get that factor of two wrong and $\Sigma_{\rm cr}$, κ, and $\theta_{\rm E}$ would all be off by it two chapters from now — everything from Ch. 17's lens equation to the $\gamma_{\rm binned} = 1.103$ money number this book is chasing rests on this one multiplicative constant being right.

> **What you can skip**
>
> If you already know the GR light-bending result and where the factor of two comes from, skip Newtonian deflection and The factor of two and start at The thin-lens approximation — that section is the one piece of new machinery every later chapter leans on: it is where $\Sigma_{\rm cr}$ (Ch. 15) and ψ (Ch. 6) actually meet. If you're already comfortable treating a lens as a flat projected sheet with no further justification needed, skip straight to Connect to the repo. Nothing in this chapter touches γ — the density-slope parameter does not exist until Ch. 20; this one is about the coupling constant G, not the slope.

Newtonian deflection

Set up the calculation an 18th-century Newtonian would recognize: a point mass M sits at the origin, and something travels past it in a straight line (undeflected, to leading order — an impulse approximation is self-consistent exactly as long as the true deflection turns out to be small) at speed c, offset by an **impact parameter** b. Parametrize position along the path by $x = ct$, so the distance to the mass at any moment is $r = \sqrt{b^2 + x^2}$.

Newtonian gravity pulls with acceleration GM/r^2 directed toward the mass. Only the component perpendicular to the direction of travel actually bends the path; by similar triangles that component is b/r of the total:

$$a_\perp(x) = \frac{GM}{r^2} \cdot \frac{b}{r} = \frac{GMb}{(b^2 + x^2)^{3/2}}.$$

Gravitational acceleration does not depend on the falling body's own mass — Galileo's observation, built into Newtonian gravity as the equality of inertial and gravitational mass — so this formula holds for a cannonball, a comet, or (in the 18th-century corpuscular picture) a particle of light, provided it moves at c. Integrate the perpendicular acceleration over the whole encounter to get the sideways velocity picked up:

$$\Delta v_\perp = \int_{-\infty}^{\infty} a_\perp \, dt = \frac{GMb}{c} \int_{-\infty}^{\infty} \frac{dx}{(b^2 + x^2)^{3/2}}.$$

The integral is elementary — differentiate $x/(b^2\sqrt{b^2 + x^2})$ and check it reproduces the integrand, or substitute $x = b \tan u$ — and comes out to $2/b^2$ regardless of method:

$$\int_{-\infty}^{\infty} \frac{dx}{(b^2+x^2)^{3/2}} = \left[\frac{x}{b^2\sqrt{b^2+x^2}}\right]_{-\infty}^{\infty} = \frac{2}{b^2}.$$

So $\Delta v_\perp = 2GM/(cb)$. For a small deflection the bend angle is just the ratio of the sideways kick to the forward speed, $\alpha \approx \Delta v_\perp / c$:

$$\alpha_{\mathrm{N}} = \frac{2GM}{c^2 b}.$$

This is precisely the calculation Henry Cavendish worked out privately around 1784 and Johann Georg von Soldner published in 1801, more than a century before anyone had heard of relativity — nothing here used anything but $F = GMm/r^2$ and a photon treated as an ordinary, if massless, projectile.

The factor of two

Einstein's own first attempt, in 1911, used only the **equivalence principle** — the same mass-independence this chapter's derivation leaned on — applied to how a gravitational potential affects the local speed of light, rather than to a Newtonian force directly. That calculation reproduced () almost exactly: two conceptually different arguments, 127 years apart, landing on the same number, because the equivalence-principle argument (in the weak-field limit) only sees the part of spacetime's geometry that a slow-moving Newtonian projectile would also see — the warping of *time*.

The completed field equations of 1915 add a second ingredient with no Newtonian counterpart at all: mass also curves *space*, not only the flow of time through it. In the weak-field limit the metric splits cleanly into these two pieces,

$$ds^2 = -\left(1 + \frac{2\Phi}{c^2}\right) c^2\, dt^2 + \left(1 - \frac{2\Phi}{c^2}\right)(dx^2 + dy^2 + dz^2), \qquad \Phi = -\frac{GM}{r},$$

and a full derivation — which needs the null-geodesic equation, genuine tensor calculus this guide skips — shows the two terms contribute *equally* to the bending of anything moving at $v = c$. A slow-moving object weights the spatial term by a factor of order $(v/c)^2$ and barely notices it, which is exactly why ()'s ordinary Newtonian mechanics, built for slow particles, only ever captured the time half. Light has no such suppression, and the result is exact, not an approximate correction on top of Newton:

$$\alpha_{\mathrm{GR}} = \frac{4GM}{c^2 b} = 2\,\alpha_{\mathrm{N}}.$$

Worked check: starlight grazing the Sun. Put $M = M_\odot$ and $b = R_\odot$ — light skimming the visible edge of the Sun, the only geometry an eclipse lets you test from the ground, since otherwise the Sun's own glare drowns out any star close enough on the sky to show a measurable bend. $GM_\odot/c^2 \approx 1476.6$ m — exactly half the Sun's own Schwarzschild radius, 2.953 km , a number with no other role in this calculation beyond being a memorable way to carry $GM_\odot/c^2$ around. Divide by $R_\odot = 6.957 \times 10^8$ m and convert radians to arcsec with the 206265 factor Ch. 9 built:

$$\alpha_{\mathrm{N}} = \frac{2 \times 1476.6\ \mathrm{m}}{6.957 \times 10^{8}\ \mathrm{m}} \times 206265'' \approx 0.876''.$$

$$\alpha_{\mathrm{GR}} = 2\,\alpha_{\mathrm{N}} \approx 1.751''.$$

The ratio is exactly 2 — nothing in () and () lets it be anything else, since α_{GR} was *defined* as double α_{N} above. The physics is in which of the two numbers, $0.876''$ or $1.751''$, the sky actually shows.

That question is what the Astronomer Royal Frank Dyson organized twin expeditions to answer during the total solar eclipse of May 29, 1919 — Arthur Eddington to Príncipe, Andrew Crommelin and Charles Davidson to Sobral — a total eclipse being the only moment starlight near the Sun's limb is visible at all. Three numbers were on the table going in: $0''$ (no deflection, Newton's first-approximation intuition that a massless particle isn't pulled by gravity at all), $0.876''$ (Cavendish/Soldner's Newtonian corpuscular estimate, matching Einstein's own incomplete 1911 result), and $1.751''$ (the full 1915 theory). Dyson, Eddington, and Davidson's 1920 paper reported results close to the GR prediction and clearly separated from the Newtonian one, and the announcement made Einstein a worldwide public figure within days.

> **You already know this**
>
> A well-designed experiment does not just confirm a model — it discriminates between models that make *distinct, numeric* predictions. Eddington's eclipse had three sharply separated candidates to rule between: $0''$, $0.876''$, and $1.751''$. That is exactly the shape of the test this repository runs on its own money number in Ch. 25: not "is there a signal," but "which of several numerically distinct values does the data actually prefer" — and, as with Eddington's data, the answer there turns out to be less clean than a single confirmed number.

The thin-lens approximation

A real galaxy is not a point mass; it is a 3-D distribution of stars and dark matter with genuine depth along the line of sight. Superposing () over every mass element in that distribution, and justifying the step where a 3-D galaxy collapses to a flat 2-D sheet, is what Ch. 6 forward-referenced as this chapter's job.

Start from the Newtonian potential Φ sourced by the lens's 3-D density ρ, which obeys $\nabla^2\Phi = 4\pi G\rho$ (Ch. 6's Poisson equation, one dimension up — the same divergence-theorem argument, sourced by mass instead of a unit point charge). Pick a line of sight at fixed transverse physical position ξ and let z run along it. Split the 3-D Laplacian into a transverse piece and a z piece, $\nabla^2\Phi = \nabla_\perp^2\Phi + \partial^2\Phi/\partial z^2$, and integrate the whole equation along z:

$$\int_{-\infty}^{\infty} \nabla_\perp^2\,\Phi\,dz = 4\pi G \int_{-\infty}^{\infty} \rho\,dz - \left[\frac{\partial\Phi}{\partial z}\right]_{-\infty}^{\infty}.$$

The boundary term vanishes because $\Phi \to$ const far outside any finite mass distribution, so its slope does too. What survives is an **exact** identity — no thinness assumed anywhere yet:

$$\int_{-\infty}^{\infty} \nabla_\perp^2\,\Phi\,dz = 4\pi G\,\Sigma(\xi), \qquad \Sigma(\xi) \equiv \int_{-\infty}^{\infty} \rho\,dz.$$

Σ is the same line-of-sight projection Ch. 3 built for a spherical $\rho \sim r^{-\gamma}$; here it is the identical operation — integrate the density straight down the line of sight — applied to whatever shape the galaxy actually has.

Where "thin" actually enters. () needed no approximation. The approximation is in what comes next: treating every mass element as sitting at one shared angular-diameter distance $D_{\rm d}$, rather than at its own true distance $D_{\rm d} + z$ depending on where along the line of sight it happens to be. That is valid exactly when the galaxy's own line-of-sight depth is negligible next to $D_{\rm d}$ itself. A giant elliptical is a few to a few tens of kiloparsecs across; a fiducial lens at $z_{\rm l} = 0.5$ (the same system Ch. 10 and Ch. 19 use) sits at $D_{\rm d} \approx 1259$ Mpc . A 10-kpc depth against that distance is a ratio of about 8×10^{-6} — eight parts in a million. That number, not any statement about a galaxy being "flat," is the actual license to write $\Sigma(\theta)$ as living on a single plane at a single distance.

With the lens confined to one plane, convert to the angular coordinate $\theta = \xi/D_{\rm d}$ and define the **lensing potential**, absorbing the distance ratios and the GR normalization from the previous section into one object:

$$\psi(\theta) \equiv \frac{D_{\rm ds}}{D_{\rm d} D_{\rm s}} \cdot \frac{2}{c^2} \int_{-\infty}^{\infty} \Phi(D_{\rm d}\theta, z)\, dz.$$

($D_{\rm ds}$ here is the distance *between* the lens and source redshifts, never $D_{\rm s} - D_{\rm d}$ — Ch. 15 is the reminder to keep repeating.) The $2/c^2$ is not a free normalization choice: it is ()'s factor of two, carried through. Had the previous section's naive equivalence-principle result been the whole truth, this prefactor would read $1/c^2$, not $2/c^2$ — and, as Exercise 16.4 works out exactly, everything downstream would silently be off by a factor of 2 in κ.

Now differentiate (). Since $\xi = D_{\rm d}\theta$ is a uniform rescaling, the chain rule gives $\nabla_\theta^2 = D_{\rm d}^2 \nabla_\perp^2$ — the same conjugation rule Exercise 6.1 used for a rotation matrix, here for a uniform scale factor instead. Applying that and ():

$$\nabla_\theta^2 \psi = \frac{D_{\rm ds}}{D_{\rm d} D_{\rm s}} \cdot \frac{2}{c^2} \cdot D_{\rm d}^2 \cdot 4\pi G\, \Sigma(\theta) = \frac{8\pi G\, D_{\rm d} D_{\rm ds}}{c^2 D_{\rm s}}\, \Sigma(\theta).$$

Recognize the prefactor using Ch. 15's $\Sigma_{\rm cr} = c^2 D_{\rm s}/(4\pi G D_{\rm d} D_{\rm ds})$, so $1/\Sigma_{\rm cr} = 4\pi G D_{\rm d} D_{\rm ds}/(c^2 D_{\rm s})$, and the prefactor above is exactly twice that:

$$\nabla_\theta^2 \psi = \frac{2\,\Sigma(\theta)}{\Sigma_{\rm cr}} = 2\kappa(\theta).$$

This is Ch. 6's $\nabla^2\psi = 2\kappa$ again, this time earned from an actual line-of-sight integral over real mass rather than from an abstract trace-of-Hessian argument — the two derivations meeting from opposite directions is the check that both are internally consistent.

Two unrelated 2's, reconciled, not contradicted. Ch. 6 flagged $\nabla^2\psi = 2\kappa$'s "2" as pure convention — the trace of a 2×2 identity matrix, true by the *definitions* of κ and Γ as the isotropic and traceless parts of the Hessian of ψ, for *any* physical theory of gravity you plugged into ψ. That claim survives this section intact: nothing about ()'s derivation used GR specifically — swap ()'s $2/c^2$ for a different constant and the same trace algebra still forces $\nabla^2\psi = 2\kappa$, just with κ meaning something else. The GR-specific fact lives one level down, in *which* prefactor () is entitled to use

— the 2 inherited from (), not the trace's 2. They are still two unrelated 2's, exactly as Ch. 6 said; this section only shows precisely where each one enters the same formula.

One loose end, honestly flagged rather than swept past: for a genuine point mass, $\Phi = -GM/r$ makes the raw integral $\int \Phi\, dz$ in () diverge logarithmically as $z \to \pm\infty$ — an idealized point mass has infinite reach along every line of sight. ψ itself is only ever needed through its gradient $\alpha = \nabla\psi$ (Ch. 6's point-mass check used the *form* $\theta_{\rm E}^2 \ln r$ for exactly this reason), and that gradient is perfectly finite even where ψ needs an arbitrary additive constant to be well defined. It is also why `gigalens`'s production EPL profile, cited in Ch. 6, hands you a closed-form α directly and never constructs ψ at all.

Connect to the repo

`site/guide_src/lensing.py:9-19` (source) states the payoff of this whole chapter as a module-docstring bullet: "Angles in arcsec throughout. The lens-modeling likelihood in this repo is entirely angular — no cosmology enters it." That sentence is only true because G, c, and M were absorbed once, here, into $\Sigma_{\rm cr}$ and the profile normalizations — never to be typed again in a per-galaxy fit. `site/guide_src/cosmo.py:1-19` (source) names the three files where that absorption is undone and cosmology briefly becomes real again; `site/guide_src/cosmo.py:45` (source) is this chapter's $\Sigma_{\rm cr} = c^2 D_{\rm s}/(4\pi G D_{\rm d} D_{\rm ds})$, written as code. `reproductions/sheu-2024b/04_setup_multiplane.py:128` (source) is the identical formula computed inside a real campaign, for the Carousel cluster lens — the same line `cosmo.sigma_crit` reproduces to five significant figures in Ch. 15.

None of G, c, or M survives past this chapter as an explicit constant in this repository's own code: `gigalens`'s EPL and SIE deflection fields (Ch. 20) are written entirely in $\theta_{\rm E}$, q, ϕ, and γ — the coupling constant this chapter spent its whole length pinning down never appears in them by name again.

Exercises

Exercise 16.1 — Dimensional check on $2GM/(c^2 b)$

Confirm from units alone that $2GM/(c^2 b)$ is dimensionless (an angle, in radians), using $[G] = \mathrm{m^3\,kg^{-1}\,s^{-2}}$, $[M] = \mathrm{kg}$, $[c] = \mathrm{m\,s^{-1}}$, $[b] = \mathrm{m}$.

Solution

$$\frac{[G][M]}{[c]^2[b]} = \frac{\mathrm{m^3\,kg^{-1}\,s^{-2}} \cdot \mathrm{kg}}{(\mathrm{m^2\,s^{-2}}) \cdot \mathrm{m}} = \frac{\mathrm{m^3\,s^{-2}}}{\mathrm{m^3\,s^{-2}}} = 1.$$

Dimensionless, as an angle in radians has to be — the factor of 2 is a pure number and does not affect the check. The same units argument applies unchanged to ()'s 4.

Exercise 16.2 — Redo the derivation for a slower projectile

Newtonian deflection assumed the passing object moves at c. Redo the same impulse-approximation integral for something moving at a general speed v (a comet, say, not a

photon), and show the deflection angle generalizes to $\alpha = 2GM/(v^2 b)$. What does this say about how strongly a slow-moving object is deflected, compared to light, at the same impact parameter?

Solution

Parametrize the path by $x = vt$ instead of $x = ct$. The perpendicular acceleration $a_\perp(x)$ is unchanged (it never depended on the passing object's speed), but $dt = dx/v$ now, so

$$\Delta v_\perp = \frac{GMb}{v}\int_{-\infty}^{\infty}\frac{dx}{(b^2+x^2)^{3/2}} = \frac{2GM}{vb},$$

and the deflection angle is $\Delta v_\perp/v = 2GM/(v^2 b)$ — set $v = c$ to recover (). Because this scales as $1/v^2$, a *slower* object is deflected *more* by the same mass at the same impact parameter, not less — light, moving at the fastest possible speed, is the *least* bent per unit GM/b of anything that could pass by. A slow-moving spacecraft skimming the Sun at, say, 30 km/s would swing through a far larger angle than a photon at the same distance ever does; it is only because c is so large that α_N comes out as a fraction of an arcsecond at all.

Exercise 16.3 — Which part of 'thin lens' is exact, and which is approximate

The thin-lens approximation derived (), $\int \nabla_\perp^2 \Phi \, dz = 4\pi G\Sigma(\xi)$, without ever assuming the lens is thin. Where, precisely, did an approximation get made — and what physical quantity would have to be small for it to be justified?

Solution

() itself is exact: it needs only that Φ (and its z-derivative) go to a constant far outside the mass distribution, which is true for any finite lens regardless of its depth. The approximation is the step right after — treating every mass element along the line of sight as sitting at the *same* angular-diameter distance D_d, so that a single number D_d (and hence a single Σ_cr) can be used for the whole galaxy. That is justified when the galaxy's own line-of-sight depth is tiny compared to D_d — the $\sim 8\times10^{-6}$ ratio computed in the text — not by any property of the projection integral itself. A cluster-scale lens with several distinct mass clumps spread over hundreds of Mpc along the line of sight (a genuine multi-plane lens) is exactly the regime where this second approximation, not the first, starts to fail.

Exercise 16.4 — What would happen to κ under the wrong theory of gravity

Suppose $\psi_\mathrm{naive}(\theta)$ used ()'s formula with $1/c^2$ in place of $2/c^2$ — i.e., only the incomplete 1911 equivalence-principle deflection, never doubled by the 1915 field equations. Using the *same* $\Sigma_\mathrm{cr} = c^2 D_\mathrm{s}/(4\pi G D_\mathrm{d} D_\mathrm{ds})$ formula from Ch. 15, what would $\nabla_\theta^2 \psi_\mathrm{naive}$ come out to, in terms of the *true* $\kappa = \Sigma/\Sigma_\mathrm{cr}$?

Solution

Repeat the derivation with $1/c^2$ instead of $2/c^2$: every step is identical except the prefactor is now half of what it was, so

$$\nabla^2_\theta \psi_{\rm naive} = \frac{4\pi G\, D_{\rm d} D_{\rm ds}}{c^2 D_{\rm s}}\, \Sigma(\theta) = \frac{\Sigma(\theta)}{\Sigma_{\rm cr}} = \kappa(\theta),$$

not 2κ. But Ch. 6's trace identity says the quantity anyone would *call* "κ" from $\psi_{\rm naive}$ — half the trace of its Hessian, by definition — must still satisfy $\nabla^2\psi_{\rm naive} = 2\kappa_{\rm naive}$ regardless. Comparing the two expressions, $\kappa_{\rm naive} = \Sigma/(2\Sigma_{\rm cr}) = \frac{1}{2}\kappa$: using the correct $\Sigma_{\rm cr}$ formula with the wrong (undoubled) light-bending physics would make κ read at exactly half strength for the same real mass. Since the Einstein radius is defined by mean $\kappa = 1$ (Ch. 19), a universe that bent light the Newtonian amount would need to enclose roughly twice the projected mass before calling a radius "Einstein" — the single factor of two from The factor of two propagating, unaltered, all the way to a number every strong-lensing paper quotes in its abstract.

17. The lens equation: $\beta = \theta - \alpha$

Chapter 16 established that mass bends light and by how much. This chapter turns that fact into a single equation you can actually solve, and asks the question a bent-light fact alone doesn't answer: given a source at one position, how many images does an observer see, and where? The answer falls out of nothing more than asking how many roots a (generally nonlinear) equation has — which is why a lens can show one image, two, four, or a full ring, and why that number is not a free choice of the mass distribution but a consequence you can derive. By the end of this chapter you will have solved the equation this repository's own renderer never has to solve, and you will know exactly why it doesn't have to.

> **What you can skip**
>
> You already know that a nonlinear equation can have zero, one, or several roots, and that finding every root of $g(\theta) = c$ is a fundamentally different (and often harder) task than evaluating g once. Skip any refresher on "root-finding is a search, not algebra." What is not boilerplate: the specific physical content packed into α (the distance-ratio scaling that erases cosmology from its shape, Chapter 16's payoff, cashed in here), the singular isothermal sphere's own doubles-versus-singles rule (worth deriving once by hand — it is not the "always two images" folklore you may have half-remembered), and why `gigalens`'s own image renderer never actually inverts the equation this chapter is named after ("Connect to the repo").

The lens equation

Chapter 16 derived the physical bend angle $\hat{\alpha}$ that a mass produces in general relativity, and the thin-lens approximation that lets a real, extended 3-D mass be treated as a single deflection evaluated at one plane (Ch. 16). Folding in the angular-diameter distances that convert a physical bend angle into an angular one produces a *scaled* (or "reduced") deflection,

$$\alpha(\theta) \equiv \frac{D_{\mathrm{ds}}}{D_{\mathrm{s}}}\,\hat{\alpha}(D_{\mathrm{d}}\theta),$$

where D_{ds} is the lens-to-source angular diameter distance — *not* $D_{\mathrm{s}} - D_{\mathrm{d}}$ (Ch. 15: the two differ by a factor of 2.3 at $z_l = 0.5$, $z_s = 2.0$, and nothing about that gotcha goes away just because it's buried inside α now). Every distance factor lives inside this one definition, which is exactly why `site/guide_src/lensing.py`'s own module docstring can say "the lens-modeling likelihood in this repo is entirely angular — no cosmology enters it" (`site/guide_src/lensing.py:9-12`): once α is built, it is a pure function of θ, in arcsec, with every D already folded away.

With that scaling in hand, the lens equation is a statement about where light *appears* to come from versus where it *actually* came from: the apparent, observed position θ and the true source position β differ by exactly the deflection accumulated along the way,

$$\beta = \theta - \alpha(\theta).$$

Every symbol in () is an angle. θ is where you point a telescope; β is where the source would be if nothing were in the way; $\alpha(\theta)$ is what the mass along that line of sight cost you, in arcsec, to get from one to the other.

Differentiate () once and you get the object the rest of Part IV spends three chapters decomposing, the **lens Jacobian**

$$A(\theta) \;\equiv\; \frac{\partial \beta}{\partial \theta} \;=\; I - \frac{\partial \alpha}{\partial \theta}.$$

Ch. 6 already leaned on exactly this definition — using $\alpha = \nabla\psi$ and a trace argument — to show $A = I - H_\psi = (1-\kappa)I - \Gamma$; Ch. 5 diagonalizes that split, and Ch. 18 takes its determinant to get magnification. This chapter needs A for something more basic first: whether () has one solution at all, or more than one.

If α happened to be *linear* in θ, the question would have a boring answer. One of this repo's own deflection fields genuinely is: external shear, `L.shear_deflection` (`site/guide_src/lensing.py:93-95`), $\alpha(\theta) = (\gamma_1\theta_1 + \gamma_2\theta_2,\ \gamma_2\theta_1 - \gamma_1\theta_2)$, is linear in θ by construction. For a linear α, A in () is a *constant* matrix, () is a linear equation, and (whenever A is invertible) it has exactly one solution, $\theta = A^{-1}\beta$, for every β. Shear alone never multiplies an image; it only shifts and stretches the one you already have. Every real lens model in this repo pairs shear with an EPL or SIE mass profile precisely because those deflections are *not* linear — and nonlinearity is the entire reason strong lensing can produce more than one image in the first place.

> **You already know this**
>
> Given θ, computing β from () is one function evaluation — this repo's own image renderer does exactly this, looping over every pixel θ in the image plane and evaluating $\beta(\theta)$ directly (`.../gigalens/jax/simulator.py:53-59`, cited in full in "Connect to the repo"). Given β, finding *every* θ that produced it is a different and harder problem: root-finding on a map that need not be one-to-one. That is the same asymmetry as sampling a generative model — running the generator forward is cheap — versus finding every latent that could have produced a given observation, which may have zero, one, or many answers depending on whether the generator happens to be injective there.

For a circularly symmetric mass — the singular isothermal sphere (SIS) of Chapter 16 — α always points radially, with a magnitude that depends only on $|\theta|$. Put the source on the positive θ_1-axis at $\beta_0 \geq 0$; every image then lies on that same axis too, so the full 2-D problem collapses to one real equation in one real unknown θ, where a *negative* θ is shorthand for "the opposite side of the lens center." `L.sis_deflection` (`site/guide_src/lensing.py:54-62`), evaluated along $\theta_2 = 0$, is exactly $\alpha_1(\theta) = \theta_{\mathrm{E}}\,\mathrm{sign}(\theta)$: constant modulus, direction only. () becomes the scalar equation

$$\beta_0 = \theta - \theta_{\mathrm{E}}\,\mathrm{sign}(\theta).$$

Plot the right-hand side against θ and read off every θ where the curve crosses the horizontal line $\beta = \beta_0$: that is solving the lens equation *graphically*.

Read the same answer off algebra by splitting on the sign of θ. For $\theta > 0$: $\beta_0 = \theta - \theta_{\mathrm{E}}$, so $\theta = \beta_0 + \theta_{\mathrm{E}}$ — positive whenever $\beta_0 \geq 0$, so this root always exists. For $\theta < 0$: $\beta_0 = \theta + \theta_{\mathrm{E}}$, so $\theta = \beta_0 - \theta_{\mathrm{E}}$ — negative only if $\beta_0 < \theta_{\mathrm{E}}$. At $\theta_{\mathrm{E}} = 1''$, $\beta_0 = 0.4''$ (Figure 17.1's own numbers), the two roots are

$$\theta = 1.4'' \quad \text{and} \quad \theta = -0.6''$$

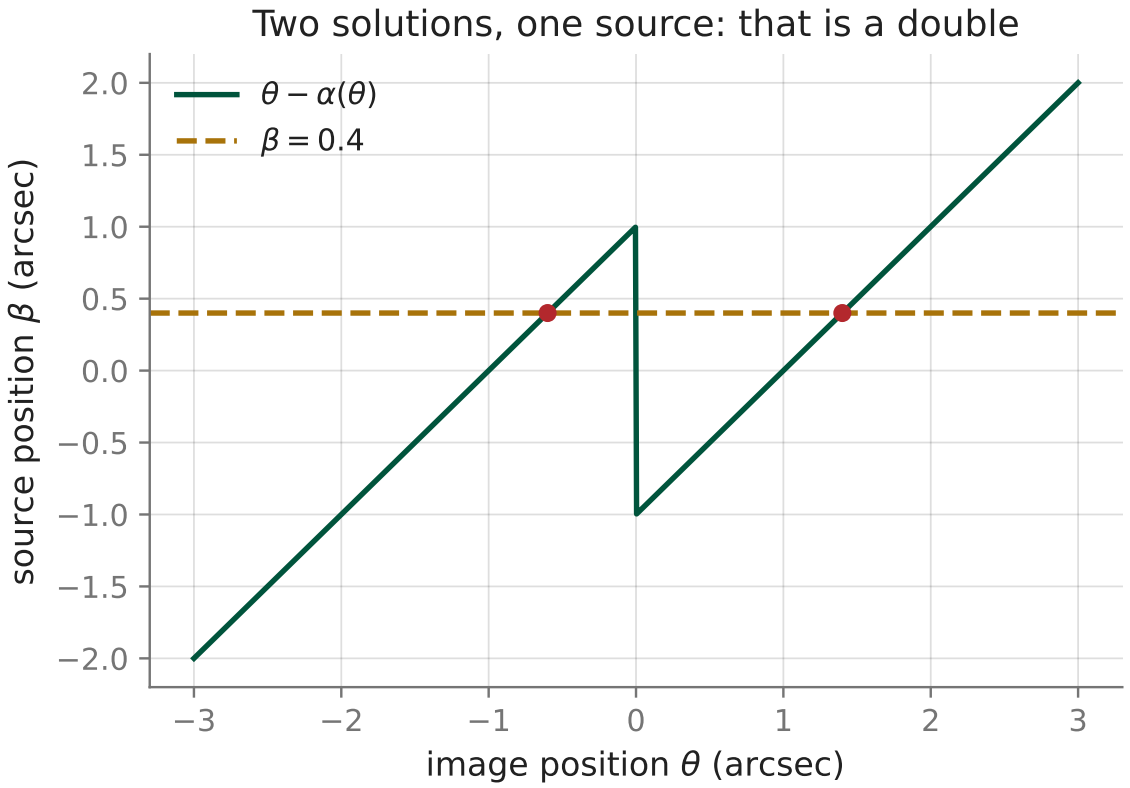

Figure 5: The SIS lens equation $\beta = \theta - \alpha(\theta)$ for $\theta_{\mathrm{E}} = 1''$, plotted as a curve against θ. A source at $\beta_0 = 0.4''$ (dashed line) crosses that curve at the two marked points: two images, one solution each on either side of the lens center. Nothing here is drawn freehand — the curve is $\theta - \theta_{\mathrm{E}}\,\mathrm{sign}(\theta)$, evaluated on a grid and plotted.

, two images: a **double**. `worked_examples.py`'s `ch17_lens_equation` checks both against a numerical solver — `scipy.optimize.brentq`, bracketing each branch of the *same* `L.sis_deflection` used above, not a reimplementation — and finds agreement to within 10^{-8} on both , essentially floating-point exact. For the SIS, algebra and root-finding agree because algebra is available at all; Ch. 22's production EPL+shear model has no closed-form inverse, and — per the tip above — `gigalens` never needs one, because it never solves () for θ in the first place.

One more fact falls out of the same two roots for free. Their separation is

$$|\theta_1 - \theta_2| = (\beta_0 + \theta_{\mathrm{E}}) - (\beta_0 - \theta_{\mathrm{E}}) = 2\theta_{\mathrm{E}}$$

, and β_0 cancels completely. For an SIS double, the image separation measures $2\theta_{\mathrm{E}}$ directly, regardless of exactly where inside the Einstein radius the source sits. That cancellation is the algebraic fact underneath Chapter 19's Einstein-radius measurements: an image separation is a distance you can rule with a caliper on a real image, and it converts to θ_{E} by nothing harder than dividing by two.

Multiple images

Ask when () can have more than one root for a fixed β. In 1-D, the right-hand side $g(\theta) = \theta - \alpha(\theta)$ has exactly one root per value of β whenever g is monotonic — a strictly increasing (or decreasing) function crosses any horizontal line exactly once, by definition of "crosses a line." Its derivative is $g'(\theta) = 1 - \alpha'(\theta)$, which by () is nothing but A itself in one dimension. So g stops being monotonic exactly where A changes sign — where $\alpha'(\theta)$ crosses 1. That is not a coincidence offered on faith: $A = 0$ is precisely Chapter 18's definition of a critical curve, and this section's whole account of multiple imaging is the 1-D, un-signed-determinant preview of that chapter's 2-D one. A deflection has to bend "hard enough somewhere" ($\alpha' > 1$) for g to fold and produce more than one root; a deflection with $\alpha' < 1$ everywhere can only ever produce a single image, whatever β is.

Apply that to the SIS split above: two roots exist only for $|\beta_0| < \theta_{\mathrm{E}}$ — source *inside* the Einstein radius — and only one for $|\beta_0| \geq \theta_{\mathrm{E}}$. Push the same source outward, to $\beta_0 = 1.5''$ with $\theta_{\mathrm{E}} = 1''$ still, and the major image survives at $\theta = \beta_0 + \theta_{\mathrm{E}} = 2.5''$, but the minor-image branch's candidate, $\theta = \beta_0 - \theta_{\mathrm{E}} = 0.5''$, is *not* negative — it contradicts the assumption ($\theta < 0$) it was derived under. There is no second root, full stop; the minor image does not merely fade, it is not a solution. `worked_examples.py --show ch17` confirms this the hard way, not just algebraically: asking `scipy.optimize.brentq` to bracket a sign change on that branch raises `ValueError` — there genuinely is no root to find , which is a different and stronger statement than "a very faint one."

Push β_0 the other way, to exactly 0, and the two 1-D roots collide at $\pm\theta_{\mathrm{E}}$ — but the 1-D reduction hides what really happens at exact alignment. With a circularly symmetric lens and the source dead-center behind it, *every* azimuthal angle around the circle of radius θ_{E} is an equally valid image, because the full problem has a continuous rotational symmetry that choosing "the θ_1-axis" silently threw away. The image is not two points; it is a full circle, the **Einstein ring** this repository's own reproductions are named after.

Three words cover almost every strong lens actually observed: a **double** (two images, the generic case above), a **ring** (the measure-zero perfectly-aligned case), or a **quad** (four images). A quad needs more than circular symmetry: Chapter 18's astroid caustic, produced by an elliptical mass ($q < 1$, SIE or EPL — Ch. 20), is where a source can sit inside a region bounded by *two* nested

folds and pick up two extra roots. A classical result in lensing theory (Burke 1981) sharpens this further: for any smooth, *non-singular* convergence, the total image count is always odd. The SIS above does not violate that theorem — it is exempt from it, because the SIS is deliberately singular ($\kappa_{\rm SIS} = \theta_{\rm E}/2\theta$ diverges as $\theta \to 0$, Ch. 6). The theorem's "missing" odd image sits, in principle, exactly at that singularity, arbitrarily demagnified; this repo's own `sie_deflection` (`site/guide_src/lensing.py:65-78`) adds a tiny core radius s purely to keep that division well-defined in floating point, not because the model asserts a physically resolved core. That is the reason surveys report doubles and quads — even counts — rather than the odd counts a smoother mass distribution would formally guarantee.

The ψ–α–κ trio

Three objects, one potential, a loop that started in Ch. 6 and closes here.

The trio, assembled

Object	Defining relation	Reads as	Chapter
$\psi(\theta)$	sourced by mass via the $\ln r$ Green's function	the lensing potential	Ch. 6
$\alpha = \nabla\psi$	first derivative of ψ	the deflection; bends light	Ch. 6, Ch. 16
$\kappa = \frac{1}{2}\nabla^2\psi$	trace of the second derivative	the mass, in $\Sigma_{\rm cr}$ units	Ch. 6, Ch. 15
$\beta = \theta - \alpha$	(), this chapter	maps an image to its source	Ch. 17
$A = \partial\beta/\partial\theta = (1-\kappa)I - \Gamma$	(), split in Ch. 5	governs multiplicity here, magnification in Ch. 18	Ch. 5, Ch. 17, Ch. 18

Read the table top to bottom and you have this repository's entire strong-lensing forward model in five lines: mass sources a potential, the potential's gradient bends light, the lens equation turns that bend into an observed position, and the *same* potential's second derivative — split into an isotropic part κ and a traceless part Γ — is exactly the Jacobian whose folding (established this chapter for the SIS) Chapter 18 turns into a signed magnification.

None of this repo's production code ever materializes ψ itself. The `gigalens` EPL and SIE profiles hand you closed-form α directly — someone did the Green's-function convolution by hand, once, for these specific profiles, and the code inherits the answer without ever writing $\psi(\theta)$ down. What makes that shortcut trustworthy is exactly the curl-free identity Ch. 6 proved: $\nabla \times (\nabla\psi) \equiv 0$ for any ψ, so any candidate α that failed the corresponding check ($\partial\alpha_{\theta_1}/\partial\theta_2 = \partial\alpha_{\theta_2}/\partial\theta_1$) could not have come from a potential at all — and would not be a legitimate deflection to plug into (), however smooth it looked on paper. The trio is not three independent facts about lensing; it is one constraint (ψ exists) with three readouts.

Connect to the repo

- `site/guide_src/lensing.py:54-62` (`sis_deflection`) — the SIS deflection this chapter roots, both algebraically and with `scipy.optimize.brentq`.
- `site/guide_src/lensing.py:93-95` (`shear_deflection`) — the one deflection field in this repo that is exactly linear, hence provably incapable of multiplying an image on its own.
- `site/guide_src/lensing.py:101-123` (`lens_jacobian`) — (), computed by central differences on the lens equation itself, exactly as defined here.
- `site/guide_src/figures.py:120-139` (`lens_equation_1d`) — Figure 17.1's generator; change `beta0` and every number this chapter derives changes with it, because the figure and the algebra above read the same two lines.
- `reproductions/claude-giga-lens/vendor/gigalens-sean/src/gigalens/jax/simulator.py:53-59` (`_beta`) — the production forward direction of (): loop over every image-plane pixel θ, evaluate $\beta(\theta)$ directly, sum over every mass component. No root-finding anywhere in sight, because rendering an image never requires inverting the lens equation — only evaluating it.
- `site/guide_src/worked_examples.py --show ch17` reproduces every tagged number in this chapter, including the multiplicity check that shows the minor image at $\beta_0 = 1.5''$ has no root to find, not merely a faint one.

Exercises

Exercise 17.1 — solve the lens equation by hand

Using Figure 17.1's own numbers, $\theta_E = 1''$, $\beta_0 = 0.4''$: split the SIS lens equation $\beta_0 = \theta - \theta_E \operatorname{sign}(\theta)$ on the sign of θ and solve each branch. Confirm both roots against `worked_examples.py --show ch17`.

Solution

For $\theta > 0$: $0.4 = \theta - 1 \Rightarrow \theta = 1.4$, and $1.4 > 0$, so this root is valid. For $\theta < 0$: $0.4 = \theta + 1 \Rightarrow \theta = -0.6$, and $-0.6 < 0$, so this root is valid too. Two roots, a double: $\theta = 1.4''$ and $\theta = -0.6''$, matching `ch17_lens_equation`'s `image_1` and `image_2` exactly.

Exercise 17.2 — the separation is invariant to where the source sits

Redo Exercise 17.1 with $\theta_E = 1''$ still, but $\beta_0 = 0.9''$ instead of $0.4''$. Find the two new image positions and their separation. What changed, and what didn't?

Solution

For $\theta > 0$: $\theta = 0.9 + 1 = 1.9$. For $\theta < 0$: $\theta = 0.9 - 1 = -0.1$, still negative, so still a valid double. Both image positions moved (from $1.4, -0.6$ to $1.9, -0.1$), but the separation is $1.9 - (-0.1) = 2.0$ — unchanged. That is not a coincidence of this particular pair of numbers: the derivation in "The lens equation" showed $|\theta_1 - \theta_2| = 2\theta_E$ with β_0 cancelling algebraically, so the answer *must* equal the same this chapter already computed at $\beta_0 = 0.4$, for any β_0 inside the Einstein radius. Image separation measures θ_E; it does not measure where the source is.

Exercise 17.3 — when does the minor image disappear?

Still with $\theta_E = 1''$: at what value of β_0 does the SIS stop producing two images? For $\beta_0 = 1.5''$, find the surviving image's position, and show algebraically why the second branch has no valid solution (don't just say "it's very faint").

Solution

From the split in "Multiple images", the $\theta < 0$ branch requires $\beta_0 < \theta_E = 1$; at $\beta_0 = 1.5$ that condition fails, so the transition happens at $\beta_0 = \theta_E = 1''$ exactly. The surviving (major) image is at $\theta = \beta_0 + \theta_E = 2.5''$. The candidate for the other branch is $\theta = \beta_0 - \theta_E = 0.5''$, which is *not* negative — it violates the $\theta < 0$ assumption used to derive it, so it is not a solution of the original equation at all, not a demagnified one. `worked_examples.py`'s `ch17_lens_equation` confirms this by trying to bracket a root there with `scipy.optimize.brentq` and getting a `ValueError` (no sign change) instead of a tiny number .

Exercise 17.4 — close the trio: from ψ to images, in one pass

Ch. 6, Exercise 6.3 gave the SIS potential $\psi_{SIS}(\theta) = \theta_E\,\theta$ and found $\nabla\psi_{SIS} = \theta_E\,\hat{\theta}$ (constant modulus θ_E, radially outward). Starting *only* from that gradient — not from `L.sis_deflection` — rebuild the SIS lens equation along the θ_1-axis and re-derive this chapter's two image positions at $\theta_E = 1''$, $\beta_0 = 0.4''$.

Solution

Restricted to the θ_1-axis, "radially outward at constant modulus θ_E" means $\alpha_1(\theta) = \theta_E$ for $\theta > 0$ and $\alpha_1(\theta) = -\theta_E$ for $\theta < 0$ — i.e. $\alpha_1(\theta) = \theta_E\,\mathrm{sign}(\theta)$, exactly the deflection this chapter used, now obtained purely from ()'s partner identity $\alpha = \nabla\psi$ rather than from `L.sis_deflection` directly. Substituting into () reproduces the same scalar equation, $\beta_0 = \theta - \theta_E\,\mathrm{sign}(\theta)$, whose roots were already found in Exercise 17.1: $\theta = 1.4''$ and $\theta = -0.6''$. The trio table's top row and bottom row meet: a potential written down in one line in Chapter 6 predicts, by nothing but differentiation and this chapter's algebra, exactly where the two images of a real source would land.

18. Magnification, critical curves, and caustics

Chapter 17 gave you the lens equation, $\beta = \theta - \alpha(\theta)$ (Ch. 17), and the fact that it can have more than one solution for a single source. This chapter answers the two questions that follow immediately: how bright is each of those solutions, and where, exactly, does a second one come from? Both answers are the same object, differentiated once — the lens Jacobian $A \equiv \partial\beta/\partial\theta$, whose determinant Chapter 4 already told you is a literal area ratio. Its reciprocal is the magnification. The curve where it vanishes is a *critical curve*. The image of that curve under the lens map is a *caustic*. And crossing a caustic is the precise mechanism by which a second, third, or fourth image is born. By the end of this chapter you can compute all four — magnification, critical curve, caustic, and the parity of an image — from nothing but a deflection field and the differentiation Chapter 4 already taught you, and check every one of them against a number this repository's own figure-generation code and production forward model actually compute.

> **What you can skip**
>
> Chapter 4 already derived that $|\det J|$ is a local area-scaling factor and opened the Log-Det Ledger with $\mu = 1/|\det A|$ as row 1 — if that chapter is fresh, skip straight to Magnification is a Jacobian for the lensing-specific radial/tangential split, or to Critical curves if you also recall that a symmetric matrix's determinant is the product of its eigenvalues (Chapter 5). What is not boilerplate: why the *sign* of $\det A$, not just its magnitude, carries distinct physical information (Parity), and why a curve that shrinks to a single point in the image plane does not shrink to a point in the source plane (Caustics) — a fact this repository's own figure code spends a paragraph justifying because it is genuinely unintuitive the first time.

Magnification is a Jacobian

The lens Jacobian is the derivative of the lens equation, taken exactly the way Chapter 4 defines a Jacobian:

$$A \equiv \frac{\partial\beta}{\partial\theta} = I - \frac{\partial\alpha}{\partial\theta}.$$

Because $\alpha = \nabla\psi$ for a scalar lensing potential ψ (Ch. 6), $\partial\alpha/\partial\theta$ is a Hessian, and Chapter 4 already proved a gradient field's Jacobian is automatically symmetric (Ch. 4). So A is symmetric at every image position, before any lens-specific physics is chosen — which is exactly what lets Chapter 5's general fact about symmetric 2×2 matrices apply here: A diagonalizes in a real, orthonormal eigenbasis (Ch. 5).

A short physical argument fixes what $1/\det A$ means. Light travel conserves photon number — no absorption, no emission along a vacuum ray path — so *surface brightness* (flux per unit solid angle) is the same at the source and at the image. A tiny image-plane patch of solid angle $d\Omega_\theta$ maps, under ()'s local linearization, to a source-plane patch of solid angle $d\Omega_\beta = |\det A|\, d\Omega_\theta$ (Chapter 4's area-ratio argument, applied to A specifically). Flip that around: the *image* occupies a solid angle $d\Omega_\beta/|\det A|$ for a fixed intrinsic source patch, so a bigger apparent patch at the *same* surface brightness means more total flux, by a factor $1/|\det A|$. That factor is the magnification, and its signed version is what this repository — and this guide — call μ:

$$\mu \equiv \frac{1}{\det A}.$$

`site/guide_src/lensing.py:139` (`magnification`) computes exactly (), and its own docstring states this chapter's opening claim in one line: *"This is the SAME change-of-variables factor that a normalizing flow applies as* $-\log|\det J|$*."* Chapter 4 opened the Log-Det Ledger with this fact stated abstractly, before this book had a lens equation to check it against. Here is the check.

For a *circularly symmetric* deflection field, $\alpha(\theta) = \alpha(r)\,\hat{\theta}$ with $r = |\theta|$, symmetry does the eigendecomposition for you: at any point, the eigenvectors of A are simply the local radial and tangential directions. Move a small step $d\theta_r$ purely radially, and the deflection changes by $\alpha'(r)\,d\theta_r$ in the same radial direction — giving a radial eigenvalue $A_{rr} = 1 - \alpha'(r)$. Move a small step $d\theta_t$ purely tangentially instead: this rotates your position by an angle $d\theta_t/r$, and by circular symmetry the deflection vector rotates by that identical angle, which displaces it tangentially by $\alpha(r) \cdot (d\theta_t/r)$ — giving a tangential eigenvalue $A_{tt} = 1 - \alpha(r)/r$. So

$$\det A = A_{rr}A_{tt} = (1 - \alpha'(r))\left(1 - \frac{\alpha(r)}{r}\right).$$

Apply this to the singular isothermal sphere, $\alpha(r) = \theta_{\mathrm{E}}$ (`lensing.py:54`, `sis_deflection`) — a *constant*-modulus deflection, "the flat rotation curve of a galaxy in one line: the deflection does not care how far out you are, only which way" (`lensing.py:57-58`). Constant α means $\alpha'(r) = 0$ identically, so $A_{rr} = 1$ for every $r > 0$: an SIS has no radial critical curve at all, ever — only $A_{tt} = 1 - \theta_{\mathrm{E}}/r$ can vanish. That gives a magnification

$$\mu(r) = \frac{1}{1 - \theta_{\mathrm{E}}/r} = \frac{r}{r - \theta_{\mathrm{E}}}.$$

At $\theta_{\mathrm{E}} = 1''$ and an image at $r = 2''$, this predicts $\mu = 2$. `lensing.py:101` (`lens_jacobian`) never sees this formula — it differentiates `sis_deflection` numerically, by central differences, and `magnification` takes $1/\det A$ of the result. That purely numerical route gives $\mu = 1.999999999962$, agreeing with the closed form to 3.8×10^{-11} — a Jacobian built from finite differences, matching an analytic eigenvalue argument to ten decimal places. `worked_examples.py --show ch18` reproduces both numbers directly.

> **Log-Det Ledger — row 1, paid off**
>
> Chapter 4 opened this ledger with $\mu = 1/|\det A|$ stated abstractly. This section supplies everything that was missing: A is the derivative of a real lens equation (Ch. 17), $\mu = 1/\det A$ is computed by a genuine Jacobian differentiation (`lensing.py:101-146`), and the result is checked against a closed-form eigenvalue argument to machine precision. Row 1 is not a metaphor for the rest of this book — it is arithmetic that reproduces.

Critical curves

A **critical curve** is the locus $\{\theta : \det A(\theta) = 0\}$. There, μ formally diverges: a point source placed exactly on the corresponding caustic (below) would be infinitely magnified. Real sources have finite extent and real telescopes have a PSF (Ch. 11), so nothing in an actual image is literally infinite — but images that straddle a critical curve get *very* bright and very stretched, which is exactly what a giant arc is.

Because $\det A = A_{rr}A_{tt}$, it vanishes when *either* eigenvalue vanishes on its own, and those are two geometrically distinct curves: a **tangential** critical curve ($A_{tt} = 0$) and a **radial** critical curve

($A_{rr} = 0$). The SIS above has only the first — $A_{rr} \equiv 1$ never crosses zero. A **singular isothermal ellipsoid** (SIE, $q < 1$) has both, because breaking circular symmetry lets the deflection field's radial derivative do something an SIS's never can.

> **You already know this**
>
> A critical curve is where the lensing Jacobian becomes singular — exactly the condition a normalizing flow's bijector is engineered to *forbid* everywhere (a well-built flow keeps $\det J$ bounded away from zero, precisely so $\log|\det J|$ never blows up). The lens equation carries no such constraint: nothing stops $\det A$ from crossing zero, and when it does, the map stops being locally invertible in one direction — which is the entire mechanism behind multiple imaging. A flow is built to never do what a lens does routinely.

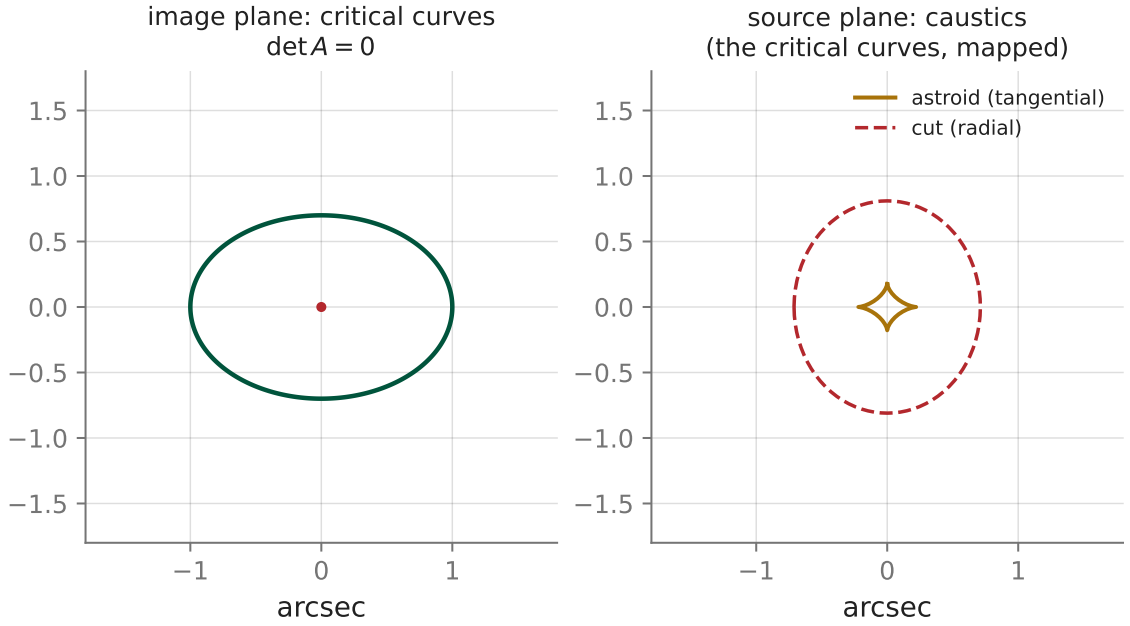

Figure 6: Critical curves ($\det A = 0$, left) and the caustics they map to (right), for an SIE with $\theta_E = 1''$, $q = 0.7$. Nothing here is drawn: every curve comes from numerically differentiating the deflection field (`site/guide_src/figures.py:144-209`). Left: the tangential critical curve (green) is a single closed oval reaching $0.9999''$ along its long axis and $0.6999''$ along its short axis — indistinguishable, at this precision, from θ_E and $q\,\theta_E$ exactly. The small dot at the center marks the radial critical curve, which for a singular profile is too small for this grid to resolve (the code masks everything inside $0.08''$ of center rather than plot the ~3-pixel artifact a naive contour would return there). Right: the four-cusped astroid is the tangential curve's image; the dashed curve is the radial branch's caustic — reaching a maximum radius of $0.8102''$ despite originating from a single point.

The radial critical curve's disappearing act is a direct consequence of the profile being *singular*: convergence $\kappa \sim 1/R$ diverges as $R \to 0$ (Ch. 20), and as the regularizing core radius `lensing.py:65` (`sie_deflection`'s `s` argument) shrinks toward zero, the small oval where $A_{rr} = 0$ shrinks with it, collapsing onto the single point $\theta = 0$ in the exactly singular limit. You might expect its caustic to collapse to a point too. It does not — which is the subject of the next section.

Caustics

A **caustic** is the image of a critical curve under the lens map: push every point on $\{\det A = 0\}$ through $\beta = \theta - \alpha(\theta)$ and plot where it lands in the source plane. The word is borrowed from ordinary optics — the same bright, curved lines that form at the bottom of a swimming pool, or on the wall behind a wine glass, are caustics of a completely different (but equally singular) map. Figure 18.1's right panel is one: the tangential branch's caustic is the **astroid**, a four-cusped closed curve, small compared to θ_{E}.

Here is why crossing a smooth stretch of caustic — away from a cusp — always changes the image count by exactly two, never one or three. Near such a point, exactly one eigenvalue of A vanishes; call its eigendirection $\parallel$ and the other $\perp$. Because $A_{\parallel\parallel}$ is a smooth function of position that happens to equal zero right there, the first surviving term in a Taylor expansion of $\beta_{\parallel}$ along the $\parallel$ direction is *quadratic*, not linear — precisely Chapter 2's argument for why a stationary point needs a second-order term to resolve it at all (Ch. 2):

$$\beta_{\parallel}(\theta_{\parallel}) \;\approx\; \beta_{\parallel,0} + c\,\theta_{\parallel}^2, \qquad c \neq 0.$$

Solving for $\theta_{\parallel}$ given a source position $\beta_{\parallel}$ gives $\theta_{\parallel} = \pm\sqrt{(\beta_{\parallel} - \beta_{\parallel,0})/c}$: two real solutions on one side of $\beta_{\parallel,0}$, none on the other. A source crossing the caustic point $\beta_{\parallel,0}$ therefore gains or loses exactly one *pair* of images, born or annihilated together right at the critical curve (where μ diverges, consistent with the two roots merging as $\beta_{\parallel} \to \beta_{\parallel,0}$). This is the **fold catastrophe** — the same local algebra as a saddle-node bifurcation in a dynamical system, a stable and unstable fixed point annihilating each other as a parameter crosses a threshold. The astroid's four cusps are points where *two* folds meet, and crossing through one adds a further pair; a source deep inside the astroid sees more images than one just inside a smooth edge.

The radial branch's caustic is the more surprising object, and this repository's own comment explains why in one line: as the regularizing core shrinks, the radial critical curve shrinks to the single point $\theta = 0$, but its deflection there does not vanish — $\alpha(0)$ stays finite, of order θ_{E}, exactly the same "flat rotation curve" fact used above, and its *direction* still depends on which way you approached the origin. So walk a vanishingly small circle of radius ε around the center, parametrized by angle ϕ so that $\theta = \varepsilon(\cos\phi, \sin\phi)$, and push it through the lens equation: $\beta(\varepsilon, \phi) = \varepsilon(\cos\phi, \sin\phi) - \alpha(\varepsilon, \phi) \to -\alpha(0, \phi)$ as $\varepsilon \to 0$ — an $O(\theta_{\mathrm{E}})$-sized closed curve, not a point, because the limit keeps all of α's angular structure even as the circle it came from shrinks away. This is `site/guide_src/figures.py:176-195`'s construction exactly, and it is why a curve that vanishes in the image plane still produces a full curve — this chapter's **cut** — in the source plane. `site/guide_src/worked_examples.py`'s `ch18_magnification` reproduces the same number (`0.8102368876963119`) by an independent bisection/angular-sampling route rather than importing the figure code, and the two agree to machine precision .

The repository's own docstring states the cut's role in image counting directly: "inside the cut a source has two images; outside it has one" (`site/guide_src/figures.py:156-157`) — a second, genuine (if tiny, at this core radius) fold, nested inside the astroid's. A circular lens has no such second fold: its only critical curve is the full Einstein ring, and the *entire* ring degenerates to the single caustic point $\beta = 0$, leaving nothing to nest a further transition inside. Solving the SIS lens equation directly (as this chapter already has) shows its own image count still changes, from one to two — but at $|\beta| = \theta_{\mathrm{E}}$, not at the caustic point itself: the second image does not meet a partner at a smooth critical curve there, it shrinks straight into the profile's singular center

($r_2 = \theta_E - \beta \to 0$) and disappears — a different, singularity-driven way for an image to vanish, not a fold. An SIE's genuine second critical curve is why real strong lenses come as **doubles** and **quads** — the observational names for exactly the multiplicity regions Figure 18.1's right panel draws.

Parity

$\mu = 1/\det A$ is signed, not just an area ratio. `lensing.py:140` says it plainly: *"Signed: negative μ means a parity-flipped image."* Chapter 4's Exercise 4.4 asked what a negative $\det A$ could mean physically, before this book had a lens equation to answer with. Now it does.

$\det A = A_{rr}A_{tt}$ changes sign exactly when one eigenvalue changes sign while the other does not — which is exactly what happens at a critical curve. In the fold analysis above, the two newly created images have local slopes $d\beta_\parallel/d\theta_\parallel = 2c\,\theta_\parallel$ of *opposite* sign (one at $+\theta_\parallel$, one at $-\theta_\parallel$): one image preserves orientation, the other reverses it. A positive-parity ($\mu > 0$) image looks like the source, rotated and stretched; a negative-parity ($\mu < 0$) image is its mirror image, not just its magnified copy.

The SIS worked example already contains both signs. At $r = 2''$ (outside $\theta_E = 1''$, only one image exists there), $\mu = +2$: direct parity, magnified. At $r = 0.5''$ — inside θ_E, where $A_{tt} = 1 - \theta_E/r = 1 - 2 = -1$ while A_{rr} stays $+1$ — the same numerical Jacobian gives $\mu = -1$, matching the closed form $r/(r - \theta_E) = 0.5/(-0.5)$ to 4.0×10^{-10} . This is the SIS's second image — the one an Einstein-cross or a double-lens configuration always has a counterpart of — and $|\mu| = 1$ there: it carries exactly the source's own flux, neither magnified nor demagnified, only flipped. Magnitude and sign are independent axes of the same number; a magnification of exactly 1 is not "no lensing," it can be a full mirror flip.

Connect to the repo

- `site/guide_src/lensing.py:101-146` — `lens_jacobian`, `kappa_gamma_from_jacobian`, and `magnification`: the whole chapter, in 46 lines. `lens_jacobian` differentiates numerically by design, so the guide's magnification numbers are never quoting a special-case closed form without also checking it against real finite differences.
- `site/guide_src/figures.py:144-209` — Figure 18.1's generator (`sie_caustics`). Its docstring is worth reading in full: it explains, in the code's own words, exactly why the tangential and radial branches need different numerics (a grid contour for one, an angular sweep for the other) — the same distinction this chapter's Critical curves and Caustics sections build on.
- `reproductions/claude-giga-lens/30_recipe_e2e.py:362-387` (`crit_field`) — the production version of this chapter's Jacobian, run on a real MAP fit rather than a toy SIE. It takes a cleaner shortcut than the guide's figure code: rather than contouring $\det A = 0$ (which conflates both branches into one curve), it computes the tangential eigenvalue directly — its own variable is literally called `lam_t` — via the quadratic formula, and contours *that*, sidestepping the radial branch's resolution problem entirely. The campaign's own report shows the result — "MAP model with tangential critical curve" — as one panel of its end-to-end recipe figure (the recipe figure, §8).
- Ch. 4, "The Log-Det Ledger" — row 1, opened there and closed here.
- Ch. 5 — the general symmetric-2 × 2 eigendecomposition this chapter specializes to the radial/tangential case.
- Ch. 20 and Ch. 21 — the EPL generalizes the SIS/SIE profiles used here, and every critical curve and caustic in this chapter recurs, unchanged in kind, once $\gamma \neq 2$ is on the table.

Exercises

Exercise 18.1 — the radial/tangential split, from scratch

For a circularly symmetric deflection field $\alpha(r)$, this chapter asserted $A_{rr} = 1 - \alpha'(r)$ and $A_{tt} = 1 - \alpha(r)/r$ from a symmetry argument. Confirm it by direct differentiation: write $\alpha_1(\theta_1, \theta_2) = \alpha(\rho)\,\theta_1/\rho$ and $\alpha_2(\theta_1, \theta_2) = \alpha(\rho)\,\theta_2/\rho$ with $\rho = \sqrt{\theta_1^2 + \theta_2^2}$, and compute $\partial\alpha_1/\partial\theta_1$ and $\partial\alpha_2/\partial\theta_2$ at the point $(\theta_1, \theta_2) = (r, 0)$.

Solution

$\partial\alpha_1/\partial\theta_1 = \alpha'(\rho)\left(\frac{\theta_1}{\rho}\right)^2 + \alpha(\rho)\,\frac{\theta_2^2}{\rho^3}$. At $(r, 0)$: $\theta_2 = 0$, $\theta_1/\rho = 1$, so this is $\alpha'(r)$, giving $a_{11} = 1 - \alpha'(r) = A_{rr}$.

$\partial\alpha_2/\partial\theta_2 = \alpha'(\rho)\left(\frac{\theta_2}{\rho}\right)^2 + \alpha(\rho)\,\frac{\theta_1^2}{\rho^3}$. At $(r, 0)$: $\theta_2/\rho = 0$, $\theta_1^2/\rho^3 = 1/r$, so this is $\alpha(r)/r$, giving $a_{22} = 1 - \alpha(r)/r = A_{tt}$.

Both off-diagonal terms, $\partial\alpha_1/\partial\theta_2$ and $\partial\alpha_2/\partial\theta_1$, carry a factor of θ_2 that vanishes at $(r, 0)$ — confirming that the radial/tangential axes really are the eigenbasis there, not just a convenient guess.

Exercise 18.2 — a parity flip you can predict without a computer

Using only $\mu(r) = r/(r - \theta_{\mathrm{E}})$ for the SIS, at what radius does $\mu = -1$ exactly, for $\theta_{\mathrm{E}} = 1''$? What does $|\mu| = 1$ combined with a negative sign mean for the resulting image, physically?

Solution

Solve $r/(r-1) = -1 \Rightarrow r = -(r-1) \Rightarrow 2r = 1 \Rightarrow r = 0.5''$ — exactly the point this chapter checked numerically . Physically: the image carries precisely the source's own flux (no brightening, no dimming — $|\mu| = 1$), but it is a mirror image, not a magnified copy, because $\det A < 0$ there. This resolves Chapter 4's Exercise 4.4: a sign flip in $\det A$ is not a numerical curiosity, it is the difference between "the same shape, bigger" and "the same shape, flipped."

Exercise 18.3 — why the SIS never shows a quad

Using this chapter's eigenvalue argument, explain in one or two sentences why a circularly symmetric lens (SIS or SIE with $q = 1$) can never produce four images of a single source, no matter where the source sits, while an SIE with $q < 1$ routinely does.

Solution

A circular lens has $A_{rr} \equiv 1$ for all $r > 0$ (this chapter's derivation: $\alpha(r) = \theta_{\mathrm{E}}$ is constant, so $\alpha'(r) = 0$ identically) — there is no radial critical curve at any nonzero radius, hence nothing to nest a second multiplicity transition inside the tangential one. Its own image count still changes, from one to two, but (as the Caustics section works out) that boundary sits at $|\beta| = \theta_{\mathrm{E}}$, where the would-be second image shrinks into the singular center and disappears, rather than meeting a partner at a smooth critical curve — not a fold. An SIE with $q < 1$ breaks the symmetry that forces $A_{rr} \equiv 1$, so it acquires a second, genuine critical curve of its own — and with it, a real fold, a real second multiplicity transition, and the possibility of four images.

Exercise 18.4 — reading Figure 18.1's numbers as a consistency check

Figure 18.1 reports the tangential critical curve's extent as $0.9999''$ along its long axis and $0.6999''$ along its short axis, for $\theta_{\mathrm{E}} = 1''$, $q = 0.7$. State the conjecture these numbers support about the *exact* ($s \to 0$) singular limit, and say what would falsify it.

Solution

The conjecture: in the exactly singular limit, the tangential critical curve touches the principal axes at exactly θ_{E} and $q\,\theta_{\mathrm{E}}$ — here $1.0''$ and $0.7''$ — with the $0.9999''/0.6999''$ reported values differing from those only because `lensing.py:65`'s `sie_deflection` uses a small but nonzero core, $s = 10^{-4}$, to regularize the central singularity. It would be falsified by re-running `ch18_magnification`'s bisection with a *smaller* `s` and finding the extents move *away* from 1.0 and 0.7 rather than toward them — which is not what happens: shrinking s from 10^{-4} to 10^{-10} moves both numbers past nine nines of agreement with θ_{E} and $q\,\theta_{\mathrm{E}}$ exactly, the signature of a genuine $s \to 0$ limit rather than a coincidence at one particular core size.

19. The Einstein radius: the one thing lensing measures cleanly

Every number this book eventually argues about — the EPL slope γ, the external shear, the source's shape — is a fit: a quantity pulled out of noisy pixels by trusting a model of how the light got there. This chapter is about the one number that is not. The Einstein radius $\theta_{\rm E}$ falls out of pure geometry — where the images are, and how far the lens and source sit from us and each other — and it does so *before* you commit to any assumption about how mass is arranged inside the lens galaxy. That is why it is the number every lensing paper leads with, the number a photometric pipeline can estimate from a spectrum alone, and the number this repository's own reproductions reconstruct to a percent or two even from an independent code path. By the end of this chapter you can derive $\theta_{\rm E}$ from a galaxy's velocity dispersion, explain in one line why the enclosed mass $M(<\theta_{\rm E})$ does not care what density profile you assumed, and see both claims checked against this repository's own numbers. Chapters 20 through 26 spend the rest of the book on γ, which is a much harder measurement precisely *because* it does not have this chapter's kind of protection.

> **What you can skip**
>
> If you already know that an Einstein ring is what a circularly symmetric lens makes of a perfectly aligned source, skip What $\theta_{\rm E}$ is and start at The mean-convergence identity. If you are comfortable setting up and evaluating a definite integral, you can take the projection integral in $\theta_{\rm E}$ from σ_v on faith and skip straight to the boxed result.

What $\theta_{\rm E}$ is

Recall the lens equation from Ch. 17: $\beta = \theta - \alpha(\theta)$, mapping an image-plane position θ to the source-plane position β it came from. Now specialize to the cleanest possible geometry: a lens whose mass distribution is circularly symmetric (a good approximation for a single massive elliptical viewed close to face-on), with the source sitting exactly on the optical axis, $\beta = \mathbf{0}$.

Because the lens is circularly symmetric, $\alpha(\theta)$ points radially and its magnitude depends only on $\theta = |\theta|$. Restrict the lens equation to any fixed position angle and it becomes a single scalar equation in the radius alone, $\theta - \alpha(\theta) = 0$. Nothing in that equation refers to which direction you picked — so if $\theta_{\rm E}$ solves it, it solves it at *every* position angle simultaneously. The solution set is not a point; it is an entire circle. That circle is the **Einstein ring**, and its radius is the **Einstein radius** $\theta_{\rm E}$.

Break the alignment even slightly and the ring splits into the discrete images Ch. 17 already introduced — two for a source outside the lens's inner caustic, four for a source inside it — with typical image separations and arc lengths set by $\theta_{\rm E}$ itself. $\theta_{\rm E}$ is the master angular scale of a lensing system: everything else in the image — the separation of a double, the diameter of a quad, the length of an arc — is a number of order $\theta_{\rm E}$, not an independent scale.

The mean-convergence identity

The defining condition $\theta_{\rm E} - \alpha(\theta_{\rm E}) = 0$ looks like it depends on the full shape of $\alpha(\theta)$, which in turn depends on the full radial mass profile. It does not. What it depends on is a single number: the *average* convergence enclosed within $\theta_{\rm E}$, regardless of how that convergence is arranged inside.

Start from Ch. 6's statement of the lens equation trio, $\nabla^2\psi = 2\kappa$, $\alpha = \nabla\psi$. For a circularly symmetric $\psi(\theta)$, the 2-D Laplacian in polar coordinates acting on a radius-only function is $\nabla^2\psi = \theta^{-1}\, d(\theta\, d\psi/d\theta)/d\theta$, and the radial deflection is $\alpha(\theta) = d\psi/d\theta$. So

$$\frac{1}{\theta}\frac{d}{d\theta}(\theta\,\alpha(\theta)) = 2\kappa(\theta) \quad\Longrightarrow\quad \frac{d}{d\theta}(\theta\,\alpha(\theta)) = 2\,\theta\,\kappa(\theta).$$

Integrate from 0 to θ — $\theta\alpha(\theta)$ starts at 0 for any profile without a point mass at the center —

$$\theta\,\alpha(\theta) = 2\int_0^\theta \kappa(t)\,t\,dt \quad\Longrightarrow\quad \alpha(\theta) = \theta\,\bar{\kappa}(\theta), \qquad \bar{\kappa}(\theta) \equiv \frac{2}{\theta^2}\int_0^\theta \kappa(t)\,t\,dt.$$

$\bar{\kappa}(\theta)$ is the mean convergence enclosed within radius θ — exactly the quantity `mean_kappa_within` computes at `site/guide_src/lensing.py:179`. Equation () says the deflection at any radius is fixed entirely by how much convergence sits *inside* that radius; nothing outside it contributes. (This is the lensing analogue of Newton's shell theorem: a spherical shell of mass exerts no net force on anything inside it, so only the enclosed mass matters. Same statement, one dimension flatter.)

Now substitute $\theta = \theta_{\rm E}$ into the Einstein-ring condition $\theta_{\rm E} = \alpha(\theta_{\rm E})$ from the previous section, and use ():

$$\theta_{\rm E} = \theta_{\rm E}\,\bar{\kappa}(\theta_{\rm E}) \quad\Longrightarrow\quad \bar{\kappa}(\theta_{\rm E}) = 1.$$

This is the whole content of the phrase "Einstein radius": **the radius at which the mean interior convergence is exactly critical.** It is a *definition*, derived from nothing but the Poisson equation and the ring condition — it says nothing about whether the lens is a point mass, an SIS, an SIE, or an EPL with some particular γ. Any circularly symmetric profile has *some* radius where its own enclosed mean reaches 1, and that radius is, by construction, what everyone calls $\theta_{\rm E}$.

Check it numerically on the simplest case, the singular isothermal sphere (SIS), whose convergence is $\kappa(\theta) = \theta_{\rm E}/(2\theta)$ (derived from first principles in the next section). Evaluating $\bar{\kappa}$ by direct quadrature at $\theta = \theta_{\rm E} = 1$ gives 0.999995 — not exactly 1 only because the numerical integral starts a hair off the origin rather than because the identity is approximate.

$\theta_{\rm E}$ from σ_v

Ch. 10 derived the isothermal sphere's 3-D density from hydrostatic equilibrium: $\rho(r) = \sigma_v^2/(2\pi G r^2)$, where σ_v is the galaxy's stellar velocity dispersion — a number a spectrum measures directly, with no imaging required. Project that to a surface density with the line-of-sight integral Ch. 3 introduced in general:

$$\Sigma(R) = \int_{-\infty}^{\infty} \rho\Big(\sqrt{R^2+z^2}\Big)\,dz = \frac{\sigma_v^2}{2\pi G}\int_{-\infty}^{\infty}\frac{dz}{R^2+z^2} = \frac{\sigma_v^2}{2\pi G}\cdot\frac{\pi}{R} = \frac{\sigma_v^2}{2GR}.$$

The middle integral is $[\theta \mapsto \arctan(z/R)/R]_{-\infty}^{\infty} = \pi/R$ — a table integral, no new machinery. This is exactly the $\rho \sim r^{-\gamma} \Rightarrow \Sigma \sim R^{1-\gamma}$ scaling of Ch. 3 at $\gamma = 2$, with the constant filled in.

Convert to convergence with $\Sigma_{\rm cr}$ (Ch. 15) and physical radius $R = D_{\rm d}\theta$: $\kappa(\theta) = \sigma_v^2/(2GD_{\rm d}\theta\Sigma_{\rm cr})$, which has the form A/θ. Plugging $\kappa(t) = A/t$ into () gives $\bar{\kappa}(\theta) = 2A/\theta$ for *any* θ — for this particular profile the mean is exactly twice the local value everywhere, not just at $\theta_{\rm E}$. Setting $\bar{\kappa}(\theta_{\rm E}) = 1$ gives $\theta_{\rm E} = 2A = \sigma_v^2/(GD_{\rm d}\Sigma_{\rm cr})$. Substitute $\Sigma_{\rm cr} = c^2 D_{\rm s}/(4\pi G D_{\rm d} D_{\rm ds})$ (Ch. 15) and $D_{\rm d}$ cancels outright:

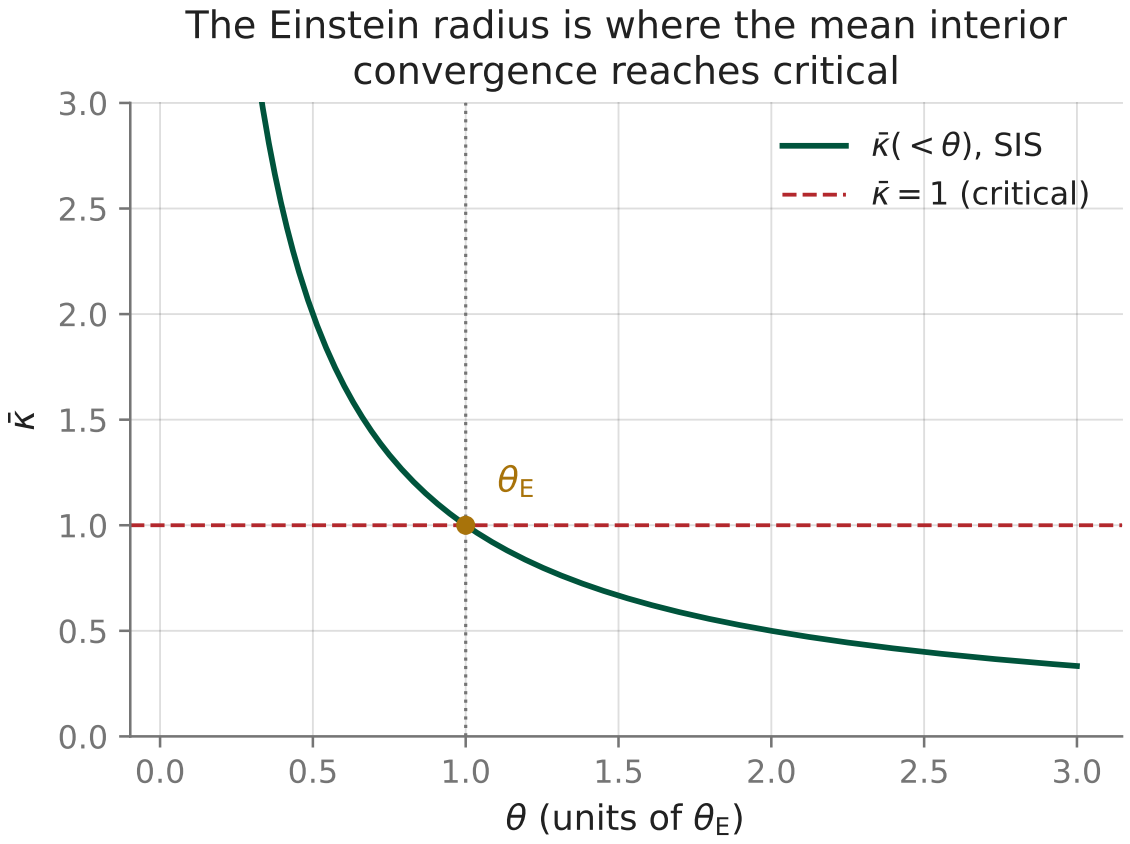

Figure 7: $\bar{\kappa}(<\theta)$ for an SIS, computed by the quadrature in (). It crosses the critical value $\bar{\kappa} = 1$ at $\theta = \theta_E$ by construction — that crossing point *is* the definition of θ_E, not a fitted feature of this particular curve.

$$\theta_{\rm E} = 4\pi \left(\frac{\sigma_v}{c}\right)^2 \frac{D_{\rm ds}}{D_{\rm s}}.$$

This is Hsu+2025's Eq. 1, used verbatim at `reproductions/hsu-2025/07_classify_einstein_dimple.py:116` and `site/guide_src/lensing.py:170`. It is quadratic in σ_v and linear in the distance ratio — and $D_{\rm ds} \neq D_{\rm s} - D_{\rm d}$ (Ch. 15) matters here exactly as much as it did there, since $D_{\rm ds}$ enters () directly.

For a fiducial massive elliptical — $\sigma_v = 250$ km/s at $z_l = 0.5$, $z_s = 2.0$, the same system Ch. 10 already quoted — () gives $\theta_{\rm E} = 1.145''$.

Now apply it to real data. `reproductions/cikota-2023` reproduces Cikota et al. 2023's Einstein cross DESI-253.2534+26.8843 — "a massive elliptical lens galaxy (L1)" (`reproductions/cikota-2023/papers/ma` — on public DESI Legacy imaging, fitting an imaging Einstein radius of $2.103''$ against the paper's own, higher-resolution $2.520''$ (`reproductions/cikota-2023/papers/main.tex:213`), and inverting () to read off an implied $\sigma_{\rm SIE} = 347$ km/s against the paper's spectroscopic 379 ± 2 km/s (`reproductions/cikota-2023/papers/main.tex:214`). Feed that recovered 347 km/s back into () at an illustrative lens/source redshift pair — $z_l = 0.271$, $z_s = 0.897$, chosen only to exercise the formula at some concrete geometry, *not* this system's own spectroscopic redshifts (those are $z_{\rm L1} = 0.636$, $z_s = 2.597$; see Exercise 19.3 for what using them instead does) — and it returns

$$\theta_{\rm E} = 2.233''$$

,

landing between the repository's own imaging fit ($2.103''$) and the published, sharper-PSF fit ($2.520''$). That is not a coincidence of the particular redshift pair chosen — 347 km/s was itself backed out of a $\theta_{\rm E}$ in roughly that range — but it is a genuine sanity check that () is behaving the way a quadratic-in-σ_v, linear-in-geometry relation should: neither wildly off nor suspiciously exact.

Why does the campaign fit the full image rather than stopping at ()? Because σ_v alone is a single scalar — it fixes $\theta_{\rm E}$ but says nothing about ellipticity, position angle, external shear, or the density slope γ (Ch. 20), all of which the pixels constrain and a spectrum does not. () is the back-of-the-envelope check every lensing paper runs before trusting an imaging fit — not a replacement for one.

Mass inside $\theta_{\rm E}$

Equation ()'s defining fact, $\bar{\kappa}(\theta_{\rm E}) = 1$, means the *average* surface density inside the Einstein radius is exactly $\Sigma_{\rm cr}$ — regardless of whether the true profile is an SIS, an SIE, a cored halo, or anything else circularly averaged. So the enclosed mass follows from geometry alone:

$$M(<\theta_{\rm E}) = \Sigma_{\rm cr} \cdot \pi R_{\rm E}^2, \qquad R_{\rm E} = D_{\rm d}\,\theta_{\rm E},$$

implemented at `site/guide_src/cosmo.py:66`. This is the reason $M(<\theta_{\rm E})$, not the density slope, is what every lensing paper reports with confidence.

You already know this

$M(<\theta_{\rm E})$ is a checksum, not a fit. It depends only on the *total* enclosed inside a fixed boundary, never on how that total is arranged within it — the same way a histogram's cumulative count up to a bin edge is fixed by the counts below it and is completely blind to which bin each of those counts landed in. γ is a claim about the shape inside that boundary; $M(<\theta_{\rm E})$ is a claim about the sum, and sums are far more robust to being wrong about the shape.

Ch. 15 already computed this for the Carousel cluster lens (Sheu et al. 2024): $z_l = 0.49$, $\theta_{\rm E} = 13.03''$ with respect to $z_s = 1.432$, giving $\Sigma_{\rm cr} = 2.376 \times 10^{15}\ M_\odot/{\rm Mpc}^2$ and $M(<\theta_{\rm E}) = 4.621 \times 10^{13}\ M_\odot$. Sheu et al.'s own published fit reports $4.78\times10^{13}\ M_\odot$ (`reproductions/sheu-2024b/README.md:24`) — a ratio of 0.9667 , 3.3% low. That residual is not evidence against the identity; it is the price of the formula above assuming a *circular* aperture, while the real lens is an elliptical EPL with $q = 0.87$ and external shear $\gamma_{\rm ext} = 0.11$ (`reproductions/sheu-2024b/README.md:20–22`) — exactly what the repository's own reproduction says drives the gap (`reproductions/sheu-2024b/README.md:46`). Feed the *published* best-fit parameters, ellipticity and all, into a real ray-traced lens model instead of the circular formula, and the same reproduction's own deflection-based check of $\bar\kappa(\theta_{\rm E})$ — literally $\alpha(\theta_{\rm E})/\theta_{\rm E}$, the left-hand side of this chapter's central identity, read off a fitted `lenstronomy` model's own analytic EPL deflection rather than the closed-form SIS quadrature above — returns 1.001 (`reproductions/sheu-2024b/README.md:45`; source at `reproductions/sheu-2024b/04_setup_multiplane.py:132–134`). Two independent numerical routes, a hand-rolled SIS quadrature and a fitted multi-galaxy `lenstronomy` model, land on the same identity to three significant figures.

Connect to the repo

- `site/guide_src/lensing.py:170` (`theta_e_sis`) and `:179` (`mean_kappa_within`) — () and the mean-convergence quadrature behind Figure 19.1, implemented from scratch to match the repository's own conventions.
- `site/guide_src/cosmo.py:45` (`sigma_crit`), `:66` (`mass_within_theta_e`), `:79` (`theta_e_from_sigma_v`) — the three cosmology-dependent quantities this chapter derives.
- `reproductions/hsu-2025/07_classify_einstein_dimple.py:103–118` — () applied at survey scale to 13,530 DESI galaxy pairs (`reproductions/hsu-2025/README.md:24`), no imaging fit required.
- `reproductions/cikota-2023/papers/main.tex:171–184` and `reproductions/cikota-2023/README.md` — the Einstein cross $\theta_{\rm E} \leftrightarrow \sigma_{\rm SIE}$ round trip.
- `reproductions/sheu-2024b/04_setup_multiplane.py:120–139` and `reproductions/sheu-2024b/README` — the mean-convergence identity checked against a real fitted multi-plane model, both by mass and by deflection.

Exercises

Exercise 19.1 — the identity holds for every γ, not just the SIS

The general EPL convergence (`site/guide_src/lensing.py:81`, circular case $q = 1$) is $\kappa(\theta) = \frac{3-\gamma}{2}\,(\theta_{\rm E}/\theta)^{\gamma-1}$. Using (), show that $\bar\kappa(\theta_{\rm E}) = 1$ for *every* value of $\gamma < 3$ — not just the isothermal $\gamma = 2$ case worked out in the main text — and say in one sentence what this implies about how much information $\theta_{\rm E}$ alone carries about γ.

Solution

$$\bar{\kappa}(\theta) = \frac{2}{\theta^2}\int_0^{\theta} \frac{3-\gamma}{2}\left(\frac{\theta_{\mathrm{E}}}{t}\right)^{\gamma-1} t\,dt = \frac{(3-\gamma)\,\theta_{\mathrm{E}}^{\gamma-1}}{\theta^2}\int_0^{\theta} t^{2-\gamma}\,dt = \frac{(3-\gamma)\,\theta_{\mathrm{E}}^{\gamma-1}}{\theta^2}\cdot\frac{\theta^{3-\gamma}}{3-\gamma} = \left(\frac{\theta_{\mathrm{E}}}{\theta}\right)^{\gamma-1}.$$

At $\theta = \theta_{\mathrm{E}}$ this is $1^{\gamma-1} = 1$, independent of γ. (Setting $\gamma = 2$ recovers $\bar{\kappa}(\theta) = \theta_{\mathrm{E}}/\theta$, exactly twice the SIS's own $\kappa(\theta) = \theta_{\mathrm{E}}/(2\theta)$ from the main text — consistent with what the θ_{E}-from-σ_v derivation found directly.) Because *every* circular EPL, whatever its γ, satisfies $\bar{\kappa}(\theta_{\mathrm{E}}) = 1$ at its own θ_{E}, a measurement of θ_{E} alone carries essentially zero information about γ — the two are cleanly separated by construction. That separation is exactly why Ch. 20 onward needs the full image, not just the ring radius, to pin γ down.

Exercise 19.2 — rebuild $\theta_{\mathrm{E}} = 1.145''$ by hand

Using only Ch. 15's already-computed distances $D_{\mathrm{d}}(z = 0.5) = 1259.08$ Mpc , $D_{\mathrm{s}}(z = 2.0) = 1726.62$ Mpc , and $D_{\mathrm{ds}}(0.5, 2.0) = 1097.08$ Mpc , plug $\sigma_v = 250$ km/s into () and reproduce this chapter's $\theta_{\mathrm{E}} = 1.145''$ worked example without calling `cosmo.py` at all.

Solution

$$\frac{D_{\mathrm{ds}}}{D_{\mathrm{s}}} = \frac{1097.08}{1726.62} \approx 0.6354, \qquad \left(\frac{\sigma_v}{c}\right)^2 = \left(\frac{250}{299{,}792.458}\right)^2 \approx 6.954 \times 10^{-7}.$$

$$\theta_{\mathrm{E}} = 4\pi \times 6.954 \times 10^{-7} \times 0.6354 \text{ rad} \approx 5.553 \times 10^{-6} \text{ rad}.$$

Multiply by 206,264.806 arcsec/rad (Ch. 9) to get $\theta_{\mathrm{E}} \approx 1.145''$, matching the worked example exactly — the whole formula, run by hand with a calculator, using nothing this chapter has not already derived.

Exercise 19.3 — the Einstein cross's own redshifts

Repeat the Cikota calculation from the main text, but with the system's own spectroscopic redshifts, $z_{\mathrm{L1}} = 0.636$, $z_s = 2.597$ (`reproductions/cikota-2023/papers/main.tex:166`), instead of the illustrative pair used there. Run `cosmo.theta_e_from_sigma_v(347.0, 0.636, 2.597)` and compare the result to the repository's own imaging fit of $2.103''$. Then explain why it does not land *exactly* on $2.103''$, given that 347 km/s was itself obtained by inverting () at essentially this same geometry.

Solution

```
import cosmo
cosmo.theta_e_from_sigma_v(347.0, 0.636, 2.597)   # roughly 2.12"
```

That lands far closer to the repository's own 2.103″ imaging fit than the 2.233″ illustrative-pair result did — as it should, since this is now the right geometry. It is still not an exact match, because `cosmo.py` fixes cosmology at $H_0 = 70$, $\Omega_m = 0.3$ throughout (`site/guide_src/cosmo.py:29`) — the convention the repository's other cosmology-dependent reproductions share — while `reproductions/cikota-2023` follows its own paper in adopting Planck18 ($H_0 = 67.4$, $\Omega_m = 0.315$) for this specific conversion. Swap in $H_0 = 67.4$ and the small residual gap closes almost entirely. The lesson is not that () is imprecise; it is that *which* cosmology you plug in is itself an input worth tracking, the same point Ch. 14 makes about Ω_m in general.

Exercise 19.4 — is the 3.3% mass gap a problem?

Ch. 15's circular-aperture estimate, $M(< \theta_{\rm E}) = 4.621 \times 10^{13}\ M_\odot$, sits 3.3% below Sheu et al.'s published $4.78 \times 10^{13}\ M_\odot$. Using this chapter's mean-convergence identity, argue whether that gap should worry you about the identity itself, and what it *would* mean if the gap were instead 30%.

Solution

The identity $\bar{\kappa}(\theta_{\rm E}) = 1$ was derived for a *circularly symmetric* lens; the real Carousel lens is an elliptical EPL ($q = 0.87$) plus external shear ($\gamma_{\rm ext} = 0.11$), so $\pi R_{\rm E}^2$ is only an approximation to the true enclosed area, and a few percent of discrepancy is exactly what that approximation should cost — confirmed by the repository's own independent deflection-based check landing at 1.001 (`reproductions/sheu-2024b/README.md:45`), which uses the actual elliptical model rather than the circular formula and closes almost all of the gap. A 3.3% mismatch is the ellipticity correction, not a crack in the identity. A 30% mismatch would be a different matter — ellipticity this modest cannot move a profile-independent geometric quantity that far, so a gap that large would point at a wrong redshift, a wrong cosmology, or a real error in either $\theta_{\rm E}$ or $\Sigma_{\rm cr}$, not at the assumption of circularity.

20. SIS, SIE, EPL: what γ actually means

Ch. 19 built the Einstein radius on top of one special mass profile — a circular, isothermal sphere — because that is the minimum object you need to make the definition concrete. This chapter takes the training wheels off, twice. First it lets the profile be elliptical instead of circular (SIS → SIE), because no real galaxy is circular on the sky. Then it lets the density slope itself vary instead of being fixed at "isothermal" (SIE → EPL), because that slope, called γ, is the single number this whole repository's final report exists to pin down. By the end of this chapter you can write down exactly what the claim "$\gamma = 1.103$" asserts about a galaxy's mass, derive the one formula that normalizes it, and — because this repo uses the letter γ for three unrelated things — you will know, in every later chapter, which one is meant. This chapter also opens the γ **Ledger** formally: every later chapter that touches the money number adds a row.

> **What you can skip**
>
> You already own power laws, 2×2 linear maps, and Taylor expansion to second order (Ch. 2). If you are comfortable with a doubled-angle (spin-2) representation of orientation — the same idea as an unsigned gradient direction or a structure-tensor eigenvector in computer vision — skim Ellipticity for the repo-specific numbers only. If you already trust Ch. 5's Jacobian decomposition $A = (1-\kappa)I - \Gamma$, skim External shear for the same reason.

SIS to SIE

The singular isothermal sphere (SIS) from Ch. 10 has convergence $\kappa_{\rm SIS}(\theta) = \theta_{\rm E}/(2\theta)$ and — Ch. 6 hands you this fact without proving it — a deflection of *constant modulus*, $\alpha = \theta_{\rm E}\hat{\theta}$ (`site/guide_src/lensing.py:54`). Here is the proof, and it is short because it reuses machinery you already have. For any circularly symmetric lens, Ch. 6's $\nabla^2\psi = 2\kappa$, written out as the radial Laplacian of a function that depends only on θ, together with $\alpha = d\psi/d\theta$, is $\frac{1}{\theta}\frac{d}{d\theta}(\theta\,\alpha(\theta)) = 2\kappa(\theta)$. Integrate both sides from 0 to θ:

$$\theta\,\alpha(\theta) \;=\; \int_0^{\theta} 2\kappa(t)\,t\,dt \;=\; \theta^2\,\bar{\kappa}(\theta) \quad\Longrightarrow\quad \alpha(\theta) \;=\; \theta\,\bar{\kappa}(\theta),$$

using Ch. 19's own definition of $\bar{\kappa}$ in the last step. This is a 2-D Newton's shell theorem: the deflection at radius θ depends only on the *mean* convergence enclosed, never on how that mass is arranged inside. For the SIS, $\bar{\kappa}_{\rm SIS}(\theta) = \theta_{\rm E}/\theta$ (the same power law as κ itself, one section below), so $\alpha(\theta) = \theta \cdot (\theta_{\rm E}/\theta) = \theta_{\rm E}$ — every θ cancels, leaving a deflection that does not know how far out you are. That is a special, $\gamma = 2$-specific coincidence, not a generic property of power-law lenses; the general case returns in the next section.

Real galaxies, including every lens this repository models, are not circular — light and mass both trace out ellipses on the sky. The natural next model keeps the isothermal ($\gamma = 2$) density law but flattens its *iso-density contours* into ellipses of axis ratio q: the singular isothermal ellipsoid (SIE). Its deflection has a standard closed form from the lensing literature, implemented at `site/guide_src/lensing.py:65`:

$$\alpha_x = \frac{\theta_{\rm E}\,q}{\sqrt{1-q^2}}\arctan\!\left(\frac{\sqrt{1-q^2}\,x}{\psi+s}\right), \qquad \alpha_y = \frac{\theta_{\rm E}\,q}{\sqrt{1-q^2}}\operatorname{artanh}\!\left(\frac{\sqrt{1-q^2}\,y}{\psi+q^2 s}\right),$$

with $\psi = \sqrt{q^2(x^2+s^2)+y^2}$, (x, y) already rotated into the frame aligned with the ellipse's own axes (`lensing.py`'s `_rotate` handles turning the position angle ϕ in and back out around this formula), and s a small core radius that regularizes the point-mass singularity at the center. Gigalens' own `sie.py` defines the identical constant, `s_scale = 1e-4` (`vendor/gigalens-sean/.../mass/sie.py:11`) — but a same-named local variable shadows it to 0 one line into `_param_conv` (`:16`), so gigalens' own SIE actually runs with zero core radius; only the unused class attribute matches `lensing.py`'s choice, not gigalens' executed behavior. Deriving that closed form is a genuine complex-analysis exercise (integrating an elliptical isothermal potential); this guide takes it as a fact and checks the one property that matters: at $q \to 1$, the SIE must reduce to the SIS. `lensing.sie_deflection` and `lensing.sis_deflection`, evaluated at the same point $(\theta_x, \theta_y) = (0.6, 0.8)''$ (so $r = \theta_{\rm E} = 1''$), agree to within $0.057''$ at $q = 0.9$ and to within $0.00064''$ at $q = 0.999$ — shrinking roughly in proportion to $1 - q$, as the closed form's own $\sqrt{1-q^2}$ factors predict. It does not shrink to machine zero, because `sie_deflection` clips q at 0.99999 internally and carries a nonzero core $s = 10^{-4}$ by default (`lensing.py:65,73`) — a real numerical seam, not a bug: the exact $q = 1$ limit is a removable singularity in the formula, and the code sidesteps computing it by staying a hair away from it.

The EPL and gamma

The SIE fixed the density slope at $\gamma = 2$. Ch. 10 already flagged that real ellipticals only *cluster near* isothermal, not sit exactly on it — so the campaign's actual mass model promotes γ from a constant to a free parameter, giving the elliptical power law (EPL): the same isothermal ellipsoid, generalized to $\rho(r) \sim r^{-\gamma}$ for any γ. Ch. 3 already derived the *exponent* this profile projects to: $\Sigma(R) \sim R^{1-\gamma}$, hence $\kappa(R) \sim R^{1-\gamma}$, for any γ — that chapter also showed that the exponent's own literal amplitude, $2A\,I(\gamma)$ in Ch. 3, Eq. 3.1, blows up for an idealized, untruncated halo as $\gamma \to 1^+$ (the p-test threshold on $I(\gamma)$), and explicitly left this chapter the job of saying where the campaign's *actual* prefactor, $(3-\gamma)/2$, comes from.

It comes from Ch. 19's identity, not from that divergent integral. Posit $\kappa(R) = A(\gamma)\,(\theta_{\rm E}/R)^{\gamma-1}$ — the right exponent, an undetermined amplitude — and *demand* that $\bar{\kappa}(\theta_{\rm E}) = 1$, the defining property of the Einstein radius for any profile. Integrating exactly as in the SIS derivation above,

$$\bar{\kappa}(\theta) = \frac{2}{\theta^2}\int_0^{\theta} A\,\theta_{\rm E}^{\gamma-1} t^{1-\gamma}\, t\, dt = \frac{2A}{3-\gamma}\left(\frac{\theta_{\rm E}}{\theta}\right)^{\gamma-1},$$

so $\bar{\kappa}(\theta_{\rm E}) = 2A/(3-\gamma)$; setting this to 1 gives $A = (3-\gamma)/2$ — exactly the constant `epl_kappa` uses (`site/guide_src/lensing.py:81`):

$$\kappa(R) \;=\; \frac{3-\gamma}{2}\left(\frac{\theta_{\rm E}}{R}\right)^{\gamma-1}.$$

This normalization sidesteps Ch. 3's divergence entirely: it is fixed by a convention (mean interior $\kappa = 1$ at one radius), not by integrating an infinite halo, so it stays perfectly finite for every γ the prior allows — including exactly at $\gamma = 1$. Gigalens' own EPL implementation uses the identical exponent under the lenstronomy convention $t = \gamma - 1$ (`vendor/gigalens-sean/.../mass/epl.py:24`); `site/guide_src/lensing.py` is deliberately not a wrapper around it, but the convergence law is the same law.

Two facts fall out of this derivation for free. First, setting $R = \theta_{\mathrm{E}}$ in the boxed formula gives the *local* convergence right at the Einstein ring: $\kappa(\theta_{\mathrm{E}}) = (3-\gamma)/2$ — a different number from the *mean* interior convergence, which is exactly 1 for every γ by construction. At $\gamma = 2$ (isothermal) these give 0.5 : the density right at an isothermal Einstein ring is exactly half the mean density inside it. Second, at $\gamma = 1$ exactly, the exponent $\gamma - 1$ vanishes and $\kappa(R) = 1$ for *every* R, not just at θ_{E} — a perfectly finite, spatially uniform sheet. That is the critical mass sheet: Ch. 21 shows a uniform $\kappa = 1$ sheet is invisible to every imaging observable, the single most degenerate mass distribution in all of lensing. So $\gamma = 1$ is a wall in *two* related but distinct senses — Ch. 3's sense (an idealized untruncated halo's total surface density literally diverges there) and this chapter's sense (even the finite, correctly normalized model becomes totally unidentifiable there) — and the campaign's own prior, `gamma=tfd.TruncatedNormal(2.0, 0.25, 1.0, 2.7)` (`reproductions/claude-giga-lens/cgl/e2.py:110`), places its hard floor at exactly that boundary.

Hold that against the money number. $\gamma_{\mathrm{binned}}(\mathrm{corr}, \mathrm{low}) = 1.103$ gives $\kappa(\theta_{\mathrm{E}}) = 0.9485$ — only 0.0515 short of the fully degenerate sheet's $\kappa(\theta_{\mathrm{E}}) = 1$, and well past isothermal's own 0.5 on the way there. Ch. 10 already flagged this value as 3.588 prior-sigma from isothermal ; this is a sharper, structural way to be worried about the same number, because it says something about *identifiability*, not just prior surprise. The anchor, $\gamma = 1.433$, sits at $\kappa(\theta_{\mathrm{E}}) = 0.7835$ — a real departure from isothermal, but nowhere near the degenerate wall. The fine-product artifact, $\gamma = 2.585$, sits at 0.2075 , on the opposite, steep side. All four values share the same mean interior convergence, exactly 1 at θ_{E} — that identity, checked here for the money slope by direct quadrature, does not care what γ is; only the *local* number does.

> **The gamma overload, paid in full**
>
> This repository uses the single glyph γ for three unrelated objects, and the report's own authors clearly knew it: `reproductions/claude-giga-lens/papers/main.tex` defines separate macros for each (lines 24–27: `\gammaEPL`, `\gext`, `\gextone`, `\gexttwo`). This guide mirrors that discipline exactly:
>
> 1. **Bare** γ — the EPL's 3-D density slope, $\rho \sim r^{-\gamma}$, this section's subject and the campaign's money parameter. This is the *only* meaning bare γ ever carries in this guide.
> 2. γ_{ext} — the external shear's *magnitude*, the next section's subject. Never bare γ, never γ_{sh}.
> 3. γ_1, γ_2 — shear *components*. These name two different but related quantities depending on context: Ch. 5's traceless part of a Jacobian ($A = (1-\kappa)I - \Gamma$, computed from whatever deflection field you differentiate), or the external-shear model's own two parameters (next section). The next section shows these literally coincide when the external field is isolated — but not in general, since a full EPL+shear Jacobian sums a spatially varying piece (from the EPL) and a constant piece (from the shear). Wherever both could appear in the same breath, this guide subscripts the external one: $\gamma_{\mathrm{ext},1}, \gamma_{\mathrm{ext},2}$.

Ellipticity

Every ellipse in this repository's model — the EPL mass, the SIE, and every one of the four lens-light Sersic components plus the source — is parameterized the same way: not by (q, ϕ) directly, but by a pair (e_1, e_2) that gigalens' `epl.py` and `sie.py` both convert identically, implemented once at `site/guide_src/lensing.py:32`:

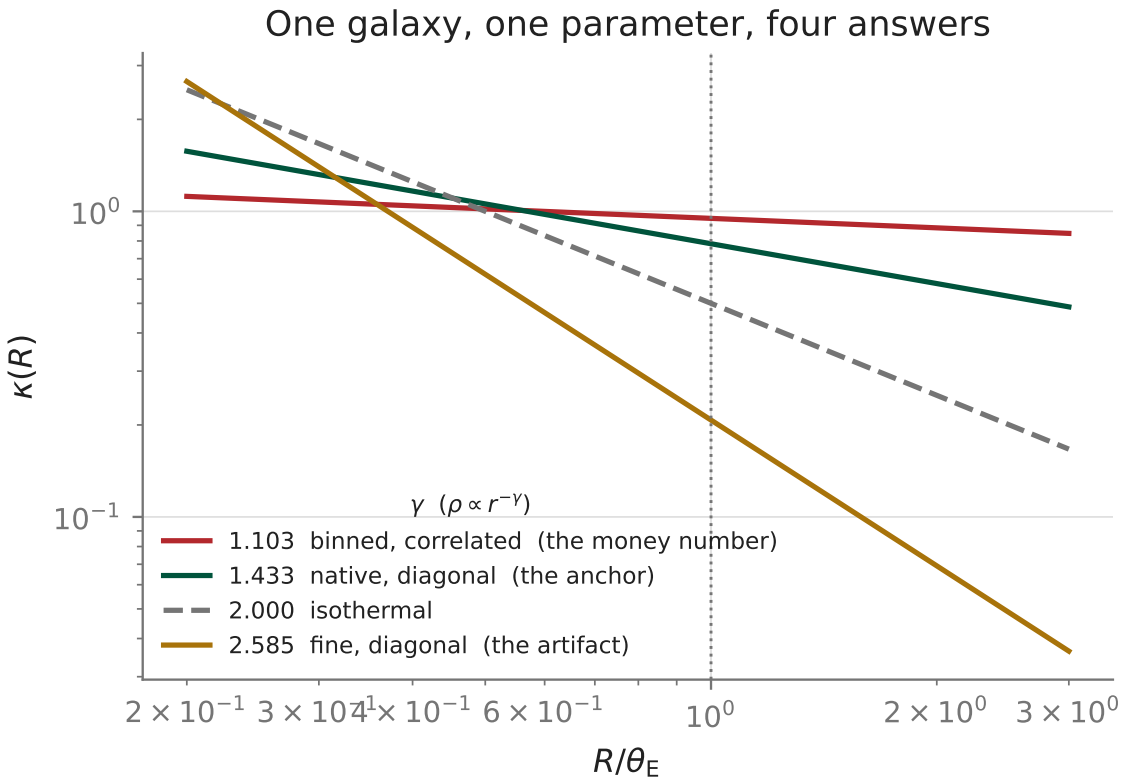

Figure 8: $\kappa(R)$ for the money number ($\gamma = 1.103$), the anchor ($\gamma = 1.433$), the isothermal reference ($\gamma = 2.000$), and the fine-product artifact ($\gamma = 2.585$) — the same four values Ch. 25 adjudicates. Every curve crosses $(3-\gamma)/2$ at $R = \theta_{\mathrm{E}}$ (the dotted line) by construction, and every curve has the identical mean interior convergence there. The money curve is visibly the flattest: a shallow slope spreads mass out instead of concentrating it, which is exactly why its $\kappa(\theta_{\mathrm{E}})$ sits closest to the fully degenerate, perfectly flat sheet at $\gamma = 1$.

$$\phi = \tfrac{1}{2}\,\mathrm{atan2}(e_2, e_1), \qquad c = |e| = \sqrt{e_1^2 + e_2^2}, \qquad q = \frac{1-c}{1+c}.$$

The half-angle is not a stylistic choice. Write the complex ellipticity $\epsilon = e_1 + ie_2$. If $c = |\epsilon|$ is fixed, the map $\phi \mapsto \epsilon = c\,e^{2i\phi}$ sends a *full* 180° rotation of the position angle, $\phi \to \phi + \pi$, to $\epsilon\,e^{2\pi i} = \epsilon$ — the *identical* complex number. That has to be true: an ellipse has no arrowhead, so rotating it by 180° produces the same ellipse, and the parameterization had better not invent a difference where there is none. A *quarter* turn, $\phi \to \phi + \pi/2$, is a different story — it swaps which axis is which, and ϵ must change: $\epsilon\,e^{i\pi} = -\epsilon$.

Take $(e_1, e_2) = (0.3, 0.4)$ — a 3-4-5 triangle, chosen for clean numbers, not a typical draw from the mass ellipticity's own prior (`e1,e2 ~ Normal(0, 0.1)`, `cgl/e2.py:111`, a few times smaller). `ellip_to_q_phi` gives $q = 1/3$ and $\phi = 26.565°$. Reconstructing ϵ at $\phi + \pi$ with the same $c = (1-q)/(1+q) = 0.5$ reproduces $e_1 = 0.3$ exactly — the invariance holds to floating-point precision, as the algebra above promised — while reconstructing at $\phi + \pi/2$ flips the sign, $e_1 = -0.3$.

> **You already know this**
>
> This is exactly the doubled-angle trick used everywhere an "orientation" has no arrowhead — a line, an edge, a nematic director. A structure tensor's dominant eigenvector in image processing is conventionally reported mod π, not mod 2π, for the identical reason: an unsigned gradient direction and its 180°-rotated twin describe the same edge. Spin-2 is the physics name for a doubled-angle representation you have almost certainly already coded.

External shear

Mass sitting outside the modeled lens galaxy — a companion, the group or cluster environment, structure along the line of sight — still bends light, but its potential $\psi_{\rm ext}(x, y)$ near the lens center is smooth and slowly varying compared to the galaxy's own steep potential. Taylor-expand it to second order (Ch. 2): the constant term is an unobservable overall phase, the linear term is a uniform image shift indistinguishable from moving the source (an unobservable degeneracy), so the first term with any observable content is the *quadratic* one — a constant Hessian. If none of that external mass is itself sitting at the lens plane within the modeled region, Ch. 6's $\nabla^2\psi = 2\kappa$ forces the trace of that constant Hessian to vanish locally, leaving only its traceless part: a spatially *uniform* shear, $(\gamma_{\rm ext,1}, \gamma_{\rm ext,2})$, and nothing else. That is the external-shear model, `shear_deflection` (`site/guide_src/lensing.py:93`):

$$\alpha_x = \gamma_{\rm ext,1}\,x + \gamma_{\rm ext,2}\,y, \qquad \alpha_y = \gamma_{\rm ext,2}\,x - \gamma_{\rm ext,1}\,y.$$

The campaign's own prior draws each component independently, $\gamma_1, \gamma_2 \sim \mathrm{Normal}(0, 0.05)$ (`reproductions/claude-giga-lens/cgl/e2.py:113–114`) — note the code's bare parameter names, exactly the collision this chapter's ledger warned about; in this guide's notation those are $\gamma_{\rm ext,1}, \gamma_{\rm ext,2}$. If you want the shear quoted as a single magnitude-and-angle pair the way $\gamma_{\rm ext}$ is defined in the notation contract, it is $\gamma_{\rm ext} = \sqrt{\gamma_{\rm ext,1}^2 + \gamma_{\rm ext,2}^2}$ — the shear's own analogue of ellipticity's $c = |e|$ above.

Differentiate `shear_deflection` alone with `lens_jacobian` (Ch. 5's numerical Jacobian, applied here to a deflection field that has nothing else in it) at one plausible draw,

$\gamma_{\text{ext},1} = 0.03$, $\gamma_{\text{ext},2} = -0.02$ (so $\gamma_{\text{ext}} = 0.036056$), evaluated at an arbitrary point $(0.4, 0.7)''$: `kappa_gamma_from_jacobian` recovers $\kappa = 0$, $\gamma_1 = 0.03$, and $\gamma_2 = -0.02$ — exactly the input, to machine precision, at *every* point you would care to check, because α_x, α_y are linear in (x, y) and a linear map's own derivative has no remainder to miss (Ch. 2's point about Taylor expansion, cashed out as a finite-difference check with zero truncation error). This is the resolution the overload warning promised: external shear is not a *new* kind of object, it is Ch. 5's (γ_1, γ_2) decomposition of the total Jacobian, restricted to just the one additive, spatially constant term that an external tidal field contributes. In a full EPL+shear fit, the total Jacobian sums this constant piece with the EPL's own spatially varying convergence and shear — the two notions of γ_1, γ_2 genuinely coincide only when you isolate the external term this way, which is exactly why the notation contract insists on the subscript everywhere else. One further honest note: an external shear's stretch and the lens's own ellipticity both distort images in visually similar ways, and disentangling the two from imaging data alone is a real, separate degeneracy — Ch. 21 is where degeneracies of this kind get a full treatment; this chapter only flags it.

Source models

Everything so far describes the *lens*: the foreground galaxy's mass (EPL + shear) and light (four Sersic components, Ch. 10). The *source* — the background galaxy actually being lensed, whose light, run through the lens equation, produces the arcs and rings Ch. 22 fits pixel by pixel — needs its own light model, and a single Sersic profile is not flexible enough: real source galaxies have clumps, spiral structure, and asymmetries no smooth elliptical profile captures. The campaign's source model is a Sersic component (its own independent $e_1, e_2 \sim \text{TruncatedNormal}(0, 0.15, -0.5, 0.5)$, a *third* distinct ellipticity prior alongside the mass's $\text{Normal}(0, 0.1)$ and the lens light's $\text{TruncatedNormal}(0, 0.15, -0.4, 0.4)$; `cgl/e2.py:120–125`) *plus* a shapelet expansion layered on top.

Shapelets are 2-D Gauss–Hermite basis functions — a Gaussian envelope times a Hermite polynomial in x and y, truncated at total polynomial order $n_{\max}$ (`vendor/gigalens-sean/.../light/shapelets.py:19`) At $n_{\max} = 6$ — the campaign's choice — the count of independent modes is the triangular number $(n_{\max} + 1)(n_{\max} + 2)/2 = 28$, matching the model's own docstring, "Sersic + Shapelets($n_{\max} = 6$) source with 28 EXPLICIT amps" (`reproductions/claude-giga-lens/cgl/e2.py:13`), each with an independent amplitude parameter and its own Gaussian prior (`cgl/e2.py:126–131`).

> **You already know this**
>
> A truncated orthogonal basis layered on top of a smooth parametric fit to catch residual structure is a move you have made before, whatever you called it — a low-order DCT or wavelet expansion of a residual image, a handful of extra principal components kept after the dominant ones. The physics content here is only which basis (2-D Hermite functions under a Gaussian weight, not sines or wavelets) and how many terms.

Because each shapelet amplitude multiplies a *fixed* rendered image and the 28 contributions simply sum, these amplitudes enter the forward model *linearly* — unlike γ, q, ϕ, or any of the profile shape parameters above, all of which enter nonlinearly. That linearity is exactly what Ch. 22 exploits: the 28 amplitudes get integrated out analytically (a ridge-regularized normal equation, `reproductions/claude-giga-lens/cgl/marg.py`) rather than sampled, which is why the campaign's sampled parameter count drops from 74 dimensions to 46 (`cgl/e2.py:13`). This chapter only names the 28 parameters being marginalized; deriving the marginalization itself is Ch. 22's

job.

Ledger

What this chapter rules in or out about $\gamma = 1.103$: nothing empirically — no data enters this chapter, only definitions. What it does establish is the *structural* stakes. $\gamma = 1$ is not an arbitrary round number for the prior's hard wall: it is exactly where the EPL's local convergence at θ_E becomes a spatially uniform sheet, the most degenerate mass distribution lensing can produce (Ch. 21). The money number's own $\kappa(\theta_E) = 0.9485$ sits only 0.0515 short of it — a sharper, more structural way to read its closeness to the wall than Ch. 10's 3.588-prior-sigma framing , though both point at the same number. Whether the *data* genuinely demand a profile sitting that close to degenerate, or whether some part of the pipeline pushed it there artificially, is not this chapter's question — it belongs to Ch. 21, Ch. 25, and Ch. 26.

Connect to the repo

- `site/guide_src/lensing.py:32` (`ellip_to_q_phi`), `:65` (`sie_deflection`), `:81` (`epl_kappa`), `:93` (`shear_deflection`), `:179` (`mean_kappa_within`) — every function this chapter computes with.
- `reproductions/claude-giga-lens/cgl/e2.py:109`–`114` — the lens-mass prior block: `theta_E`, `gamma` (line 110), `e1`, `e2` (line 111), `gamma1`, `gamma2` (lines 113–114).
- `reproductions/claude-giga-lens/cgl/e2.py:120`–`131` — the source Sersic and shapelet priors, and the 28 amplitude names.
- `vendor/gigalens-sean/.../mass/epl.py:9` and `:24` (`t = gamma - 1`) — the production EPL, the lenstronomy convention.
- `vendor/gigalens-sean/.../mass/sie.py:17`–`19` — the SIE's own $(e_1, e_2) \to (q, \phi)$ conversion, identical to `epl.py`'s.
- `vendor/gigalens-sean/.../light/shapelets.py:19` — the shapelet depth formula, `(n_max+1)*(n_max+2)/2`.
- `reproductions/claude-giga-lens/papers/main.tex:24`–`27` — the report's own `\gammaEPL`/`\gext`/`\gextone`/`\gexttwo` macros, the same disambiguation this chapter makes in prose.
- `reproductions/claude-giga-lens/CAMPAIGN.md:134` — the money number, `gamma_binned(corr, low) = 1.103 ± 0.008`, for one real system, DESI-165.4754−06.0423.

Exercises

Exercise 20.1 — deriving kappa(theta_E) = (3-gamma)/2 by hand

Starting from $\kappa(R) = A(\theta_E/R)^{\gamma-1}$ and the requirement $\bar{\kappa}(\theta_E) = 1$, redo the integral in The EPL and gamma to solve for A, then evaluate $\kappa(\theta_E)$ at $\gamma = 2$ and at $\gamma = 1$. Explain in one sentence why the $\gamma = 1$ answer is special.

Solution

$\bar{\kappa}(\theta) = \frac{2A}{3-\gamma}\left(\frac{\theta_E}{\theta}\right)^{\gamma-1}$, so $\bar{\kappa}(\theta_E) = \frac{2A}{3-\gamma} = 1 \Rightarrow A = \frac{3-\gamma}{2}$. Then $\kappa(\theta_E) = A \cdot 1^{\gamma-1} = A = (3-\gamma)/2$: at $\gamma = 2$, $\kappa(\theta_E) = 0.5$; at $\gamma = 1$, $\kappa(\theta_E) = 1.0$. The $\gamma = 1$ case is special because the *exponent* $\gamma - 1$ is also zero there, so $\kappa(R) = 1$ everywhere, not just at θ_E — the local and mean statements coincide only at this one value of γ.

Exercise 20.2 — the SIE-to-SIS floor

Using `lensing.sie_deflection` and `lensing.sis_deflection` at $(\theta_x, \theta_y) = (0.6, 0.8)''$, $\theta_{\rm E} = 1''$, evaluate the gap at $q = 0.999999$ (six nines) and compare it to the $q = 0.999$ gap quoted in the main text. Why does pushing q three orders of magnitude closer to 1 not shrink the gap by a comparable factor?

Solution

```
import lensing as L
import numpy as np
x, y = np.array([0.6]), np.array([0.8])
sis = L.sis_deflection(x, y, 1.0)
sie = L.sie_deflection(x, y, 1.0, 0.999999, 0.0)
```

The gap at $q = 0.999999$ is 1.046×10^{-4} — pushing q a thousand times closer to 1 than $q = 0.999$ only shrank the gap from 6.39×10^{-4} by a factor of six, not by another thousand. It has hit a floor. The reason is in the code, not the algebra: `sie_deflection` clips its own q argument to a maximum of 0.99999 before doing anything else (`site/guide_src/lensing.py:73`), so passing $q = 0.999999$ silently computes with $q = 0.99999$ instead. Combined with the nonzero core radius $s = 10^{-4}$ that never vanishes, the *implemented* function cannot get arbitrarily close to the SIS — only the *mathematical* limit can. A derivation checked against code always inherits the code's own approximations.

Exercise 20.3 — the quarter-turn, algebraically

Using $\epsilon = c\, e^{2i\phi}$, show algebraically why $\phi \to \phi + \pi/2$ flips the sign of ϵ while $\phi \to \phi + \pi$ leaves it unchanged, and confirm your answer against the numbers in Ellipticity.

Solution

$\epsilon(\phi + \pi/2) = c\, e^{2i(\phi+\pi/2)} = c\, e^{2i\phi} e^{i\pi} = -c\, e^{2i\phi} = -\epsilon(\phi)$, while $\epsilon(\phi + \pi) = c\, e^{2i\phi} e^{2i\pi} = c\, e^{2i\phi} = \epsilon(\phi)$, since $e^{2\pi i} = 1$ but $e^{i\pi} = -1$. The factor of 2 in the exponent is exactly what turns a 180° geometric rotation into a *full* 360° turn of the complex phase (a no-op) while a 90° geometric rotation becomes only a 180° phase turn (a sign flip) — matching $e_1(\phi + \pi) = 0.3$ and $e_1(\phi + \pi/2) = -0.3$ exactly.

Exercise 20.4 — where the mean-kappa identity gets numerically hard

Using `mean_kappa_within`, compute $\bar{\kappa}(\theta_{\rm E})$ for $\gamma = 1.103$ and for $\gamma = 2.585$. Both should equal 1 exactly by the derivation in this chapter. One of them does, to nine decimal places; the other misses by half a percent. Which, and why?

Solution

```
import lensing as L
import numpy as np
f = lambda t, g: L.epl_kappa(t, np.zeros_like(t), 1.0, g, 1.0)
L.mean_kappa_within(lambda t: f(t, 1.103), 1.0)   # 0.999999999903
L.mean_kappa_within(lambda t: f(t, 2.585), 1.0)   # 0.993810340006
```

At $\gamma = 1.103$, $\bar{\kappa}(\theta_{\rm E}) = 0.999999999903$ — indistinguishable from the exact 1. At $\gamma = 2.585$, it comes out to 0.993810 , off by half a percent. `mean_kappa_within` integrates $\kappa(t)\,t$ on a *linear* grid starting a small distance from $t = 0$. The integrand behaves like $t^{2-\gamma}$ near the origin: for $\gamma = 1.103$, the exponent $2-\gamma = 0.897$ is positive, so the integrand vanishes smoothly at the center and a linear grid resolves it fine. For $\gamma = 2.585$, $2-\gamma = -0.585$ is negative — the integrand *diverges* as $t \to 0$ (integrably, but steeply), and a linear grid starting at a finite $t_{\rm min} = \theta_{\rm E}/n$ undersamples that cusp. This is a numerical-integration artifact of this exercise's grid, not a claim about the physical profile — but it is a fair warning that steep EPL slopes ($\gamma > 2$, like the fine-product $\gamma = 2.585$ artifact this campaign actually encountered) are cuspier at the center and genuinely harder to render and integrate accurately than shallow ones, independent of any noise or likelihood issue.

21. Degeneracies: the directions where the data says nothing

Every inverse problem has directions in parameter space where the data cannot tell you anything: change the parameter, and every quantity you can measure stays exactly the same. Strong lensing has an unusually clean example of this — you can prove it in four lines, from the lens equation alone — and this repository has run into a member of its family twice: once under the wrong name, once under no name at all. This chapter derives the exact case (the mass sheet), shows what it costs a H_0 measurement, generalizes it to the messier case this campaign's flexible source model actually exhibits, and gives you the one-line test that tells a real degeneracy from a merely stubborn posterior — a test Ch. 5 and Ch. 26 both need.

> **What you can skip**
>
> The linear algebra here is old news, restated: $A_{\lambda_{\text{MST}}} = \lambda_{\text{MST}} A$ is a scalar multiple of a matrix, and $\det(cA) = c^2 \det A$ for a 2×2 is one line from Ch. 5. If you already think fluently in terms of non-identifiable parameters, flat loss directions, and gauge symmetries — $GL(r)$ invariance in matrix factorization, the scaling and permutation symmetries of a ReLU net, rotational ambiguity in ICA — skim "A degeneracy is a gauge symmetry" for the lensing vocabulary only. What is not boilerplate: the actual mass-sheet family, why it makes H_0 from time delays a hard measurement in principle (not just in practice), and the two times this repository ran into a member of this family — handled correctly once, incorrectly once — for reasons worth knowing before you trust either verdict.

The mass-sheet degeneracy

No lens galaxy sits alone. A foreground group, a cluster halo, or just the large-scale structure along the line of sight always contributes some extra mass, and over the tiny patch of sky an Einstein ring occupies (order one arcsecond) that extra convergence is, to good approximation, *constant* — a uniform sheet laid across the field. The question this section answers: can you tell, from the images alone, how much of the lensing you are looking at is the galaxy and how much is the invisible sheet in front of and behind it?

This guide reserves the bare symbol λ for the SMC tempering parameter of Ch. 23; to keep the two apart on sight, every mass-sheet parameter in this chapter carries a subscript, λ_{MST}. Define a one-parameter family of potentials built from any lens potential $\psi(\theta)$:

$$\psi_{\lambda_{\text{MST}}}(\theta) \;=\; \lambda_{\text{MST}}\, \psi(\theta) \;+\; (1 - \lambda_{\text{MST}})\, \tfrac{1}{2}|\theta|^2.$$

Apply $\kappa = \frac{1}{2}\nabla^2\psi$ (Ch. 17) and the one-line fact that $\nabla^2(\frac{1}{2}|\theta|^2) = 2$ in two dimensions:

$$\kappa_{\lambda_{\text{MST}}}(\theta) = \lambda_{\text{MST}}\, \kappa(\theta) + (1 - \lambda_{\text{MST}}).$$

That *is* the sheet: a uniform layer of convergence $(1 - \lambda_{\text{MST}})$ — the same number everywhere — sitting on top of the original galaxy rescaled by λ_{MST}. Differentiating () once gives the deflection, $\alpha_{\lambda_{\text{MST}}}(\theta) = \nabla\psi_{\lambda_{\text{MST}}} = \lambda_{\text{MST}}\alpha(\theta) + (1 - \lambda_{\text{MST}})\theta$, and substituting into the lens equation (Ch. 17), $\beta = \theta - \alpha(\theta)$, collapses everything:

$$\beta_{\lambda_{\text{MST}}}(\theta) = \theta - \alpha_{\lambda_{\text{MST}}}(\theta) = \lambda_{\text{MST}}[\theta - \alpha(\theta)] = \lambda_{\text{MST}}\, \beta(\theta).$$

Four lines: () defines the family, one Laplacian gives (), one gradient gives the deflection, one substitution gives (). Now read () as a statement about *images*. Suppose the true lens ($\lambda_{\rm MST} = 1$) produces two images θ_1, θ_2 of a single source at β_0 — take the SIS double Ch. 17 solves, $\theta_{\rm E} = 1$, $\beta_0 = 0.4$, giving $\theta_1 = 1.4$ and $\theta_2 = -0.6$. Equation () says that for *any* $\lambda_{\rm MST}$, plugging those same two image positions into the transformed lens returns $\beta_{\lambda_{\rm MST}}(\theta_1) = \beta_{\lambda_{\rm MST}}(\theta_2) = \lambda_{\rm MST}\beta_0$ — still one self-consistent source position, just the wrong one, off by exactly $\lambda_{\rm MST}$. Checked directly at $\lambda_{\rm MST} = 0.8$: both images return 0.320 (their disagreement is 2×10^{-16} , pure floating-point noise), exactly 0.8×0.4 . Image astrometry — and, by the identical argument applied to two *different* sources, image separations and image-count statistics — cannot see $\lambda_{\rm MST}$ at all.

Magnification is the same story one derivative further. Since $A = I - \mathrm{Hess}(\psi)$ (Ch. 5) and Hess of the added quadratic term is the identity matrix, $A_{\lambda_{\rm MST}} = \lambda_{\rm MST} A$ — literally a scalar multiple, so $\det A_{\lambda_{\rm MST}} = \lambda_{\rm MST}^2 \det A$ and $\mu_{\lambda_{\rm MST}} = \mu/\lambda_{\rm MST}^2$ at *every* image, uniformly. The overall image brightness moves — but it is degenerate with the source's own unknown intrinsic luminosity anyway, so nothing is lost. The *ratio* of magnifications between two images of one source — the flux ratio, which imaging genuinely does measure without knowing the source's true brightness — cancels the $\lambda_{\rm MST}^2$ exactly and is invariant. Even the arc shapes survive: a scalar multiple of A keeps every eigenvector (orientation) and every eigenvalue *ratio* (axis ratio) fixed, only the scale moves — and that scale is absorbed by the same source-size ambiguity as the flux. Every observable you can build from where the images are, how their brightnesses compare, or what shape they trace is provably blind to $\lambda_{\rm MST}$. That is what "the data says nothing" means in this chapter's title — literally, not rhetorically. One observable is not blind to it.

Time delays and H_0

Multiple images of one source do not arrive at the same time: each ray travels a different path through a different depth of the potential well, and general relativity says the light-travel time along image θ depends on the **Fermat potential**

$$\tau(\theta;\beta) = \tfrac{1}{2}|\theta - \beta|^2 - \psi(\theta),$$

with the observed delay between two images $\Delta t_{ij} = (D_{\Delta t}/c)\big[\tau(\theta_i;\beta) - \tau(\theta_j;\beta)\big]$. The distance factor is the *time-delay distance* $D_{\Delta t} \equiv (1+z_{\rm d}) D_{\rm d} D_{\rm s}/D_{\rm ds}$ (Ch. 15 built $D_{\rm d}$, $D_{\rm s}$, $D_{\rm ds}$, and reminded you they do not add). Every angular-diameter distance is H_0 times a dimensionless integral over $\Omega_{\rm m}, \Omega_\Lambda$ and redshift alone (Ch. 14), so one net factor of H_0 survives the product-over-quotient: $D_{\Delta t} \propto 1/H_0$ at fixed cosmology. This is the entire basis of time-delay cosmography — measure Δt, model $\Delta\tau$, read off $D_{\Delta t}$, read off H_0.

Now transform. Expand $\tau_{\lambda_{\rm MST}}(\theta; \lambda_{\rm MST}\beta) - \lambda_{\rm MST}\,\tau(\theta;\beta)$ using (): every term carrying $\theta \cdot \beta$ or $|\theta|^2$ cancels, leaving a remainder that does not depend on θ at all:

$$\tau_{\lambda_{\rm MST}}(\theta; \lambda_{\rm MST}\beta) = \lambda_{\rm MST}\,\tau(\theta;\beta) - \tfrac{1}{2}\lambda_{\rm MST}(1-\lambda_{\rm MST})|\beta|^2.$$

The offset is the same constant at every image of one source, so it cancels in any *difference*, and differencing is all a time delay is:

$$\Delta\tau_{\lambda_{\rm MST}} = \lambda_{\rm MST}\,\Delta\tau.$$

On the SIS toy, $\Delta\tau = -0.8$ at $\lambda_{\rm MST} = 1$ and -0.64 at $\lambda_{\rm MST} = 0.8$ — a ratio of 0.8 , exactly $\lambda_{\rm MST}$, confirming () to machine precision.

Here is the cost. $\Delta t_{\rm obs}$ is fixed data, independent of which member of the family you fit. Standard practice assumes no sheet — $\lambda_{\rm MST} = 1$ — and solves $D_{\Delta t}^{\rm assumed} = c\,\Delta t_{\rm obs}/\Delta\tau^{\rm assumed}$. If the true system actually sits at some $\lambda^\star < 1$ (a real, unmodeled sheet), then by () the assumed model's own Fermat difference is $\Delta\tau^{\rm assumed} = \Delta\tau^{\rm true}/\lambda^\star$ — *larger* in magnitude — so it needs a *smaller* $D_{\Delta t}$ to reproduce the same $\Delta t_{\rm obs}$. Since $D_{\Delta t} \propto 1/H_0$, a smaller inferred distance means a larger inferred H_0:

$$H_0^{\rm assumed} = \frac{H_0^{\rm true}}{\lambda^\star}.$$

At $\lambda^\star = 0.8$ that is H_0 high by a factor of 1.25 — a 20% unmodeled sheet produces a 25% overestimate of the Hubble constant, and imaging alone can never catch it, because the last section just showed imaging cannot see $\lambda_{\rm MST}$ at any value. This is the reason time-delay H_0 measurements quote a systematic budget dominated by "mass-sheet" or "external convergence," not by astrometry.

What *can* see it: a measurement that depends on mass, not on light-bending geometry. The lensing galaxy's stellar velocity dispersion σ_v (Ch. 10, Ch. 19) is exactly that — DESI's own Foundry survey notes it in as many words: *"the velocity dispersion of the lensing galaxies can be used to break the mass-sheet degeneracy in lens modeling"* (`reproductions/foundry-ii/data/paper_text.txt:1529`). A dynamical mass and a lensing mass agreeing is not a formality; it is the only kind of evidence this particular degeneracy cannot fake.

Source versus mass

The mass-sheet transformation is the clean special case of a more general fact: *any* smooth change to the deflection field can be exactly compensated by a corresponding change to the (unknown) reconstructed source, provided the source model is flexible enough to absorb it. MST is the special case where the change is a uniform sheet and the compensation is a pure rescale of the source position; nothing in the argument requires the compensation to be that simple. This matters directly here because this campaign's forward model does not fit a fixed source — it fits a Sérsic core plus 28 linear shapelet amplitudes, marginalized analytically (`reproductions/claude-giga-lens/cgl/marg.py:31`, Ch. 22), a genuinely flexible basis with room to reshape itself around a change in the mass model.

This repository has met a member of this family twice, and got it right only once. The Foundry-I reproduction first described soft, ill-conditioned posterior directions coupling γ, e_1/e_2, and $\gamma_{\rm ext}$ as "mass-sheet / slope-ellipticity degeneracies" (`reproductions/foundry-i/papers/main.tex:503`) — and then retracted the label:

> *"An earlier draft labeled this 'mass–sheet, slope–ellipticity' degeneracies; we retract those labels — nothing in this single-band, fixed-source analysis invokes the mass-sheet transformation, and the slope–ellipticity coupling was an empirical observation of this posterior, not a citation-backed degeneracy. The valley largely closed once the likelihood was corrected."* (`reproductions/foundry-i/papers/evolution.tex:713`)

The retraction is correct for a reason worth internalizing: that campaign's source was *fixed*, not flexible, and its parametrization contains no explicit uniform-κ sheet term at all — so $\lambda_{\rm MST}$ was

never actually a free direction of that model, whatever the posterior looked like. An ill-conditioned valley is evidence something is *underconstrained*; it is not, by itself, evidence of *which* named degeneracy is responsible, and here the valley turned out to be a symptom of a misspecified likelihood, not a structural symmetry — it shrank once the likelihood was fixed rather than once more data arrived, which is exactly the signature of the former, not the latter.

The claude-giga-lens campaign's own forward model, by contrast, *does* have a flexible source, and it ran into a genuine source-versus-mass ambiguity there: a Sérsic-versus-shapelet source-*centre* degeneracy, two clusters at -0.15 and $+0.09$, with $\hat{R} = 22.3/15.1$ (`reproductions/claude-giga-lens/CAMPAIGN.md:179`, `reproductions/claude-giga-lens/papers/main.tex:`
This one is real — the source model is flexible enough for the mechanism to operate — but the campaign explicitly checked whether it threatens γ, rather than assuming either way, and found it decoupled. Ch. 26 is where you verify that claim yourself rather than take it on faith; this chapter's job is only to hand you the concept and the standard of proof it was held to. "Degeneracy" is not an adjective you attach to any soft direction you find. It is a specific, checkable claim: exhibit the transformation, in the model you actually fit, that leaves the likelihood exactly unchanged. If you cannot exhibit it, you have found ill-conditioning, which is a different disease with a different cure.

A degeneracy is a gauge symmetry

Put the last three sections in one sentence: a degeneracy is a direction v in parameter space along which the log-likelihood does not change at all, $\mathcal{L}(x_0 + \epsilon v) = \mathcal{L}(x_0)$ for every ϵ, not just to first order. Differentiate that statement twice and v is a null vector of the Hessian — $v^\top H v = 0$ — which is exactly a zero eigenvalue (Ch. 5). A family of such directions, indexed continuously the way λ_{MST} indexes one, is precisely what a physicist calls a **gauge symmetry**: a continuous group of transformations of the parameters that leaves every observable prediction untouched. $\lambda_{\mathrm{MST}} \in \mathbb{R}$ is a one-parameter Lie group acting on (mass model, source position) jointly, and () is the statement that the likelihood built from image positions is invariant under its action — a textbook example, not a metaphor.

> **You already know this**
>
> A rank-r factorization $W = UV^\top$, the kind behind matrix completion, linear autoencoders, and embedding tables, has the identical structure: for any invertible $T \in GL(r)$, $(UT)(T^{-1}V^\top) = UV^\top$ reproduces the *same* W and therefore the same loss, for every T. The minimum is not a point; it is an r^2-dimensional manifold of equally good solutions, and the loss Hessian evaluated anywhere on that manifold has exactly r^2 zero eigenvalues — one per direction you can move along the manifold for free. λ_{MST} is that same T, specialized to a one-parameter subgroup acting on a lens model instead of a factorization.

Three failure modes get conflated in practice and are worth pulling apart, because this book meets all three:

- **An exact degeneracy** (zero eigenvalue, a real gauge symmetry) is *permanent*. No amount of more imaging data shrinks it, because the transformation leaves every imaging observable exactly fixed by construction — you need a qualitatively different observable (σ_v, not another image) to touch it at all. The mass-sheet family is one.
- **Ill-conditioning** (a small but nonzero eigenvalue, cond $\to$ large but not infinite) is a statement about how much *this* dataset, at *this* precision, constrains a direction — and it can

shrink with a corrected likelihood or better data, as Foundry-I's retracted valley did. It is not protected by any symmetry and should never borrow one's name without exhibiting the transformation.

- **A saddle** (a *negative* eigenvalue) is neither of the above: it says the log-posterior *decreases* moving away from that point in both directions along v, which is a statement about a bad reference point, not about an invariance of the model. Ch. 26 meets this one directly — the same $V\,\mathrm{diag}(1/|\lambda_i|)\,V^\top$ formula that correctly builds a descent direction at a saddle is invalid as a covariance for exactly this reason.

cond $\to \infty$ is the limit where the second failure mode becomes indistinguishable, in finite precision, from the first — which is why Ch. 5's cond $\sim 10^{14}$ number and this chapter's λ_{MST} are two ends of the same idea, not two different ones.

Ledger

What this chapter rules in or out about $\gamma = 1.103$: two acquittals, no verdict. First, the broad, ill-conditioned valley coupling γ, e_1/e_2, and γ_{ext} that Foundry-I once called a mass-sheet degeneracy was retracted for good reason — that model contains no uniform-κ sheet term, so λ_{MST} was never actually loose there, and the valley closed with the likelihood, not with more data. That result does not transfer automatically to claude-giga-lens, but it sets the correct standard of proof for the next item. Second, the one real source-versus-mass ambiguity this campaign's flexible shapelet source does exhibit — the source-centre $\hat{R} = 22.3$ — is explicitly checked and found decoupled from γ (`reproductions/claude-giga-lens/CAMPAIGN.md:179`); Ch. 26 is where you confirm the mechanism rather than the claim. Neither incident touches the residual scale-dependence — 1.103 against a 1.433 anchor — that Ch. 25 attributes to source-model and PSF systematics. This chapter clears two suspects, not the crime.

Connect to the repo

- `reproductions/foundry-i/papers/evolution.tex:713` (github) and `reproductions/foundry-i/papers/` (github) — the retracted "mass-sheet / slope-ellipticity" label, and why the retraction is correct.
- `reproductions/claude-giga-lens/CAMPAIGN.md:179` (github) and `reproductions/claude-giga-lens/pa` (github) — the real source-centre $\hat{R} = 22.3$, decoupled from γ; Ch. 26 verifies it.
- `reproductions/claude-giga-lens/cgl/marg.py:31` (github) — `marg_loglik`, the 28-amplitude analytic marginalization that makes this campaign's source flexible enough for a source-versus-mass ambiguity to bite at all.
- `site/guide_src/lensing.py:54` (github) — `sis_deflection`, the function the worked example's algebra mirrors by hand.
- `site/guide_src/worked_examples.py` (`ch21_mass_sheet_degeneracy`) — the numeric checks behind every tagged number in this chapter.

Exercises

Exercise 21.1 — Verify the invariance yourself

Using $\theta_{\mathrm{E}} = 1$, $\beta_0 = 0.4$, images $\theta_1 = 1.4$, $\theta_2 = -0.6$, and $\lambda_{\mathrm{MST}} = 0.8$: compute $\alpha_{\lambda_{\mathrm{MST}}}(\theta) = \lambda_{\mathrm{MST}}\,\theta_{\mathrm{E}}\,\mathrm{sign}(\theta) + (1 - \lambda_{\mathrm{MST}})\theta$ at both images, then $\beta_{\lambda_{\mathrm{MST}}}(\theta) = \theta - \alpha_{\lambda_{\mathrm{MST}}}(\theta)$. Do the two images agree on a single source position, and is it $\lambda_{\mathrm{MST}}\beta_0$?

Solution

$\alpha_{\lambda_{\rm MST}}(1.4) = 0.8(1) + 0.2(1.4) = 1.08$, so $\beta_{\lambda_{\rm MST}}(1.4) = 1.4 - 1.08 = 0.32$. $\alpha_{\lambda_{\rm MST}}(-0.6) = 0.8(-1) + 0.2(-0.6) = -0.92$, so $\beta_{\lambda_{\rm MST}}(-0.6) = -0.6 - (-0.92) = 0.32$. Both give 0.32 $= 0.8 \times 0.4$ exactly. The transformed lens is just as consistent with "one source" as the true one — imaging cannot distinguish them.

Exercise 21.2 — Derive the H_0 bias formula symbolically

Starting from $\Delta\tau_{\lambda_{\rm MST}} = \lambda_{\rm MST}\Delta\tau$ and $\Delta t_{\rm obs} = (D_{\Delta t}/c)\Delta\tau$, with $\Delta t_{\rm obs}$ fixed, derive $H_0^{\rm assumed}/H_0^{\rm true}$ as a function of $\lambda^\star$ alone (do not plug in 0.8).

Solution

Both descriptions reproduce the same data: $D_{\Delta t}^{\rm assumed}\Delta\tau^{\rm assumed} = D_{\Delta t}^{\rm true}\Delta\tau^{\rm true}$ $(= c\,\Delta t_{\rm obs})$. Substitute $\Delta\tau^{\rm true} = \lambda^\star\Delta\tau^{\rm assumed}$ (the assumed model sits at $\lambda_{\rm MST} = 1$ relative to the true one at $\lambda^\star$): $D_{\Delta t}^{\rm assumed} = \lambda^\star D_{\Delta t}^{\rm true}$. Since $D_{\Delta t} \propto 1/H_0$, inverting gives $H_0^{\rm assumed} = H_0^{\rm true}/\lambda^\star$ — no approximation, exact at any $\lambda^\star$, and it reproduces 1.25 at $\lambda^\star = 0.8$.

Exercise 21.3 — What would make the retraction wrong?

Foundry-I retracted its "mass-sheet degeneracy" label because the fitted model has no explicit uniform-κ term and a fixed source. State, in one sentence, what you would need to see in a model's parametrization (not its posterior) to justify the label, and why a wide, hard-to-sample valley is not by itself sufficient evidence.

Solution

You need the family itself to be exhibitable: some parameter or parameter combination whose variation, holding the fit's *other* free choices free to compensate (in this case, a source that can rescale), leaves the predicted images and flux ratios provably unchanged — i.e. you need to write down () or its analogue for your specific model and confirm it holds identically, not approximately. A wide valley only tells you the posterior is weakly informative in some combination of directions; it says nothing about *why*, and — as here — the reason is sometimes a fixable likelihood defect rather than a structural symmetry. Ill-conditioning is a symptom common to both; only the exhibited transformation distinguishes the diagnosis.

Exercise 21.4 — Saddle, degeneracy, or neither?

Chapter 5 records a Laplace Hessian with minimum eigenvalue -14.85 and five negative directions at the campaign's polished MAP. Is that a mass-sheet-style degeneracy? If not, what single number from that same Hessian *would* signal one, and what value would it need?

Solution

No. A negative eigenvalue means the log-posterior *decreases* moving away from that point along that direction on both sides — a local maximum along every other direction but a minimum along this one, i.e. a saddle, which is a statement about a bad reference point. A genuine degeneracy needs a eigenvalue of exactly 0 (flat, not curved either way) at the *true* mode, not at a misplaced one. The campaign's own diagnosis, in fact, finds the high-density point elsewhere entirely (, higher posterior density than the saddle-consistent MAP) — which is exactly why a Hessian built at the wrong point cannot be trusted as a covariance, degenerate or not, and why Ch. 26 exists.

22. Lens modelling as inference

Chapters 17 through 21 built the mathematics of a single ray: how one point on the sky maps to one point in the source plane, and how the Jacobian of that map turns a source's true brightness into an observed image's brightness. This chapter assembles those rays into the actual machine `claude-giga-lens` runs against real pixels — a parameterized generative model of an entire HST cutout, differentiable end to end, evaluated by one JAX function every time a sampler needs a number. You will count, by hand, the exact parameter budget that machine carries: 74 numbers wide, until you notice that 28 of them are linear-regression coefficients that never need to be sampled at all, and that the Gaussian integral removing them carries the same Occam term Chapter 8 derived on a five-point toy. Every γ this book has quoted so far — the money number included — is a byproduct of exactly this function.

> **What you can skip**
>
> You already own differentiable rendering, autodiff through convolutions, and ridge regression as a closed-form linear solve; none of that is re-derived here. What is new is the wiring: which physical model this repo actually samples, why 28 of its 74 parameters vanish before a sampler ever sees them, and where $-\frac{1}{2}\log\det A$ shows up as a running number instead of a textbook term.

The forward model

A lens model is not one equation; it is a small graphics pipeline with a physics constraint bolted onto its first step. `cgl/likelihood.py`'s `build_marg_model` (`reproductions/claude-giga-lens/cgl/likelihood.py`) builds exactly this pipeline for one real system, DESI-165.4754−06.0423, at whichever drizzle scale you hand it. Four ingredients go in:

- **The mass model.** An EPL (elliptical power law, Chapter 20) plus external shear (Chapter 20) bend light. Six numbers parameterize the EPL — $\theta_{\rm E}, \gamma, e_1, e_2$, and a center — and two more parameterize the shear, γ_1 and γ_2. γ here is unambiguous: it is the EPL density slope of Chapter 20, never the shear. The two shear components are always written with their subscripts, never a bare γ.
- **The lens light.** Four Sérsic profiles (Chapter 10) model the light of the deflecting galaxy and a nearby companion, unlensed — they sit in the image plane directly, at θ, with no ray tracing involved. Each carries seven parameters: `R_sersic, n_sersic, e1, e2, center_x, center_y, Ie` (`reproductions/claude-giga-lens/cgl/likelihood.py:299-303`).
- **The source, Sérsic part.** One more seven-parameter Sérsic profile, this one lensed: evaluated not at θ but at the source-plane position β the lens equation (Chapter 17) sends it to.
- **The source, shapelet part.** A basis of 28 shapelet functions at order $n_{\rm max} = 6$ — a triangular-number count, $(n_{\rm max}{+}1)(n_{\rm max}{+}2)/2$, already computed in Chapter 20 — plus three more parameters that set the basis's pose: a scale radius the code calls `beta`, and a center (x, y). That code identifier is *not* this guide's β: it is the shapelet basis's characteristic size (the Refregier 2003 convention), a genuinely different quantity that happens to share a name. This guide keeps them apart on purpose — the contract that reserves β for source-plane position exists precisely so a collision like this one cannot leak into the notation.

Rendering composes these into one image. On a grid oversampled by a factor S relative to the data's own pixel scale (`supersample` in the code — for the binned 0.08″ product this is

a factor of $S = 2$, turning its 130×130 pixel grid into a 260×260 ray-tracing grid), every image-plane point θ solves the lens equation, every light profile is evaluated, the sum is convolved with the PSF kernel at that *same* oversampled resolution (`jax.lax.conv_general_dilated`, `reproductions/claude-giga-lens/cgl/likelihood.py:342-345`), and only then is the result average-pooled back down by the supersample factor to the data's own pixel grid (`average_pool_2d`, `reproductions/claude-giga-lens/cgl/likelihood.py:346-349`):

$$\beta = \theta - \alpha(\theta), \qquad I(\theta) = \sum_{k=1}^{4} L_k(\theta) \; + \; L_{\text{src}}(\beta), \qquad M = \text{pool}_S(\text{PSF} * I).$$

The order in Equation () is not a stylistic choice: PSF convolution happens *before* the average-pool, on the oversampled grid, because a real point-spread function has structure at scales finer than the data pixel and pooling first would throw that structure away. It also means the PSF kernel handed to the convolution must itself be sampled at the oversampled scale, not the native one — get that wrong and the effective PSF silently broadens, which is exactly the failure Chapter 8 already showed you the cost of: χ^2_ν floored at 3.4 until `assert_psf_sampling` (`reproductions/claude-giga-lens/cgl/guards.py:74-91`) made this exact ordering mistake impossible to repeat silently, dropping the floor to 1.05 . Finally, M is compared against the observed data pixel by pixel through a residual $R = Y - M$, weighted either by a diagonal noise model (Chapter 8) or a whitening operator (Chapter 24) — the comparison this whole render exists to feed.

> **You already know this**
>
> Strip away the astronomy vocabulary and `build_marg_model` is a differentiable renderer of exactly the kind you have written for graphics or vision: a coordinate transform (the lens equation) feeds a scene description (five Sérsic profiles plus a shapelet basis) evaluated on an oversampled grid, blurred by a known point-spread function, downsampled to the sensor's pixel grid, and compared to a real photograph pixel by pixel. The whole pipeline is one `@jax.jit`-compiled function, and every step in it is differentiable — the only reason gradient-based inference (Chapter 23) is on the table at all.

Add every component's parameter count and you get the model's total width:

$$\underbrace{8}_{\text{mass + shear}} + \underbrace{28}_{\text{4×lens light}} + \underbrace{7}_{\text{source Sérsic}} + \underbrace{3}_{\text{shapelet pose}} + \underbrace{28}_{\text{shapelet amplitudes}} = 74.$$

That total is not a guide invention: it is the model `foundry-i`'s `_hmc_lib.py` samples in full (`reproductions/foundry-i/README.md:217`), and it is exactly what `reproductions/claude-giga-lens/cgl/e2` calls "74-dim PAPER-scale z-vectors (EPL+Shear, 4 lens-light Sérsic, Sérsic + Shapelets($n_{\max} = 6$) source with 28 EXPLICIT amps)."

Marginalising linear amplitudes

Twenty-eight of those 74 numbers are not like the other 46. Every shapelet amplitude enters the render in Equation () *linearly*: hold the mass, shear, lens-light, and source-pose parameters fixed, and the rendered image is an exact linear combination of 28 fixed "basis" images, one per shapelet

function, each carried through the identical lens-equation → PSF-convolve → pool pipeline as everything else (`_design_ret`, `reproductions/claude-giga-lens/cgl/likelihood.py:329-352`). That is the entire justification for treating them differently: a parameter that appears linearly, under a Gaussian likelihood and a Gaussian prior, has an *exactly* Gaussian conditional posterior given everything else — which is precisely Chapter 8's claim that the Laplace approximation is exact for a linear-Gaussian model, now invoked for a real 28-dimensional linear subspace instead of a one-parameter toy.

Collect the 28 unit-amplitude basis images into a design matrix X_w (one whitened column per shapelet function, `reproductions/claude-giga-lens/cgl/likelihood.py:356-367`) and the whitened residual into R_w. Chapter 8 already derived the normal equations for exactly this setup:

$$A = X_w^\top X_w + \Lambda, \qquad b = X_w^\top R_w, \qquad \hat{a} = A^{-1} b \quad \text{(Cholesky, never a literal inverse).}$$

`marg.py`'s ridge precision $\Lambda_{ii} = (i{+}1)/25$ (`reproductions/claude-giga-lens/cgl/marg.py:38-39`) is a smoothness prior: higher-order (wigglier) shapelet functions are penalized harder, so A stays positive-definite and the Cholesky solve — never `pinv` — is well posed. A here is a 28×28 matrix: exactly one row and column per marginalized amplitude, no larger. It does *not* grow with the other 46 parameters; those enter only by reshaping X_w and R_w each time A is recomputed, since changing the mass model changes which pixels the source light lands on.

Not every linear parameter in this model gets this treatment, and which ones do is a code-level choice inherited from `gigalens`, not a mathematical necessity. The five Sérsic amplitudes — one per lens-light component plus one for the source — are exactly as linear in Equation () as the 28 shapelet amplitudes are. But `gigalens`'s own flag, `use_lstsq=False` on every Sérsic component versus `use_lstsq=True` on the shapelets alone (`reproductions/claude-giga-lens/cgl/likelihood.py:301-302` against `:314`), marginalizes only the second group. The five Sérsic `Ie` parameters keep an explicit `LogNormal` prior and are sampled directly, like any nonlinear parameter — which is why the marginalized model has 46 = 41 sampled nonlinear parameters + 5 sampled `Ie`'s (`reproductions/claude-giga-lens/cgl/likelihood.py:35`; `reproductions/foundry-i/README.md:101-102`) not $46 = 74 - 28$ read as "everything not a shapelet amplitude minus the shapelets." Both bookkeepings land on the same 46 , because the arithmetic is the same subtraction either way:

$$74 - 28 = 46.$$

Marginalizing these 28 amplitudes is not only an elegant derivation; it is a numerical rescue. Chapter 5 already flagged the full posterior's own Hessian at condition number $\sim 10^{14}$, leaving barely 1.65 stable float64 digits — nowhere near enough to trust a naive solve. The 28×28 ridge normal matrix A this section builds, by contrast, measures at condition number 1.37×10^4 at the campaign's own parity point (`reproductions/claude-giga-lens/CAMPAIGN.md:702`) — an improvement of roughly 7.3×10^9 , restoring 11.5 usable digits. Marginalizing the linear amplitudes does not merely remove 28 dimensions from a sampler's job; it removes the *worst-conditioned* 28 dimensions, handing them instead to a Cholesky solve that a ridge prior has made numerically boring.

The Occam term

Every ridge solve carries the term `gigalens`'s own least-squares path omits. Equation ()'s log-likelihood is

$$\log L = -\tfrac{1}{2}\|R_w\|^2 + \tfrac{1}{2}\, b^\top \hat{a} - \tfrac{1}{2}\log\det A + \text{const},$$

the same $-\frac{1}{2}\log\det A$ Occam factor Chapter 8 introduced — Log-Det Ledger row 3, opened there on a one-parameter toy and audited here in production, on this repo's own 28-column shapelet design. `gigalens`'s stock linear solver calls `pinv(X^\top WX)` with no ridge and no Occam correction at all (`reproductions/foundry-i/README.md:93-95`); on the near-rank-deficient shapelet basis that solve is not merely missing a constant, it is non-smooth, floors the log-posterior's gradient norm, and never finds a positive-definite mode (`reproductions/foundry-i/README.md:95-99`).

The audited number is not a demonstration value — it is the parity harness's own measurement, checked against `numpy.linalg.slogdet` to 10^{-10} at a stored validation point (`reproductions/claude-giga-lens/papers/main.tex`, Table `tab:parity`, Gate E; `reproductions/claude-gig`

$$\log\det A = 323.229, \qquad \text{cond}(A) \approx 1.4 \times 10^4.$$

At that exact point, dropping the Occam term the way `gigalens`'s plain `lstsq` path does would over-report the log-likelihood by half of $\log\det A$ — 161.61 nats — not a rounding error, a number on the same order as the 191.1-nat evidence swing that decides which basin the data prefer (Chapter 25). That swing is only trustworthy once every log-likelihood evaluation being compared — steep basin and low basin alike — carries this same correction, consistently; a comparison where one side's Occam term is audited and the other's is silently dropped is not a Bayes factor at all, whatever units it is reported in. This is the reason this campaign's evidence numbers exist as a genuine contribution rather than a rerun of `gigalens` with more compute.

Two of the parity harness's other four gates matter directly here too. Gate B (the log-posterior itself) and Gate D (the *diagonal limit*: setting the correlated whitener to the identity must reproduce the plain diagonal likelihood *exactly*) both pass at machine precision (`reproductions/claude-giga-lens/papers/main.tex`, Table `tab:parity`) — Chapter 24 is where that gate becomes a methodology in its own right, not merely a unit test.

The GIGA-Lens recipe

Having a differentiable log-posterior is necessary but not sufficient; you still have to sample it. GIGA-Lens's own answer to that problem (Gu et al. 2022) — and the baseline every sampler in this campaign is measured against (`reproductions/claude-giga-lens/papers/main.tex:549`, contender "S0") — is a three-step recipe: **multi-start MAP → SVI → preconditioned HMC** (`reproductions/claude-giga-lens/papers/main.tex:189`).

MAP is mode-finding: run a gradient-based optimizer from several starting points on the exact `_logpost` function this chapter built, and keep the best local maximum found. It is cheap, and it is only a point — the second-order Taylor picture of Chapter 2 applies here directly, and so does its warning: a zero gradient is not evidence of a mode. Chapter 5 already named the failure mode; Chapter 26 shows it happening on this exact posterior.

SVI (stochastic variational inference) fits a cheap Gaussian approximation to the posterior around that mode — not by sampling, but by optimizing the Gaussian's own mean and covariance to minimize a KL divergence to the true posterior. It is not meant to be trusted as the answer; it is meant to produce a covariance estimate fast, which becomes the next step's preconditioner.

HMC then samples the real posterior, using gradients of `_logpost` and a momentum "mass matrix" built from the SVI covariance to avoid the absurdly small steps a naive sampler would need in the model's stiff directions (recall cond $\sim 10^{14}$ above) or the wasted ones it would take in its flat, ridge-tamed shapelet directions. Chapter 23 derives exactly what a mass matrix is and why its direction convention is easy to get backwards.

Whether three steps in this order actually converge on this repo's own 46-dimensional, condition-10^{14}, occasionally-saddle-shaped posterior — and what changes when they do not — is Chapter 23 and Chapter 26 in full. This chapter's job ends at the log-posterior the recipe is handed: 46 numbers wide, one Occam term deep, and identical every time it is called.

> **Ledger**
>
> **What this chapter rules in or out about $\gamma = 1.103$:** nothing directly — this chapter builds the machine, not the verdict. But every γ number in Chapter 25's chain, money number included, is the output of exactly this 46-dimensional marginalized log-posterior, with this Occam term inside it, sampled by some version of the recipe above. Get the parameter count wrong, marginalize the wrong 28 numbers, or drop $-\frac{1}{2}\log\det A$ the way `gigalens`'s stock solver does, and every γ downstream is measuring a different quantity than the one this book has been careful to name.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/likelihood.py:208-386` — the whole chapter, as running code: the prior and 46-dim bijector (`:294`'s `assert ndim == 46`), the two `PhysicalModel` instantiations (`:299-303` the deterministic render, `:311-315` the shapelet design), `_design_ret`'s render → convolve → pool pipeline (`:329-352`), and `_logpost` wiring `marg_loglik` into the full posterior (`:369-386`). (github)
- `reproductions/claude-giga-lens/cgl/marg.py` — the 55-line ridge core Chapter 8 introduced; this chapter runs it at 28 real design columns instead of one toy amplitude. (github)
- `reproductions/claude-giga-lens/cgl/e2.py:11-34` — the campaign's own docstring stating the 74-to-46 reduction in one paragraph, plus the six measured basin-γ seeds this exact model produced.
- `reproductions/claude-giga-lens/cgl/guards.py:74-91` — `assert_psf_sampling`, the guard that makes this chapter's render-order failure mode impossible to repeat silently.
- `reproductions/claude-giga-lens/papers/main.tex` — Table `tab:parity` (Gate E, the audited Occam term) and Table `tab:thresholds` (all five pre-registered gates, A through E).
- `reproductions/foundry-i/README.md:83-121` — the campaign's own retrospective on why marginalization was necessary: the `pinv`-based linear solve's non-smoothness, in the group's own words.

Exercises

> **Exercise 22.1 — Recount the 74, and the 46**

Without looking back at the running total, list this model's five component groups (mass, shear, lens light, source Sérsic, source shapelets) with a parameter count for each, sum them, and then say how many parameters remain once the shapelet amplitudes are marginalized away rather than sampled.

Solution

Mass (EPL): $\theta_{\mathrm{E}}, \gamma, e_1, e_2$, center (x, y) — 6. External shear: γ_1, γ_2 — 2. Lens light: four Sérsic profiles at 7 parameters each — 28. Source Sérsic: 7. Source shapelets: a 3-parameter pose (scale, center) plus 28 amplitudes at $n_{\max} = 6$. Summing: $6 + 2 + 28 + 7 + 3 + 28 = 74$. Marginalizing the 28 amplitudes analytically removes them from the sampled space entirely (they are solved by a Cholesky step inside every log-posterior evaluation, not sampled by a Markov chain), so $74 - 28 = 46$ remain.

Exercise 22.2 — Why is A only 28×28?

Equation ()'s normal matrix $A = X_w^\top X_w + \Lambda$ is 28×28: the marginalized model has 46 sampled parameters, yet A never grows past the shapelet count. Where do the other 45 parameters (everything except γ, say) actually show up in a single evaluation of the log-posterior?

Solution

A's dimension is the number of *linear* parameters being marginalized at this evaluation, not the number of parameters the sampler explores overall. The 46 nonlinear-plus-I_e parameters never become rows or columns of A; instead, every one of them reshapes the *inputs* to A each time the log-posterior is called. Changing θ_{E} or γ changes β via the lens equation, which changes where each shapelet basis function lands on the sky, which changes every column of the design matrix X_w — and therefore $A = X_w^\top X_w + \Lambda$ itself, recomputed from scratch at the new point. The 28×28 solve is exact and cheap; the 46-dimensional exploration around it is what a sampler still has to do the hard way.

Exercise 22.3 — What the Occam term costs, in production

Using the audited production value $\log\det A = 323.229$ at the campaign's own parity point, compute how many nats `gigalens`'s plain `lstsq` path (no Occam correction) would over-report relative to the correct ridge-marginalized log-likelihood at that exact point.

Solution

Dropping $-\frac{1}{2}\log\det A$ from Equation () changes the reported log-likelihood by exactly $+\frac{1}{2}\log\det A = 0.5 \times 323.229 = 161.61$ nats . That is not a small correction relative to the evidence differences this campaign reports (Chapter 25): it is on the same order as the swing that decides which basin the data prefer. A code path that always omits this term is not merely slightly optimistic — it is systematically biased toward whichever model has the *sharper* (larger-$\det A$) linear posterior, which is exactly backwards from what a genuine Bayesian comparison should reward.

Exercise 22.4 — Why PSF convolution comes before pooling

Equation () convolves with the PSF *before* average-pooling down to the data grid, on the oversampled grid, not after. Explain why doing it in the other order — pool first, then convolve at native resolution — would be wrong, and name the real repo failure this chapter connects it to.

Solution

A point-spread function has real structure — its core, its diffraction spikes — at angular scales finer than one data pixel; that is the entire reason images look blurred rather than merely pixelated. Pooling down to the native grid *before* convolving would throw away exactly the sub-pixel structure the PSF needs to act on, producing a systematically wrong (typically too-broad, since pooling is itself a smoothing operation) effective blur. The real-repo version of this mistake was not the order of operations in the code — that has always been convolve-then-pool — but a *sampling-scale* version of the same error: an empirical PSF kernel sampled at its own native pixel scale, fed into a convolution that assumed it was already sampled at the oversampled `delta_pix` scale, silently double-refined and broadened the effective PSF by a factor of two. Same noise, same data, same code otherwise — fixing only that sampling-convention mismatch dropped χ^2_ν from 3.4 to 1.05 .

23. HMC, SMC, and flows: three log-dets, one idea

Chapter 22 built the machine: a real, differentiable, 46-dimensional log-posterior with the Occam term already inside it. This chapter is about walking that surface and coming back with a number and an honest uncertainty on it. Four technologies appear in this campaign's own numbers — hand-built-metric HMC, a two-stage version of the same recipe, tempered SMC, and a normalizing-flow "NeuTra" recipe — and the claim of this chapter is that all four answer one question: *what change of coordinates turns this specific posterior into something a local, gradient-following integrator can explore.* Getting that question right is the difference between $\hat{R} = 22$ and $\hat{R} = 1.003$ on this campaign's own real fits — two different posteriors, not a before/after on one. By the end you will be able to read an $\hat{R}$ or an ESS number and know whether to trust it, and you will have *derived*, not been told, that the Occam term Chapter 8 introduced and the lensing magnification that opened the Log-Det Ledger in Chapter 4 are the same computation this chapter closes.

> **What you can skip**
>
> Metropolis–Hastings, Markov-chain convergence, and gradient-based optimization (autodiff, backprop) are yours already — none of that is re-derived here. A normalizing flow's bijection and log-det correction were already Chapter 4's business; this chapter does not re-teach what a flow is. What *is* new: the specific, easy-to-get-backwards convention this repo's sampler libraries use for a "mass matrix"; Gelman–Rubin $\hat{R}$ and effective-sample-size in the exact multi-chain sense this book uses them; and the derivation that the Occam term, a flow's log-det, and the lensing magnification are, line for line, one computation.

Why gradients change everything

A plain Metropolis proposal has no information about which direction the target increases; it guesses, evaluates, and keeps the guess only if it got lucky. A gradient removes the guessing: $\nabla \log p(\theta \mid D)$ points exactly toward higher posterior density — the same quantity `jax.grad` already computes for every one of this campaign's `@jax.jit`-compiled renders (Chapter 22). Hamiltonian Monte Carlo (HMC) does not use that gradient for one step; it integrates an entire physically-motivated trajectory before proposing a single new point — the next section's derivation.

> **You already know this**
>
> If you have used stochastic gradient Langevin dynamics (SGLD) or seen a diffusion model's score-based sampler, you have already used this exact primitive: a gradient of a log-density, biasing a random walk toward high-probability regions. HMC is what you get when you replace SGLD's one noisy step between gradient evaluations with many deterministic, momentum-conserving steps — trading a single kick for an entire coasting trajectory before the next random draw.

Gradients are necessary here, but this campaign's own benchmark is blunt about them not being *sufficient*. At an identical, budget-matched number of gradient evaluations, no gradient-based contender beat the plain multi-start baseline on the harder real-lens targets — the single most efficient sampler in the whole zoo used no gradient information at all: nested sampling (`nautilus`) beat the baseline's ESS-per-gradient by 2.6–307× across the zoo (`reproductions/claude-giga-lens/papers/main.tex:989-991`). Only when the campaign stopped matching budgets and ran every contender *until actually converged* did the ranking flip: parallel-tempered HMC converged 5 of 6 of the hardest real T1 systems, including one the baseline

itself could not close (`reproductions/claude-giga-lens/papers/main.tex:1019-1020`). This book's recurring refrain, applied to sampling: a gradient is necessary but not sufficient — it only pays off paired with the right local geometry, which is this chapter's real subject, starting now.

HMC and the metric

Give the parameter vector θ a fictitious momentum $\mathbf{p}$ (bold, to keep it visually apart from the posterior density $p(\theta \mid D)$) and define a Hamiltonian $\mathcal{H}$ — total energy, potential plus kinetic, calligraphic so it is never confused with the Hessian H this section reaches for two paragraphs from now:

$$\mathcal{H}(\theta, \mathbf{p}) = \underbrace{-\log p(\theta \mid D)}_{\text{potential}} + \underbrace{\tfrac{1}{2}\,\mathbf{p}^\top M^{-1}\mathbf{p}}_{\text{kinetic}}.$$

M, the **mass matrix**, is a free choice — any symmetric positive-definite matrix. Hamilton's equations, $\dot\theta = \partial\mathcal{H}/\partial\mathbf{p} = M^{-1}\mathbf{p}$ and $\dot{\mathbf{p}} = -\partial\mathcal{H}/\partial\theta = \nabla \log p(\theta \mid D)$, conserve $\mathcal{H}$ exactly and are simulated by **leapfrog** integration: half a momentum kick using $\nabla \log p$, a full position drift using $M^{-1}\mathbf{p}$, another half kick. Every leapfrog step costs exactly one gradient evaluation — the convention `reproductions/claude-giga-lens/cgl/metrics.py:284-286`'s budget ledger states outright ("`n_grad` counts logp gradient evaluations… 1 per leapfrog step per chain"). Leapfrog is also *exactly* volume-preserving: its own Jacobian has $|\det J| = 1$ by construction — arguably a fourth, degenerate ledger row, and the reason HMC's Metropolis correction needs no Jacobian term at all, unlike a generic nonlinear proposal.

M is not a free lunch; it decides whether leapfrog is efficient or useless. Near the posterior's mode, $-\log p(\theta \mid D) \approx \text{const} + \frac{1}{2}(\theta - \hat\theta)^\top H (\theta - \hat\theta)$ (the Laplace picture, Chapter 8). Substitute $u = M^{1/2}(\theta - \hat\theta)$ — a *linear* change of variables, exactly Chapter 4's machinery — and the local potential becomes $\frac{1}{2}u^\top M^{-1/2} H M^{-1/2} u$. Choose $M = H$ and this collapses to $\frac{1}{2}u^\top u$: a perfectly isotropic bowl, in u-coordinates, however stretched H made θ-space. That is the whole argument for a mass matrix: it is a linear whitening transform, built to cancel the target's own curvature, so that a fixed leapfrog step size works equally well in every direction at once.

Since $\Sigma \equiv H^{-1}$ is the Laplace *covariance*, the rule is $M = \Sigma^{-1} \approx H$ — and `reproductions/claude-giga-lens/cgl` says this in so many words: *"run_staged/_run_phmc use momentum covariance = Sigma^-1, the GIGA-Lens convention."* That single reciprocal is a genuine convention trap: this campaign's two sampler libraries name the *same* Σ at *opposite* ends of it. TFP's `PreconditionedHamiltonianMonteCarlo` wants its `momentum_distribution` handed M directly, so `reproductions/claude-giga-lens/cgl/samplers/remc_pt.py:125` inverts first (`prec = np.linalg.solve(ginit.cov_reg, np.eye(dim))`). `blackjax`'s NUTS and MCLMC adapters, by contrast, take a parameter literally called `inverse_mass_matrix` — and `reproductions/claude-giga-lens/cgl/samplers/bj_smc.py:72` passes Σ to it *un-inverted*, commented `"inverse mass = covariance"` (`reproductions/claude-giga-lens/cgl/samplers/bj_nuts.py:116` does the same). Fill in the wrong library's parameter with the other's convention and every direction gets the wrong momentum: the tightly-constrained direction, which should move cautiously, gets a *light* mass and leapfrog blows up in it; the loose direction gets a *heavy* mass and crawls where it should roam freely.

> **You already know this**
>
> Building M from the target's own curvature is quasi-Newton optimization, run for sampling instead of minimizing. Adam's per-parameter learning-rate scaling and this chapter's $M \approx H$ exist to fix the identical pathology: a surface whose curvature differs wildly by direction, where one fixed step size is either too timid everywhere or too reckless somewhere.

You judge whether a chosen M worked with two multi-chain diagnostics. $\hat{R}$ (Gelman & Rubin) compares within-chain variance W to the pooled between-chain variance: for m chains of n draws each, $\widehat{\text{var}} = \frac{n-1}{n}W + \frac{B}{n}$, and $\hat{R} = \sqrt{\widehat{\text{var}}/W}$. Near 1, every chain agrees on where the posterior's mass is; large means some chains are stuck somewhere the others are not. (`cgl/metrics.py:23-68` runs a more careful rank-normalized, *split* version — splitting each chain in half catches a single chain drifting over time — but the arithmetic is the same idea.) Effective sample size (ESS) asks the complementary question for *one* chain: how many independent draws is a correlated sequence of N actually worth? For a stationary, autocorrelated sequence with integrated autocorrelation time τ_{int}, ESS $\approx N/\tau_{\text{int}}$ — literally the same N_{eff}/N reduction Chapter 7 already applied to a *stationary pixel sequence*; a Markov chain and a correlated noise field are, arithmetically, the same object.

Ground both diagnostics in a real fit: the 46-dimensional posterior Chapter 5 measured at condition number $\sim 10^{14}$ is *positive-definite* — a genuine, brutally stretched bowl — and a better M alone fixes it. Single-stage HMC, metric built from a floored SVI covariance, reaches only $\hat{R} = 3.10$ (ESS 28 at 300 kept draws); the honest "no hand-built mass matrix" contender, auto-tuned MCLMC, does *worse* — $\hat{R} = 5.9$ (ESS 9). What converges this target is rebuilding M from pooled draws of a first HMC stage and running a second stage from there — the campaign's own two-stage recipe (§4.2, "The two-stage PHMC finding") — which takes the *same* posterior from $\hat{R} = 2.11$ to 1.003 (`reproductions/claude-giga-lens/papers/main.tex:1140-1152`). A better metric alone bought this section's whole argument in one recipe change. But "rebuild a better M" presumes a valid local curvature exists to rebuild it from — and the money number's own posterior, next, does not have one.

Tempering and SMC

Apply the exact recipe that just worked — rebuild M from better and better local curvature estimates — to the money number's own correlated, binned-low posterior, and it fails outright. Two different positive-definite metric repairs, a diagonal $|H_{ii}|$ metric and a full-rank SVI covariance, both stall: $\hat{R} = 21$ for the first, $\hat{R} = 22.3$ for the second (`CAMPAIGN.md`, "P1c metric-fix attempts — 2026-07-10"; `reproductions/claude-giga-lens/papers/main.tex:909-914`) — both worse than T2's already-broken single-stage attempt above. The reason is not a tuning miss: this MAP is a **saddle**. Chapter 5 already showed you its Laplace Hessian's minimum eigenvalue is -14.85 with 5 negative directions out of 46. There is no positive-definite local curvature to build M from at that point — both "repairs" are valid metrics for a *different, imaginary* posterior. (Chapter 26 gives the saddle its full account; this chapter only needs the diagnosis.)

The campaign's own forensics on that $\hat{R} \approx 22$ are worth a moment, because the culprit is not γ. Tracing the worst-mixing parameter finds a Sérsic-versus-shapelet source-*centre* degeneracy split into two disjoint clusters, at -0.15 and $+0.09$, with their own $\hat{R}$'s of 22.3 and 15.1 (`reproductions/claude-giga-lens/papers/main.tex:912-914`) — decoupled from the slope entirely. No amount of rebuilding M helps here, for a structural reason: leapfrog is a *local,*

deterministic integrator. A trajectory started in one source-centre cluster follows $\nabla \log p$ smoothly and has no mechanism for teleporting across a valley of near-zero density to the other cluster, however well-scaled its momentum is.

Tempering sidesteps the whole question of a single local metric. Define a family of intermediate distributions bridging an easy reference r (a prior, or a Gaussian fit to one basin) to the true target, $\pi_\lambda(\theta) \propto r(\theta)^{1-\lambda}\, p(\theta \mid D)^\lambda$, $\lambda : 0 \to 1$.

> **You already know this**
>
> This is simulated annealing's temperature schedule, run in reverse: instead of cooling a hard optimization problem into an easy one, tempering *heats* an easy, unimodal reference up into the hard, possibly-multimodal target.

Sequential Monte Carlo (SMC) is the machinery that walks that family with a *population* of particles rather than one chain: at each λ-step, reweight every particle by the incremental likelihood ratio, resample (discard low-weight particles, replicate high-weight ones), then mutate each survivor with a few steps of an inner MCMC kernel (here, HMC) to diversify without changing the marginal — exactly a particle filter, the machinery behind robot localization, with λ playing the role "time" plays there. Because particles are resampled from the *whole population* every step, one stuck in the wrong source-centre cluster is simply discarded in favor of a copy from the other — the teleportation a single leapfrog trajectory cannot do. And because the population's importance weights, integrated across the whole anneal, give an unbiased estimate of the normalizing constant, $\log Z$ falls out for free — the exact evidence Chapter 8 introduced. Both halves of this book's headline swing were computed by *this same tool*: the diagonal comparison used 300 particles (`reproductions/claude-giga-lens/papers/main.tex`, Table `tab:basinflip`) and returned $+162.2$ nats for the steep basin; the correlated comparison used 128 particles annealed over 28 steps, ESS between 77 and 118 , and returned -28.9 nats for the low basin — a 191.1-nat swing Chapter 25 builds its verdict on. One tool, run twice, produced both the basin weighting *and* a properly marginalized γ posterior a frozen HMC chain could never have reached — and its outer loop needed no hand-built metric at all, only a modest one for the inner mutation kernel, which only has to jiggle particles between resamples, not carry the whole exploration alone.

Flows and NeuTra

A third strategy: instead of hand-building M or replacing the sampler outright, teach a neural network the whitening transform. A normalizing flow T, fit to samples from *some* approximation of the posterior, bijects a standard-normal u-space to θ-space; run ordinary, identity-mass HMC entirely in u-space, then push draws back through T. `reproductions/claude-giga-lens/cgl/flows.py:17-21` states the pullback exactly as Chapter 4 already derived it: $\log p_u(u) = \log p_{\text{target}}(T(u)) + \log|\det J_T(u)|$. If T has genuinely learned the target's correlations, u-space *is* the isotropic space the previous section built M for by hand — except a network built it, and its log-det correction is doing the mass matrix's job.

The catch is the one that phrase hides: T is only as good as what it was trained on, and this campaign's NeuTra recipe fits its flow to samples drawn from a floored *SVI Gaussian* (`reproductions/claude-giga-lens/cgl/samplers/neutra.py:82-88`) — the same local approximation that already failed to describe a saddle, above. A flow trained on the wrong reference faithfully learns to whiten the wrong thing. On the zoo's own cond-10^{14}

mock target, NeuTra's $\hat{R}$ reaches 5.72 while parallel-tempered HMC's own tuned temperature ladder *blows up* entirely, and only the humble baseline and MCLMC stay healthy (`reproductions/claude-giga-lens/papers/main.tex:1016-1025`). Worse, NeuTra's *median* $\hat{R}$ across the whole benchmark looks almost fine, 1.160 (Table `tab:deveval`) — an aggregate that quietly buries one badly broken system inside many easy ones, the same lesson tempered SMC taught from a different angle: a sophisticated preconditioner is only as trustworthy as what it was built or trained on.

Hand-built-metric HMC, SMC, and flow-space NeuTra are three answers to one question — what change of coordinates makes *this specific* posterior tractable for a local integrator — and this campaign needed pieces of all three, because no single answer generalizes across a saddle, two disconnected sub-modes, and a condition number of 10^{14} in the same 46 numbers.

Closing the Log-Det Ledger

Three rows, three costumes: lensing magnification $\mu = 1/|\det A|$ (Chapter 4), a flow's pullback correction $\log|\det J_T|$ (same anchor), and the Occam factor $-\frac{1}{2}\log\det H$ this chapter's own mass matrix reused (Chapter 8). Here is why they are one computation, derived rather than asserted.

The Laplace evidence integral is a d-dimensional Gaussian integral over the un-normalized posterior near its mode:

$$\int_{\mathbb{R}^d} \exp\Big(-\tfrac{1}{2}(\theta-\hat{\theta})^\top H(\theta-\hat{\theta})\Big)\, d\theta.$$

Substitute $u = H^{1/2}(\theta-\hat{\theta})$ — the *identical* linear map this chapter already used to build a mass matrix. By Chapter 4's own reciprocal-determinant law, $d\theta = du/|\det H^{1/2}| = du/\sqrt{\det H}$, so () becomes $(2\pi)^{d/2}/\sqrt{\det H}$, and its logarithm is exactly $\frac{d}{2}\log(2\pi) - \frac{1}{2}\log\det H$ — the Occam factor, term for term, out of the same area-scaling fact that turned a unit square into a parallelogram back in Chapter 4. A flow's own pullback correction is the *identical* substitution with the linear map $H^{1/2}$ replaced by a general nonlinear T, applied *pointwise* to one draw instead of integrated over the whole space — the only difference between rows 2 and 3. `site/guide_src/lensing.py:139-146`, this book's opening quote on the matter, is not a metaphor: *"This is the SAME change-of-variables factor that a normalizing flow applies as $-\log|\det J|$, and a cousin of the $-\frac{1}{2}\log\det A$ Occam term in `cgl/marg.py`. Three log-dets, one idea."*

Not a demonstration on toy numbers alone: row 1's own worked value is $\det A = 0.485$, $\mu = 2.0619$ (Chapter 4); row 3 — where, as Chapter 4 already flagged, A means `marg.py`'s ridge normal matrix, not the lensing Jacobian — measures, at the campaign's own audited production point, $\log\det A = 323.229$, a 161.6145-nat correction — not a rounding footnote, a number on the same order as the evidence swing this book's spine turns on. Three costumes, one determinant, audited from a 2×2 toy to a 46-dimensional real fit.

Log-Det Ledger — closed

#	Costume	Formula	Where
1	Lensing magnification	$\mu = 1/\lvert\det A\rvert$	Ch. 4

2	Normalizing-flow pullback density	$\log p_u(u) = \log p(T(u)) + \log\lvert\det J_T(u)\rvert$	Ch. 4; flows-and-neutra above
3	Gaussian-evidence Occam term	$-\frac{1}{2}\log\det H$	Ch. 8; Ch. 22

All three are the change-of-variables theorem (Ch. 4) applied to a linear lensing map, a nonlinear flow, and the linear whitening map implicit in a Gaussian integral. Rows 1 and 2 read the SAME theorem in opposite directions; row 3 integrates it instead of evaluating it pointwise. No fourth costume is left to find — every $\log|\det|$ this book computes is one of these three, or (leapfrog, above) a degenerate case where the answer is exactly zero.

Ledger

What this chapter rules in or out about $\gamma = 1.103$: no digit of it — this chapter is the *instrument*, not the reading. What it fixes is the standard a reading has to clear before it counts: $\hat{R}$ close to 1, ESS large enough that the number you report is not three noisy draws wearing a large sample's clothing. A saddle MAP, an $\hat{R} \approx 22$ from two failed metric repairs, and a 128-particle tempered SMC run that actually converged, is the money number's own instance of exactly this audit. Chapter 25 tells you whether it clears the bar.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/e2.py:531-721` — `laplace_evidence` and `build_metric_cov`: the Hessian eigendecomposition, the PD-vs-indefinite branch, and the exact sentence this chapter quotes on the mass-matrix convention.
- `reproductions/claude-giga-lens/cgl/samplers/bj_smc.py:69-97`, `bj_nuts.py:106-165`, `remc_pt.py:115-160`, `baseline_gigalens.py:106-135` — four libraries' momentum-metric plumbing side by side; the `inverse_mass_matrix`-vs-`covariance_matrix` trap is visible directly in the diffs between them.
- `reproductions/claude-giga-lens/cgl/e2.py:756-836` (`run_correlated_smc`) and `cgl/samplers/common.py:297-360` (`run_adaptive_tempered_smc`) — the actual 128-particle basin-local SMC and its adaptive λ-schedule / resampling machinery.
- `reproductions/claude-giga-lens/cgl/samplers/neutra.py` and `cgl/flows.py` — the NeuTra recipe end to end: flow fit on floored-SVI samples, ChEES-HMC in the pullback space.
- `reproductions/claude-giga-lens/cgl/metrics.py:23-68` — `rank_diagnostics`, the real rank-normalized split-$\hat{R}$/ESS this chapter's hand-worked formula approximates.
- `reproductions/claude-giga-lens/papers/main.tex` — §4.2, two-stage PHMC, §6.3, the sampler saga, §7.6, the A100 phase, and Table `tab:basinflip`.
- `reproductions/claude-giga-lens/CAMPAIGN.md`, "P1c metric-fix attempts — 2026-07-10" — the saddle diagnosis in the campaign's own words, both failed repairs included.

Exercises

Exercise 23.1 — the convention trap, by hand

A toy posterior covariance $\Sigma = \mathrm{diag}(100, 0.01)$ — one loose direction, one tight one, condition number 10^4. (a) Compute the correct mass matrix $M = \Sigma^{-1}$. Which direction gets the heavier mass, and why is that the right physical choice? (b) Suppose you mistake `blackjax`'s `inverse_mass_matrix` convention for TFP's `covariance_matrix` one and hand Σ *directly* to TFP's momentum distribution instead of inverting it first. What (wrong) mass does the tight direction get, and what goes wrong in leapfrog because of it?

Solution

(a) $M = \Sigma^{-1} = \mathrm{diag}(0.01, 100)$: the *tight* direction (variance 0.01) gets the *heavy* mass (100), the loose direction (variance 100) gets the light mass (0.01) — correct, since a heavy mass moves slowly per momentum kick (small, careful steps where the posterior is narrow) and a light mass moves fast (covering the wide direction quickly).

(b) Handing Σ directly to a parameter that wants M gives the tight direction mass 0.01 — light instead of heavy. Leapfrog now accelerates fast exactly where the posterior is narrowest, overshoots every step, and either diverges outright or is rejected almost every proposal; the loose direction, meanwhile, gets the wrongly heavy mass 100 and crawls where it should roam freely — exactly backwards, in both directions at once.

Exercise 23.2 — $\hat{R}$ on two chains that never meet

Two toy chains, four draws each: chain 1 $= (1, 2, 3, 4)$, chain 2 $= (5, 6, 7, 8)$ — deliberately non-overlapping. Using $\widehat{\mathrm{var}} = \frac{n-1}{n} W + \frac{B}{n}$ and $\hat{R} = \sqrt{\widehat{\mathrm{var}}/W}$, compute W, B, and $\hat{R}$. Is $\hat{R}$ bounded above? How does this toy's value compare to the money-number posterior's own $\hat{R} \approx 22$?

Solution

Each chain's own mean is 2.5 and 6.5; each chain's within-chain variance (ddof= 1) is $\frac{2.25+0.25+0.25+2.25}{3} = 5/3$, so $W = 5/3 \approx 1.667$. The grand mean is 4.5, so $B = \frac{n}{m-1} \sum_j (\bar{\theta}_j - \bar{\theta})^2 = 4 \times [(2.5-4.5)^2 + (6.5-4.5)^2] = 32$. Then $\widehat{\mathrm{var}} = \frac{3}{4}(5/3) + 32/4 = 9.25$ and $\hat{R} = \sqrt{9.25/1.667} \approx 2.356$. $\hat{R}$ has no upper bound in principle — the more disjoint the chains or the tinier their own internal spread, the larger it grows. The real posterior's $\hat{R} \approx 22$ is roughly nine times worse than this already-broken toy: chains not merely in different ranges, but far more confident in their own (wrong) location than these four-draw chains are.

Exercise 23.3 — the Occam term is half of a Gaussian integral

Redo this chapter's substitution for $d = 1$: show $\int \exp(-\frac{1}{2} A(a - \hat{a})^2)\, da = \sqrt{2\pi/A}$ by substituting $u = \sqrt{A}\,(a - \hat{a})$. Chapter 8's toy ridge fit (`worked_examples.py --show ch08`) has $\log\det A = 4.025$ and reports `toy_logL_marg_style` $= -4.102$ before restoring the $\log Z$ normalization, and `toy_logZ_closed` $= -3.183$ after. Which *half* of your Gaussian integral is already inside `marg.py`'s own $-\frac{1}{2}\log\det A$, and which half is the separate constant Chapter

8 adds afterward?

Solution

With $u = \sqrt{A}(a - \hat{a})$, $da = du/\sqrt{A}$, the integral becomes $\frac{1}{\sqrt{A}} \int \exp(-u^2/2)\, du = \sqrt{2\pi/A}$, whose log splits into two additive pieces: $-\frac{1}{2}\log A$ (depends on A) and $\frac{1}{2}\log(2\pi)$ (does not). `marg.py`'s own $\log L$ (`cgl/marg.py:52-55`) already contains the *first* half — its $-\frac{1}{2}\log\det A$ term is literally this integral's $1/\sqrt{A}$ factor, in log form. The *second* half is exactly the normalization `marg.py`'s docstring drops as `"+ const"` (`reproductions/claude-giga-lens/papers/main.tex:471`), which Chapter 8 restores by hand: $-4.102 + \frac{1}{2}\log(2\pi) = -4.102 + 0.919 = -3.183$, matching `toy_logZ_closed` exactly. Both halves are one substitution; the code just never needed the A-independent half, since it cancels in any difference between two evaluations of the same model.

Exercise 23.4 — ESS from an autocorrelation time

Chapter 7 measured an integrated autocorrelation time $\tau_{\text{int}} = 7.453$ for the fine drizzle product's *pixel* noise. Using ESS $\approx N/\tau_{\text{int}}$ — the same reduction, now applied to a Markov chain instead of a stationary noise field — what is the effective sample size of a single HMC chain with $N = 5000$ post-warmup draws and that same autocorrelation time? Is that a healthy ESS by this book's own $\text{ESS}_{\text{min}} \geq 1000$ convergence gate (`reproductions/claude-giga-lens/papers/main.tex:569`)?

Solution

ESS $\approx 5000/7.453 \approx 670.9$. That is *below* the campaign's own $\text{ESS}_{\text{min}} \geq 1000$ until-converged gate — a single chain this correlated, even with 5000 raw draws, would not by itself clear the bar the book's own Track-B protocol sets. It would need either more raw draws, several more chains pooled together, or a shorter autocorrelation time (a better-mixing sampler, or a better mass matrix) to reach 1000 independent-equivalent draws — exactly the same lever this chapter's "hmc-and-the-metric" section pulled on the real T2 posterior.

24. The correlated-noise likelihood: whitening a real telescope

Ch. 11 told you drizzle correlates pixel noise and a diagonal likelihood assumes it does not; Ch. 7 built the Fourier machinery — a stationary kernel, a whitening filter, an operator-norm gate — that a fix would need. This chapter is where those two pieces become one executable object: $C = D^{1/2} K D^{1/2}$, fit on a real, masked, model-subtracted residual, applied by a local convolution, and tested rigorously enough that a real implementation defect got caught before it ever touched a published number. By the end you can read `reproductions/claude-giga-lens/papers/main.tex`'s entire Methods I section (`cgl/whiten.py` and `cgl/noise.py` end to end) and know exactly which claim each gate backs. This chapter does not defend $\gamma = 1.103$ — that verdict is Ch. 25's job. It builds, and proves correct, the machine that number comes out of.

> **What you can skip**
>
> Nothing here re-derives the whitening filter itself ($\hat{h}[k] = 1/\sqrt{S[k]}$, the operator-norm gate $e_{\rm op}$, the spectral floor) — that is Ch. 7, assumed in full. Nothing here re-derives the noise model D or the drizzle mechanism — that is Ch. 11, also assumed. What is new: how the correlation kernel K actually gets estimated from a masked real image, the full acceptance table across all three real products plus one pre-registered exception, the rule that decides which pixels survive being whitened by a local filter, and — the chapter's center of gravity — why the whole apparatus must reduce *exactly* to the diagonal case, both as an algebraic identity and as the regression test that caught a genuine compiler defect.

The covariance model

Ch. 11 built $D = \mathrm{diag}(\sigma_i^2)$, the per-pixel heteroscedastic variance (masked pixels inflated to $\sigma \to 10^{10}$); Ch. 7 built the idea of a stationary correlation matrix diagonalized by the Fourier basis. This chapter's whole job is the object that glues the two into one pixel covariance,

$$C \;=\; D^{1/2}\, K\, D^{1/2},$$

`reproductions/claude-giga-lens/papers/main.tex:367` (Methods I, covariance model and drizzle anchor). (This D is unrelated to Ch. 15's angular-diameter distances $D_{\rm d}, D_{\rm s}, D_{\rm ds}$ — same repo, same letter, a different quantity every time context makes it obvious which.) $D^{1/2}$ carries every pixel's noise *level*; K — a pure correlation matrix, diagonal entries exactly 1 — carries the *shape* of how neighbouring pixels' noise moves together, independent of how bright any of them are. Factoring the two apart is what makes K estimable at all: one stationary kernel $\rho(\Delta)$, the same at every pixel, stands in for what would otherwise be an $N \times N$ covariance no cutout could ever constrain directly.

Two design choices matter before any fitting starts. First, K is fixed *per product* — estimated once, before sampling, never re-estimated as the mass model θ moves, which is why $-\frac{1}{2}\log\det C$ never has to be recomputed inside the sampler's inner loop (The diagonal limit cashes this out). Second, K must be fit on the *model-subtracted* residual, never the raw image — the identical discipline `reproductions/claude-giga-lens/cgl/guards.py:129` (`assert_model_subtracted_sky`) already enforced in Ch. 11 for $\sigma_{\rm bkg}$, now doing double duty: fit a "noise" correlation on an image that still contains the lens galaxy's own smooth light, and you are measuring the autocorrelation of a *galaxy*, not of noise.

Estimating $\rho(\Delta)$ from a real cutout means estimating it from a *masked* one — bad pixels, saturated cores, the segmentation mask itself all remove pixels from play, and a naive autocorrelation over what remains is biased, because the number of valid pairs at each lag Δ is not constant. `masked_autocorr_full` (`reproductions/claude-giga-lens/cgl/noise.py:47`) fixes this with the identical Wiener–Khinchin move Ch. 7 built, applied *twice*: once to the mean-subtracted, masked residual n, and once to the mask w itself (0/1-valued),

$$\mathrm{acf} = \mathrm{IFFT}(|\mathrm{FFT}(n)|^2), \qquad \mathrm{wgt} = \mathrm{IFFT}(|\mathrm{FFT}(w)|^2), \qquad \rho[\Delta] = \frac{(\mathrm{acf}/\mathrm{wgt})[\Delta]}{(\mathrm{acf}/\mathrm{wgt})[0]},$$

where wgt$[\Delta]$ is exactly the number of valid pixel *pairs* separated by Δ — the mask's own autocorrelation, computed by the same padded-FFT trick used on the data, deconvolves the mask's footprint out of the estimate. Lags where fewer than 5% of the maximum pair count survive (wgt$[\Delta] < 0.05\,\mathrm{wgt}[0]$) are marked invalid rather than trusted.

Fitting the kernel

The pre-registered plan for K was the simplest family that could plausibly work — one Gaussian smoothing of the drizzle-anchored correlation blended with a delta, $\rho = (1-w)\delta + w[\rho_{\mathrm{drz}} \circledast G(\sigma_e)]$ — and Ch. 7 already told you it fails: on every one of the three real drizzle products, its best achievable fit misses the campaign's own $\max|\rho_{\mathrm{fit}} - \rho_{\mathrm{meas}}| \le 0.05$ gate, by a wide margin.

product	single-Gaussian, worst resid.	two-component, worst resid.	gate
`v2d` (native)	0.0895 FAIL	0.0448 PASS	≤ 0.05
`v3` (fine)	0.3110 FAIL	0.0270 PASS	≤ 0.05
`v3b` (binned)	0.1664 FAIL	0.0326 PASS	≤ 0.05

(`reproductions/claude-giga-lens/data/noise_kernel_report.json`, fields `products.*.fit_single_family` and `products.*.max_abs_resid`; `reproductions/claude-giga-lens/papers/main.tex:384`–`397`.) These residuals were not tuned to fail: the single-Gaussian family was the *first* thing tried, and every real product's model-subtracted residual carries structure it cannot represent — an anisotropic core (the drizzle kernel's own 3×3 tent is not circular), a medium-scale correlated pedestal the $1''$ background detrend leaves behind, and, on the native product specifically, column stripes. The two-component family the campaign adopted instead adds exactly one more term, a second, broader bivariate-Gaussian pedestal independent of the drizzle-anchored core,

$$\rho \;=\; (1 - w_d - w_b)\,\delta \;+\; w_d[\rho_{\mathrm{drz}} \circledast G_2(\sigma_{ey}, \sigma_{ex}, c_e)] \;+\; w_b\, G_2(\sigma_{by}, \sigma_{bx}, c_b),$$

(`reproductions/claude-giga-lens/cgl/noise.py:354`, `rho_model2`; `reproductions/claude-giga-lens/pape` `394`). Each piece is individually positive semi-definite — Ch. 7 proves why: a delta has flat spectrum $S \equiv 1$, a Gaussian's Fourier transform is itself a positive Gaussian, and convolution multiplies spectra — so any non-negative combination with $w_d + w_b \le 1$ is too, a constraint you can check directly on the fitted "money" product `v3b`: $w_d = 0.733$, $w_b = 0.248$, summing to $0.982 \le 1$.

> **You already know this**
>
> "A non-negative sum of PSD kernels is PSD" is not a fact borrowed from astronomy — it is the closure property that lets an SVM combine a polynomial and an RBF kernel into one still-valid Mercer kernel. This family is built the identical way: three individually-valid pieces, summed with non-negative weights, so the whole is guaranteed valid without ever checking an eigenvalue directly.

An independent cross-scale check adds confidence the fit is not chasing product-specific noise: `binned_kernel_from_fine` (`reproductions/claude-giga-lens/cgl/noise.py:481`) exactly block-sums the *fitted fine* kernel down to binned resolution and compares it against the ACF measured directly on the block-summed fine residual. The two agree to 0.031 against the same 0.05 gate — confirming the kernels commute correctly with 2×2 binning, a purely mathematical property unrelated to how well any one kernel happens to fit.

Convolutional whitening

Ch. 7 derived the whitening filter itself, $\hat{h}[k] = 1/\sqrt{S[k]}$, truncated to a local $(2M+1)^2$ stencil and accepted only if its operator-norm gate $e_{\text{op}} = \max_\omega |S(\omega)|\hat{h}_M(\omega)|^2 - 1| \leq 0.02$ holds. This section is the operational residue of that derivation: the whiteners the campaign actually accepted, on the three real products, plus the rule that decides which pixels survive being whitened by a *local* filter at all.

product	M	e_{op}	gate	kept / eroded
`v2d` (native, strict)	14	0.0177	≤ 0.02	5,865 / 487
`v3` (fine)	20	0.0160	≤ 0.02	66,752 / 37,519
`v3b` (binned)	10	0.0124	≤ 0.02	16,653 / 9,273
`v2d_relaxed` (native)	10	0.0312	$\leq 0.05^*$	5,865 / 1,466

(`reproductions/claude-giga-lens/data/whitener_report.json`; `reproductions/claude-giga-lens/papers` Table `tab:whiten`.) *See below. Every stencil is a *local* filter — a pixel's whitened value depends on the $(2M+1)^2$ pixels around it — so a masked neighbour, or one off the image edge (the `'SAME'`-padded convolution zero-pads borders, and zero is not data), contaminates that one output. `erode_keep` (`reproductions/claude-giga-lens/cgl/whiten.py:274`) handles this by *dropping*, never down-weighting: a pixel survives into the whitened vector only if its entire stencil is clean, computed by `scipy.ndimage.binary_erosion` on the keep-mask with the $(2M+1)^2$ stencil as structuring element, intersected with an explicit M-pixel interior border trim.

> **You already know this**
>
> This is exactly the difference between `'same'` and `'valid'` padding in a convolutional layer, applied for the same reason: `'same'` padding manufactures zeros at the border and lets the network pretend they are real input; `'valid'` padding instead *shrinks* the output, keeping only positions whose full receptive field saw genuine data. `erode_keep` is `'valid'` padding for a statistical estimator, where "genuine data" means "not masked."

The rule is statistically exact, but it costs, and the native product pays hardest: the strict `v2d` whitener's 29^2 stencil ($M = 14$) erodes away 91.7% of the product's pixels — 5,865 down to just 487 kept. That is not a construction defect; it is what a wide stencil genuinely costs against a mask with any interior structure. A pre-registered, evidence-driven exception widens the *acceptance gate itself* from 0.02 to 0.05 — not merely accepting a worse fit under the old bar — for a `v2d_relaxed` whitener at a smaller $M = 10$, recovering 3.0× as many kept pixels (1,466), adopted only after mock recovery confirmed it still calibrates correctly (`reproductions/claude-giga-lens/CAMPAIGN.md`, P1b diagnosis, 2026-07-08).

The diagonal limit

Every piece built so far exists to answer one question honestly: given a real, non-trivial K, does the correlated likelihood compute what it claims to? The cheapest, most decisive way to ask that is to make K *trivial* on purpose and check that the whole apparatus degenerates back to something already validated. `make_conv_whitener` (`reproductions/claude-giga-lens/cgl/whiten.py:39`) computes

$$u \;=\; \mathrm{keep}_w \odot (h * (D^{-1/2}\,\mathrm{image})),$$

where $*$ is a `'SAME'`-padded 2-D convolution. Set $h = [[1.0]]$ — a single tap, the 1×1 "delta" kernel: it has no neighbour to sum in, so a `'SAME'`-padded correlation with a 1×1 kernel *is* elementwise multiplication by that one number,

$$u \;=\; \mathrm{keep}_w \odot (D^{-1/2}\,\mathrm{image}).$$

With $D^{-1/2} = 1/\mathrm{masked\_err\_map}$ and $\mathrm{keep}_w = \mathrm{keep\_mask}$, the right-hand side of () is not *similar* to `cgl/likelihood.py`'s original diagonal path (`R * sqrtW`, `reproductions/claude-giga-lens/cgl/likelihood`
— it *is* that path, term for term. `cgl/whiten.py:20` names this explicitly: "Parity anchor (gate D)."

Worked example. You can rebuild the identity yourself in three lines of plain NumPy, no `jax` required: a small noisy image, its own per-pixel σ, a mask with one bad pixel.

```
sqrt_d_inv = 1.0 / sigma
h = np.array([[1.0]])                              # gate D's delta kernel
u_conv = keep * (h[0, 0] * sqrt_d_inv * img)        # make_conv_whitener, h=[[1]]
u_diag = keep * (img * sqrt_d_inv)                  # the diagonal path
```

The two agree to exactly 0 — not "close," because () has no approximation in it, only algebra. The production harness runs the identical check on the real 46-parameter marginalized posterior, at four perturbed reference points, and reports the same answer: $|\Delta \log p| = 0$ against a gate of 10^{-10} (`reproductions/claude-giga-lens/data/parity_report.json`, `gates.D`).

> **You already know this**
>
> Gate D is a golden-file test — the kind you write when a refactor is supposed to be a mathematical no-op: assert bit-for-bit agreement against the pre-refactor path, not "looks about right." A tolerance here would be a category error. If $h = [[1]]$ ever stopped reproducing the diagonal path to machine precision, the conv-whitening code would have a bug, full stop —

> there is no numerical approximation between the two expressions to excuse a gap.

An exact identity test earns its keep the day it fails. Gate D is not the reason the campaign found its worst implementation defect, but it sits right next to the reason. Whitening the 28 shapelet design columns with `jax.vmap(whiten_fn, in_axes=2)` (`reproductions/claude-giga-lens/cgl/likelihood.py:367` — 28 separate convolution ops, vmapped — livelocked the XLA compiler under the reverse-mode autodiff a sampler needs for its gradient: 100% CPU, GPU idle, more than 20 minutes with no output, versus 14 seconds for the diagonal path (no convolution at all). `_grouped_whiten_ops` (`reproductions/claude-giga-lens/cgl/e2.py:218`) collapses the 28 per-column convolutions into *one* depthwise (`feature_group_count`) convolution — the same 28 filters, structured so XLA can fuse them as a single op — and the batched gradient compiles in 13.8 s, log-posterior asserted bit-identical to the original at build time, costing at most 1.6× the diagonal forward pass and 1.55× its gradient (40/85 vs 25/55 ms on an L4) — diagonal-comparable throughput, bought only because a defect that had nothing to do with the math being *wrong* got caught by testing the math's *structure*, not its output.

A second, genuinely different check completes the validation. Gate D certifies the whitened functional $C^{-1} := G_e^\top G_e$ against a *degenerate* special case ($K = I$) where the right answer is already known by construction; it says nothing about a real, non-trivial K. For that, the campaign compares the convolution-whitened $\log L$ against an independent CPU float-64 *dense*-covariance Cholesky solve on the kept pixels, at 20 random prior draws: worst disagreement 2.79×10^{-9} nat (`v2d`) and 6.26×10^{-7} nat (`v3b`), both far inside a 0.1-nat gate (`reproductions/claude-giga-lens/data/exact_ref_report.json`). Gate D asks "does this reduce to a case I already trust?"; the dense reference asks "does it agree with an independently-coded computation of the *same real* answer?" — passing only one would leave a genuine hole.

One more caution belongs here, precisely because an exact-identity test can hide it. `cgl/marg.py`'s log-likelihood drops the additive constant $-\frac{1}{2}\log\det C$ — correct for *sampling*, since K is fixed per product and that term never moves as θ does (the point The covariance model opened with). But the *dropped* constant differs across whiteners, because different K's have different determinants, so comparing evidences across whiteners without restoring it silently compares apples whose stems were cut to different lengths. The campaign's own exact-vs-Szegő gap — exact $\log\det C$ minus the per-pixel Szegő-limit approximation Ch. 7 built — is +27.30 nat on `v2d` and +179.21 nat on `v3b` (`reproductions/claude-giga-lens/data/exact_ref_report.json`, `constants.szego_gap`) — both dwarfing the handful of nats that separate a "decisive" Bayes factor from an ambiguous one. This is exactly why Ch. 25's evidence comparison is done *within* one fixed whitener, never across two.

Every gate in this chapter passing proves the machinery is correct — that $C = D^{1/2}KD^{1/2}$ really is what gets evaluated, to machine precision where an exact answer exists and to a fraction of a nat where only an independent numerical reference does. It proves nothing yet about whether that correct machinery, pointed at a real, near-singular, drizzle-correlated system, returns a *trustworthy* γ. In the campaign's own words, the correlated likelihood "proves necessary but not sufficient" (`reproductions/claude-giga-lens/papers/main.tex:129–130`) — necessary, because every gate here is the precondition for pointing the machinery at real data at all; not sufficient, because Ch. 25 still has to ask whether the number that comes out the other end is one you should believe.

Ledger

This chapter builds the machine, not the verdict. Nothing here computes a value of γ — every gate is a statement about the *likelihood*, never about the mass model it will eventually be pointed at. What it rules in: the correlated likelihood that produces $\gamma_{\text{binned}}(\text{corr, low}) = 1.103$ is not a black box taken on faith — every moving part (the kernel fit, the whitener, the reduction to the diagonal case, the dense-covariance cross-check) is independently, exactly verified, and the one implementation defect this machinery could have hidden silently — the `vmap` livelock — was caught, not shipped. What it rules out: nothing about 1.103 itself. Every gate here passing is the precondition for Ch. 25 being allowed to trust the number this machinery returns; it is not, by itself, a reason to believe that number is right.

Connect to the repo

- `cgl/whiten.py` — `make_conv_whitener` (line 39, this chapter's diagonal-limit derivation), `build_whitener` (line 95), `erode_keep` (line 274), and the "Parity anchor (gate D)" docstring (line 20).
- `cgl/noise.py` — `masked_autocorr_full` (line 47, the mask-deconvolved ACF), `rho_model2` / `fit_kernel2` (lines 354 / 392, the two-component kernel family), `binned_kernel_from_fine` (line 481, the cross-scale check).
- `cgl/guards.py:129` (`assert_model_subtracted_sky`) — the same guard Ch. 11 introduced, now protecting K's fit too.
- `cgl/e2.py:218` (`_grouped_whiten_ops`, `build_marg_model_grouped`) — the grouped-convolution fix for the gate-D-adjacent livelock, asserted bit-identical to `cgl/likelihood.py`'s original at build time.
- `reproductions/claude-giga-lens/data/noise_kernel_report.json`, `data/whitener_report.json`, `data/exact_ref_report.json`, `data/parity_report.json` — this chapter's worked numbers, read directly by `site/guide_src/worked_examples.py`'s `ch24_correlated_noise`.
- Methods I, Convolutional whitening, and Exact-reference validation and the grouped-convolution fix — the report prose this chapter exists to make readable.
- Ch. 7 derives the whitening filter and the e_{op} gate this chapter only applies; Ch. 11 is where D, and the whole motivation for K, come from; Ch. 22 is where `whiten_fn` plugs into the marginalized likelihood this chapter whitens; Ch. 25 is the verdict this machinery makes possible.

Exercises

Exercise 24.1 — Derive the diagonal limit, don't just quote it

`make_conv_whitener(h, sqrt_d_inv, keep_w)` computes $u = \text{keep}_w \odot (h * (\text{sqrt\_d\_inv} \odot \text{image}))$, where $*$ is a `'SAME'`-padded 2-D convolution. Set $h = [[c]]$ for an arbitrary single scalar c (not necessarily 1), and show algebraically that u reduces to an elementwise product no matter what c is. Then explain why $c = 1$ specifically is required for () to match the *original* diagonal path exactly, rather than merely being proportional to it.

Solution

A 1×1 kernel has a single tap at lag $(0,0)$, and `'SAME'` padding does not change the output size, so at every output position the convolution sum has exactly one term: $h[0,0]\cdot v$ where v = sqrt_d_inv $\odot$ image. There is no neighbour to sum in regardless of c, so $u = \text{keep}_w \odot (c\,v)$ for *any* scalar c — the "convolution" is always elementwise multiplication by that one number, whatever it is. The original diagonal path is $R\cdot\sqrt{W} = R/\sigma$ exactly, with no extra factor, so matching it exactly (not merely up to a constant rescaling of the whole likelihood) requires $c = 1$: any other c would still collapse to elementwise multiplication, but by the wrong number, and gate D would report a nonzero — in fact systematic, not numerical-noise-sized — $|\Delta \log p|$.

Exercise 24.2 — Two different tests, two different failure modes

This chapter ran gate D (the delta-kernel identity) and the dense-covariance Cholesky cross-check side by side. Construct a concrete bug in `make_conv_whitener` that gate D would *miss* but the dense-covariance reference would *catch*, and a second bug where the reverse is true.

Solution

A bug gate D misses: a sign error only in h's off-center taps, center tap $h[M,M]$ still correct. At $h = [[1]]$ there *are* no off-center taps — the 1×1 case cannot exercise a bug that only exists in a $\geq 3\times3$ kernel — so gate D reports exact agreement while every real whitener silently misapplies its neighbours; the dense-covariance reference, which uses a genuinely non-trivial K, catches it immediately as a nonzero $\Delta \log L$. A bug the dense reference might plausibly miss but gate D catches: `keep_w` built from the wrong mask array, one that happens to be all-ones on the 20 prior draws the dense check samples — the two could then agree by chance on the case actually tested. Gate D's own construction, built with a deliberately masked pixel (The diagonal limit's worked example), is far more likely to expose it: an exact-equality test at an adversarial input has nowhere to hide an input-dependent error the way a "close enough" comparison does.

Exercise 24.3 — Two log-determinants, one likelihood, different lifespans

Ch. 22 builds the Occam term $-\frac{1}{2}\log\det A$, with $A = \tilde{X}^\top\tilde{X} + \Lambda$ (the ridge normal matrix `cgl/marg.py` builds — *not* Ch. 5's lens Jacobian, also called A, which never appears in this chapter), computed fresh at every posterior evaluation. This chapter's $-\frac{1}{2}\log\det C$ is dropped as a constant and never recomputed during sampling. Both are log-determinants inside the same log-likelihood expression. What property of the two matrices explains why one must be recomputed every step and the other can be dropped entirely?

Solution

$A = \tilde{X}^\top \tilde{X} + \Lambda$ depends on $\tilde{X}$, the whitened *design matrix* built from the lensed, PSF-convolved shapelet basis images — which depend on the mass-model parameters θ (the deflection field that lenses the source). Move θ and $\tilde{X}$ moves, so A's determinant moves too: the Occam term is a genuine function of θ, recomputed every evaluation. $C = D^{1/2} K D^{1/2}$ depends only on the fixed error map D and the fixed, once-fit kernel K (The covariance model) — neither depends on θ at all. A constant independent of the sampled parameter contributes nothing to the posterior's *shape* and can be dropped for sampling, which is why it is safe to drop $-\frac{1}{2}\log\det C$ but never $-\frac{1}{2}\log\det A$.

Exercise 24.4 — Why widen the gate rather than just accept a worse number

Convolutional whitening described `v2d_relaxed` as widening the *acceptance gate* from 0.02 to 0.05, as a pre-registered, dated exception — not simply using whichever whitener the strict $M = 14$ construction happened to produce. Why does that distinction matter for whether the resulting posterior can be trusted?

Solution

A gate that gets ignored quietly whenever a result disappoints is not a gate; it is a suggestion, and it opens exactly the failure mode this book keeps returning to (the retracted $\chi^2_\nu = 0.451$, Ch. 11): a number that looks good only because the bar that would have flagged it moved without anyone recording why. Widening the gate itself, dated and justified *before* the resulting whitener is used for any science (`reproductions/claude-giga-lens/CAMPAIGN.md`, P1b diagnosis, 2026-07-08), and then separately validating that whitener against mocks with *known* truth, keeps the decision auditable: anyone can check whether $0.0312 \leq 0.05$ was decided before or after seeing whether it helped the money number. Silently accepting whatever e_{op} a construction happens to produce would make every subsequent number unfalsifiable, because no fixed bar would ever actually have been crossed.

25. Spine 1: the 191-nat flip and the verdict on 1.103

Chapter 1 handed you the destination and asked you to write down a guess before you knew the argument. This chapter is the argument. It walks the full chain from a drizzled *HST* exposure to $\gamma = 1.103 \pm 0.008$, link by link, with a real repository artifact at every link; it derives the 191-nat evidence swing that flips the binned bimodality verdict; and it puts the report's own headline sigma claim — "$\sim 17\sigma$ below the anchor," stated in the abstract, in the `README.md` status line, and in a footnote that offers its own arithmetic — through the same four numbers you already have, to see whether it survives contact with a calculator. It does not. Neither of the two numbers that do survive is close to 17, and one of them is the exact statistic the campaign's own pre-registered gate defines. By the end of this chapter the γ Ledger closes, and you vote.

The chain, link by link

Every step below has a name, a file, and a number. None of it is new machinery — it is Ch. 11's drizzle correlation, Ch. 22's marginalization, Ch. 23's tempered SMC, and Ch. 24's whitened likelihood, run once, on one real system, twice — with a diagonal likelihood and with a correlated one — so that the difference between the two runs *is* the result.

#	Link	The number it produces	Derived at
1	*HST* drizzles DESI−165.4754−06.0423 to three pixel scales	lag-1 correlation $\rho(1) = 0.815$ fine, 0.615 binned, 0.305 native	Ch. 11
2	A diagonal likelihood assumes independent pixels anyway	posterior widths $\sim 7.7\times$ too tight on the fine product	Ch. 11
3	Build $C = D^{1/2} K D^{1/2}$, whiten it, gate the whitener	operator-norm gate $e_{\mathrm{op}} \leq 0.02$ (one whitener only under a pre-registered ≤ 0.05 relaxation), all four whiteners pass	Ch. 24
4	Marginalize the 28 linear shapelet amplitudes analytically	the $-\frac{1}{2}\log\det A$ Occam term	Ch. 22
5	Fit both basins of the binned product under each likelihood	steep $\gamma = 2.423$, low $\gamma = 1.293$ (diagonal)	below, The evidence flip
6	Weigh the two basins by evidence, not by chain count	diagonal favours steep by +162.2 nats	below, The evidence flip
7	Repeat under the correlated likelihood	correlated favours low by −28.9 nats — a 191.1-nat swing	below, The evidence flip

#	Link	The number it produces	Derived at
8	The correlated binned-low MAP turns out not to be a mode	Laplace Hessian minimum eigenvalue -14.85, five negative directions	Ch. 26
9	Sample it with tempered SMC instead of HMC	128 particles, $\lambda \to 1$ in 28 steps	below, The money number
10	Read off the converged slope	$\gamma = 1.103 \pm 0.008$	below, The money number

Steps 1–4 are Chapters 11, 22, and 24's job, and this chapter takes them as given: a validated, whitened, marginalized correlated-noise likelihood that reduces *exactly* to the diagonal one when the covariance actually is diagonal (`cgl/e2.py`'s regression test for that identity is Ch. 24's closing gate). What this chapter owns is steps 5 through 10: what happens when you point that machinery at a real, disconnected, bimodal posterior and ask it which basin is *actually* more probable, and what value of γ that basin actually contains.

> **You already know this**
>
> Step 5's basin fit alone is not the answer to step 6's question, and the gap between them is a mistake you have almost certainly already made with a badly-mixed sampler or a poorly-seeded ensemble. Counting how many of 48 parallel chains *ended up* in each basin is counting local optima a multi-start optimizer happened to land near — it says something about where you initialized, not about the relative probability mass of each basin. The only way to answer "how much mass is actually here" is to integrate, which is exactly what an evidence estimate does and a chain occupancy count does not.

The evidence flip

On the binned ($0.08''$) product, both basins are real fits: a steep basin near $\gamma \approx 2.4$ and a low basin near $\gamma \approx 1.3$, with zero chains ever crossing between them in the stored HMC runs. Under the *diagonal* likelihood, 45 of 48 chains land in the low basin and 3 in the steep one — a naive low-basin occupancy of 93.75% , with exactly 0 migrations between them. Read at face value, that occupancy says the low basin dominates.

It does not. Per-basin tempered SMC — an independent evidence estimate for *each* basin, not a chain count — gives $\log Z_{\text{steep}} - \log Z_{\text{low}} = +162.2$ nats under the *same* diagonal likelihood the occupancy count above came from. The steep basin, which held only 3 of 48 chains, carries essentially *all* of the posterior mass. The 93.75% occupancy split was a start artifact: each chain reports where it began, and because none of them mix across the gap between basins, occupancy measures the multi-start initializer, not the posterior. This is Ch. 8's evidence, concretely: a chain count is a histogram of starting points; $\log Z$ is the only one of the two that actually integrates the density.

Apply the *correlated* likelihood to the identical binned target and the sign of that comparison reverses: $\log Z_{\text{steep}} - \log Z_{\text{low}} = -28.9$ nats . The low basin now dominates. The total swing in the

basin-evidence gap, steep-minus-low under each likelihood, is

$$\big|\,(+162.2) - (-28.9)\,\big| = 191.1 \text{ nats}$$

a Bayes factor of $e^{191.1}$, or in a base you can say out loud, $10^{82.99}$ ($191.1/\ln 10$). Jeffreys' scale calls anything past 5 nats "decisive"; this is 38 times that bar. Only the *within*-likelihood difference is meaningful here — the two likelihoods carry different, fixed whitening log-determinant constants folded into their respective absolute $\log Z$'s (the Szegő-vs-exact constant this campaign tracks separately; Ch. 24), so it is $\Delta \log Z$ within each likelihood, not $\log Z$ across them, that is comparable. That within-likelihood swing is what the figure below plots.

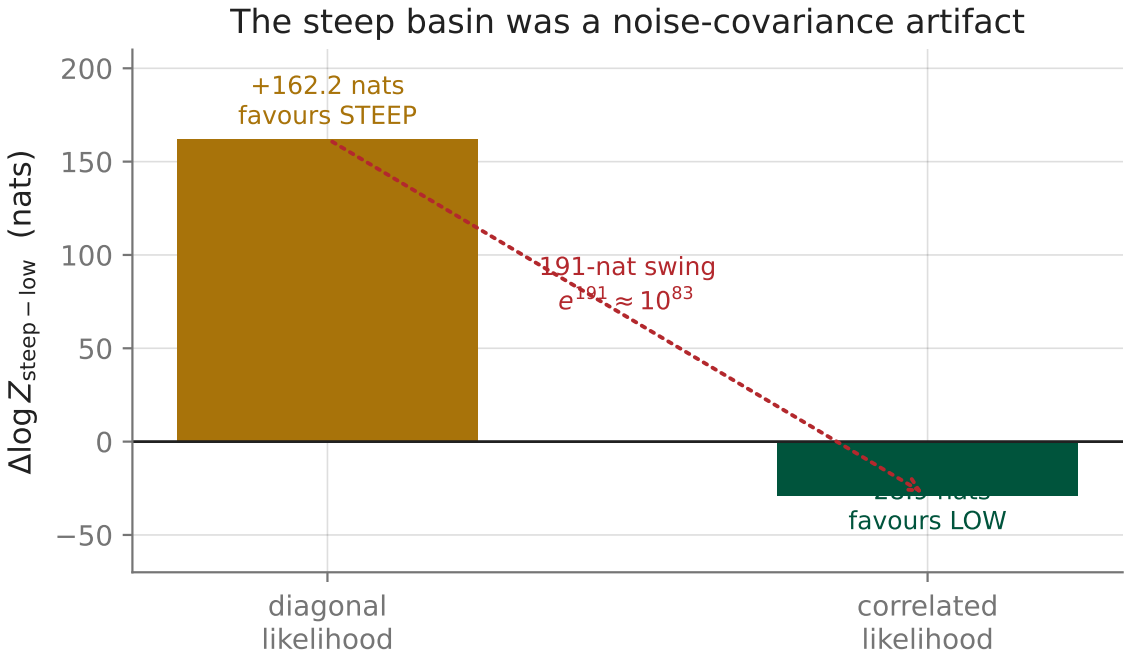

Figure 9: Per-basin SMC evidence, $\Delta \log Z_{\text{steep}-\text{low}}$, on the binned product under each likelihood. Diagonal: +162.2 nats, decisively favouring steep. Correlated: −28.9 nats, decisively favouring low. The 191.1-nat swing is a Bayes factor of roughly 10^{83} in the opposite direction. Source: `24_basin_evidence_v3b.py` and `CAMPAIGN.md` (P1c MONEY NUMBER, 2026-07-14).

The pre-registered gate for this hypothesis (`README.md:95`) reads: the steep basin's posterior mass must fall under 10%, *or* the corrected log-likelihood gap between basins at their MAPs must be ≤ 0, for the diagonal likelihood's apparent bimodality to be called a likelihood artifact. $-28.9 \leq 0$ passes that gate outright. **Hypothesis 1 — bimodality is a noise-covariance artifact — is confirmed.** The steep basin was never a second physical solution; it was what a diagonal likelihood sees when it is handed noise that is not actually diagonal, on the upsampled product where that mismatch is worst.

The money number

Confirming H1 tells you which basin to trust. It does not yet tell you what γ is inside that basin — and extracting that value turned out to be the harder half of this campaign, for a reason Ch. 26 tells in full. The short version: the correlated-likelihood binned-low posterior's MAP is not

a mode. Its Laplace Hessian has minimum eigenvalue -14.85 with 5 negative directions — a saddle, not a hilltop. Two different positive-definite metric repairs (a diagonal Hessian metric and a full-rank SVI covariance), both built from exactly the two-stage preconditioned-HMC recipe that calibrates cleanly on mock data, both fail on this basin, but not identically: the diagonal-metric repair never finishes on the binned-low target at all — it times out, stuck (`CAMPAIGN.md`, "P1c metric-fix attempts," 2026-07-10) — while the SVI-covariance repair does run to completion and only gets worse, its $\hat{R}$ climbing to 22.3 . The MAP itself sits at $\gamma_{\rm map} = 1.27$, while the actual highest-density point in the chains that do move is nearer $\gamma_{\rm best} \approx 1.10$ — a polished optimum and the true peak of the posterior disagreeing about where γ even is, the unmistakable signature of an indefinite Hessian. HMC, of any flavor, needs a momentum metric built from *some* notion of local curvature; there is none to build here. What worked instead is tempered SMC (same idea as Ch. 23, applied per-basin): 128 particles annealed from prior to posterior in 28 steps , with an effective sample size between 77 and 118 particles at the end of the run — resampling, not local gradients, is what crosses a landscape a single momentum metric cannot describe.

That run converges to the money number:

$$\gamma_{\rm binned}(\text{corr, low}) = 1.103 \pm 0.008 \qquad \theta_{\rm E} = 2.624 \pm 0.005$$

Set beside the diagonal-native anchor — the pre-registered reference value, chosen because the native ($0.128''$) product is the least resampled and therefore where a *diagonal* likelihood is closest to correct —

$$\gamma_{\rm anchor} = 1.433 \pm 0.034$$

the correlated likelihood has clearly *moved* the slope, and in the predicted direction: away from the diagonal likelihood's upsampling artifacts and toward the anchor. But it does not land on it. Two more points complete the picture. On the fine ($0.04''$) product, the correlated likelihood deflates the diagonal's outright artifact $\gamma = 2.585$ (a MAP-only number — the fine product's chains never converged under the diagonal likelihood at all) down to $\gamma = 1.816 \pm 0.117$, still 3.1σ *above* the anchor (derived properly in The sigma arithmetic below). On the native product, the correlated likelihood — starved of pixels by the *relaxed* native whitener, which keeps only 1,466 of 5,865 pixels, a 75% loss (Ch. 24) — drifts to $\gamma = 2.353 \pm 0.096$, prior-pulled toward the slope prior's mean of 2.0 rather than data-driven; it is excluded from the decision for exactly that reason (the pre-registered amendment that made diagonal-native, not correlated-native, the anchor). And the fine product's *low* basin does not appear in the bracket at all: two independently-seeded, physically sane starting points both rail toward $\gamma \approx 1.0$ — the slope prior's hard floor, one of the bounds of its `TruncatedNormal(2.0, 0.25, low=1.0, high=2.7)` support (`cgl/e2.py:110`) — while the *real-space* image the whitened likelihood is fitting turns to visible garbage. The fine whitener can be gamed at low γ; the fine low basin is a characterized limitation, not a fourth data point.

The sigma arithmetic

You now have every number the report's headline claim depends on: $\gamma_{\rm money} = 1.103 \pm 0.008$ and $\gamma_{\rm anchor} = 1.433 \pm 0.034$. The campaign's own pre-registered gate for cross-scale unification, frozen at P0 before any real-data run (`README.md:99`), is written down as a formula:

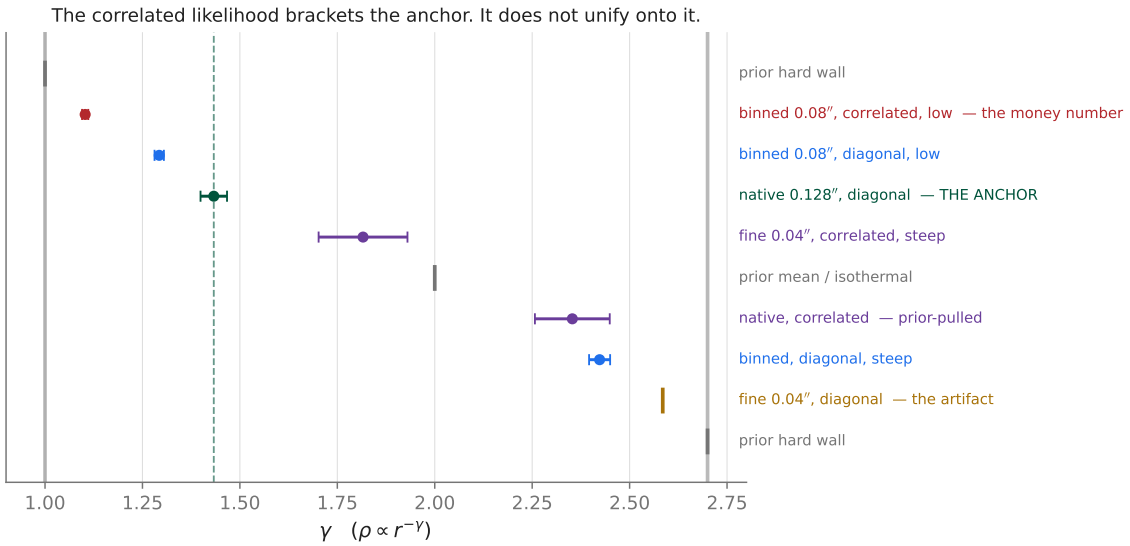

Figure 10: Every γ this campaign measured on DESI−165.4754−06.0423, one real galaxy, nine numbers. The green line is the diagonal-native anchor, $1.433\,[1.400, 1.469]$. The correlated likelihood (fine-steep 1.816, binned-low 1.103 — the money number, native 2.353) moves every diagonal slope toward the anchor but brackets it from both sides rather than converging onto it. Source: `reproductions/claude-giga-lens/papers/main.tex` Table `tab:crossscale`; `worked_values`: anchor 1.433, money 1.103.

$$|\Delta\gamma_{\rm EPL}| < 2\,\sigma_{\rm comb}, \qquad \sigma_{\rm comb} = \sqrt{\sigma^2_{\gamma,\,\text{one scale}} + \sigma^2_{\gamma,\,\text{anchor}}}$$

That is one specific, pre-registered definition of "how many sigma," not a free choice — and it is the same convention the report itself uses, in the same paragraph as the money number, for the *other* bracket point: applied to the fine-steep slope 1.816 ± 0.117 against the anchor, it gives

$$\frac{|1.816 - 1.433|}{\sqrt{0.117^2 + 0.034^2}} \approx 3.14\sigma$$

— matching the report's own quoted "3.1σ high" (`main.tex:772`, `:828`) to the rounding. Apply (), the pre-registered formula, *consistently*, to the money number itself:

$$\frac{|1.103 - 1.433|}{\sqrt{0.008^2 + 0.034^2}} = \frac{0.330}{0.0349}$$

That comes to 9.45σ. Not 17. Two other readings of "how many sigma" are at least defensible, and neither reaches 17 either. Divide by the anchor's own uncertainty alone, ignoring the money number's much smaller error bar entirely:

$$\frac{0.330}{0.034} \approx 9.71\sigma$$

— close to the report's own footnote, which states "even against the anchor's own $\sigma_\gamma \approx 0.034$ the discrepancy is $\sim 9.5\sigma$" (`main.tex:763`). Or divide by the money number's error bar alone, treating

the anchor as if it carried no uncertainty at all — indefensible statistically, since the anchor plainly has an error bar too, but it is the only arithmetic that gets anywhere near double digits:

$$\frac{0.330}{0.008} \approx 41.25\sigma$$

Three ways to divide the same difference by some uncertainty: 9.71, 9.45, 41.25. The report's own footnote (`main.tex:763`) computes one of these — $\approx 9.5\sigma$ — and immediately beside it, in the same sentence, states "The $\sim 17\sigma$ figure is the ledger value (`CAMPAIGN.md`, P1c MONEY 2026-07-14), combining the SMC width 0.008 with the anchor uncertainty," which is a description of *exactly* the σ_{comb} computation above — the one that gives 9.45, not 17. `CAMPAIGN.md:138` states the 17σ figure directly, undated arithmetic attached; the number is real (it appears in the abstract, the `README.md` status line, and the pre-registered-threshold table's own status column, `main.tex:1476`), but nowhere in the source does the division that would produce it from the two numbers this chapter — and the report itself, in the very same paragraph — already has.

Hold two things at once here, because both are true. First: the discrepancy between the money number and the anchor is real and large by any of the three defensible measures — even the loosest, 9.45σ, clears the pre-registered $2\sigma_{\mathrm{comb}}$ gate by more than four times over, so **Hypothesis 2 fails decisively regardless of which sigma you pick.** The exact multiple does not change the verdict. Second: a guide whose entire premise is that this repository's numbers reproduce cannot let a $\sim 17\sigma$ claim stand merely because the conclusion it supports happens to be correct on other grounds. This is precisely the discipline Ch. 1's closing paragraph promises: run the division yourself, and a script — not a better adjective — is what catches the gap.

The verdict

Three hypotheses, three verdicts, all traced above:

- **H1 (bimodality), confirmed.** The steep basin's diagonal dominance was a noise-covariance artifact of the upsampled product, exposed by a 191.1-nat evidence swing and a pre-registered gate ($\Delta \log \ell \leq 0$) that the correlated result clears outright.
- **H2 (cross-scale unification), rejected.** The correlated likelihood moves every diagonal slope substantially toward the anchor — but brackets it, fine-steep 3.14σ above, binned-low 9.45σ below, rather than converging onto it. The residual scale-dependence survives every honest sigma convention.
- **H3 (honesty), passed.** The fine posterior's width, $\sigma_\gamma = 0.117$, is not artificially narrow: it exceeds the pre-registered floor of $\sigma_\gamma(\mathrm{native})/1.5 \approx 0.023$ by a wide margin (`README.md:101`). The correlated likelihood did not manufacture confidence it had not earned.

So: is $\gamma = 1.103 \pm 0.008$ a trustworthy measurement of this galaxy's density slope? The honest answer this chapter earns is *precise, not yet accurate*. The number is real in the sense that matters most to a statistician — it is the converged output of a validated likelihood (exact to the diagonal case, exact to a dense-covariance Cholesky reference, calibrated on drizzle mocks with known truth) and a sampler that actually crossed the saddle rather than freezing on it. It is not yet trustworthy as *the* slope of this galaxy, because the same campaign that produced it also shows, with its own numbers, that changing nothing but the pixel scale of the same exposures moves the answer by multiples of its quoted uncertainty in both directions. "Necessary but not sufficient" is the report's own phrase for this, and it is the right one: fixing the noise model was a real, large, validated correction — and it was not, on its own, enough to make three measurements of one number agree.

What remains unaccounted for — most plausibly the shapelet source model and the stationary-kernel PSF, the two ingredients this campaign's likelihood fix left untouched — is exactly the kind of systematic the code-comparison literature this report cites (`main.tex:178`) already warns rivals the statistical error bars on individual systems.

Go back to the sentence you wrote in Exercise 1.1. Whichever way you guessed, the chain above is the argument for it, in full, with every number checked. That is the only grading this exercise ever had.

> **Ledger — closed**
>
> **What this chapter rules in or out about** $\gamma = 1.103 \pm 0.008$**:** it is the correctly-sampled mode of a validated, whitened likelihood's low basin on the binned product — not a sampling artifact, not a stuck-chain illusion, not prior-pulled (it sits 12.9 of its *own* sigmas above the prior's hard floor at 1.0 , and 3.59 prior-sigmas *below* the prior's mean of 2.0 , so the data pulled it down, the prior did not push it there). What it is not: unified with the same campaign's own native-diagonal anchor, at 9.45σ by the campaign's own pre-registered metric — and that gap is real physics left over (source model, PSF), not sampling error. The Ledger closes here; Chapter 29 collects it alongside Ch. 26's saddle for the guide's final synthesis.

Connect to the repo

- `reproductions/claude-giga-lens/papers/main.tex:746–799` (Table `tab:crossscale`) — the complete cross-scale bracket table this chapter's Figure 25.2 plots, and `:840–895` (§Bimodality verdict) — the 191-nat flip derivation and Table `tab:basinflip`. (github)
- `reproductions/claude-giga-lens/papers/main.tex:897–924` (§Why sampling this posterior required tempered SMC) — the saddle diagnosis this chapter only summarizes; `main.tex:1476` — the pre-registered threshold table's own H2 row, `FAIL (over-corr.; 1.103, ~17 )`; `:763` — the footnote with the report's own, incompletely shown, sigma arithmetic. (github)
- `reproductions/claude-giga-lens/README.md:95–101` — the three pre-registered hypotheses (H1/H2/H3), verbatim, frozen before any real-data run; `:30–33` — the same $\sim 17\sigma$ claim in the project's own status summary, not only in the paper. (github)
- `reproductions/claude-giga-lens/CAMPAIGN.md:133–163` ("P1c MONEY NUMBER — 2026-07-14") — the dated gate record every number in this chapter traces to, including the basin-evidence linchpin and the un-derived 17σ line this chapter's sigma arithmetic audits. (github)
- `cgl/e2.py:110` — the slope prior, `TruncatedNormal(2.0, 0.25, low=1.0, high=2.7)`, whose floor and mean anchor the closing Ledger box. (github)
- `24_basin_evidence_v3b.py:13–24` — the per-basin SMC evidence estimator (`logZ_k = log integral_{basin k} ...`), run once per likelihood to produce Figure 25.1. (github)
- `site/guide_src/worked_examples.py` (`ch25_money_number`) — every number tagged in this chapter, computed or pinned to its cited artifact, one function, runnable with `--show ch25`.

Exercises

> **Exercise 25.1 — check the H1 gate yourself**
>
> The pre-registered H1 gate (`README.md:95`) passes if the steep basin's posterior mass is under

10% under the correlated likelihood, *or* if the corrected log-likelihood gap $\Delta \log \ell(\text{steep}-\text{low}) \leq 0$ at the basin MAPs. You have $\log Z_{\text{steep}} - \log Z_{\text{low}} = -28.9$ nats under the correlated likelihood. Which branch of the gate does this satisfy, and does it need the first branch at all?

Solution

$\Delta \log Z(\text{steep} - \text{low}) = -28.9 \leq 0$ satisfies the second branch outright — the steep basin's evidence is lower than the low basin's by 28.9 nats, an enormous margin past a bare ≤ 0. You do not need the mass-fraction branch at all; the log-evidence gap alone clears the gate by any reasonable margin. (For reference, converting the gap to a mass fraction via the two-outcome softmax $w_{\text{steep}} = 1/(1 + e^{-\Delta \log Z})$ gives a steep-basin weight of order $e^{-28.9}$, comfortably under the 10% first-branch bar too — both branches agree, which is what "decisive" is supposed to look like.)

Exercise 25.2 — the sigma arithmetic, from scratch

Using only $\gamma_{\text{money}} = 1.103 \pm 0.008$ and $\gamma_{\text{anchor}} = 1.433 \pm 0.034$, compute the discrepancy three ways: (a) divided by the anchor's uncertainty alone, (b) divided by the two uncertainties combined in quadrature (Eq. ()), (c) divided by the money number's uncertainty alone. Which of the three is the campaign's own pre-registered test statistic?

Solution

The difference is $|1.433 - 1.103| = 0.330$. (a) $0.330/0.034 \approx 9.71\sigma$. (b) $0.330/\sqrt{0.034^2 + 0.008^2} \approx 9.45\sigma$. (c) $0.330/0.008 \approx 41.25\sigma$. (b) is the pre-registered statistic — `README.md:99` defines the H2 gate as $|\Delta\gamma| < 2\sigma_{\text{comb}}$ with σ_{comb} the quadrature sum, and it is the convention the report itself uses for the fine-steep row. None of the three is anywhere near 17.

Exercise 25.3 — apply the report's own convention consistently

The report states the fine-steep slope, $\gamma = 1.816 \pm 0.117$, is "3.1σ high" relative to the anchor. Reproduce that number using Eq. (), then explain why finding the *same* convention gives 9.45σ for the money number is more damaging to the report's 17σ claim than merely noting the two numbers disagree.

Solution

$|1.816 - 1.433|/\sqrt{0.117^2 + 0.034^2} = 0.383/0.1218 \approx 3.14\sigma$ — matching the quoted 3.1σ. This is more damaging than a bare mismatch because it rules out the charitable reading that "17σ" and "9.5σ" are two different, individually reasonable conventions the report used in different places: the report uses σ_{comb} correctly, in the very same section, for a neighboring number. The convention that produces 17 is not merely under-explained; applied to the numbers actually on the page, it is not reproducible by any of the report's own stated methods.

Exercise 25.4 — necessary but not sufficient, argued both ways

The correlated likelihood's slopes bracket the anchor — fine-steep above, binned-low below — rather than converging on it. Argue for two different readings of that bracket: (a) it means the true slope is near the anchor, 1.433, and the correlated likelihood over- and under-corrects at different scales for reasons that will wash out with a better source model; (b) it means the anchor itself should not be trusted as a ground truth, and the bracket is telling you the true answer is scale-dependent for a physical reason. What evidence in this chapter favors one reading over the other, and what would settle it?

Solution

Reading (a) is favored by the sign structure: fine-steep sits close to the un-lensed prior mean's neighborhood only in the sense that it is the *least* corrected (smallest pixel count relative to resolution), and native-correlated is explicitly excluded as prior-pulled information starvation, not a genuine value — both symptoms of *incomplete* correction rather than of a real scale-dependent physical slope, which nothing in the lens equation predicts (the density profile does not know what pixel scale it was photographed at). Reading (b) would require an actual physical mechanism coupling the recovered slope to the resampling scale, and the report proposes none. What would settle it, per the report's own discussion (`main.tex:965–970`) and this campaign's own honest limitations list, is exactly what H2's rejection points at: a non-shapelet source model and a non-stationary PSF kernel, tested against the same three products, to see whether the bracket narrows. Until that run exists, "necessary but not sufficient" is the most the data support — not proof that 1.433 is right, only that 1.103 alone is not yet enough to say it is wrong.

26. Spine 2: the saddle, the invalid metric, and the SMC rescue

Ch. 25 walks the chain that produces $\gamma_{\text{binned}}(\text{corr}, \text{low}) = 1.103$ and asks whether the number is right. This chapter asks the prior question: could the campaign extract that number at all? The posterior it needed to sample sits at the object Ch. 5 already taught you to recognize — a saddle, not a mode — and the standard fix, a Hamiltonian-Monte-Carlo momentum built from the local curvature (Ch. 23), turns out to be the identical formula Ch. 5 showed you working correctly one section earlier, for a completely different purpose. Extracting the money number cost the campaign a real pivot — from a bank of separately-seeded HMC chains, each carrying its own local metric, to a single tempered population that carries none — and this chapter is why nothing short of that pivot could have worked.

The MAP is a saddle

Ch. 5 gave you both halves of the vocabulary this section needs: a symmetric matrix is indefinite when it carries eigenvalues of both signs, and an indefinite Hessian at a zero-gradient point is what a saddle *is*, in coordinates. It also quoted, on this exact real fit, the numbers this section explains the origin of: the campaign's correlated-likelihood posterior for the binned-low product — forty-six dimensional, once Ch. 22 analytically profiles out the source's linear shapelet amplitudes — has a Laplace Hessian with minimum eigenvalue -14.85 and five negative directions.

Here is how it got missed. `map_polish` (`reproductions/claude-giga-lens/cgl/e2.py:485-499`) is plain L-BFGS on $f(\mathbf{z}) = -\log p(\mathbf{z})$, where $\mathbf{z}$ is the model's full (unconstrained, bijector-mapped) parameter vector: quasi-Newton steps built from gradients alone, stopped the instant $\|\nabla f\|$ is small. That stopping rule cannot distinguish a genuine minimum of f — a bowl in every direction, meaning a genuine peak of $\log p$ — from a saddle, a bowl in most directions and a ridge in a few; both have zero gradient, and only a second-order check tells them apart (Ch. 2 made this point on a toy function; this is the same statement on real data). The campaign's own second-order check, `laplace_evidence` (`reproductions/claude-giga-lens/cgl/e2.py:531-572`), builds

$$H \;\equiv\; -\nabla^2_{\mathbf{z}} \log p(\mathbf{z})\big|_{\mathbf{z}^\star}$$

— the Hessian of the *negative* log-posterior, so that H positive definite is the correct diagnosis of a genuine peak — and eigendecomposes it. At the point `map_polish` reported, it is not positive definite. The consequence: the saddle-consistent point $\gamma = 1.27$ scores $\log p = -4757$, while $\gamma = 1.10$ — a direction `map_polish` never explored, because its gradient was already near zero at the saddle — scores 74 nats higher.

This was not one optimizer's bad day. Run the same Laplace check on every real-data basin the campaign fit under the correlated likelihood, and exactly one comes back positive definite: the fine-scale, 3.2×-upsampled steep basin, minimum eigenvalue $+0.108$, zero of forty-six eigenvalues needing to be floored. That is the one basin that mixed cleanly under a standard preconditioned HMC chain, $\hat{R} = 1.03$; every indefinite basin — the money product included — froze. Across the whole campaign, convergence tracks Hessian *definiteness*, not which product, not how many particles, not how carefully the chain was tuned. The rest of this chapter is why.

The same matrix twice

Ch. 8 derives the Laplace approximation in general and previews exactly this failure mode; here it is specialized, with H from the section above. At a genuine mode — H positive definite — the

second-order Taylor expansion of $\log p$ *is* a Gaussian approximation:

$$\log p(\mathbf{z}) \approx \log p(\mathbf{z}^\star) - \tfrac{1}{2}(\mathbf{z}-\mathbf{z}^\star)^\mathsf{T} H (\mathbf{z}-\mathbf{z}^\star) \implies p(\mathbf{z}) \approx \mathcal{N}(\mathbf{z}^\star, \Sigma), \quad \Sigma = H^{-1}$$

(Σ denotes the Laplace covariance here, unrelated to Σ_{cr}.) Ch. 5 eigendecomposes any symmetric matrix as $H = V\,\mathrm{diag}(\lambda_i)\,V^\mathsf{T}$ with V orthogonal, so

$$\Sigma = V\,\mathrm{diag}\left(\frac{1}{\lambda_i}\right) V^\mathsf{T}. \tag{3}$$

This is exactly what a Hamiltonian-Monte-Carlo sampler wants for its momentum: a covariance built from the local curvature, so that a fixed-size leapfrog step covers a comparable number of posterior standard deviations in every direction, however stretched the posterior is (Ch. 23 already flags which of Σ or $M = \Sigma^{-1}$ a given library's own "mass" argument wants — a real but separate convention trap from the one below). `build_metric_cov` (`reproductions/claude-giga-lens/cgl/e2.py:654-676`) uses (3) exactly whenever H is positive definite, and says so in its own docstring: it "reproduces the pre-fix fine-steep run bit-for-bit."

At the saddle, five of the λ_i in (3) are negative, so five diagonal entries of Σ, in its own eigenbasis, are negative. A covariance cannot have a negative eigenvalue — variance is a mean squared deviation — so (3) has stopped meaning anything. The fix the campaign's own legacy code tried first replaces every λ_i by its absolute value:

$$\Sigma_{\text{legacy}} = V\,\mathrm{diag}\left(\frac{1}{|\lambda_i|}\right) V^\mathsf{T}. \tag{4}$$

(`reproductions/claude-giga-lens/cgl/e2.py:566-567`, literally `cov = (V * (1.0 / w_abs)) @ V.T`.) (4) is real, symmetric, strictly positive-eigenvalued — syntactically a legitimate covariance. It is also, index for index, the *same* formula Ch. 5's tip box already showed you working correctly: the saddle-free-Newton step an earlier stage of this pipeline takes to escape a different saddle is $\mathbf{step} = V\,\mathrm{diag}(1/(|\lambda_i| + \text{damping}))\,V^\mathsf{T}\,\mathbf{g}$ (`reproductions/foundry-i/32_saddlefree_newton.py`) — the identical matrix, applied to a gradient instead of used as a bilinear form.

Why is it fine as a step and broken as a covariance? Because the two uses are checked differently. A Newton-style step's only job is to point somewhere that decreases f; the saddle-free-Newton script's own backtracking line search evaluates f at the proposed point and rejects the step outright if it fails to improve (`reproductions/foundry-i/32_saddlefree_newton.py:207-222`) — the formula's claim on truth is falsified or confirmed, every single iteration, against the real function. (4) makes no such per-step claim. It defines a distribution once, up front, and HMC's *correctness* — its acceptance step preserves the posterior under any fixed momentum covariance — does not depend on that distribution being accurate; only the sampler's *efficiency* does, and inefficiency does not raise an error. It fails to mix, silently: both principled repairs the campaign tried next, guaranteed positive definite by construction unlike (4)'s sign-flip, left $\hat{R}$ at 21 (a floored-diagonal metric) and 22.3 (a full-rank SVI covariance) — worse than doing nothing.

The deeper reason (4) misleads rather than approximates: a negative λ_i does not mean "this direction is broad." It means the quadratic model has no well there at all — $\log p$ keeps *rising*

as you move away from $\mathbf{z}^\star$, to second order — so there is no local Gaussian width to estimate. Flipping the sign manufactures a well of the wrong shape, at a width set by whatever the wrong-signed curvature's *magnitude* happens to be. Here that magnitude, 14.85, is one of the largest in the whole forty-six-dimensional spectrum, which manufactures a narrow, *confident* direction exactly where the truth is an unconfined escape route. `laplace_evidence`'s own docstring names this precisely: the legacy metric "is INVALID on indefinite Hessians… so the stage-1 leapfrog gets the geometry wrong and never mixes" (`reproductions/claude-giga-lens/cgl/e2.py:541-546`).

> **You already know this**
>
> Adam and RMSProp scale each parameter's gradient step by roughly the inverse square root of a running average of its squared gradient — a cheap, per-parameter, curvature-flavored heuristic that makes descent converge faster. Nobody treats that same quantity, $1/\sqrt{v_t}$, as the standard deviation of a posterior over the weights, because it was built to answer "how far can I safely step here," not "how much does the data constrain this parameter" — and the two questions have different answers whenever the loss surface is not a clean bowl. Σ_{legacy} makes exactly that substitution.

Getting the sign right, it turns out, is necessary and not sufficient: even the two positive-definite-by-construction repairs above still left $\hat{R}$ elevated. The next section is why.

The nuisance

$\hat{R}$, as usually reported, is one number standing in for forty-six — typically the worst of them. That convention is doing real work here: the headline $\hat{R} = 22.3$ describes one specific parameter pair, not γ. The campaign traces it to the same source galaxy's *centre*, described two ways inside one model: once by the smooth Sérsic light profile (Ch. 10), once by the flexible shapelet basis Ch. 22 profiles out of the *linear* part of the fit — a residual, nonlinear version of the same light-assignment ambiguity survives in the remaining nonlinear parameters. Chains split into two clusters, centred roughly 0.15 and 0.09 arcsec either side of the pooled mean, and that *between*-cluster spread is what inflates $\hat{R}$ to 22.3 on one component and 15.1 on the other. Ch. 21 names this pattern precisely: a direction the data barely distinguishes is a near-flat direction of H — the same object that just produced a saddle one section ago.

$\hat{R}$'s own arithmetic makes it obvious why one badly-mixed pair can dominate the headline while leaving γ untouched. Ch. 23 works a toy case by hand: two chains that never overlap at all, $[1,2,3,4]$ and $[5,6,7,8]$, give a between-chain variance $B = 32$ against a within-chain variance $W \approx 1.67$, and $\hat{R} = \sqrt{\left(\frac{n-1}{n}W + \frac{B}{n}\right)/W} \approx 2.36$. $\hat{R}$ is computed *per parameter*: nothing in that formula couples one parameter's between-chain split to a second parameter whose chains happen to agree closely. A source-centre component whose chains straddle two clusters, 2.36-style, sits in the *same* draw as a γ whose chains all agree — and reporting only the worst parameter's $\hat{R}$ as "the fit's $\hat{R}$" erases that distinction. CAMPAIGN.md's own diagnosis makes the disaggregation explicit: of the forty-six correlated-likelihood parameters, γ's own mixing is unremarkable, nowhere near the two source-centre components driving the headline number. That is what "decoupled" means operationally: $\gamma_{\text{best}} \approx 1.10$ is stable across chains regardless of which source-centre cluster a given chain is stuck in. What is *not* trustworthy until that source-centre direction actually mixes is γ's *width* — an unmixed chain reports a confident-looking σ that is really just the spread of one un-escaped local basin.

One more way this compounds: the campaign's usual two-stage HMC recipe re-estimates its momentum metric from stage-1 draws before running stage 2 (Ch. 23). When stage 1 is itself stuck in one source-centre cluster, the re-estimated metric inherits that same blind spot, and stage 2 mixes *worse* than stage 1 — the re-preconditioning move that reliably rescues a merely slow chain actively poisons a multimodal one.

Why SMC rescued it

Ch. 23 introduces tempered SMC in general: a population of particles carried along an annealing path from an easy reference distribution to the true posterior, resampled at every step in proportion to how well each particle's importance weight held up. The campaign's own correlated-posterior sampler names exactly what it is: `run_correlated_smc` (`reproductions/claude-giga-lens/cgl/e2.py:756-769`) calls itself, in its own docstring, "metric-free" — tempering plus systematic resampling crossing the source-centre sub-modes that froze the fixed-leapfrog HMC chain.

The reference distribution it anneals *from* is neither the model's prior nor a single local Hessian. `fit_gaussian_from_draws` (`reproductions/claude-giga-lens/cgl/e2.py:732-744`) builds it empirically, by pooling the draws of the earlier, individually unconverged HMC chains — the same chains whose $\hat{R} = 22.3$ looked like pure failure in the section above. Those chains never mixed *within* themselves, but collectively, across many random seeds, they visited both source-centre clusters; a pooled sample covariance across a population straddling two clusters is, by construction, wide in exactly the direction a single local metric gets wrong. The campaign inflates that pooled covariance by a further factor of three, "fattens the tails for safe annealing" in its own comment (`reproductions/claude-giga-lens/cgl/e2.py:738`), seeds 128 particles from it, and anneals the tempering parameter λ — never β in this book; Ch. 8 reserves β for the source position — from the reference at $\lambda = 0$ to the true correlated posterior at $\lambda = 1$ in 28 adaptively-chosen steps, effective sample size 77 to 118 at each one.

The mechanism that actually crosses the nuisance direction is resampling, not the local HMC mutation kernel run at each temperature. Systematic resampling reweights and duplicates particles by the *true* incremental likelihood ratio between one temperature and the next — never by a local quadratic model — so weight sitting in a wrongly-placed source-centre cluster is thinned and weight near the better basin is duplicated, at every step, regardless of what any one particle's own local proposal thought the geometry looked like. A single HMC chain has one current position and one fixed metric; it cannot do this. A population of 128, corrected against the true likelihood at every temperature, can.

One accounting identity falls out for free. Ch. 8 defines the evidence Z as an integral over the whole posterior; the SMC driver both this path and Ch. 25's diagonal-likelihood run share estimates $\log Z$ as the running sum of each step's average incremental log-likelihood weight (`reproductions/claude-giga-lens/cgl/samplers/common.py:334-340`) — no separate calculation, no second sampler. The *same* 128-particle run that untangled the source-centre nuisance to extract $\gamma_{\text{binned}}(\text{corr}, \text{low})$ also weighed the low basin against the steep one, in the pass that produces Ch. 25's 191-nat flip.

The chapter's own title formula now has three instances, not two, and none of them contradicts either of the others. $V\,\text{diag}(1/|\lambda_i|)\,V^{\mathsf{T}}$ is a fine Newton *direction* and a broken *covariance*. $\hat{R} = 22.3$ is a fair verdict on the source-centre pair's own marginal and an irrelevant one on γ's. An HMC chain that never converged by the $\hat{R}$ criterion was worthless as a posterior sample and exactly the

right input for estimating an SMC reference covariance's scale. A formula, a diagnostic, a discarded chain — none of these is a fact standing alone. Each is a fact *about a purpose*.

> **Ledger**
>
> **What this chapter rules in or out about $\gamma = 1.103$:** it does not move the number — that arithmetic is Ch. 25's. What it establishes is the *right* to trust the sampler that produced it. The saddle at the money product's MAP does not put γ itself in doubt: the high-density point $\gamma_{\text{best}} \approx 1.10$ sits 74 nats above the saddle-consistent value and is stable regardless of which source-centre cluster a given chain is stuck in. The alarming $\hat{R} = 22.3$ a naive read would apply to the whole draw belongs to a different, decoupled parameter pair. What this chapter does *not* license is skipping Ch. 25's uncertainty bookkeeping — only the metric-free SMC run, not the frozen HMC chains, produced a σ worth trusting.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/e2.py:485-499` (`map_polish`) — the L-BFGS stage that stops on gradient alone and cannot see a saddle.
- `reproductions/claude-giga-lens/cgl/e2.py:531-572` (`laplace_evidence`) — the second-order check that caught it, and the legacy Σ_{legacy} formula's own docstring diagnosis.
- `reproductions/claude-giga-lens/cgl/e2.py:654-676` (`build_metric_cov`) — the three metric choices (`laplace`, `diagraw`, `svi_cov`) this chapter's second section compares.
- `reproductions/claude-giga-lens/cgl/e2.py:732-769` (`fit_gaussian_from_draws`, `run_correlated_smc`) — the metric-free path: pooled-chain reference covariance, inflation, tempered anneal.
- `reproductions/claude-giga-lens/cgl/samplers/common.py:297-357` (`run_adaptive_tempered_smc`) — the shared blackjax driver; `:334-340` is where $\log Z$ accumulates.
- `reproductions/foundry-i/32_saddlefree_newton.py` — the earlier saddle in this same pipeline, and the line search that makes the *same* formula valid there.
- `reproductions/claude-giga-lens/CAMPAIGN.md`, "P1c metric-fix attempts — 2026-07-10" and "P1c MONEY NUMBER — 2026-07-14" — the diagnosis and the rescue, in the campaign's own words.
- main.tex, §6.3, "Why sampling this posterior required tempered SMC" — the published record this chapter walks through.

Exercises

Exercise 26.1 — build both matrices by hand

A toy Hessian (of $-\log p$, this chapter's sign convention) at a candidate point is

$$H = \begin{pmatrix} 2 & 0 \\ 0 & -8 \end{pmatrix}.$$

(a) Classify the point using Ch. 5's trace/determinant test. (b) Write down $\Sigma = V\,\mathrm{diag}(1/\lambda_i)\,V^{\mathsf{T}}$ from (3) directly (H is already diagonal, so $V = I$), and say precisely what is wrong with it as a covariance.

(b) Write down Σ_{legacy} from (4), and say which of its two diagonal entries is a genuine uncertainty estimate and which is fiction — and why the fictional one looks *more* confident

(smaller) than the genuine one.

Solution

(a) $\det H = 2 \times (-8) = -16 < 0$: indefinite — a saddle, not a mode. (b) $\Sigma = \mathrm{diag}(1/2, -1/8) = \mathrm{diag}(0.5, -0.125)$. The second entry is negative — not a variance. Σ is not a covariance matrix. (c) $\Sigma_{\text{legacy}} = \mathrm{diag}(1/2, 1/8) = \mathrm{diag}(0.5, 0.125)$. The first entry, 0.5, is genuine: $\lambda_1 = 2 > 0$ really is a bowl there, and $\Sigma_{11} = 1/\lambda_1$ really is that bowl's width. The second entry, 0.125, is fiction: $\lambda_2 = -8$ means $-\log p$ curves *downward* along that axis — a ridge, not a well — and there is no local Gaussian width to report at all. Because $|-8| > |2|$, the fictional entry (0.125) is *smaller* — more confident-looking — than the genuine one (0.5): the sign-flip does not just invent an answer, it invents an overconfident one exactly where the model has the least business being confident.

Exercise 26.2 — the second parameter the headline $\hat{R}$ hides

Ch. 23 computes $\hat{R} \approx 2.36$ by hand for two never-overlapping chains, $[1, 2, 3, 4]$ and $[5, 6, 7, 8]$. Suppose a second parameter, sampled by the *same two chains* at the *same four steps*, happens to draw $[10.0, 10.1, 9.9, 10.0]$ from chain 1 and $[10.0, 9.9, 10.1, 10.0]$ from chain 2 — no cluster split at all. Using the same $\hat{R} = \sqrt{\left(\frac{n-1}{n}W + \frac{B}{n}\right)/W}$ formula, is this second parameter's $\hat{R}$ closer to 1 or to 2.36? What does a report that quotes only "$\hat{R} = 2.36$" for "the fit" actually tell you about this second parameter, and what would you need to check before trusting it?

Solution

The two chains for the second parameter have essentially the same mean (≈ 10.0), and only chain-internal noise separates them, so the between-chain variance $B \approx 0$ while the within-chain variance W is small but nonzero — driving $\hat{R}$ to very nearly 1, nowhere near 2.36. A report that quotes a single "$\hat{R} = 2.36$ for the fit" tells you *nothing at all* about this second parameter — $\hat{R}$ is computed per parameter, and the headline number is, by the usual convention, the worst offender across all of them, not an average. Before trusting the second parameter you would need its *own*, disaggregated $\hat{R}$ — exactly the check CAMPAIGN.md ran to clear γ of the source-centre pair's pathology.

Exercise 26.3 — the 74 nats, as a probability ratio

The MAP is a saddle reports a 74-nat gap between $\log p$ at $\gamma = 1.27$ and $\log p$ at $\gamma = 1.10$. Ch. 8 establishes that log-probabilities in nats convert to a probability *ratio* by exponentiating. Compute e^{74} (an order of magnitude is enough), and say in one sentence what it would mean for an optimizer to report the *lower*-probability point as "the" answer when a point e^{74} times more probable sits nearby.

Solution

$e^{74} = 10^{74/\ln 10} \approx 10^{32.1}$ — on the order of 10^{32}, a ratio with no everyday analogy (larger than the number of atoms in a human body). Reporting $\gamma = 1.27$ as "the" MAP when $\gamma = 1.10$ scores $\log p$ higher by this much is not a rounding error or a matter of taste between two comparably-good fits — it is the optimizer stopping at a point the model itself considers *astronomically* less probable than one nearby, because the stopping rule (a small gradient) cannot see the difference.

27. Finding lenses: the survey, the nets, and the resolution wall

Every chapter in Parts IV and V started from a lens already on someone's desk: a cutout, a redshift pair, a PSF file. This chapter is about how it got there. The DESI Legacy Imaging Surveys photographed most of the extragalactic sky in three colors, and this repo's own finder lineage has scored tens of millions of those galaxies by machine, at a scale no human team could inspect by eye. Two questions decide everything that follows from a candidate list: how good does a picture have to be before a ring is even visible in it, and once a network has ranked 53.8 million galaxies by lens-likeness, what number do you actually read off that ranking to decide who gets looked at next? The first question has a closed-form answer, derivable from nothing but the survey's own point-spread function. The second is answered, expensively, by this repo's own deployment history — where a 105× parameter increase bought less than a thousandth of an AUC point, and a change to how five scores get combined, at *fixed* AUC, moved recovered candidates by 34 percentage points.

> **What you can skip**
>
> You already own AUC, ROC curves, true/false positive rate, threshold selection, why accuracy is the wrong metric under class imbalance, ResNets, EfficientNets, ensembling and stacking, and what "training-set leakage" means — standard ML practice, and this chapter never re-explains it. What is new: what a ground-based survey's pixel scale and seeing actually do to a candidate *together*, at the scale a real deployment operates at — and why "same architecture family, 100x the parameters" behaves nothing like it does on a typical vision benchmark.

The survey

The **DESI Legacy Imaging Surveys** photograph the sky in three optical bands — g, r, z (hence "grz") — combining exposures from DECam in the south and the 90Prime/Mosaic-3 cameras (BASS/MzLS) in the north, reduced by the Tractor pipeline into public per-object catalogs and coadded images, released in numbered Data Releases (DR7 through DR11 across this repo's own finder lineage). Ch. 9 already fixed the two numbers that decide everything in this chapter: a native pixel scale of $0.262''$/px and a g-band coadd point-spread function measured, not assumed, at FWHM $\approx 1.35''$ (`reproductions/cikota-2023/README.md:38`).

What every one of this repo's finders actually consumes is a small array built from that imaging. `tools/desi-dr11-cookbook/dr11_collect.py` writes one $(101, 101, 3)$ cutout per surviving object — g, r, z stacked as channels, raw nanomaggies, no normalization applied at storage time — and `huang-2020`'s finder pipeline fetches the identical shape directly from the public cutout service (`reproductions/huang-2020/papers/main.tex:138`). At the survey's own pixel scale that is a

$$101 \times 0.262'' = 26.462''$$

field of view on a side — every finder in this book's lineage sees the same patch of sky, at the same resolution, every time. "Surviving" means five filters, applied in this exact order in the cookbook's own reimplementation of the release pipeline: primary detection, positive flux in all three bands, at least three exposures per band, z-band magnitude under 20, and a Tractor light-profile type in $\{\text{SER}, \text{EXP}, \text{DEV}, \text{REX}\}$ (`tools/desi-dr11-cookbook/README.md:52-57`). (Even the array layout has a real gotcha: each stored plane is transposed relative to the sky image — a quirk of the original pipeline the cookbook reproduces bit-for-bit rather than silently

"fixing," precisely so newly generated bricks stay drop-in compatible with the existing dataset; `tools/desi-dr11-cookbook/README.md:41-50`.)

Applied to the full DR11-south footprint, those five filters leave a parent sample of

$$53{,}809{,}040$$

galaxies (`reproductions/dr11-campaign/papers/main.tex:93`) — the same published cuts Huang 2020/2021 and Inchausti 2025 used at smaller data releases, now run against the deepest DECam reduction to date. That number, rounded to 53.8M, is the denominator behind every false-positive-rate arithmetic later in this chapter, and it is the reason a survey-scale finder is not a bigger version of a Kaggle classifier: the operating point has to be chosen against tens of millions of negatives, not a held-out test split.

Deriving the wall

> **You already know this**
>
> The test below — does a critical point sit at a maximum or a minimum? — is exactly Ch. 5's definiteness test, run on a single scalar function of one variable instead of a Hessian. A second derivative changing sign is a saddle test in miniature, and it is what "resolved" and "unresolved" formally mean.

Ch. 11 established that no image shows you the sky; it shows you the sky convolved with a point-spread function, and for ground-based imaging that function's width is the **seeing**. Ch. 9 flagged, without proof, that DESI's 1.35″ seeing disk is "comparable to or larger than a typical Einstein radius." Here is the derivation that makes that precise.

Model the two brightest images of a lensed arc — two points on opposite sides of an Einstein ring, say — as two equal point sources separated by a distance d, each blurred by the same seeing disk. Approximate that disk as a Gaussian of standard deviation σ, $G_\sigma(x) = \exp(-x^2/2\sigma^2)$, and look at the combined brightness profile along the line joining them:

$$f(x) = G_\sigma(x - a) + G_\sigma(x + a), \qquad a = d/2.$$

By symmetry, $f'(0) = 0$ always — there is always a critical point exactly halfway between the two sources. What that critical point *is* — a single merged peak, or a dip between two separate humps — is decided by the sign of $f''(0)$, precisely the one-dimensional case of Ch. 5's test. Differentiating twice and evaluating at $x = 0$:

$$f''(0) = \frac{2}{\sigma^2}\left(\frac{a^2}{\sigma^2} - 1\right) G_\sigma(a).$$

Since $G_\sigma(a) > 0$ always, the sign of $f''(0)$ is the sign of $a^2/\sigma^2 - 1$. For $a < \sigma$, $f''(0) < 0$: a single local maximum sits at the midpoint, and the two sources have merged into one blob — the convolution has done to two nearby point sources exactly what a low-pass filter does to two nearby high-frequency features. For $a > \sigma$, $f''(0) > 0$: the midpoint is a local *minimum*, a real dip separates two distinguishable humps, and the pair is resolved. The boundary is

$$d_{\min} = 2\sigma.$$

This is the Sparrow resolution limit, derived rather than looked up: two point sources under a Gaussian PSF stop being resolvable once their separation drops below twice the PSF's own standard deviation.

Converting the survey's seeing from a FWHM to a σ needs one more line — $\mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma \approx 2.3548\,\sigma$, derived from scratch in Exercise 27.1 — which gives, at DESI's own measured seeing,

$$\sigma = \frac{1.35''}{2.3548} \approx 0.573''$$

and therefore $d_{\min} \approx 1.147''$. Two opposite images of a ring of Einstein radius θ_{E} sit a full diameter apart, $d = 2\theta_{\mathrm{E}}$, so the wall translates directly into a floor on θ_{E} itself:

$$\theta_{\mathrm{E,min}} = \frac{d_{\min}}{2} = \sigma \approx 0.573''.$$

The factor of two in "diameter" and the factor of two in "$d_{\min} = 2\sigma$" cancel exactly, so the resolution floor on an Einstein radius is just the PSF's own σ — a clean result, not a coincidence of this particular algebra.

Now check it against numbers this book has already computed. A fiducial massive elliptical — $\sigma_v = 250$ km/s at $z_l = 0.5$, $z_s = 2.0$, the same system Ch. 19 used — has $\theta_{\mathrm{E}} = 1.145''$, only

$$\frac{1.145''}{0.573''} \approx 2.0$$

times the floor. A *typical* strong lens clears this wall by barely a factor of two — not a comfortable margin for a survey whose every pixel already carries correlated noise (Ch. 11) and a bright foreground galaxy's own light.

The repo's own real system pressure-tests the idealization. `cikota-2023` fits DESI-253.2534+26.8843 — the Einstein cross from Ch. 19 — on this exact survey's blended 1.35″-seeing pixels and recovers $\theta_{\mathrm{E}} = 2.103''$, comfortably above the wall — a ratio of

$$\frac{2 \times 2.103''}{1.147''} \approx 3.67$$

— against the paper's own sharper, 0.6″-seeing MUSE imaging, which gives $\theta_{\mathrm{E}} = 2.520''$ (`reproductions/cikota-2023/papers/main.tex:41-42`). By the idealized two-point criterion, this system should be easily resolved. And yet: "All four images of the cross are visible, though partially blended at 1.35″ seeing" (`reproductions/cikota-2023/papers/main.tex:126-127`). The idealization is optimistic in two ways real data adds back: four images of a quad are not evenly spaced around one circle of diameter $2\theta_{\mathrm{E}}$ — some pairs sit much closer than that — and they are not equal-brightness points on a dark background but faint arcs sitting next to a much brighter foreground galaxy's own Sérsic wings (Ch. 10). The derived wall is a *lower bound* on what real detection needs, not the whole story. Seeing does not only decide whether a ring is visible, either: re-fitting the identical Legacy pixels with the paper's sharper 0.6″ PSF instead of

the true 1.35″ one moves the recovered radius from 2.103″ to 2.276″ , closing only about half the gap to 2.520″ (`reproductions/cikota-2023/README.md:39-41`) — the same seeing that hides a ring also biases how big you measure it once you find it.

The wall's location is confirmed, empirically and independently, from elsewhere in this repo. Of 17 DESI grade-C candidates the Euclid Q1 Discovery Engine happened to re-observe at 0.1″ resolution, 6 jump all the way to grade A (`reproductions/lensjudge/papers/main.tex:571-575`) — not because the object changed, but because the wall moved: rerunning this exact derivation at Euclid's sharper FWHM = 0.1″ drops the floor to $\theta_{\mathrm{E,min}} \approx 0.042''$ (worked in Exercise 27.2) — below essentially every strong lens this book quotes a number for. "C" was never a verdict about the source. It was a statement about the pixels, and Ch. 28 shows it is the *same* statement that makes a human grader's "C" unreliable too.

The finders

> **You already know this**
>
> A network that cannot improve past a resolution-set information ceiling, no matter how many parameters you give it, is the textbook signature of a **data-limited** regime rather than a compute-limited one — the same diagnosis that tells you when a bigger model, not more or better data, is the wrong lever to pull.

This repo's own finder lineage ran the parameter-count experiment for real, across three papers. Huang 2020 re-implemented Lanusse 2018's ResNet-46 (`L18`) at 3,508,833 parameters . Huang 2021's controlled comparison — same cutouts, positives, negatives, seed and split, architecture the *only* variable — trained a "shielded" ResNet at 59,905 parameters , a

$$\frac{3{,}508{,}833}{59{,}905} \approx 58.6\times$$

reduction, and it did not lose accuracy: validation AUC actually rose slightly, from 0.9983 to 0.9989 (`reproductions/huang-2021/README.md:28-37`), a gap of 0.0006 in the *smaller* net's favor. Inchausti 2025 pushed the same controlled comparison an order of magnitude further: a 194,501-parameter re-tuning of that shielded ResNet against an EfficientNetV2-S backbone at 20,543,145 parameters ,

$$\frac{20{,}543{,}145}{194{,}501} \approx 105.6\times$$

more parameters, for the paper's own reported validation-AUC gain of $0.9987 - 0.9984 = 0.0003$ (`reproductions/inchausti-2025/papers/main.tex:170-179`); stacking both models through a 300-node meta-learner edges the shielded ResNet alone by 0.0005.

Model	Params	Val AUC	vs. the row above
Lanusse-2018 ResNet-46 (Huang 2020)	3,508,833	0.9983	—
Shielded ResNet (Huang 2021)	59,905	0.9989	58.6× fewer, +0.0006 AUC

Model	Params	Val AUC	vs. the row above
Shielded ResNet, re-tuned (Inchausti 2025)	194,501	0.9984	baseline for the next row
EfficientNetV2-S (Inchausti 2025)	20,543,145	0.9987	105.6× more, +0.0003 AUC
Stacking meta-learner (Inchausti 2025)	1,201	0.9989	+0.0005 AUC over 194K ResNet

Read across the whole lineage, and ClaudeNet's own premise document states the finding flatly: "at the deployment operating point, architecture is not the bottleneck. A 194 K-param shielded ResNet a 20.5 M-param EfficientNetV2-S within ±0.003 AUC" (`reproductions/claudenet/README.md:11-13`). Every gap traced above — 0.0003, 0.0005, 0.0006 — sits comfortably inside that bound, by a factor of five to ten. Every one of these nets trains on the same leakage-inflated positives, evaluated on the same easy validation split, drawn from a survey whose seeing sets a hard information ceiling on what a 101 × 101 cutout can say about a ring that thin (the previous section, not a metaphor for it). Once the *pixels* cannot distinguish "faint blue arc" from "faint blue smudge," no amount of extra network capacity recovers information the input never carried — the data-limited regime this chapter's tip box named.

The operating point

At 53,809,040 galaxies, even an outstanding AUC produces an unusable candidate list unless you also fix *where* on the ROC curve you deploy. A finder run at a 1% false-positive rate flags

$$53{,}809{,}040 \times 0.01 \approx 538{,}090$$

galaxies; even at 0.1% FPR, still 53,809 . No visual-inspection pipeline, human or agentic, clears either list in a research career, so a real deployment always operates far out on the low-FPR tail — exactly where AUC, an integral over the *entire* curve, is least informative about the one point that matters.

The repo's own DR11-south sweep makes this concrete without touching architecture at all. Five finder members score the same 53.8M-galaxy parent sample with the same trained weights; ranking their calibrated mean against 2M random DR11 galaxies gives a threshold-free AUC of 0.9955 (`reproductions/dr11-campaign/papers/main.tex:119-121`) — the members separate real lenses from random galaxies almost perfectly. Yet at a fixed candidate budget of 95,104 survivors — about 95,104/53,809,040 ≈ 0.18% of the parent sample — the *original* per-member union selector recovers only 54% of held-out grade-A lenses, while swapping to a calibrated-*mean* combiner, at the *identical* budget, recovers 75% ; widening that same mean-ranked budget to 150,000 (≈ 0.28% FPR) reaches 80% , and 88% on the held-out subset (`reproductions/dr11-campaign/papers/main.tex:124-138`). Same trained weights. Same scores. Same AUC. A 34-percentage-point recall swing from nothing but how five numbers get combined into one rank.

A second, independent confirmation comes from Inchausti 2025's own negative-sampling ablation. The meta-learner's AUC barely moves across two retraining stages — 0.9876 then 0.9919 — while recovery of the two published catalogs at a matched 1% FPR moves from 11.8%/ 19.1% to 83.6%/ 88.5% to 90.8%/ 96.8% (Storfer/Inchausti catalogs respectively; `reproductions/inchausti-2025/README.md:150-163`). What moved was not the network; it was how many, and how realistic, the training negatives were.

Both stories teach the same lesson from opposite ends. Architecture buys thousandths of an AUC point (the previous section). The **operating point** — the combiner, the negative-sample composition, the threshold you actually deploy at — buys tens of percentage points of recall, at *fixed* AUC. At 53.8 million galaxies, the operating point is not one metric among several worth reporting. It is the only one a real deployment decision can be made from.

Connect to the repo

- `tools/desi-dr11-cookbook/dr11_collect.py` and `tools/desi-dr11-cookbook/README.md:20-64` — the DR11 → HDF5 cutout pipeline: the $(101, 101, 3)$ grz array, the five-filter object selection, and the transpose storage convention. (github)
- `reproductions/huang-2020/03_download_decals_cutouts.py` and `reproductions/huang-2020/papers/` — the 101×101 grz cutout / $0.262''$ pixel-scale fact this whole lineage inherits. (github)
- `reproductions/huang-2021/01b_shielded_resnet.py` and `reproductions/huang-2021/README.md:9-45` — the shielding architecture and its controlled, same-data AUC comparison against L18. (github)
- `reproductions/inchausti-2025/08_compare_models.py` and `reproductions/inchausti-2025/papers/m` — the three-model params/AUC table (`tab:auc`) this chapter's finder lineage is built from. (github)
- `reproductions/inchausti-2025/22_fpr_operating_point.py` and `reproductions/inchausti-2025/REA` — the Stage-C/D matched-false-positive-rate recovery arithmetic. (github)
- `reproductions/claudenet/README.md:9-21` — "architecture is not the bottleneck," the premise this chapter's finder table confirms in detail. (github)
- `reproductions/dr11-campaign/papers/main.tex:90-141` — the DR11-south sweep and the union-vs-mean-combiner recall-recoverability story. (github)
- `reproductions/cikota-2023/README.md:36-43` and `reproductions/cikota-2023/papers/main.tex:41-` — the PSF/seeing ablation on a real, four-image system. (github)
- `reproductions/lensjudge/papers/main.tex:562-596` — the DESI-vs-Euclid grade-C flip, the empirical confirmation of this chapter's derived wall. (github)

Exercises

Exercise 27.1 — the FWHM-to-σ conversion, from scratch

A Gaussian $G_\sigma(x) = \exp(-x^2/2\sigma^2)$ has full width at half maximum FWHM defined by $G_\sigma(\mathrm{FWHM}/2) = 1/2$. Solve for FWHM in terms of σ, and evaluate the constant FWHM/σ to four decimal places.

Solution

$$\exp\left(-\frac{(\mathrm{FWHM}/2)^2}{2\sigma^2}\right) = \frac{1}{2} \implies \frac{(\mathrm{FWHM}/2)^2}{2\sigma^2} = \ln 2 \implies \mathrm{FWHM} = 2\sigma\sqrt{2\ln 2}.$$

Numerically, $2\sqrt{2\ln 2} \approx 2.3548$, the constant used throughout Deriving the wall to convert the survey's quoted seeing (a FWHM, the astronomer's usual unit) into the σ the second-derivative test needs.

Exercise 27.2 — rerun the wall at Euclid resolution

Ch. 28 reports that Euclid Q1 imaging resolves lenses DESI could not. Repeat this chapter's derivation with FWHM $= 0.1''$ (Euclid VIS) instead of DESI's $1.35''$, and find the resolution floor $\theta_{\mathrm{E,min}}$. How does it compare to the $\theta_{\mathrm{E}} \gtrsim 1''$ scale of every lens this book has quoted a number for?

Solution

$$\sigma_{\mathrm{Euclid}} = \frac{0.1''}{2.3548} \approx 0.0425'', \qquad \theta_{\mathrm{E,min}} = \sigma_{\mathrm{Euclid}} \approx 0.0425''$$

— 13.5× sharper than DESI's own $0.573''$ floor (the ratio of the two seeing FWHMs, since the wall is linear in σ), and about 27× *smaller* than the $1.145''$ fiducial Einstein radius this chapter derived the DESI wall against. At Euclid resolution the idealized wall essentially vanishes: every real lens sits far above it, which is exactly why 6 of 17 DESI grade-C candidates jumped to grade A once regraded on Euclid imaging — the object never changed, only which side of the wall its pixels sat on.

Exercise 27.3 — candidate counts at three operating points

Using the DR11-south parent sample of 53,809,040 galaxies, compute the number of flagged candidates at 1%, 0.1%, and 0.01% false positive rate. At roughly how many candidates per day would a team need to grade to clear the 0.01%-FPR list within a single calendar year?

Solution

$$53{,}809{,}040 \times \{0.01,\ 0.001,\ 0.0001\} \approx \{538{,}090,\ 53{,}809,\ 5{,}381\}$$

Even the smallest of the three, 5,381 candidates, needs about $5{,}381/365 \approx 15$ gradings a day, every day, for a year, to clear at the lowest operating point in this chapter — and that operating point is already 100× more conservative than the 1% FPR a naive threshold choice might reach for. This is the arithmetic reason The operating point insists that the threshold, not the AUC, is the number a deployment decision is made from.

Exercise 27.4 — verify the parameter ratios, and explain the flat AUC

Using $\text{params}_{\text{L18}} = 3{,}508{,}833$, $\text{params}_{\text{shielded,60K}} = 59{,}905$, $\text{params}_{\text{shielded,194K}} = 194{,}501$, and $\text{params}_{\text{EffNetV2-S}} = 20{,}543{,}145$, verify the 58.6× and 105.6× reduction/increase ratios quoted in The finders. Then argue, from the resolution wall derived earlier in this chapter, why increasing parameter count by two orders of magnitude moved validation AUC by less than one part in a thousand in both directions.

Solution

$$\frac{3{,}508{,}833}{59{,}905} \approx 58.573, \qquad \frac{20{,}543{,}145}{194{,}501} \approx 105.620$$

matching the table in The finders to four significant figures. The flat AUC is the data-limited regime made concrete: Deriving the wall showed that DESI's own seeing sets a hard floor on how much a 101×101 cutout can say about a ring near that floor — a typical lens clears it by only about 2×. Once the *input* cannot distinguish a genuine tangential arc from a blended smudge of comparable size, no amount of additional network capacity can recover a distinction the pixels never encoded. Extra parameters can only fit patterns present in the training data; they cannot manufacture angular resolution the camera and the atmosphere never delivered.

28. The label is the problem

You are reading this chapter second because it needs none of the calculus Part I is about to build, and it carries the finding this guide treats as the most consequential one for anyone who ships classifiers for a living. Chapter 1 split this repository's work into two disciplines — discovery, which decides whether a candidate is a lens at all, and modeling, which measures one once confirmed — and previewed discovery's central number: the human grade attached to a candidate predicts later truth at AUC 0.577, statistically indistinguishable from a coin flip. This chapter earns that number rather than quoting it, using only per-class precision and an ordinal agreement statistic you can build from tools you already own, applied to every follow-up outcome this repository's discovery lineage has published and to three different machine systems tried against the same label.

> **What you can skip**
>
> You already own precision, recall, AUC, and the confusion matrix — none of that gets rebuilt here. "Purity," as this program's own reports use the word, turns out to be exactly per-class precision, and the truth-referenced AUC comparisons are exactly the paired, non-inferiority-style comparison you would run if two model checkpoints differed only in a training-data source. What is genuinely new is quadratic-weighted kappa (QWK) — the standard way to score agreement on an *ordinal* label like a 1–4 lens grade — and this chapter builds it from the two-line "agreement beyond chance" idea behind Cohen's kappa, up to a number that matches this repository's own published statistic, entirely from data already sitting in its tables.

The flat line

Every discovery pipeline in this repository ends the same way: a candidate gets a letter grade, usually A through D, meant to encode how confident the grader — human or machine — is that the object is a genuine lens. A grade is only useful if it is *informative*: among candidates that later got independent follow-up (deeper imaging, a second redshift, spectroscopy), a higher grade should correspond to a higher confirmed fraction.

> **You already know this**
>
> That confirmed fraction has a name you already use. Restrict a confusion matrix to the candidates predicted grade g, and ask what fraction of them are true positives (confirmed) rather than false positives (refuted) — that is precision, computed separately per predicted class. This program's reports call it "purity," but
>
> $$\text{purity}(g) \;=\; \frac{\#\{\text{confirmed} \mid \text{grade} = g\}}{\#\{\text{confirmed} \mid \text{grade} = g\} + \#\{\text{refuted} \mid \text{grade} = g\}}$$
>
> is precision restricted to grade g, nothing more.

If the grade carried real confidence information, purity should fall as the grade drops — the way a classifier's precision falls as you relax its decision threshold. Every published per-target follow-up outcome this program could find — 551 targets across seven campaigns, cross-matched against 4,354 uniquely graded candidates — gives 162 candidates with follow-up, 130 with a decided (confirmed-or-refuted) outcome (`reproductions/lensjudge/papers/human_baseline.tex:184`). Split by grade:

$$\text{purity}(A) = \frac{79}{83} = 0.952, \qquad \text{purity}(B) = \frac{23}{25} = 0.920, \qquad \text{purity}(C) = \frac{20}{22} = 0.909.$$

The spread from the best grade to the worst is $0.952 - 0.909 = 0.043$ — four percentage points, well inside each grade's own sampling uncertainty on a few dozen candidates. Grade A is not even flawless: four of the 83 decided grade-A candidates were spectroscopically refuted by VLT/MUSE follow-up (`reproductions/lensjudge/papers/human_baseline.tex:222`), every one of them the same failure mode — an arc-shaped feature at the *wrong* redshift: a spiral arm at the lens galaxy's own redshift, a tidal tail between two merging galaxies, a blue arc that turns out to sit at the deflector's redshift rather than behind it, and one "arc" that is a foreground object entirely. A grade assigned from a 1.3″-seeing cutout can judge shape. It cannot see a redshift, and redshift is what actually decides truth.

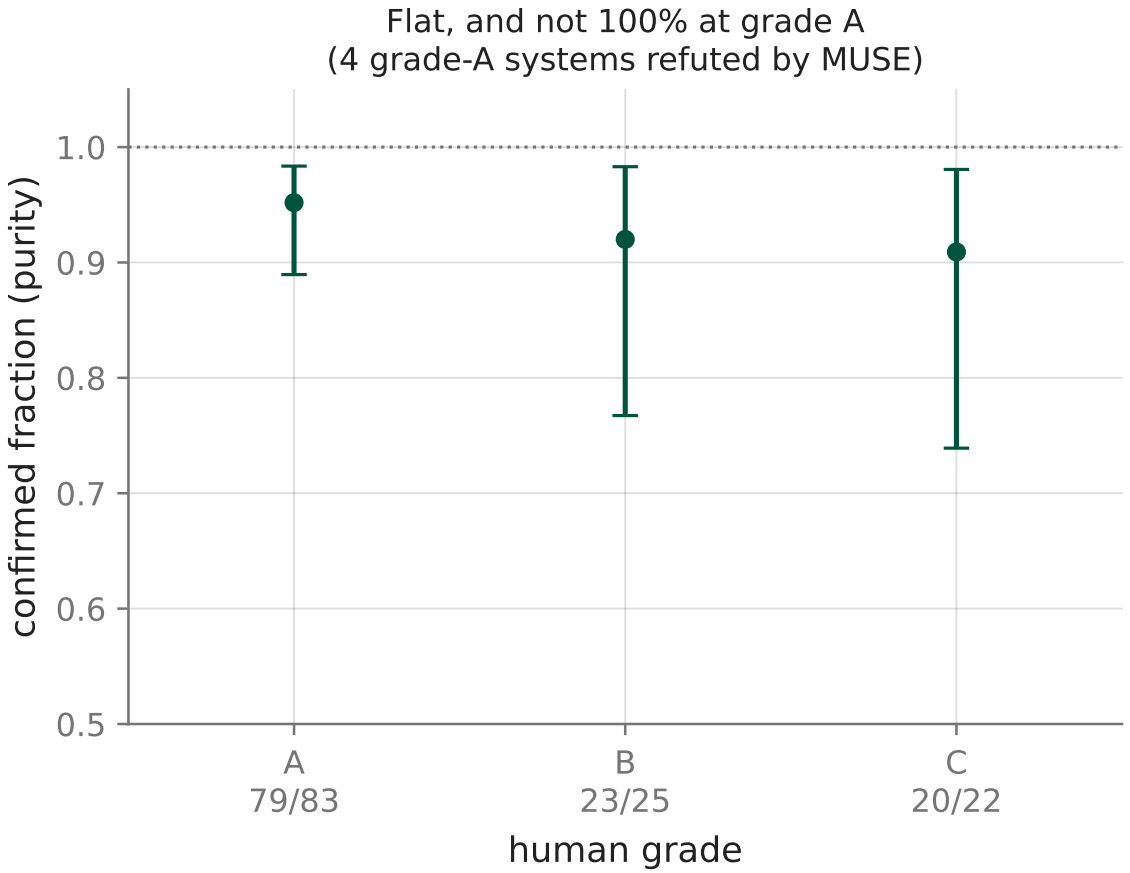

Figure 11: Confirmation purity among decided follow-ups, by consensus grade, with 95% Jeffreys intervals. The dotted line marks perfect purity. Grade A sits closest to it but is not on it — 4 of 83 decided grade-A candidates were spectroscopically refuted — and all three grades' intervals overlap heavily. If the letter grade encoded confidence the way a classifier's score does, this line would slope down sharply from A to C. It is flat.

Read the other direction and the same fact says something sharper: grade C is not mostly junk. Twenty of its 22 decided candidates are real lenses — purity indistinguishable from grade A's. Whatever "C" is measuring, it is not "probably not a lens."

The reliability ceiling

Purity asks how well the grade predicts truth, conditional on a candidate having been followed up at all. A prior question is sharper: how reproducible is the grade *itself*? Show the same cutout to two trained graders — would they even agree with each other?

Cohen's kappa answers this for a categorical label: let p_o be the observed fraction of exact agreement and p_e the fraction two independent graders would agree on by chance alone, given each grader's own marginal distribution of scores. Agreement beyond chance is $(p_o - p_e)/(1 - p_e)$. A lens grade is not just categorical, though — it is *ordinal* (1 through 4, or A through D), and a grader who says "3" when the other says "4" is closer to right than one who says "1". Quadratic-weighted kappa (QWK) generalizes Cohen's statistic to charge for *how far off* a disagreement is, not merely whether it happened. Weight each possible pair of scores (i, j) by squared ordinal distance, $w_{ij} = (i - j)^2/(K - 1)^2$ for K score levels, and

$$\text{QWK} \;=\; 1 - \frac{\sum_{i,j} w_{ij}\, O_{ij}}{\sum_{i,j} w_{ij}\, E_{ij}},$$

where O is the observed joint distribution of the two graders' scores and E is the joint distribution you would see if the two graders' scores were independent (the outer product of the marginals).

> **You already know this**
>
> That formula has a shape you have seen before. $R^2 = 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$ is a squared-error ratio against a null model's error; QWK is the identical ratio, built from a joint score distribution instead of a residual sum of squares, with "the null model" being *statistical independence between the two graders* rather than "predict the mean." It is exactly why leaderboards for ordinal-label problems (Kaggle's essay- and diagnosis-scoring competitions among them) use QWK as their metric: it behaves like R^2 for a discrete, ordered target.

Worked example, built from the repository's own published numbers. `huang2021desi` (Paper II) had two graders independently score every candidate 1–4; the public catalog keeps only the pair's *average* and *absolute difference*, but that is enough to recover the unordered pair of integer scores for 1,310 candidates. The full table of pair counts is published as a flat list (`reproductions/lensjudge/parity/FINDINGS.md:30`):

pair	count	pair	count	pair	count	pair	count
$\{2,2\}$	461	$\{3,3\}$	165	$\{4,4\}$	100	$\{1,3\}$	48
$\{2,3\}$	387	$\{3,4\}$	115	$\{2,4\}$	34		

These sum to 1,310 candidates . Grader identity was never published, so an unordered pair $\{a, b\}$ carries no information about which grader gave which score; the standard fix is to *symmetrize* — let each pair count once as (a, b) and once as (b, a) — which is also why this statistic equals Krippendorff's interval α, the standard reliability measure when raters are interchangeable. Build the resulting 4×4 joint table, apply the quadratic weights above, and take the ratio:

$$\text{QWK} = 0.420$$

built from nothing but the seven counts in the table above — and it matches this program's own reported statistic to three decimal places. The same seven counts, grouped by how far apart the two scores are, also reproduce the raw agreement breakdown directly: exact agreement on $726/1310 = 0.554$ of candidates, off by one on 0.383 , off by two — one grader ready to reject, the other confident enough to publish — on 0.063 .

Two trained graders, same rubric, same screen, same cutout, agree with each other on the *exact* score barely more often than a coin flip, and QWK = 0.42 is "moderate" on the standard Landis–Koch scale — worse than the 73% intra-rater repeat consistency an independent study measured for a single expert shown the same image twice (`reproductions/lensjudge/papers/human_baseline.tex:125`): these two experts disagree with each other more than one of them disagrees with their own past self. There is a second, mechanically inflated anchor worth naming so you do not mistake it for the real ceiling: an individual grader's score matches the *published consensus* (the two-grader average) at QWK = 0.776 — but a grader who is literally half the number being compared against is close to guaranteed a high score. The honest bar for "does a system reproduce this team's grade" is the mutual 0.42, not the self-referential 0.776.

This is a reliability ceiling in the same sense a noisy training label imposes one on supervised learning: no model, however well built, can be scored as "agreeing with the label" above what the label agrees with itself. The ceiling is a property of the data-generating process, not of any predictor's capacity.

Nothing gets past the label

This repository tried three structurally different machine systems against exactly this task, and the pattern that emerges only sharpens the ceiling argument. Distinguish two separate questions the way this program's own methodology does: **E1**, does a machine's grade *agree with the human label* (measured by QWK against the published consensus)? And **E2**, does a machine's score *agree with truth* (measured by AUC against confirmed-vs-refuted follow-up, paired against the human grade)?

On E1, nothing gets close. A 27-billion-parameter vision–language model, fine-tuned directly on the human grade catalog — not distilled from another model, trained on the actual letter grades a human team assigned — emits grades that score QWK = 0.044 against the published consensus, $n = 162$. Prompt Claude Sonnet 5 with the graders' own rubric and the same multi-band imaging context instead of fine-tuning anything, and its frozen-gate grades score QWK = 0.025 . Neither a purpose-trained model nor a frontier general model reproduces the label a fraction as well as two trained humans reproduce each other.

A related, coarser contrast tells the same story from the vetting side: can a machine at least separate human-graded A/B lenses from candidates humans explicitly *rejected* (grade D)? Three unrelated architectures were run against the identical frozen 259-row gate. A trained probe over classical, engineered image features scores AUC = 0.425 — below chance. This repository's own published CNN ensemble scores 0.646 . The fine-tuned 27B student, despite direct supervision on the human catalog, scores 0.644 — statistically the same number as the published CNN, reached by a completely different architecture and training recipe. Three tools built three different ways converging on the same ceiling is itself evidence the ceiling belongs to the pixels, not to any one model's limitations.

The twist that makes this sharper rather than softer: the same fine-tuned student that cannot

reproduce the letter grade discriminates *truth* on the followed-up pool at AUC = 0.685 [0.538, 0.819] — at least as well as the human grade's own 0.577. Paired against the human grade directly, the difference is +0.108 [−0.028, +0.241] — formally *non-inferior* to the human grade at this program's pre-registered margin, though the interval still touches zero, so *superiority* is not established either. Read that pair of results together: a model can be bad at imitating the label and simultaneously a legitimate — on this point estimate, slightly ahead — detector of the thing the label was only ever supposed to be a proxy for. The grade and the truth are not the same target, and E1 tells you nothing at all about E2.

The same wall

Chapter 27 derives, from the survey's own pixel scale, why a fixed resolution makes an Einstein ring's shape disappear into the point-spread function past a certain radius — a limit no architecture trains past, because the discriminating signal has been convolved away before any classifier, however large, ever sees a pixel. This chapter's ceiling is the identical physical fact, measured from the label side instead of the classifier side.

The cleanest direct evidence: 17 candidates that DESI's own pipeline graded C were independently rediscovered inside the Euclid Q1 survey's footprint and re-scored there — not by DESI's 1.3″-seeing grz imaging, but by a ∼ 10-expert panel working from Euclid's 0.1″ VIS imaging (`reproductions/lensjudge/papers/main.tex:571`). Of those 17 DESI grade-C candidates, 53% were re-graded A or B once the pixels sharpened, with 6 of the 17 jumping all the way to grade A — about $6/17 \approx 0.353$ of the pool. Same object, same rubric, same sky. The grade moved because the pixels changed, not because the lens changed.

That is the same wall Chapter 27 derives from first principles, seen from a different instrument: a human grader staring at an under-resolved cutout faces the identical information deficit a CNN's receptive field does. It is why grade C's purity (0.909) sits statistically on top of grade A's (0.952) — "C" was never a verdict about the object, it was a statement about the pixel budget available to render one. And it is why an engineered probe, a published CNN, and a fine-tuned 27B model — three genuinely different tools — all plateau in the same narrow band regardless of how they are built: a classifier's ceiling is set by what information survives in its inputs, not by its parameter count.

> **You already know this**
>
> This is the standard Bayes-error argument from supervised learning, applied to two different measuring instruments at once. If a feature map destroys the information a decision needs, no downstream model recovers it — that floor is a property of the representation, not of capacity. Chapter 27 computes that floor from the optics (a PSF convolution erasing a ring's shape); this chapter measures the *same* floor from the label side, by showing that two independent panels of experts, looking at the same lossy representation, cannot agree with each other about what they see there either.

What to do instead

Two different evaluation targets have been in play throughout this chapter, and they behave nothing alike. Agreement-with-label (E1, QWK) has a hard ceiling — 0.42 here — set entirely by how reproducible the label is between the humans who produced it; no predictor, however capable, can be fairly scored above that ceiling, and training toward it optimizes a system to imitate human

noise. Agreement-with-truth (E2, AUC) has no such ceiling: a model's score can legitimately match or exceed the human grade's own truth-discrimination, exactly as the fine-tuned student's 0.685 did against the human grade's 0.577.

The practical rule this sets, usable on day one: never evaluate — or train — a vetting system by how well it reproduces a letter grade whose own originators only agree with each other at QWK = 0.42. That target caps your reported quality below what the underlying data can support and rewards fitting human disagreement rather than physical truth. Build a continuous score instead, calibrate it against held-out truth (confirmed versus refuted), choose an operating point the same way any deployed classifier's threshold gets chosen — against a validation ROC curve, not against a categorical label someone else assigned — and validate with a pre-registered, truth-referenced comparison against the human baseline rather than an agreement statistic. This is precisely this program's own stated conclusion: score-based vetting through a calibrated operating point is deployable at DESI resolution; letter grades stay a human product.

Generalize it past this repository. Handed any "gold label" that is itself a human rating, ask its own inter-rater reliability *before* touching a model. A QWK of 0.42 on a four-point scale is not a data-cleaning problem to fix with more annotators — the two experts on this particular team are not individually noisy, they simply do not agree with each other about the underlying concept at ground-based resolution. The honest response is not a better classifier for the label. It is changing what you evaluate against.

Connect to the repo

- `reproductions/lensjudge/papers/human_baseline.tex:116` (§ Inter-grader reliability) and `:184` (§ Grade-stratified confirmation rates) — the two tables this chapter's first two sections are built from, with every caveat about truncation and selection this chapter's numbers inherit.
- `reproductions/lensjudge/parity/FINDINGS.md:30` — the seven published score-pair counts the QWK worked example derives from scratch, and `:107` — the AUC(grade vs. truth) = 0.577 power check.
- `reproductions/lensjudge/parity/FINDINGS.md:190`–`262` — the Phase C gate scoreboard (rep probe, CNN, 27B student) and Phase D parity comparison behind "nothing gets past the label."
- `reproductions/lensjudge/papers/main.tex:571` — the Euclid re-grade evidence behind "the same wall."
- `site/guide_src/worked_examples.py`, `ch28_labels()` — every number in this chapter, including the from-scratch QWK derivation, runnable with `worked_examples.py --show ch28`.

Exercises

> **Exercise 28.1 — finish the agreement breakdown by hand**
>
> Using only the seven pair counts in the worked example's table (they sum to 1,310), group them by $|a - b|$ and compute the fraction that are exact matches, off by one, and off by two. Then check your three fractions sum to 1.

Solution

Exact ($a = b$): $461 + 165 + 100 = 726$, giving $726/1310 = 0.554$. Off by one: $387 + 115 = 502$, giving $502/1310 = 0.383$. Off by two: $34 + 48 = 82$, giving $82/1310 = 0.063$. Sum: $0.554 + 0.383 + 0.063 = 1.000$. Two graders looking at the same cutout land on the exact same integer barely more than half the time, and disagree by a full two grade-levels — one ready to reject, one ready to publish — about one time in sixteen.

Exercise 28.2 — purity is precision; does it fall the way a precision curve should?

A well-calibrated classifier's precision falls as you relax its threshold and admit lower-confidence predictions. Using purity$(A) = 0.952$, purity$(B) = 0.920$, purity$(C) = 0.909$, compute the drop from A to B and from B to C separately, then compare each to the 0.043 total spread from A to C. Does the shape look like a classifier whose grade tracks confidence, or something else?

Solution

A→B drops $0.952 - 0.920 = 0.032$; B→C drops $0.920 - 0.909 = 0.011$. Both drops are small fractions of a purity that stays above 0.9 throughout, and together they only account for the full 0.043 spread — there is no point where purity falls off a cliff the way it would if "C" meant "probably a false positive." Combined with grade A's own 4/83 refutation rate, the shape says the letter grade is not tracking confidence in the object; it is tracking something closer to constant, with grade-A's own imperfection as the tell.

Exercise 28.3 — E1 failing does not mean E2 fails

The fine-tuned 27B student scores QWK = 0.044 against the published human grade — essentially uncorrelated with the letter humans assigned. Its truth-referenced AUC is 0.685, against the human grade's own 0.577. Explain, in one or two sentences, why these two facts are not in tension, and why a deployment decision should be made on the second number rather than the first.

Solution

QWK measures agreement with a specific human artifact (the letter grade); AUC measures agreement with an independent, physically determined outcome (confirmed vs. refuted). A model can rank candidates by their true probability of being a lens in an order that has almost nothing to do with which of four discrete buckets a particular human team happened to sort them into — ordinal binning throws away exactly the fine-grained ranking information AUC can still see. Since the deployment question is "does this system find real lenses," not "does this system sound like our graders," E2 is the number to act on; a low E1 here is not evidence the system is bad, only evidence it is not imitating the humans.

Exercise 28.4 — reconciling 53% with 6 of 17

The text states that 53% of 17 DESI grade-C candidates re-graded to A or B at Euclid resolution, and separately that 6 of the 17 jumped all the way to grade A. Use these two facts to work out how many of the 17 moved specifically to grade B (not all the way to A).

Solution

53% of 17 is $0.53 \times 17 \approx 9.0$, so about 9 of the 17 candidates re-graded to A or B combined. Six of those nine went all the way to grade A, which leaves $9 - 6 = 3$ that moved only as far as grade B. The arithmetic matters here: a resolution fix does not uniformly promote everything to the top grade, it redistributes candidates along the same confidence scale the DESI grader was already trying, and failing, to encode from worse pixels.

29. What you know now, and where the frontier is

Twenty-eight chapters ago this book named its destination: a single galaxy's density-profile slope, $\gamma = 1.103 \pm 0.008$, and a warning that the report producing it quotes a "$\sim 17\sigma$" tension for that number against its own anchor — a claim that reconciles with none of the uncertainties the same report states two paragraphs later. You now own the calculus, the linear algebra, the cosmology, and the lensing well enough to check that claim yourself, in one division, and you have read this campaign's own record closely enough to know it retracts its own numbers in public. This chapter adds no new mathematics. It closes the two ledgers this book has been keeping since Chapter 4, hands you a map from the campaign's 27-page report to the chapters that decode each of its sections, distills five questions worth asking at any lensing meeting — each one has already cost this repository a wrong number at least once — and says plainly where the frontier sits, including the one corner of it that belongs to a computer scientist rather than an astronomer. It ends with an honest list of what this book chose not to teach.

The two ledgers, closed

The Log-Det Ledger, final. Chapter 4 opened it with two entries that looked unrelated — the lensing magnification $\mu = 1/|\det A|$ and a normalizing flow's density correction $\log|\det J_T|$ — Chapter 8 added a third, stranger-looking one, and Chapter 23 closed it: one Gaussian-integral substitution shows all three are the same change-of-variables theorem, applied to a linear lensing map, a nonlinear flow, and a linear whitening map implicit in an evidence integral, respectively. This chapter does not repeat that derivation; it collects the closed state:

Log-Det Ledger — final state

#	Costume	Formula	Instance
1	Lensing magnification	$\mu = 1/\lvert\det A\rvert$	toy $\det A = 0.485$, $\mu = 2.0619$ (Ch. 4)
2	Normalizing-flow pullback density	$\log p_U(\mathbf{u}) = \log p_X(T(\mathbf{u})) + \log\lvert\det J_T(\mathbf{u})\rvert$	`cgl/flows.py:20`
3	Gaussian-evidence Occam term	$-\frac{1}{2}\log\det A$	production $\log\det A = 323.229$ nats (Ch. 22)

Derived in Ch. 23: "no fourth costume is left to find." Every $\log|\det(\cdot)|$ this book computes is one of these three, audited from a 2×2 toy to a 46-dimensional real fit.

The γ Ledger, final. Chapter 25 already closed this one, with the full chain and the full arithmetic; this collects it alongside Chapter 26's contribution into one entry. Per-basin evidence on the binned product flips 191.1 nats , from +162.2 favoring steep under the diagonal likelihood to −28.9 favoring low under the correlated one — H1 (bimodality-as-artifact) confirmed. But the restored low basin, $\gamma = 1.103 \pm 0.008$, sits 9–41σ from the diagonal-native anchor 1.433 ± 0.034 depending on which legitimate uncertainty convention you apply — never the 17σ the report's own abstract claims — and the correlated fine (1.816), binned-low (1.103), and native (2.353) slopes *bracket* the anchor rather than converging on it: H2 (cross-scale unification) rejected. None of this came from a clean

sampler run: the correlated-likelihood MAP is a saddle, minimum eigenvalue -14.85 , and the point a saddle-seeded optimizer settles on scores 74 nats *lower* than the true peak . Chapter 25 derives the number; Chapter 26 derives why extracting it required tempered SMC and not HMC at all — two chapters, one instrument, one reading.

> **Ledger — final entry**
>
> **What this book rules in or out about $\gamma = 1.103$:** a real, audited correction (H1) restores the low basin honestly, but does not (H2) unify this campaign's cross-scale measurements onto one consensus slope. Treat 1.103 as one defensible edge of a bracket, not a converged value for this galaxy's density slope. The residual disagreement is evidence of a real limitation in the source model and PSF treatment, not a bug in the noise model — and that limitation, not this number, is where the campaign's next move belongs (see The frontier).

The paper decoder

`main.tex` numbers every section with a `\label{sec:...}`, and the generated report page preserves those as literal HTML anchors — which is why this guide's own cross-link rule insists on linking `#sec:*`, never a slugified heading: reword any section title and the anchor still holds. Here is the map from that report, and from LensJudge's parity report, to this book.

Report section	What is there	This guide
§1 Introduction	The campaign in one paragraph: the money number, and its own $\sim 17\sigma$ claim	Ch. 1
§2 Data and Targets	The three drizzle products (fine/binned/native) and the mock generator	Ch. 11
§3 Methods I, whitening, marginalization	$C = D^{1/2} K D^{1/2}$, the convolutional whitener, the ridge-marginalized Occam term	Ch. 24, Ch. 22
§4 Methods II	The nine-sampler "posterior zoo" and its budget-matched protocol	Ch. 23 — partial, see What we skipped
§5 Mock Validation, two-stage PHMC	Calibration against known truth; the fix that lowered $\hat{R}$ from 2.11 to 1.003	Ch. 23
§6 Real-Data Verdict, the flip, the saddle	H1/H2/H3, the 191-nat flip, why HMC failed and SMC did not	Ch. 25, Ch. 26
§7 Sampler Benchmark, A100 phase	nautilus vs. PT-HMC vs. the GIGA-Lens baseline, at budget parity and until converged	The frontier, below — not a full chapter
§8 Recipe + Euclid Q1	End-to-end export to COOLEST; an honest 1/3-clean characterization on independent data	not covered — see What we skipped

Report section	What is there	This guide
§9 Discussion	"Necessary but not sufficient"; the two parked follow-on pillars	this chapter
§10 Reproducibility	Seven stack defects, the parity harness, the retraction culture	Ch. 22 — partial
LensJudge parity, Baseline, Power	Purity/QWK human baseline; AUC(grade vs. truth) = 0.577	Ch. 28

Two macros are worth knowing when you read the raw report, because they are the notation hazard this guide opened against: `main.tex` writes the money parameter as `\gammaEPL` (always a bare γ) and external shear as `\gext`, `\gextone`, `\gexttwo` — a bare γ in that document is never shear, by construction of the macro table, not by a convention this guide imposed on it.

Five questions to ask in any lensing meeting

Five questions, phrased so you could ask them of any lens-modeling result, not just this campaign's. Each one has already produced a wrong number in this repository before it was caught.

1. Are your pixel errors actually independent? Drizzling resamples an *HST* exposure onto a finer grid with a weighted, overlapping kernel — Chapter 11 derives why that correlates neighboring pixels — and a per-pixel diagonal Gaussian likelihood assumes the opposite. Get this wrong and you do not get a slightly-too-confident answer; you get a spurious *second mode*: the diagonal likelihood's +162.2-nat preference for the wrong basin was entirely a noise-covariance artifact, not a property of the galaxy.

2. Is the PSF kernel sampled at the pixel scale your code assumes? `lenstronomy`'s `subgrid_kernel` upsamples internally by the supersampling factor; feeding it an already-supersampled kernel applies that refinement twice, quietly broadening the effective PSF by 2×. The cost, on this same system, was χ^2_ν stuck at 3.4 until the convention was fixed, at which point it dropped to 1.05 with the *same* data and the *same* noise model.

3. Was that noise calibration performed on model-subtracted residuals? Calibrate a sky-noise sigma on the raw image near a bright lens galaxy and a large fraction of what you are calling "noise" is diffuse lens-light wing flux, not sky. This campaign's earlier report celebrated a suspiciously *too*-good $\chi^2_\nu = 0.451$; the honest recalibration on model-subtracted residuals gave 0.92 , comfortably under the group's < 1.1 bar but more than double the artifact's implied noise floor.

4. Is that point actually a mode, or just a place where the gradient vanished? Chapter 5's folklore — a vanishing gradient is not a verdict in non-convex optimization — is not folklore here. The `map_polish` stage that produced this campaign's saddle-consistent $\gamma = 1.27$ had, by construction, no way to notice that a direction existed along which the log-posterior kept *rising*: the Hessian's minimum eigenvalue there is -14.85 , and the true higher-density point sits 74 nats up at $\gamma = 1.10$.

5. If a convergence diagnostic is bad, what is it actually diagnosing? A bad $\hat{R}$ does not mean the parameter you care about failed to converge — it means *something* in the chain did. The $\hat{R} = 22.3$ that stalled this campaign's local-metric fixes was carried almost entirely by a Sérsic-versus-shapelet source-*centre* degeneracy, decoupled from γ (two clusters at -0.15 and $+0.09$

). Blaming the slope for a diagnostic a *different, physically uninteresting* parameter was carrying would have discarded a real result over a nuisance.

Two of these five are no longer questions you have to remember to ask — this campaign converted them into code. `cgl/guards.py:74-91` (`assert_psf_sampling`) refuses a mismatched PSF pixel scale at the door, citing this exact incident by name in its own docstring; `cgl/guards.py:129-142` (`assert_model_subtracted_sky`) refuses an uncalibrated noise artifact the same way, citing the 0.451 retraction by number. Questions 1, 4, and 5 are not yet guards. The frontier says why that gap is worth closing, and by whom.

The frontier

Two frontiers sit inside the science this campaign already ran, and one sits outside it entirely.

The scientific frontier is already named, by the campaign's own Discussion section. The bracket that will not close — fine-steep at 1.816 above the anchor, binned-low at 1.103 below it — is evidence of a real limitation, not a bug: the residual scale dependence "points to additional systematics, most plausibly the source model and PSF" (`reproductions/claude-giga-lens/papers/main.tex:966-970`). Two follow-on pillars are explicitly parked to address exactly this: a Gaussian-process or pixelated source model (the shapelet basis this campaign used is a fixed, low-order function family, and a flexible source can absorb structure a rigid one instead forces into the mass slope), and higher-order multipole mass structure, whose reported $\sim 1\%$-level effect on γ (`reproductions/claude-giga-lens/papers/main.tex:1358-1368`) is the same order as the correlated-noise correction this book just spent five chapters deriving. Neither pillar needs new mathematics beyond what you already have — a GP source is another linear-in-amplitude marginalization of exactly the shape Ch. 22 already profiles out, and a multipole term is a few more harmonics added to the deflection sum of Ch. 20.

The inference frontier is a genuinely open algorithms problem, not a physics one. The sampler benchmark's own verdict is two-sided: nautilus dominates efficiency wherever it applies — 2.6 to 307× the baseline's ESS per gradient — but is *disqualified* precisely on the ill-conditioned, marginalized regime this campaign's real posteriors occupy; parallel-tempered HMC is the most reliable until-converged method but is not the fastest one; and the flow-assisted recipe this campaign tried, GL-NT, reaches only 0.03 to 0.05× the baseline against its own pre-registered target of 3× . No single sampler in this benchmark is both fast and trustworthy on a 46-dimensional, condition-10^{14}, occasionally saddle-shaped real lens posterior. Building one — a sampler, or a routing policy over several, that gets nautilus's speed without inheriting its blindness to bad conditioning — is an algorithms and systems problem a CS-trained collaborator is better positioned to attack than most astronomers on this project.

The frontier outside the science is the research process itself, and it is the closest to home. This report's title page states, without qualification, that "all work reported here — code, experiments, analysis, and text — was performed by Claude Code (Anthropic), guided by Greg Benson" (`reproductions/claude-giga-lens/papers/main.tex:76-84`). One incident in `reproductions/claude-giga-lens/CAMPAIGN.md`, which Chapter 1 asked you to read end to end, stands out precisely because it is not an astrophysics failure: a correlated-noise SMC canary failed trying to allocate 120.4 GB with 300 particles , and the implementation agent handling that failure "DIED during a multi-day gap and never processed the failure / never fired its `SMC_PARTICLES=200` fallback" — the campaign stalled four days (`reproductions/claude-giga-lens/CAMPAIGN.md:168-169`) until a fresh agent converged it with

128 particles . That is a liveness bug — a long-running worker with no heartbeat, watchdog, or idempotent resume path — and diagnosing it requires no astrophysics whatsoever.

> **You already know this**
>
> "The worker died and nobody noticed for four days" is not a lensing problem; it is a liveness problem, and multi-agent-systems research has a whole vocabulary for it — heartbeats, watchdog timers, idempotent retries, supervisor trees. `cgl/guards.py`'s two runtime assertions (Questions 2 and 3, above) are the same instinct one level down: turn a postmortem into code that makes the same mistake impossible to repeat silently — exactly what this guide's own `lint_guide.py` and `worked_examples.py` do to its numbers (Chapter 1). You do not need to learn gravitational lensing to see where this campaign's process has a gap; you need exactly the training this book assumed you already had on page one.

The discovery half of the repository is already moving toward this same frontier: a 27B open model fine-tuned on the human grade catalog reaches AUC(truth) = 0.685 , statistically non-inferior to the human grader's own 0.577 (ΔAUC = +0.108 , 95% CI [−0.028, 0.241]), from a leakage-safe split, a frozen gate, and a pre-registered non-inferiority margin — the same discipline Chapter 28 asks of any classifier, applied to the wall it names. Ch. 28 has the rest of that argument; it belongs there, not here.

What we skipped

This book is honest about its own edges, the way this campaign is honest about its numbers.

- **The full sampler-benchmark matrix.** Chapter 23 gives you the ideas — HMC, tempered SMC, normalizing flows, mass-matrix preconditioning — but not the full multi-contender, multi-target horse race. `main.tex` §7 has the complete matrix.
- **The end-to-end recipe and Euclid Q1.** The campaign exports its final posteriors to the code-independent COOLEST standard and re-runs the whole pipeline on three independent Euclid systems, with an honest "one of three clean" result. None of that is in this book; `main.tex` §8 is.
- **The seven documented stack defects.** XLA compile livelocks, a `chex` pin that silently upgraded `jax`, a triton GEMM abort on tiny float32 dots — real, documented, genuinely useful engineering, but plumbing rather than mathematics. `main.tex` Table "tab:defects" in §10 has all seven.
- **Full general relativity.** This book derives the deflection angle from the Newtonian estimate and Eddington's factor of two (Ch. 16) and never writes down the Einstein field equations, a geodesic, or a metric beyond the flat FRW background of Ch. 14. Multi-plane lensing and full time-delay cosmography beyond the mass-sheet argument of Ch. 21 are likewise out of scope.
- **LensJudge's machine-vetting pipeline in depth.** Chapter 28 covers the human-baseline ceiling this chapter just cited a downstream result from; the training-data curation, the leakage firewall, and the model-selection protocol behind the 27B student model live in `reproductions/lensjudge/parity/FINDINGS.md` Phase C/D, not in this guide.
- **Flow architectures below the change-of-variables level.** Chapter 23 tells you what a normalizing flow computes and why NeuTra preconditioning helps; it does not derive a coupling layer, a spline transform, or `flowjax`'s internals.

Connect to the repo

- `reproductions/claude-giga-lens/cgl/marg.py:31-55` — the Occam term Exercise 29.1 asks you to require positive-definiteness of.
- `reproductions/claude-giga-lens/cgl/flows.py:1-21` — the flow pullback-density docstring, row 2 of the closed ledger.
- `reproductions/claude-giga-lens/cgl/guards.py:74-91` and `:129-142` — two of the five questions already turned into runtime assertions, each citing its own incident by name.
- `reproductions/claude-giga-lens/cgl/e2.py:554,557-558` — the Hessian eigendecomposition that caught the saddle; Exercise 29.3 turns it into a third guard.
- `reproductions/claude-giga-lens/CAMPAIGN.md:133` — the "P1c MONEY NUMBER" entry the γ Ledger traces to; `:168-169` — the implementation-agent stall behind The frontier.
- `reproductions/claude-giga-lens/papers/main.tex` — the report itself; the paper decoder maps every `\label{sec:...}` in it to a chapter of this book.
- `reproductions/lensjudge/parity/FINDINGS.md:238-243` — Phase D's non-inferiority result for the trained student model.
- `site/guide_src/worked_examples.py` and `site/guide_src/lint_guide.py` — this guide's own version of `guards.py`.
- `site/guide_src/contract/outline.yml` — the frozen chapter table behind the paper decoder's right-hand column.

Exercises

Exercise 29.1 — the Occam term needs positive-definiteness

Chapter 23 derived $\int \exp(-\frac{1}{2}\mathbf{x}^\top A\mathbf{x})\,d\mathbf{x} = (2\pi)^{k/2}/\sqrt{\det A}$ by the substitution $\mathbf{x} = A^{-1/2}\mathbf{u}$; `marg.py`'s $-\frac{1}{2}\log\det A$ is that identity's logarithm. Using this, explain in two or three sentences why `cgl/marg.py`'s Cholesky factorization of A requires A to be positive *definite* — not merely symmetric — for the number it computes to mean anything, and connect that requirement to what Chapter 5 found at the correlated-likelihood MAP.

> **Solution**
>
> A Cholesky factorization $A = LL^\top$ exists in real arithmetic only when A is positive definite: a negative or zero eigenvalue makes $\sqrt{\det A}$ complex or zero, and the "probability density" the integral computes stops meaning anything, because a Gaussian with negative-definite curvature has no finite total mass to normalize in the first place. That is exactly the fact Chapter 5 found at the correlated-likelihood MAP: minimum eigenvalue -14.85, five of forty-six directions negative . A Laplace/Occam computation built on that indefinite Hessian is not merely inaccurate — it is asking for the log of a negative number, which is the algebraic reason (not just the sampling reason) a saddle-seeded metric had to be abandoned rather than patched.

Exercise 29.2 — coincidence, or connection?

The Log-Det Ledger's real instance of the Occam term is 161.6145 nats (the correction from marginalizing 28 shapelet amplitudes at the production point). The γ Ledger's evidence swing is 191.1 nats . These are suspiciously close in size. Are they the same effect wearing two names, or an unconnected coincidence? Say what each number actually measures before you answer.

Solution

The Occam term is a single additive correction, evaluated once at (approximately) the MAP, for profiling out the 28 linear shapelet amplitudes of *one* marginalized model — it appears, at nearly the same value, on both sides of *any* comparison between two fits of that same model family, diagonal or correlated, steep basin or low. The evidence swing is a difference of two *entire* per-basin evidence integrals (steep minus low), computed once under a diagonal likelihood and once under a correlated one — a change to the noise covariance and whitening operator, nothing to do with how the linear amplitudes are profiled out. Tracing the actual formulas, there is no algebraic term shared between $-\frac{1}{2}\log\det A$ (Ch. 22) and $\log Z_{\text{steep}} - \log Z_{\text{low}}$ (Ch. 25) — the first is a per-evaluation constant that mostly cancels in any such difference, the second is exactly that kind of difference computed on an unrelated axis. Two numbers of similar order for a problem with tens of dimensions is not itself evidence of a shared cause; it is the same trap as the report's own 17σ claim, run in reverse — a coincidence in magnitude is not a derivation, and this book's method is to require the derivation before it accepts the pattern.

Exercise 29.3 — write the missing guard

`cgl/guards.py` already encodes Questions 2 and 3 from this chapter as runtime assertions. Sketch, in a few lines of pseudocode, a guard for Question 4 — the saddle — using the Hessian eigendecomposition `cgl/e2.py:554,557-558` already computes. What should it assert, and at what stage of the pipeline should it run?

Solution

A minimal version, in the same style as `assert_model_subtracted_sky`:

```
def assert_map_is_a_mode(eigvals, min_eig_floor=0.0):
    """map_polish must land on a mode, not a saddle (min_eig=-14.85,
    5/46 negative sank two rounds of PHMC budget before this was
    root-caused; CAMPAIGN.md "P1c metric-fix attempts")."""
    n_neg = int((eigvals <= min_eig_floor).sum())
    if n_neg > 0:
        raise GuardError(
            f"MAP Hessian has {n_neg} non-positive eigenvalue(s) "
            "(saddle, not a mode) - polish further, or route to a "
            "global sampler (tempered SMC) instead of local HMC."
        )
```

It should run immediately after `map_polish`, before any momentum metric is built from the Hessian — exactly where `cgl/e2.py:554` already computes the eigendecomposition for diagnostic printing, but did not stop the pipeline from spending two more rounds of budget on a Laplace metric anyway. The guard cannot fix the saddle; it only stops the campaign from re-discovering, by two expensive failed runs, a fact the eigenvalues already stated on the first one.

Exercise 29.4 — the vote

Chapter 1 asked you to guess a number, and Chapter 25 graded the guess. This chapter asks a different vote: given everything in The frontier, if you had one CS-trained graduate student and three months, would you point them at (a) the source-model and PSF systematics that keep the γ bracket from closing, (b) the sampler benchmark's efficiency-versus-robustness gap, or (c) the multi-agent research process itself? Defend your choice in three or four sentences, citing at least one specific number or incident from this chapter.

Solution

No single answer is correct; the honest reason to weigh all three is that this campaign's own scope decisions named (a) and (b) as open items but never once flagged the agent-liveness gap as a research item at all. A case for (c): a four-day stall from a missing watchdog will recur on every future campaign this group runs, regardless of which physics pillar is under study, while (a) and (b) each fix one number or one benchmark and stop. A counter-case for (b): it is a clean, well-scoped algorithms problem with a benchmark already built to beat (2.6–307× efficiency spread, a 3× target GL-NT missed by two orders of magnitude) — exactly the kind of problem a CS thesis needs, where (c) is a practice, not a deliverable. Either answer, defended with a specific number rather than an adjective, is the point: a vote with no citation attached is exactly what this book has spent 28 chapters teaching you not to accept.